PROBLEM FRAMING • EVIDENCE • RELIABILITY • SYSTEMS

Machine Learning

by Design

From Problem Framing to Reliable Systems

Sam Urmian

Machine Learning by Design

From Problem Framing to Reliable Systems

Sam Urmian

Researcher, SLATE: Centre for the Science of Learning & Technology

University of Bergen

sam.urmian@uib.no

Contents

Preface **v**

Mathematical Bridge: Just-in-Time Review **3**

Mathematical Bridge I: Linear Algebra for AI Thinking **3**

Mathematical Bridge II: Probability, Risk, and Learning Signals **25**

I Foundations **47**

1 Framing Learning Problems **49**

1.1 Problem-Solving Technique: Decision-First Framing 50
1.2 Opening Dilemma: A Messy Real Problem 51
1.3 Brief Orientation: Where Machine Learning Fits . 52
1.4 Prediction Is Not the Decision 54
1.5 Objective Hierarchy: Goal, Proxy, Label, Metric . 56
1.6 The Five Commitments 58
1.7 Proxy Audit: From Vague Goal to Defensible Task 60
1.8 What Data Actually Represents 61
1.9 Data-Centric ML: Improve the Evidence, Not Only the Model 62
1.10 Evaluation Preview: Unseen Data and Hidden Targets 64

1.11 A Bad Framing Memo Rewritten 65
1.12 Worked Framing Memo: Student Support 66
1.13 Framing Is Iterative, Not One-Shot 67
1.14 Brief Orientation: Learning Settings as Kinds of Feedback 68
1.14.1 Reinforcement Learning: A Different Feedback Regime 69
1.15 Common Mistakes in First Machine Learning Projects 70
1.16 Looking Ahead 70

2 Prediction, Loss, and Decision Rules 81
2.1 Opening Question: What Would Count as Good Here? 84
2.2 Reported Metric Before Training Loss 85
2.3 Training Loss as the Quantity Optimized During Learning 87
2.3.1 Regression Losses 89
2.3.2 Classification Losses 91
2.4 From Score to Action: Threshold, Ranking, and Abstention 93
2.4.1 Threshold Tradeoffs 94
2.4.2 Cost Matrices and Threshold Selection . . . 96
2.4.3 Ranking Under a Fixed Budget 98
2.4.4 Top-k and Retrieval Metrics 99
2.4.5 Abstention and Review 100
2.4.6 ROC and Precision-Recall Curves 101
2.5 Confidence Quality: Calibration 102
2.5.1 Base-Rate Shift and Why Precision Moves . 103
2.5.2 Uncertainty and Abstention 105
2.6 Validation, Cross-Validation, and Test Discipline . 106
2.6.1 Cross-Validation When Data Is Limited . . 107
2.6.2 Hyperparameter Search Without Test Leakage 108
2.7 Metric Failure Gallery 109
2.8 Chapter Artifact: A Metric and Action Worksheet 111
2.9 Looking Ahead 112

3 Generalization, Leakage, and Evaluation Discipline 117
3.1 Opening Dilemma: The Validation Score Looks Great. Can We Trust It? 118
3.2 Can the Result Survive New Data? Generalization and Overfitting 119
3.3 Protected Evaluation Workflow 120
3.4 How Experiments Lie: Leakage 122
3.5 Baseline Ladder and Sanity Checks 124
3.6 Metric Failure Gallery 125
3.7 Chapter Artifact: A Minimal Evaluation Plan . . . 127
3.8 Common Mistakes in Evaluation 128
3.9 Looking Ahead 128

4 Linear Models and Representation 133
4.1 Opening Dilemma: Is One Weighted Sum Enough? 135
4.2 Representation Is the Model 136
4.2.1 Raw Inputs and Feature Maps 137
4.2.2 Interaction Features and Basis Expansion . 139
4.2.3 Common Feature Types 140
4.3 The Generic Linear Pipeline 141
4.4 Linear Regression as the First Concrete Case . . . 143
4.4.1 Residuals and Squared Error 143
4.4.2 Minimizing the Loss: Gradient Descent . . 144
4.4.3 Worked Example: Predicting Apartment Prices 145
4.4.4 Residual Diagnostics 147
4.4.5 Linear in the Parameters, Not Necessarily in the World 147
4.5 Logistic Regression as Score-to-Probability Modeling 148
4.5.1 From Score to Probability 149
4.5.2 Worked Example: A Spam Probability . . . 151
4.6 Reading Coefficients Without Fooling Yourself . . 153
4.6.1 Scale Matters 154
4.6.2 Correlation Matters 154
4.6.3 Missing Variables Matter 156
4.7 Prediction Is Not Intervention 157
4.7.1 Confounding Changes the Story 157

4.7.2 What Stronger Causal Language Requires . 158
4.8 Controlled Simplicity: Regularization 158
4.9 Why Linear Models Are the Right First Baseline . 161
4.10 Sparse Linear Models Stay Strong in Text and Tabular Work 162
4.11 Failure Diagnosis, Not Method Shopping 163
4.12 Chapter Artifact: A First Baseline Design Sheet . 165
4.13 Common Mistakes with Linear Models 166
4.14 Looking Ahead 167

5 Structure Beyond Linearity 171
5.1 Opening Question: What Kind of Structure Is Missing? 173
5.2 A Structural Map Before the Methods 173
5.3 Method Selection by Missing Structure 175
5.4 Trees: Conditional Logic and Region-Specific Rules 177
5.4.1 Choosing a Split 178
5.4.2 A Tree and Its Regions 181
5.4.3 Depth, Overfitting, and Pruning 182
5.5 From Single Trees to Ensembles: Same Structure, More Stability 183
5.5.1 Why Averaging Helps: Variance Without Bias 183
5.5.2 Random Forests 186
5.5.3 Boosted Trees 188
5.6 Margin-Based Classification: Support Vector Machines 188
5.7 Local Similarity: When Geometry Is the Model . . 191
5.7.1 Distance Defines the Model 192
5.7.2 Worked Example: Local Similarity in Student Profiles 192
5.8 Scale, Similarity, and the Curse of Dimensionality . 195
5.9 Unsupervised Structure: Organization Without Labels 196
5.10 Clustering as Hypothesis, Not Discovery 197
5.10.1 Choosing K 200
5.11 Clustering Validity and the Danger of Naming Clusters Too Fast 201

5.12 Dimensionality Reduction as a Lens 202
5.13 Chapter Artifact: A Structure Diagnosis Sheet . . 205
5.14 Common Mistakes with Structural Methods 205
5.15 Looking Ahead 206

II Deep Learning 213

6 Optimization and Neural Networks 215
6.1 Opening Dilemma: When Engineered Structure Stops Being Enough 217
6.2 A Neural Network as a Learned Feature Pipeline . 218
6.3 Why Nonlinearity Matters 220
6.4 Worked Anchor Example: One Forward Pass, Fully Legible 221
6.5 Loss as the Teaching Signal 225
6.5.1 Why Not Optimize Accuracy Directly? . . . 226
6.5.2 Multiclass Outputs 227
6.6 Optimization as Repeated Correction 227
6.6.1 Learning Rate 229
6.6.2 Adaptive Optimization: Momentum, Adam, and AdamW 230
6.6.3 Optimization Failure Versus Generalization Failure 232
6.6.4 Mini-Batches, Initialization, and Stability . 234
6.6.5 Batch Normalization as Signal Stabilization 235
6.6.6 Debugging a Broken Training Run 236
6.7 Backpropagation as Credit Assignment 238
6.7.1 A Small Worked Gradient Example 239
6.7.2 What Changes If ReLU Is Off? 241
6.8 Generalization Returns: Regularization as Bias Toward Stable Patterns 243
6.8.1 Weight Decay, Early Stopping, Dropout, and Augmentation 244
6.9 Capacity, Data Regime, and Label Noise 245
6.10 Chapter Artifact: A Minimum Credible Neural Training Report 247
6.11 Representation Learning as the Payoff 249

6.12 Common Mistakes with Neural Networks 250
6.13 Looking Ahead 251

7 Representation Learning and Foundation Models 255
7.1 Opening Dilemma: Solve One Task, or Learn a Reusable Space? 256
7.2 Representation as a Change of Coordinates 258
7.3 What Makes a Representation Reusable? 259
7.4 Embeddings and Geometry: Useful Space, Still a Model Choice 261
7.4.1 A Tiny Embedding Example 262
7.4.2 Cosine Similarity 263
7.5 How Representations Are Learned: Supervisory Pressure 264
7.5.1 Supervised Pressure 265
7.5.2 Self-Supervised Pressure 265
7.5.3 Examples Across Modalities 267
7.5.4 Autoencoders as Reconstruction-Based Representation Learning 268
7.5.5 Multimodal Transfer as Shared Structure Across Modalities 269
7.6 Why Self-Supervision Scales 271
7.7 Transfer Begins: Pretrain, Probe, Adapt 272
7.8 Linear Probe as the First Diagnostic 273
7.8.1 When Reuse Fails or Is the Wrong Fit . . . 274
7.9 Probe, Freeze, Partial Tune, or Full Tune? 275
7.10 Adaptation Depth: Freeze, Partial Tuning, Full Tuning 277
7.11 Attention: Building Context-Dependent Representations 279
7.11.1 Worked Example: Attention as Weighted Averaging 281
7.12 Why Transformers Matter 282
7.13 Foundation Models as a Workflow Shift 283
7.13.1 Why They Are Powerful 285
7.13.2 Why They Need More Discipline, Not Less 285
7.14 Same Base Model, Different Claims 286

7.14.1 Prompting, Retrieval, and Fine-Tuning Solve Different Problems 286
7.15 Chapter Artifact: A Reuse and Adaptation Plan . 289
7.16 Common Mistakes with Representation Learning . 291
7.17 Looking Ahead 292

III Modalities 295

8 Vision: Learning from Images 297
8.1 Opening Dilemma: The Same Object Can Look Like Many Different Pixel Arrays 298
8.2 Problem-Solving Technique: Invariance Audit . . . 299
8.3 Task Formulation Before Architecture Depth . . . 301
8.3.1 Classification 301
8.3.2 Detection 301
8.3.3 Segmentation 302
8.3.4 Tracking and Other Structured Vision Tasks 303
8.3.5 Why Structured Outputs Need Structured Evaluation 303
8.3.6 A Running Case: What the Camera Actually Sees 305
8.4 Annotation Ontology and Label Ambiguity 306
8.5 Why Raw Pixels Are Not Enough 308
8.6 Convolution as a Repeated Local Question 308
8.6.1 Why Parameter Sharing Matters 309
8.6.2 Worked Example: Detecting a Vertical Edge 310
8.7 Hierarchy and Approximate Invariance 311
8.7.1 Pooling 311
8.8 Augmentation as a Label-Preserving Claim 312
8.9 Augmentation Claim Matrix 313
8.10 Modern Visual Backbones and Transfer 314
8.11 Shortcut Learning, Robustness, and Failure Analysis 314
8.11.1 Shortcut Learning 315
8.11.2 Worked Example: Slice Breakdown Reveals the Wrong Cue 315
8.11.3 Robustness and Domain Shift 317
8.11.4 Annotation Policy and Label Noise 318

8.12 Robustness by Nuisance-Factor Stress Tests 318
8.13 Deployment-Camera Realism 320
8.14 Saliency Is Diagnostic Evidence, Not Proof 321
8.15 Chapter Artifact: A Vision Design and Stress-Test Card 321
8.16 Common Mistakes in Vision 322
8.17 Looking Ahead 323

9 Language: Context and Grounding 329
9.1 Opening Dilemma: Same Words, Different Meaning; Same Meaning, Different Words 331
9.2 Problem-Solving Technique: Context Audit 332
9.3 Task Shape Before Representation Depth 334
9.3.1 Classification and Sequence Labeling 334
9.3.2 Retrieval and Ranking 335
9.3.3 Translation, Summarization, Question Answering, and Generation 336
9.4 Worked Workflow: One Message, Several NLP Jobs 337
9.5 Tokenization as a Modeling Decision 339
9.5.1 Why Tokenization Matters 339
9.6 Bag-of-Words as the First Serious Lexical Baseline 340
9.6.1 Worked Example: Same Words, Different Meaning 341
9.7 N-Grams as the Local-Order Repair 342
9.8 From Static to Contextual Meaning 343
9.8.1 Static Embeddings 343
9.8.2 Contextual Representations 343
9.9 Sequence Models and Attention as Context Builders 344
9.9.1 Recurrent Models 344
9.9.2 Attention and Transformers in NLP 344
9.10 Language Modeling as Sequence Prediction 345
9.10.1 Worked Example: A Tiny Bigram Model . 346
9.10.2 Encoder-Decoder Models as Conditional Sequence Mapping 347
9.11 Decoding, Retrieval, and Grounding as Behavior Design 349
9.11.1 Decoding Changes the Behavior 349
9.11.2 Retrieval and Grounding Change the Claim 349

9.11.3 Retrieval Contract: What Evidence Must Be Present? 351
9.11.4 Retrieval Pipeline Anatomy 351
9.11.5 Retrieval Failure Often Precedes Generation Failure 352
9.11.6 Escalation and Abstention Change the Behavior Too 353
9.12 From Language Model to Instruction-Following Model 354
9.13 Prompting as a Behavioral Interface 356
9.13.1 Prompting Interacts with Retrieval and Fine-Tuning 358
9.14 Evaluation Matched to Task 360
9.14.1 A Simple Evaluation Map 361
9.14.2 Classification and Ranking Metrics 361
9.14.3 Retrieval and Recommendation Are Top-of-List Problems 362
9.14.4 Perplexity and Likelihood 362
9.14.5 Generation and Grounding Evidence 363
9.14.6 A Practical Rubric for Grounded Answers . 363
9.14.7 Prompt Sensitivity and Evaluation Instability 364
9.14.8 Hallucination Failure Types and Response Design 364
9.15 Bias, Privacy, and Social Meaning Are Part of the Technical Problem 365
9.16 Chapter Artifact: An NLP Context-and-Grounding Card 366
9.17 Common Mistakes in NLP 367
9.18 Looking Ahead 368

10 Audio, Time, and Prediction-Time Realism 373
10.1 Opening Dilemma: Time Turns Examples Into Processes 374
10.2 Problem-Solving Technique: Prediction-Time Audit 375
10.3 Two Domains, One Audit 377
10.4 Task Shape Before Model Family 379
10.4.1 Speech Recognition 379

10.4.2 Keyword Spotting and Audio Classification 379
10.4.3 Speaker Verification 379
10.4.4 Forecasting and Anomaly Detection 380
10.5 Audio as a Representation Problem 380
10.5.1 Waveforms and Frequencies 380
10.5.2 Worked Example: A Tone Change Over Time 381
10.5.3 Time-Frequency Tradeoffs 382
10.5.4 Helpful but Incomplete 383
10.6 Time Series as Evolving Process 383
10.6.1 Trend, Seasonality, Lagged Dependence, and Regime Change 383
10.6.2 Worked Example: Lag Features by Hand . 384
10.6.3 Irregular Sampling and Missingness as Signal 385
10.6.4 Horizon Is Part of the Task Definition . . . 386
10.7 Sequence Models as Memory Choices 386
10.7.1 Hidden Markov Models and Structured State 386
10.7.2 Recurrent Models and LSTMs 386
10.7.3 Transformers and Flexible Context Access . 387
10.8 Evaluation That Respects Time 388
10.8.1 Chronological Splits 388
10.8.2 Why Random Splits Can Mislead 388
10.8.3 A Temporal Leakage Taxonomy 388
10.8.4 Available Information at Prediction Time . 389
10.8.5 Worked Example: A Split That Solves the Wrong Problem 391
10.8.6 Backtesting Regimes and Horizon-Specific Claims 392
10.9 Naive Baselines as Honesty Checks 393
10.10 Deployment Realism: Noise, Latency, and Drift . . 394
10.10.1 Noise and Channel Variation 394
10.10.2 Latency 394
10.10.3 Concept Drift 394
10.11 Chapter Artifact: A Sequential Design Card 395
10.12 Common Mistakes in Sequential Modeling 396
10.13 Looking Ahead 397

IV Evidence, Reliability, and Systems 403

11 Experiments, Error Analysis, and Scientific Claims 405

11.1 Opening Dilemma: The Score Improved, So What? 406
11.2 Problem-Solving Technique: Claim Audit 408
11.2.1 A Seven-Step Protocol for Scientific Claims 408
11.2.2 Write the Claim Sheet Before the Result Exists 410
11.2.3 Worked Claim Audit for the 0.004 Gain . . 411
11.3 Experimental Protocol as Control of Premises . . . 412
11.3.1 What Must Be Recorded 413
11.3.2 Split Logic Is Part of the Premise 414
11.4 Baselines and Controls: Better Than What? 414
11.4.1 Task-Dependent Strong Baselines 414
11.4.2 Weak Controls Produce Weak Claims . . . 415
11.5 Repeated Runs and Uncertainty: Stable or Lucky? 415
11.5.1 Why One Number Is Not Enough 416
11.5.2 Worked Example: Clean Versus Shifted Digits Across Seeds 416
11.5.3 Uncertainty Is Part of the Finding 417
11.5.4 Cross-Validation for Limited Data 418
11.5.5 Worked Example: Mean Improvement Is Not the Whole Uncertainty Story 419
11.5.6 Paired Comparison Usually Matters More Than Separate Means 420
11.5.7 Confidence Intervals and Practical Importance 422
11.5.8 Hyperparameter Budget and Selection Protocol 422
11.5.9 Multiple Comparisons and Researcher Degrees of Freedom 424
11.5.10 Human Evaluation Can Also Be an Experimental Protocol 426
11.6 From Improvement to Mechanism: Ablation 426
11.6.1 What Good Ablation Looks Like 427
11.6.2 Mechanism Evidence Is Stronger Than Bundle Evidence 427

11.6.3 Worked Example: What Actually Caused the Gain? 427
11.7 Error Analysis and Slice Diagnosis: Where Does It Fail? 428
11.7.1 Worked Example: Overall Accuracy Hides a Failure 429
11.7.2 Useful Slices 430
11.7.3 Mistake Reading Is Part of Scientific Interpretation 430
11.7.4 From Error Clusters to the Next Experiment 430
11.7.5 When the Right Conclusion Is to Change the Claim 431
11.8 Claim Scope: Local, External, and Negative Results 432
11.8.1 Local Claims Versus Broad Claims 432
11.8.2 External Validity 432
11.8.3 Negative and Partial Results 433
11.9 Chapter Artifact: An Experiment Claim Sheet . . 433
11.10 Common Mistakes in Experimental Work 435
11.11 Looking Ahead 435

12 Reliability: Bias, Robustness, Privacy, and Safety 439
12.1 Reliability Is a Pipeline Property 440
12.2 Bias as a Full-Pipeline Failure 442
12.2.1 Harm Taxonomy Before Metrics 443
12.2.2 Overall Performance Can Hide Unequal Harm 444
12.2.3 A Subgroup Reliability Dashboard 445
12.2.4 Worked Example: Accuracy Is Not the Whole Story 446
12.2.5 Fairness Metrics Are Tools, Not Magic . . . 447
12.2.6 Calibration Gaps Can Also Be Reliability Gaps 448
12.3 Robustness Under Change 449
12.3.1 Common Forms of Shift 450
12.3.2 Spurious Correlations 450
12.3.3 Worked Example: A Reliability Tradeoff . . 451
12.3.4 Stress Tests Should Be Designed, Not Improvised 452

12.4 Privacy as a Modeling Constraint 453
12.4.1 Privacy Threat Models Before Privacy Tools 454
12.4.2 Why Models Can Leak Information 455
12.4.3 Common Privacy Responses 455
12.4.4 Privacy Has Tradeoffs 455
12.4.5 Privacy Risk Lives in the Whole Data Path 456
12.5 Safety and Bounded Failure 457
12.5.1 Hazards, Not Just Errors 458
12.5.2 Safety Cases Make Bounded Failure Concrete 458
12.5.3 Human Oversight and Fallbacks 459
12.5.4 Selective Automation Requires Usable Uncertainty 460
12.5.5 Reliability Is Sometimes a Reason to Decline Deployment 461
12.6 Monitoring Reliability After Launch 461
12.7 Why Ethics and Engineering Cannot Be Separated 462
12.8 Chapter Artifact: A Reliability Review Card . . . 463
12.9 Common Mistakes in Reliability Work 464
12.10 Looking Ahead 464

13 From Models to Systems 469
13.1 Systems Thinking Changes the Question 472
13.1.1 From Model Objective to Service Objective 472
13.1.2 Service-Level Objectives Make the Objective Operational 472
13.1.3 Success Includes More Than Accuracy . . . 474
13.2 Requirements Before Training 475
13.2.1 Worked Example: Queue Capacity Changes the Threshold 475
13.2.2 Requirements Should Be Written Down Early 476
13.3 Data Contracts and Training-Serving Consistency . 477
13.3.1 Why Small Pipeline Mismatches Matter . . 477
13.3.2 Worked Example: A Retrieval Contract Bug 477
13.3.3 Data Contracts Reduce Ambiguity 478
13.4 Prediction Is Not the Decision 479
13.4.1 Thresholds Are Policies 479

13.4.2 Queueing, Review Capacity, and Human Bottlenecks 479
13.4.3 Ranking Allocates Exposure 480
13.4.4 Selective Prediction and Abstention 481
13.4.5 Worked Example: Confidence, Coverage, and Review 482
13.5 Monitoring After Deployment 483
13.5.1 What Should Be Monitored 483
13.5.2 Delayed Labels Make Monitoring Harder . . 484
13.5.3 Rollback and Recovery 484
13.6 Post-Deployment Experimentation 484
13.6.1 Shadow Mode and Canary Release 485
13.6.2 Online Experiments and Guardrails 485
13.6.3 Not Every System Should Be Randomized . 486
13.7 Human-in-the-Loop Design 486
13.7.1 Humans Need the Right Interface 486
13.7.2 Humans Also Introduce Failure Modes . . . 487
13.8 Documentation, Ownership, and Iteration 487
13.8.1 What Documentation Should Capture . . . 487
13.8.2 Iteration Should Be Scientific 488
13.9 Chapter Artifact: A System Readiness Brief 488
13.10 When Not to Build an AI System 490
13.10.1 Simple Rules Sometimes Win 490
13.10.2 The Organization May Lack the Needed Support 490
13.10.3 High Stakes Raise the Evidence Bar 490
13.11 Worked System Walkthrough: Course Support . . 491
13.11.1 Step 1: Define the Service Objective 491
13.11.2 Step 2: Define the Input Contract 491
13.11.3 Step 3: Train and Evaluate the Components 491
13.11.4 Step 4: Choose the Action Policy 492
13.11.5 Step 5: Design the Review Interface 492
13.11.6 Step 6: Monitor and Iterate 492
13.11.7 Contrast Case: A Pedestrian-Alert Service . 493
13.12 Common Mistakes in Systems Work 493
13.13 Closing Perspective 494

A Mathematical Foundations for the Main Text 499

A.1 Scalars, Vectors, and Matrices 500
A.1.1 Matrices as Datasets 501
A.1.2 Matrix Multiplication as Many Dot Products at Once 501
A.2 Summation, Means, and Indexing 502
A.2.1 Why Means Matter 502
A.3 Functions That Appear Everywhere 503
A.3.1 Logarithms and Exponentials 503
A.3.2 The Sigmoid Function 503
A.3.3 The Softmax Function 503
A.4 Probability Language 504
A.4.1 Random Variables and Distributions 504
A.4.2 Joint, Marginal, and Conditional Probability 505
A.4.3 Bayes' Rule 505
A.5 Expectation, Variance, and Covariance 506
A.5.1 Expectation 506
A.5.2 Variance 506
A.5.3 Covariance and Correlation 506
A.6 Loss Functions and Objectives 507
A.6.1 Squared Error 507
A.6.2 Log Loss and Cross-Entropy 507
A.6.3 Regularized Objectives 508
A.7 Derivatives, Gradients, and the Chain Rule 508
A.7.1 Partial Derivatives and Gradients 508
A.7.2 Gradient Descent 509
A.7.3 Why the Learning Rate Matters 509
A.7.4 The Chain Rule 509
A.8 Norms, Distance, Similarity, and Regularization . . 510
A.8.1 Norms 510
A.8.2 Distance 510
A.8.3 Cosine Similarity 511
A.9 Statistics for Evaluation 511
A.9.1 Sample Mean and Sample Standard Deviation 511
A.9.2 Standard Error and Confidence Intuition . 511
A.9.3 Confusion Matrix Language 512
A.9.4 Thresholds 512
A.9.5 Calibration 513

A.10 Linear Algebra for Representation Intuition 513
A.10.1 Linear Maps 513
A.10.2 Projection Intuition 514
A.10.3 Eigenvectors and Principal Components . . 514
A.11 A Tiny Bridge to Optimization Practice 514
A.12 How to Use This Appendix 515

Conclusion **517**

Preface

Modern AI is often taught in one of two unsatisfying ways. In the first version, the machine-learning core of the field is presented as a catalog of methods: one chapter on linear models, another on trees, another on neural networks, another on transformers. In the second version, it becomes a toolkit course: load a library, fit a model, report a score, move on. Both versions miss the deeper intellectual habit. Real model-based AI work is not mainly about collecting algorithms. It is about solving problems under uncertainty, with incomplete data, imperfect proxies, constrained decisions, and evidence that must be defended.

This book is written around that stronger idea. It does not ask first, "Which model is famous?" It asks, "What is the problem, what evidence do we have, what claim are we making, and what would justify trusting the result?" The book is therefore problem-driven rather than algorithm-driven. Methods matter, but they appear as answers to modeling dilemmas, not as museum exhibits.

This book is for undergraduate students, teachers, and serious self-learners who want a coherent first serious introduction to machine learning, evaluation, and reliable AI systems. It assumes curiosity, some programming experience, and enough mathematical maturity to follow equations when they are explained carefully. The volume includes two mathematical bridge chapters on linear algebra and on probability, risk, and learning signals, but most first-course readers should begin with Chapter 1 and then use the bridge chapters as just-in-time review whenever notation

or derivations start to feel thin. The main line is undergraduate-first. Readers who want a faster or deeper route can go further through prerequisite bridges, advanced notes, challenge exercises, and companion code.

That scope matters. This is not a full survey of artificial intelligence in the broad classical sense. It is first a machine-learning textbook with an AI-systems endgame. Topics such as search, planning, reasoning, and reinforcement learning appear only for orientation when they clarify where machine learning fits. The book's center of gravity is the machine-learning core of modern AI together with the habits needed to evaluate, stress-test, and operationalize model-based systems.

The title is broad because the problems are broad, but the shelf claim is precise. This book should be read as a machine-learning text about framing, evidence, reliability, and systems, not as a general survey of all of AI.

Suggested Reading Paths

- **First machine-learning course.** Read Chapter 1 first. Then continue in order, using the Mathematical Bridge chapters only when later chapters point back to them or when the notation starts to slow you down.
- **Math-refresh path.** Read Chapter 1 first for the problem-framing spine, then work through the two Mathematical Bridge chapters before continuing to Chapter 2.
- **One-semester teaching path.** Chapters 1–6 form the core. If time allows, add Chapter 7 on representation learning and foundation models. Then choose the modality chapter most relevant to the course (Chapters 8–10) and finish with Chapters 11–13 for experiments, reliability, and systems.

The organizing framework of the book is simple and reusable. Every serious AI project eventually has to answer seven questions:

1. What decision are we actually supporting?

2. What learning task is the closest defensible proxy for that decision?
3. What data stands in for the world, and what could bias or leak it?
4. What representation makes the relevant structure visible?
5. What is the simplest serious baseline?
6. What evidence would justify the claim?
7. How will the model output become an action inside a real system?

Chapters 1 through 3 introduce the technical core of that method through task framing, predictive judgment, and evaluation discipline. Later chapters extend the same logic outward. Experiments become arguments. Thresholds become policies. Reliability becomes a property of the whole pipeline. Deployment becomes part of the modeling problem rather than a final packaging step. The final part of the book is therefore not an ethics appendix. It is where evidence, reliability, and systems work are brought into the same technical frame.

The book also returns repeatedly to one institutional setting: course support. The early chapters use it for framing, metric choice, and baseline design. The later language, experiments, reliability, and systems chapters turn that same setting into a grounded support assistant with retrieval, escalation, and deployment constraints. The back half also cross-checks those same habits against a dashboard-camera pedestrian detector first developed in the vision chapter, so the systems endgame does not read as if it belongs only to educational NLP.

To make those habits teachable, the book uses a small set of recurring problem-solving techniques. Some are cross-cutting across the whole book, while others become especially important in modalities such as vision, language, sequential prediction, or transfer learning.

- **Decision-first framing**: start from the action and constraint, not from the model.

- **Proxy audit**: separate the real goal from the measurable target.
- **Representation audit**: ask what information the representation must preserve before blaming the model family.
- **Baseline ladder**: begin with the simplest serious baseline, then justify each increase in complexity.
- **Claim audit**: state the exact claim before interpreting the experiment.
- **Slice diagnosis**: inspect where performance fails before trusting any aggregate metric.
- **Threshold as policy**: treat thresholds, abstention, and review paths as operational choices.
- **Pipeline realism**: ask what breaks between notebook success and system success.
- **Reuse audit**: separate gains from pretrained representation, downstream head, retrieval, prompting, or fine-tuning.
- **Invariance audit**: ask what kinds of variation should and should not change the answer.
- **Context audit**: ask how much context and grounding the task really requires.
- **Prediction-time audit**: ask what information is truly available at decision time and what future information must stay forbidden.

The book uses a small set of recurring teaching devices:

- **Problem-Solving Lens**: the short framing question that tells the reader what kind of modeling decision the chapter is really about.
- **Problem-Solving Technique**: a reusable reasoning move for choosing, debugging, or redesigning a method.
- **Math Lens**: one compact derivation, theorem, or identity that explains why the method behaves as it does.

- **From Scratch**: a tiny implementation, usually in plain Python, that exposes the mechanism without relying on a library.
- **Failure Pattern**: one common way a seemingly reasonable method can mislead when the representation, metric, or claim is weak.
- **Chapter Artifact**: one memo, card, sheet, or brief that turns the chapter's reasoning into a reusable deliverable.

The companion labs carry the larger implementations. The book uses small executable fragments and short proofs so that the mechanism becomes visible before software details take over. Each chapter should also leave the reader with a reusable artifact such as a memo, claim sheet, review card, or readiness brief.

These are not slogans. They are the actual habits that connect introductory machine learning to serious practice, advanced coursework, empirical research, and deployment work. A student who learns only formulas and architectures may still fail badly on real problems. A student who learns how to frame, test, diagnose, and redesign an AI system can carry that method across domains.

The book is not trying to replace the classical references that already define much of the field's technical backbone. Texts such as Bishop [6], Hastie et al. [27], Goodfellow et al. [23], and Murphy [46] remain essential. What this book changes is the teaching order, the recurring problem-solving language, and the explicit bridge from model construction to evidence and system design.

The book is paired with companion code, but the two are designed to age differently. The book emphasizes concepts, notation, pseudocode, and durable examples. The repository carries the runnable implementations. When libraries evolve, the code can be updated without forcing the text to depend on transient APIs or fashionable tooling.

The book is organized in five parts: a mathematical bridge, foundations, deep learning, modalities, and evidence, reliability, and systems. That structure is deliberate. The bridge chapters make the notation and training language legible without turning the

book into a theorem-first text. The next chapters build the problem-solving core: framing, prediction quality, decision rules, evaluation discipline, baselines, and early model families. The middle chapters explain how representation learning changes the modeling workflow. The final chapters widen the unit of analysis from models to experiments, reliability, and systems.

Each chapter follows a common rhythm: a modeling dilemma, a problem-solving lens, worked examples, one or more of the recurring teaching devices above, common failure modes, and exercises designed to strengthen four habits of mind: checking concepts, building small solutions, diagnosing failure, and designing better systems. The goal is not to make AI easy by removing rigor. The goal is to make rigor usable.

About This Project

This project started as part of NOKI, the AI Olympiad in Norway, to train the Norwegian national team for the International Olympiad in Artificial Intelligence (IOAI). NOKI is directed by Sam Urmian, the author of this book, who led the programme during the period in which this manuscript was developed. Sam Urmian is a researcher at the University of Bergen, Norway.

The manuscript is also publicly available in the project repository at `https://github.com/mlgorithm/ml-by-desing`.

For any questions, corrections, or feedback regarding the content of this book, please contact the author at `sam.urmian@uib.no`.

Mathematical Bridge: Just-in-Time Review

Mathematical Bridge I: Linear Algebra for AI Thinking

Learning Goals

- Explain why AI work starts with a representation choice before it starts with a model choice.
- Read vectors, matrices, dot products, and shapes as modeling statements rather than as isolated symbols.
- Compute a weighted score by hand and explain what each term contributes.
- Distinguish distance, similarity, and scale, and explain why they matter for downstream model behavior.
- Interpret norms, linear maps, and projection as tools for controlling, comparing, and fitting representations.
- Produce a one-page representation card before reporting that a model family is suitable for a task.

> **Prerequisite Bridge.** Use this chapter as just-in-time review: read it now if linear-algebra notation feels rusty, or return when later chapters start leaning on vectors, matrices, and geometry. This chapter assumes comfort with high-school algebra: variables, substitution, and evaluating expressions. Calculus is

not required. Familiarity with Python lists or arrays is helpful but not required. Students who have already taken a linear algebra course should still skim this chapter because the AI-specific vocabulary (representation, coordinate, weighted evidence) will recur throughout the book.

Many students first meet linear algebra as a list of operations: add vectors, multiply matrices, solve systems. That is not the most helpful entry point for AI. In AI, linear algebra is the language that makes representations visible. It tells us what counts as one example, what counts as one coordinate, how many examples are being processed at once, what a score really is, and what kinds of similarity or geometry a method is quietly assuming.

This bridge chapter is therefore not a miniature mathematics course. It is a reading course in the mathematical objects that later chapters will use constantly. If a student can look at a vector or matrix and ask, "what real thing does each row, column, or coordinate mean here?" then the mathematics is already doing its job.

Problem-Solving Lens. Before asking which model family to use, ask how one case is represented, what each coordinate means, what counts as similarity, and what a weighted combination of the coordinates would mean in plain language.

Problem-Solving Technique. Representation audit. Before blaming or praising a model, ask four questions: what counts as one example, what each coordinate means, what information has already been discarded, and what notion of similarity the representation makes easy.

Keep the chapter artifact in view from the start. As later sections introduce shapes, weighted scores, geometry, and projection, the intended payoff is a short *representation card*: what one example is, what the coordinates mean, what geometry matters, and what a linear score would mean in plain language.

Opening Case: One Student, One Email, One Image Patch

Suppose we want to support three different AI tasks:

- detect which students may need early human outreach,
- estimate whether an email is spam,
- estimate how many animals appear in a small aerial image crop.

These tasks look different on the surface, but they share one structural step. Each task needs a way to turn one case into a mathematical object that a model can read. A student becomes a list of measurements. An email becomes word counts, sender cues, or contextual embeddings. An image crop becomes pixel intensities or learned feature maps.

That is the first linear-algebra lesson of the book. AI does not begin with magic. It begins with a decision about representation.

Task	One possible machine-readable description
Student support	Attendance rate, quiz average, missing assignments, and recent platform activity
Spam filtering	Counts of important words, sender reputation cues, link patterns, and message length
Image counting	Pixel values or local visual features extracted from a small patch

Three tasks, one shared question. What counts as one example, and what mathematical object will stand in for it?

Vectors Are Organized Descriptions

A vector is an ordered collection of coordinates. In AI language, a vector is often just one represented example.

For the student-support task, we might record four normalized

quantities for one student:

$$x = \begin{bmatrix} 0.92 \\ 0.80 \\ 0.10 \\ 0.35 \end{bmatrix}$$

where the coordinates mean:

- 0.92: attendance rate,
- 0.80: quiz average,
- 0.10: fraction of assignments missed,
- 0.35: recent platform inactivity score.

The notation matters less than the interpretation. The vector is not just "four numbers." It is one student expressed through four chosen coordinates. Another representation choice would give a different vector and therefore a different modeling problem.

> **Prerequisite Bridge.** Read x_j as "the j-th coordinate of the represented example." If the notation feels formal, translate it immediately back into the task language: attendance, quiz average, missed assignments, or inactivity.

Two practical habits begin here.

First, order matters. Once the first coordinate means attendance and the second means quiz average, we cannot silently swap them later.

Second, scale matters. A coordinate measured from 0 to 1000 behaves differently from one measured from 0 to 1. Later chapters on nearest neighbors, regularization, and optimization all depend on this simple point.

Object	Shape	What the dimensions mean
Student feature vector	$(d,)$	d features for one student
Tabular dataset matrix	(n, d)	n examples, each with d features
Bag-of-words batch	(B, V)	B documents in the batch, vocabulary size V
Token embedding sequence	(T, d)	T token positions, each with a d-dim embedding

Table 1: Shape is a modeling statement. Rows and columns have different jobs, and confusing them causes dimension errors that are easy to miss in silent failure modes.

Shapes, Batches, and Why Dimension Errors Matter

A vector has a shape. A dataset has a shape. A batch of images or sequences has a shape. Shape is not bookkeeping. Shape states what counts as one case, what counts as one feature family, and what gets processed together. Misreading shape is one of the most common sources of silent bugs in AI work.

Consider three objects that appear across later chapters.

One tabular row. A single student described by four features is a vector $x \in \mathbb{R}^4$. Its shape is $(4,)$: four numbers, one case.

A bag-of-words batch. Suppose we process 32 emails at once, each represented as a count vector over a vocabulary of 10,000 words. The batch is a matrix of shape $(32, 10000)$: 32 rows for 32 cases, 10,000 columns for 10,000 features.

A token-by-token embedding sequence. A sentence tokenized into T tokens, where each token is embedded into d dimensions, forms a matrix of shape (T, d). Here the rows are positions in time or sequence, not independent cases. Transposing would swap that meaning and break any downstream operation that assumes the row is a token position.

A useful mini-checklist before writing any matrix operation:

- What are the rows? (examples? positions? channels?)
- What are the columns? (features? vocabulary items? time steps?)
- What is repeated across the batch? (independent cases or steps of the same sequence?)
- What would break if the object were transposed?

> **Failure Box.** Dimension errors often succeed silently in Python. If a dot product or matrix multiply happens to have compatible shapes because an accidental transpose happens to work numerically, the model will train without error messages but learn from the wrong relationship. Checking what the dimensions mean before the operation—not just checking that the numbers multiply—is how practitioners catch this class of bug.

Dot Products Are Weighted Evidence

Once an example is represented as a vector, the simplest serious model asks a clean question: how much evidence does each coordinate contribute to one score?

That is the job of the dot product. If a weight vector is

$$w = \begin{bmatrix} -1.6 \\ -1.1 \\ 2.4 \\ 1.8 \end{bmatrix},$$

then the score $w^\top x$ is

$$w^\top x = (-1.6)(0.92) + (-1.1)(0.80) + (2.4)(0.10) + (1.8)(0.35).$$

Working it out term by term gives

$$w^\top x = -1.472 - 0.880 + 0.240 + 0.630 = -1.482.$$

The point of this arithmetic is not the final decimal. The point is the interpretation.

- Good attendance pushes the score down because its weight is negative.
- Strong quiz performance also pushes the score down.
- Missing assignments pushes the score up.
- Inactivity also pushes the score up.

In other words, the dot product is a weighted evidence summary.

Coordinate	Value	Weight	Contribution
Attendance	0.92	-1.6	-1.472
Quiz average	0.80	-1.1	-0.880
Assignments missed	0.10	2.4	0.240
Inactivity score	0.35	1.8	0.630

Reading rule. Multiply each coordinate by how much it matters, then add the contributions.

> **Math Lens. Why the dot product appears everywhere.** A dot product is a weighted sum, and weighted sums are the simplest way to combine many weak signals into one score. Linear regression, logistic regression, attention scores, nearest-prototype methods, and many neural-network layers all reuse this structure. The later models are richer, but the weighted-sum idea never really leaves.

From One Example to Many: Matrices As Datasets

Once there is more than one example, it is wasteful to write every case separately. A matrix packages many vectors together.

Suppose three students are represented as rows:

$$X = \begin{bmatrix} 0.92 & 0.80 & 0.10 & 0.35 \\ 0.70 & 0.62 & 0.30 & 0.55 \\ 0.98 & 0.91 & 0.00 & 0.10 \end{bmatrix}.$$

This matrix has shape 3×4: three rows, four columns. In the dataset view:

- rows are examples,
- columns are features,
- shape records how many cases and features are present.

This is one of the most useful sanity checks in practical ML. If a student thinks a dataset has n examples and d features, then the design matrix should have n rows and d columns. Many beginner bugs are simply shape misunderstandings wearing fancy clothes.

Matrix multiplication then applies one scoring rule to every row at once:

$$Xw = \begin{bmatrix} 0.92 & 0.80 & 0.10 & 0.35 \\ 0.70 & 0.62 & 0.30 & 0.55 \\ 0.98 & 0.91 & 0.00 & 0.10 \end{bmatrix} \begin{bmatrix} -1.6 \\ -1.1 \\ 2.4 \\ 1.8 \end{bmatrix} = \begin{bmatrix} -1.482 \\ -0.092 \\ -2.389 \end{bmatrix}.$$

Now the same weighted rule has produced one score for each student.

> **Math Lens.** If the rows of X are examples $x_1^\top, \ldots, x_n^\top$, then
>
> $$Xw = \begin{bmatrix} x_1^\top w \\ x_2^\top w \\ \vdots \\ x_n^\top w \end{bmatrix}.$$
>
> So matrix-vector multiplication is just repeated dot products, one row at a time. The matrix is not doing a different kind of math; it is packaging the same linear rule so it can be applied to many examples at once.

```
def linear_scores(X, w, b=0.0):
    scores = []
    for row in X:
```

```
        if len(row) != len(w):
            raise ValueError("row and weight vector must
                have the same length")
        score = b
        for value, weight in zip(row, w):
            score += value * weight
        scores.append(score)
    return scores
```

The point of this tiny function is not software convenience. It is to make the mechanism visible. A matrix is not mystical. It is a compact way to store many represented cases, and matrix multiplication applies one structured computation to every row.

Example 0.1 (Matrix-Vector Product by Hand)**.** Consider three students, each described by two features: attendance rate and number of late submissions. The design matrix and weight vector are

$$X = \begin{bmatrix} 0.90 & 1 \\ 0.60 & 4 \\ 0.80 & 0 \end{bmatrix}, \qquad w = \begin{bmatrix} -1.5 \\ 2.0 \end{bmatrix}.$$

The columns mean: first column = attendance rate, second column = number of late submissions. The weights say: good attendance lowers the risk score (-1.5), while late submissions raise it ($+2.0$).

Computing Xw row by row:

- **Student 1:** $(-1.5)(0.90) + (2.0)(1) = -1.35 + 2.0 = 0.65$
- **Student 2:** $(-1.5)(0.60) + (2.0)(4) = -0.90 + 8.0 = 7.10$
- **Student 3:** $(-1.5)(0.80) + (2.0)(0) = -1.20 + 0.0 = -1.20$

$$Xw = \begin{bmatrix} 0.65 \\ 7.10 \\ -1.20 \end{bmatrix}.$$

Student 1's weighted score is 0.65, meaning the model predicts they are moderate risk: reasonable attendance partially offsets one late submission. Student 2's score of 7.10 signals high risk because four late submissions dominate. Student 3's score of -1.20 indicates low risk: solid attendance with no late submissions.

This is the key idea: the same weight vector w is applied to every row simultaneously. The matrix does not change the structure of the computation—it only packages it so every student gets scored in one operation.

Sparse, Dense, and Missingness as Representation Facts

Representations are not only numeric. They differ in density, noise, and missingness, and those differences change how models behave.

A *sparse* representation has many zero coordinates. A bag-of-words vector for one email drawn from a vocabulary of 50,000 words will usually have fewer than 200 nonzero entries. Most words in the vocabulary simply do not appear in that message. Sparsity is not a deficiency of the representation. It is information: a coordinate being zero means the corresponding feature is absent.

A *dense* representation has very few zeros. A learned word embedding of dimension 300 will typically have all 300 coordinates populated with small but nonzero values. Dense representations are compact and support smooth geometric operations like cosine similarity, but they offer no inherent interpretability for individual coordinates.

A *partially observed* representation has coordinates that are missing for structural reasons, not random ones. A student record may have quiz scores for seven of ten weeks because a student joined late. A sensor trace may be missing every other reading because the device transmits on an alternating schedule. In each case, the absence of a value carries information about the process

Type	Example	What absence means
Sparse	Bag-of-words email vector	Word did not appear in this message
Dense	Learned word embedding	Every coordinate is informative; no zero convention
Partially observed	Student engagement record with gaps	Student may not have been enrolled yet, or event did not fire

Table 2: Representation quality includes density, which coordinates are absent, and whether absence is informative. A zero can mean "not present," "missing by design," or "unknown," and those meanings matter for modeling.

that generated the data.

Three practical lessons follow.

Sparse representations support fast inner products but can break distance-based methods. If most of two vectors' coordinates are zero, their Euclidean distance is dominated by the dimensions where both are nonzero—or by the few nonzero dimensions of one but not the other. Cosine similarity can handle this better in some cases, but neither operation treats "both missing this word" and "both present on this word" the same way.

Dense embeddings lose interpretability at the coordinate level. A single coordinate of a word embedding does not mean anything human-readable. The space as a whole may encode meaning geometrically, but the coordinates individually do not.

Missingness in tabular data is often not random. Imputing a missing value with a mean or zero assumes the value was random noise. But if the value is missing because the student dropped before week eight, or because a sensor was offline after a power failure, the imputation will inject a different kind of noise. Later chapters on reliability and deployment return to this point. For now, the lesson is that "missing" is a value in the representation, not an error to be erased before modeling.

Geometry: Distance, Similarity, and Scale

Once cases live in vectors, geometry starts to matter. The geometry is not a picture drawn after the fact. It is part of the model.

If two represented students are

$$x = \begin{bmatrix} 0.92 \\ 0.80 \\ 0.10 \\ 0.35 \end{bmatrix}, \qquad z = \begin{bmatrix} 0.88 \\ 0.78 \\ 0.12 \\ 0.40 \end{bmatrix},$$

their Euclidean distance is

$$\begin{aligned} \|x - z\|_2 = \big[&(0.92-0.88)^2 + (0.80-0.78)^2 \\ &+ (0.10-0.12)^2 + (0.35-0.40)^2\big]^{1/2}. \end{aligned}$$

This distance is small, so the two cases are close under that geometry.

Cosine similarity asks a different question. Instead of asking whether the vectors are close in absolute position, it asks whether they point in nearly the same direction:

$$\cos(\theta) = \frac{x^\top z}{\|x\|_2 \|z\|_2}.$$

This is useful when overall scale should matter less than pattern. In language or embedding-style problems, two long vectors can represent similar meaning even if one is globally larger than the other.

> **Failure Box.** Distance-based methods can fail for a boring reason: one coordinate dominates because it is measured on a much larger numeric scale. If age is stored in years but income is stored in dollars, then raw Euclidean distance mostly measures income. That is not a property of the world. It is a property of the chosen units.

Norms, Size, and Regularization Intuition

A norm measures the size of a vector. Two common norms are

$$\|w\|_1 = \sum_{j=1}^{d} |w_j|, \qquad \|w\|_2 = \sqrt{\sum_{j=1}^{d} w_j^2}.$$

Later chapters use these ideas for regularization. The reason is simple. A model with very large coefficients can become brittle, overly reactive, or too willing to fit accidental quirks in the training data. Penalizing large coefficient size is one way of expressing a preference for simpler behavior unless the data strongly demands otherwise.

Students should not hear the word *regularization* and think only "extra term in the objective." The modeling question is more useful:

> How much coefficient size, complexity, or sensitivity are we willing to tolerate before we think the fit has become unstable?

Linear Maps, Projections, and Fitting

A linear map is a rule that transforms one vector into another while preserving weighted-sum structure. In AI terms, it is a structured feature transformation.

This matters because many models do not work directly on the raw input. They work on transformed coordinates. Even when later chapters discuss embeddings or learned internal layers, one recurring question survives:

> What transformation is making the relevant structure easier to see?

Projection is the cleanest first fitting intuition. Suppose the world is more complicated than the model class we have allowed. Then training is often not finding the truth exactly. It is finding the closest allowed approximation under a chosen notion of error.

That is the useful least-squares picture. A linear model does not win because it captures every detail of reality. It wins when the best projection into the linear family is already good enough for the task.

Math Lens. For the simplest projection onto a one-dimensional direction u with $\|u\|_2 = 1$, the fitted point is

$$\hat{x} = (x^\top u)u.$$

The residual is $r = x - \hat{x}$. The key geometric fact is

$$r^\top u = 0,$$

so the leftover error is orthogonal to the direction we kept. In least squares, this is the meaning of "best fit": after projection, no remaining error points along the allowed direction.

Math Lens. A projection map is itself a linear map. If u is a unit direction, then

$$P_u(x) = (u^\top x)u$$

keeps only the component of x that lies along u. Applying it twice changes nothing:

$$P_u(P_u(x)) = P_u(x).$$

That is the algebraic version of geometric fitting: once a vector has been projected onto the allowed line, projecting again cannot improve it further.

Advanced Note. Students often hear that linear models are "simple" and stop there. A better statement is that linear models define a restricted hypothesis class with very transparent geometry. That restriction is sometimes a weakness, but it is also what makes the class interpretable, diagnosable, and often surprisingly competitive.

Example 0.2 (Projection in One Dimension)**.** Suppose we observe three data points $y = [2, 4, 6]$ and we want to fit a constant model—a single number c that stands in for all three values. Which constant is best?

The mean is $\bar{y} = (2+4+6)/3 = 4$. The residuals from the mean are

$$r = [2-4,\ 4-4,\ 6-4] = [-2,\ 0,\ 2].$$

The sum of squared residuals is

$$SS(\bar{y}) = (-2)^2 + 0^2 + 2^2 = 4 + 0 + 4 = 8.$$

Now try a different constant, say $c = 3$:

$$r = [2-3,\ 4-3,\ 6-3] = [-1,\ 1,\ 3].$$

$$SS(3) = (-1)^2 + 1^2 + 3^2 = 1 + 1 + 9 = 11.$$

The sum of squares rose from 8 to 11 when we moved away from the mean. Any other constant will produce a sum of squares no smaller than 8. In coefficient terms, the best constant is 4. In geometric terms, the data vector $[2, 4, 6]$ projects onto the constant-model class as $[4, 4, 4]$, the unique member of that class that sits closest to the data under squared error.

This is the geometry of least-squares fitting in miniature. The model class (all constant functions) is a restricted family. Training finds the member of that family that is closest to the target values. The residuals $[-2, 0, 2]$ are orthogonal to the constant direction—they average to zero, meaning no constant shift can reduce the error further.

Linear Separability and What a Hyperplane Can Actually Express

A linear score $w^\top x + b$ divides the input space into two half-spaces: examples where the score is positive and examples where it is negative. In two dimensions, that boundary is a line. In three dimensions, it is a plane. In d dimensions, it is a hyperplane.

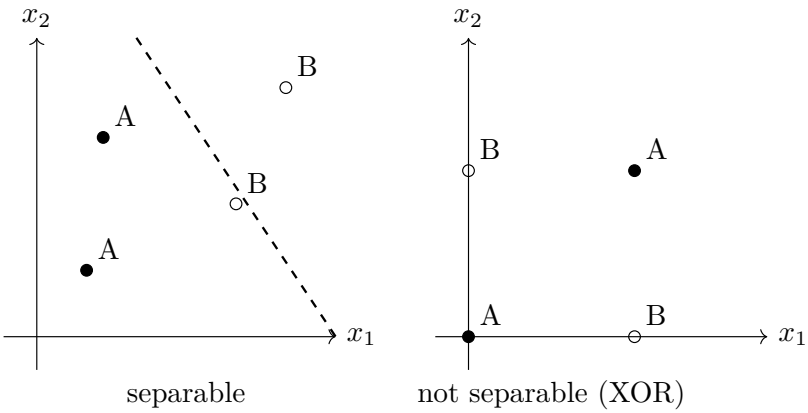


Figure 1: Left: a line separates the two classes. Right: the XOR pattern cannot be separated by any line. The filled points are class A; the open points are class B.

Understanding what a hyperplane can and cannot separate gives a concrete reason for why later chapters need trees, kernels, and neural nonlinearities. Without this picture, "nonlinearity matters" can sound like folklore. With it, the limitation becomes a specific geometric fact.

A case where a line succeeds. Consider four points in two dimensions:

$$(1,1),\ (1,2),\ (3,3),\ (4,4).$$

Suppose the first two are class A and the last two are class B. A line such as $x_1 + x_2 = 5$ separates them cleanly. Here the linear model is sufficient.

A case where a line fails: the XOR pattern. Now consider four points:

$$(0,0),\ (1,1),\ (1,0),\ (0,1).$$

Suppose $(0,0)$ and $(1,1)$ are class A, and $(1,0)$ and $(0,1)$ are class B. Any line through this plane will put at least one point on the wrong side. There is no linear decision boundary that correctly separates these four points.

The XOR pattern is not a contrived puzzle. It represents the general situation where the decision depends on an *interaction*

between features: being at position $(0, 0)$ and being at position $(1, 1)$ belong to the same class not because of either coordinate alone, but because of the combination. A linear model working on raw coordinates cannot represent that.

This is the geometric reason why interaction features, kernel methods, tree splits, and neural nonlinearities matter. Each one extends the expressible boundary beyond the flat hyperplane. Trees do it by recursive axis-aligned cuts. Kernel methods do it by implicitly working in a richer feature space. Neural networks do it by composing nonlinear transformations.

> **Advanced Note.** The formal study of which problems linear models can solve is part of the theory of VC dimension and linear dichotomies. For this book, the geometric picture is enough: a hyperplane is a powerful tool when the relevant decision boundary is roughly flat in the feature space, and a poor tool when the boundary requires reasoning about feature interactions.

Low-Dimensional Structure

Real datasets often have many coordinates but fewer truly important directions. In image, language, and tabular problems alike, some features overlap, some move together, and some mostly repeat what others already say.

The practical lesson is not that every dataset can be compressed aggressively. The lesson is that high ambient dimension does not always mean high effective complexity. This intuition later supports PCA, embeddings, representation learning, and transfer learning.

> **Math Lens.** A projection onto an allowed subspace is a linear map. For a unit direction u, define
>
> $$P_u(x) = (u^\top x)u.$$
>
> Then P_u keeps only the component of x that lies along

u. Applying it twice does nothing new:

$$P_u(P_u(x)) = P_u(x).$$

That idempotence is the algebraic version of geometric compression: once the allowed direction has been kept, projecting again does not change the answer.

Students do not need a full singular-value-decomposition treatment here. They do need the idea that useful structure can live in fewer directions than the raw coordinate count suggests.

Chapter Artifact: Representation Card

Representation Card

Field	What to write
One example is...	Name the basic case: one student, one email, one image patch, one customer session, one hour of sensor data
Coordinates are...	Name each feature family and what each coordinate means
What is lost?	State what the representation throws away or blurs
What geometry matters?	Distance, angle, overlap, ordering, or something else
What does a linear score mean?	Explain what a weighted sum would count as evidence for or against
Main representation risk	State the most likely mismatch between representation and task

If this card cannot be written clearly, later model debates are usually premature.

Common Mistakes

Mistake Checklist.

- **Treating a vector as just a bag of numbers.** A vector is a chosen representation, not a random pile of values.
- **Confusing row count with feature count.** One case and one feature family play different roles in a dataset.
- **Ignoring scale when distance or nearest neighbors matter.** Geometry depends on units.
- **Treating a large dot product as self-explanatory.** A weighted sum only matters relative to the chosen representation.
- **Thinking that more dimensions automatically mean more useful information.** Extra coordinates can just as easily add noise or redundancy.

Looking Ahead

This bridge supplied the geometric language of representation: vectors, matrices, weighted scores, distance, and projection. The next bridge adds the uncertainty language that makes those scores trainable and interpretable: probability, loss, empirical risk, and gradient signals. Together they prepare the main text, where Chapter 1 turns those mathematical objects into a defensible learning problem.

Chapter Summary

- Linear algebra matters in AI because representation is already part of the model.
- A vector is one represented case, and a matrix is many represented cases organized together.

- A dot product is a weighted evidence summary.
- Distance, similarity, and scale are modeling assumptions, not just visualization choices.
- Norms measure size and later become regularization tools.
- The main deliverable of the chapter is a representation card written before declaring that a model family is a good fit.

Study Questions

1. Why is representation choice already a modeling decision rather than a neutral preprocessing step?
2. In plain language, what does the dot product $w^\top x$ mean in a decision-support problem?
3. Why can Euclidean distance and cosine similarity lead to different judgments about what counts as "similar"?
4. What does it mean to say that fitting can be understood as projection into an allowed model class?

Exercises

1. Write a four-coordinate vector for a student-support problem and explain the meaning of each coordinate in one sentence.
2. For the vector $x = [0.9, 0.7, 0.2]^\top$ and the weight vector $w = [-2, 1, 3]^\top$, compute $w^\top x$ and explain what each contribution means.
3. Construct a 3×4 design matrix for three emails with four hand-chosen features. State clearly what the rows and columns represent.
4. Give one situation in which cosine similarity would be a better comparison than Euclidean distance, and one situation in which it would be worse.

5. Fill out a representation card for spam filtering, student support, or a simple image task. If you plan to carry one running case through the book, keep the same case and add one sentence saying which coordinates Chapter 1's framing memo would treat as acceptable inputs at prediction time.

Challenge Exercises

1. Show with a tiny numeric example that rescaling one feature can change which case is the nearest neighbor, even when all other coordinates stay the same.
2. Give two different representations for the same task, explain what each preserves, and argue which one makes a linear score more meaningful.
3. Suppose a target cannot be expressed exactly by any weighted sum of the available features. Explain why least-squares fitting can still be read as finding the closest allowed predictor.

Mathematical Bridge II: Probability, Risk, and Learning Signals

Learning Goals

- Distinguish clearly between a score, a probability, a decision rule, a pointwise loss, and a dataset-level training objective.
- Read conditional probability and expectation in plain language before manipulating formulas.
- Explain why logarithms, exponentials, sigmoid-style transformations, and cross-entropy appear naturally in ML.
- Interpret empirical risk as the average training signal over a dataset rather than as an abstract formula to memorize.
- Use a light functional-analysis viewpoint to explain why a predictor is a function and why a risk objective is a rule that assigns one number to the whole predictor.
- Read gradients and the chain rule as credit-assignment tools for reducing training loss.

> **Prerequisite Bridge.** Use this chapter as just-in-time review: read it now if probability or training-signal notation feels thin, or return when later chapters start leaning on loss, calibration, and gradients. This chapter assumes comfort with fractions, ratios,

and the idea of an average. Familiarity with logarithms is helpful; they are introduced with worked examples. Calculus (derivatives) appears only in a few short derivations, especially the gradient section, and is explained from scratch when it appears. Students who have taken probability theory before should read the chapter anyway: the AI-specific vocabulary (loss, empirical risk, gradient as credit signal) differs from the standard probability course framing.

Many students meet probability and optimization in fragments. One course teaches conditional probability. Another introduces derivatives. A machine-learning lecture suddenly writes down log loss, empirical risk, and gradient descent as if the connection were obvious. This chapter slows that chain down.

The goal is not to flood the reader with formalism. The goal is to make later training objectives readable. When a later chapter writes down a loss, a score, a calibration curve, or a gradient update, the student should be able to answer three questions immediately:

- What uncertain quantity is being modeled?
- What mistake is being penalized?
- What signal tells the parameters how to move?

Problem-Solving Lens. Separate five objects that students often collapse together: the model score, the probability interpretation of that score, the decision rule that acts on it, the pointwise loss on one example, and the dataset-level objective minimized during training.

Problem-Solving Technique. Loss as learning signal. A loss is not the final judgment of the system. It is the message training uses to say, "this prediction behaved well" or "this prediction behaved badly." The message should be interpreted before it is optimized.

Keep the chapter artifact in view from the start. As later sections separate score, probability, decision rule, pointwise loss, and dataset objective, the goal is to fill a *learning-signal card* rather than to memorize disconnected formulas.

Opening Question: What Does A Score Like 0.8 Mean?

Suppose a model outputs 0.8 for a student, an email, or a medical case. That number could mean several different things.

- It could be an uncalibrated score: larger means riskier, but the number is not yet an honest probability.
- It could be a probability estimate: about 80% of comparable cases are expected to be positive.
- It could be the input to a decision rule: act if the number is above some threshold.

Those are three different interpretations. Later modeling mistakes often begin when they are treated as the same thing.

This bridge chapter therefore starts with language. A model output is not automatically a trustworthy probability, and a probability is not automatically a decision.

Conditional Probability In Plain Language

Machine-learning prediction usually asks about uncertainty *given* observed information. The notation

$$P(y = 1 \mid x)$$

is read as "the probability that the target is positive given the observed input x."

The vertical bar means "given." That is the central operational idea.

- In spam filtering, x might be the message text and metadata.

- In student support, x might be attendance, quizzes, and submissions.
- In image classification, x might be the visual measurement.

Conditional probability matters because AI systems do not predict labels in a vacuum. They predict them given whatever evidence the representation makes available.

> **Prerequisite Bridge.** If the notation feels dense, translate it immediately into a sentence: "given the evidence we observed, how plausible is the target event?" That plain sentence is the main idea. The notation is only the compact version.

Expectation As Average Consequence

Expectation is the language of average consequence. If a quantity can vary, its expectation summarizes the average value we would expect across repeated comparable cases.

This idea appears constantly in AI:

- average loss over a training set,
- average reward in decision-making,
- average error over future unseen cases,
- average outcome under a probabilistic forecast.

Suppose a support team can contact one student. Contacting a truly struggling student creates $+5$ units of benefit, while contacting a student who did not need help costs -1 unit of attention. If a model estimates a 0.8 chance that the student truly needs help, then the expected value of contacting is

$$0.8(5) + 0.2(-1) = 4 - 0.2 = 3.8.$$

The purpose of this calculation is not to claim that life has units called 3.8. The purpose is to show how uncertain outcomes become decision-relevant through expected consequence.

Base Rates, Bayes Decisions, and Cost-Sensitive Action

Probability and action are different things. A model may produce a well-calibrated probability estimate, yet the right action can still differ across teams, deployments, and time periods, even when the probability stays the same.

The bridge from probability to action goes through three factors: the estimated probability, the costs of the available actions, and the base rate of the relevant event.

Consider the student-support example. Suppose the model assigns a probability of 0.35 to the event that a student will disengage. Is that high enough to trigger an outreach call?

That question cannot be answered from the probability alone. It depends on:

- the cost of a correct intervention: if it prevents serious academic harm, the benefit is large,
- the cost of a false alarm: staff time and student inconvenience,
- the base rate: if 30% of students routinely score near 0.35, routine outreach may be sustainable; if only 2% ever reach that level, the same policy creates a very different workload.

Example 0.3 (Same Probability, Different Action)**.** A medical triage team and a marketing team both receive a model score of 0.35. The medical team decides to escalate because missing a true case has a severe consequence and resources are available. The marketing team decides not to act because their cost-per-contact is high relative to the expected value of conversion at 35% likelihood. Same probability, different correct action. The model output did not contain the action. The action required a cost structure on top of the probability.

Bayes-optimal decisions formalize this idea. In the simplest binary setup, suppose the model score is already the calibrated posterior probability of the positive class, correct decisions carry no extra utility beyond avoiding mistakes, and the relevant costs

are C_{FP} (cost of a false positive) and C_{FN} (cost of a false negative). Then the optimal threshold τ satisfies

$$\tau = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

If false negatives are nine times more costly than false positives, the optimal threshold is 0.1, not 0.5. The default threshold of 0.5 therefore encodes an implicit equal-cost assumption that is often wrong in practice. Base-rate changes still matter, but they matter through the calibrated posterior probabilities the threshold is applied to.

> **Problem-Solving Technique. Separate three questions.** Does the model output an honest probability? What are the costs of each kind of mistake? What is the current base rate? Only when all three are answered does it make sense to ask what the action should be.

A common undergraduate mistake is to assume that the best classifier, the best probability forecaster, and the best decision policy are automatically the same object. They are not. A model with strong ranking quality may still produce poorly calibrated probabilities. A well-calibrated model may still imply a different threshold in a high-stakes setting than in a low-stakes one. And both of those may be further adjusted when the base rate changes seasonally, demographically, or across deployment contexts.

Logarithms, Exponentials, and Why They Keep Appearing

Probabilities often multiply. If a sequence of events or labels is modeled jointly, the full likelihood can become a product of many factors. Logarithms are useful because they turn products into sums:

$$\log(ab) = \log a + \log b.$$

That simple identity is one reason log loss and log-likelihood show up throughout ML. Sums are easier to analyze and optimize than

long products.

Exponentials matter for the reverse reason. They turn linear or affine scores into positive quantities that can then be normalized into probabilities. The sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

maps any real score z into a number between 0 and 1. Large positive scores map near 1, large negative scores map near 0, and $z = 0$ maps to 0.5.

> **Math Lens.** If p is the probability of class 1, then the odds are $p/(1-p)$. Taking logs gives the log-odds $z = \log\big(p/(1-p)\big)$. Solving for p gives $p = e^z/(1+e^z) = 1/(1+e^{-z})$, which is exactly the sigmoid.

Students do not need to memorize this only as a formula. They should read it as a translation rule:

> a linear score can be turned into a number between 0 and 1 through a smooth nonlinear map

In simple binary classifiers, that number is often read as an estimated class probability. That reading is useful, but it is not magic. It depends on how the model was trained and on whether its outputs are later checked for calibration.

Loss Functions As Learning Signals

A loss function is a penalty on one prediction. It says how bad the current output was on one example. Three common losses are worth reading carefully.

- **Squared error**: $(y - \hat{y})^2$. Large misses are punished strongly.
- **Absolute error**: $|y - \hat{y}|$. Misses grow linearly.
- **Log loss / cross-entropy**: penalizes assigning low probability to the true label, especially when the model is confidently wrong.

For binary classification with label $y \in \{0, 1\}$ and predicted probability p, the pointwise log loss is

$$\ell(y, p) = -\big[y \log p + (1 - y) \log(1 - p)\big].$$

If the true label is 1, this becomes $-\log p$. If the model says $p = 0.99$, the penalty is tiny; if it says $p = 0.01$, the penalty is severe. This is why log loss is the standard teaching example for "confident mistakes are expensive."

Example 0.4 (Computing Cross-Entropy Loss by Hand)**.** Two models predict the probability that a student is at risk. The true label is $y = 1$ (the student genuinely is at risk).

- **Model A** predicts $p_A = 0.8$.
- **Model B** predicts $p_B = 0.3$.

Since $y = 1$, the log loss formula reduces to $\ell = -\log p$. Computing each:

$$L_A = -\log(0.8) \approx 0.22, \qquad L_B = -\log(0.3) \approx 1.20.$$

Model A is penalized far less because it was more confident in the correct direction. Model B assigned low probability to what actually happened, so its penalty is steep.

The log makes the penalty grow sharply when the model is confidently wrong. A model that predicts $p = 0.01$ for a true positive case suffers a loss of $-\log(0.01) \approx 4.61$—more than twenty times the penalty of Model A above. This asymmetry is exactly the behavior that makes log loss the standard choice when calibrated probability estimates matter.

The right question is never just "which loss is standard?" The better question is:

> What kind of wrong behavior does this loss care about, and what kind might it still permit?

For example, squared error reacts strongly to outliers. Absolute error is less sensitive. Log loss cares a great deal about confidence. A model that says 0.99 and is wrong is punished much more harshly than a model that says 0.60 and is wrong.

Loss	What it sees	Main modeling message
Squared error	Numeric discrepancy	Large misses matter much more than small ones
Absolute error	Numeric discrepancy	A few very large misses do not dominate as strongly
Log loss	Probability assigned to the truth	Confident mistakes are especially serious

Practical Note. The formulas above apply to binary classification with $y \in \{0, 1\}$. Many tasks have more than two classes. Chapter 6 introduces the standard multiclass extension: each class gets a raw score (a *logit*), softmax converts the vector of logits into probabilities that sum to one, and the loss penalizes the log-probability assigned to the true class. The binary version here is worth internalizing first because it makes every idea—confident mistakes, proper scoring, calibration—visible with the fewest symbols.

Proper Scoring Rules and Honest Probability Forecasts

Not every loss encourages a model to report its true beliefs. A *proper scoring rule* is one that makes honest forecasting the safest strategy: if a model genuinely believes the probability of an event is p, then the loss is minimized in expectation by reporting exactly p rather than some distorted value.

Log loss is the classic example. Suppose the true probability of a positive outcome is $p^* = 0.7$. A model that reports $\hat{p}$ incurs expected log loss

$$\mathbb{E}[\ell] = -p^* \log \hat{p} - (1 - p^*) \log(1 - \hat{p}).$$

Taking the derivative with respect to $\hat{p}$ and setting it to zero gives $\hat{p} = p^*$. Reporting the true probability minimizes expected log loss. Shading the report in either direction increases expected loss.

Accuracy, by contrast, is not a proper scoring rule for probabilities. Under accuracy, any probability above 0.5 produces the same binary prediction of class 1. A model is not penalized for being overconfident at 0.95 when the true probability is 0.7; both produce the same prediction. So accuracy gives no incentive to calibrate.

> **Math Lens.** A scoring rule $S(\hat{p}, y)$ is *proper* if $\mathbb{E}_{y\sim p^*}[S(p^*, y)] \leq \mathbb{E}_{y\sim p^*}[S(\hat{p}, y)]$ for all $\hat{p}$, with equality only when $\hat{p} = p^*$ in the strictly proper case. Log loss and Brier score are proper. Accuracy-based rewards are not. Proper rules are the right losses to use when calibrated probabilities matter for downstream decisions.

Why does this matter for AI practice? When a model output drives a threshold decision, the calibration of that output determines whether the threshold can be set reliably from a stated probability. A model that is 0.82 overconfident at high scores will, under a threshold-as-policy interpretation, flag many cases that a properly calibrated model would not flag. The team may then tighten the threshold empirically, but without understanding that the underlying model probabilities are systematically distorted, they cannot trust that the threshold will transfer to a new semester, a new patient cohort, or a new deployment context.

> **Practical Note.** When selecting a training loss, ask whether the loss rewards honest probability forecasts or merely correct binary predictions. For any application where the probability score will drive a downstream decision, prefer log loss or Brier score over accuracy as the training signal.

From One Example To Many: Empirical Risk

The ideal quantity we would like to minimize is future expected loss on the population of cases the system will actually face. We

almost never know that quantity directly. What training can compute is the average loss on the sample in hand, used as a practical stand-in for the unknown population risk.

> **Math Lens.** If f is a predictor and $\mathcal{D}$ is the data-generating process, then the population risk is
>
> $$R(f) = \mathbb{E}_{(X,Y)\sim\mathcal{D}}\big[\ell(Y, f(X))\big],$$
>
> while the empirical risk on a sample $(x_i, y_i)_{i=1}^n$ is
>
> $$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(y_i, f(x_i)\big).$$
>
> The sample average is the stand-in because it is the quantity we can compute now, and if the sample is representative it estimates the future loss we care about.

Training does not optimize one example at a time forever. It optimizes an objective over a dataset. If the model is f_θ, the pointwise loss is ℓ, and the dataset is $(x_i, y_i)_{i=1}^n$, then a standard training objective is

$$\mathcal{L}_{\text{train}}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(y_i, f_\theta(x_i)\big).$$

This is empirical risk: the average observed loss on the available sample.

The operational reading is more important than the notation:

- make a prediction on each training example,
- score that prediction with a pointwise loss,
- average the penalties,
- change the parameters to reduce the average.

This is the bridge from one mistake to a trainable objective.

Population Risk Versus Empirical Risk

Training computes one quantity. The quantity we actually care about is different.

The *empirical risk* is the average loss over the examples in hand. We can compute it. It changes as we update the parameters, and we minimize it during training.

The *population risk* is the expected loss over all the cases the system will face in deployment: future students, future emails, future images from a new camera. We cannot compute it directly. We have only a sample of those cases, and even that sample may not perfectly represent the future.

This gap is one of the deepest ideas in the book. Its consequences reach every chapter:

- a model that achieves very low training loss may still have high population risk if it has memorized accidents of the training sample,
- a model that generalizes well has low population risk even if its training loss is not minimal,
- validation and test sets are our only window onto population risk, which is why contaminating them with training decisions is so damaging.

Example 0.5 (Why Low Training Loss Is Not Yet Evidence)**.** Suppose a model memorizes every training example perfectly: for each student in the training set it has learned an exact rule that reproduces the label. Its training loss is zero. But when the model meets a new student—one it has never seen—it has no generalizable rule to fall back on. Its population risk may be high. The perfect training loss was a mirage: it said something about the sample but nothing about the distribution.

The correct mental discipline is to treat training loss as a signal about optimization and population risk as the actual goal. Every time someone says "the model improved," they should mean population risk improved, as estimated by held-out performance.

Saying "training loss went down" without that translation is not yet evidence of learning in the useful sense.

> **Problem-Solving Technique. Population risk discipline.** Whenever training loss is mentioned as evidence of learning, ask: what do the held-out numbers say? If only training loss has been reported, the argument is incomplete.

A Gentle Functional View Of Loss Functions

This next section does not require graduate-level analysis. It asks for one conceptual shift: stop looking only at the coordinates of θ, and start looking at the whole predictor that training is selecting.

This is the right place for one mathematical shift that many students never get stated clearly enough. A model is not only a parameter vector. It is also a function.

A Predictor Is A Function

When we write $f_\theta(x)$, the parameter vector θ matters, but the real object used at prediction time is the mapping $x \mapsto f_\theta(x)$. The deployed system receives an input and returns an output. In that sense, the trained model is a predictor function.

This viewpoint matters because later questions such as generalization, regularization, and hypothesis-class choice are easier to think about at the function level than at the parameter level.

Pointwise Loss Versus Risk Functional

The loss $\ell(y, \hat{y})$ is pointwise. It looks at one target and one prediction. The empirical risk, by contrast, takes the whole predictor and returns one number that summarizes how it behaved across the dataset.

That is the useful light functional-analysis idea in this book:

> a risk objective is a rule that assigns one scalar value to a whole predictor

Seen this way, the formulas above are not just notation changes. One is the ideal quantity defined by the real world; the other is the computable approximation used during training.

Only after the student understands that sentence does the term *functional* become useful.

Hypothesis Classes As Sets Of Allowable Predictors

Training is not choosing from all imaginable functions. It is choosing from a restricted class:

- all linear predictors on chosen features,
- all trees up to some depth,
- all neural networks of a chosen architecture,
- all models produced by a certain pretraining and adaptation pipeline.

This class is the hypothesis class. The class matters because every training objective answers the question, "which predictor in this allowed family looks best under the chosen risk?"

Norms, Complexity, and Regularization

Regularization becomes easier to interpret once predictors are seen as members of a function class. Penalizing coefficient size is one way of saying:

> among predictors that fit the data similarly well, prefer the less extreme one

In linear models this often appears as a penalty on coefficient norms. In broader function-class language, it is a preference for predictors that are simpler, smoother, or less sensitive according to a chosen measure of size.

Projection Intuition For Least Squares

Least squares can be read as a projection idea. The target outputs may not be matched perfectly by any predictor in the al-

lowed linear class. Training then finds the closest allowed predictor under squared error.

That is a deeper way to read regression. The model is not recovering every detail of the world. It is finding the best approximation available inside the chosen family.

> **Advanced Note.** This section does *not* need graduate functional analysis. The useful undergraduate level for this book is enough: predictor as function, risk as a rule on predictors, norm as size or complexity measure, and projection as fitting intuition. This book does not need Hahn–Banach, compactness theorems, or an abstract study of Banach spaces to make loss functions readable.

Gradients And Optimization

Once the training objective is defined, optimization asks how to reduce it. A derivative tells how one quantity changes when another changes slightly. A gradient collects those directional sensitivities across many parameters.

If the loss is $\mathcal{L}(\theta)$, then the gradient

$$\nabla_\theta \mathcal{L}$$

points in the direction of increasing loss. To reduce the loss, gradient descent moves the parameters in the opposite direction:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L},$$

where η is the learning rate.

The right mental model is simple:

- change a parameter slightly,
- see how the loss responds,
- collect those responses,
- move against the direction that increases loss.

```
theta = initial_parameters()
for step in range(num_steps):
    gradient = grad(training_loss, theta)
    theta = theta - learning_rate * gradient
```

The pseudocode is short because the mechanism is short.

Example 0.6 (One Gradient Descent Step by Hand)**.** Let the loss function be $L(w) = (w-3)^2$, a simple parabola with minimum at $w = 3$. Start at $w = 1$ with learning rate $\eta = 0.4$.

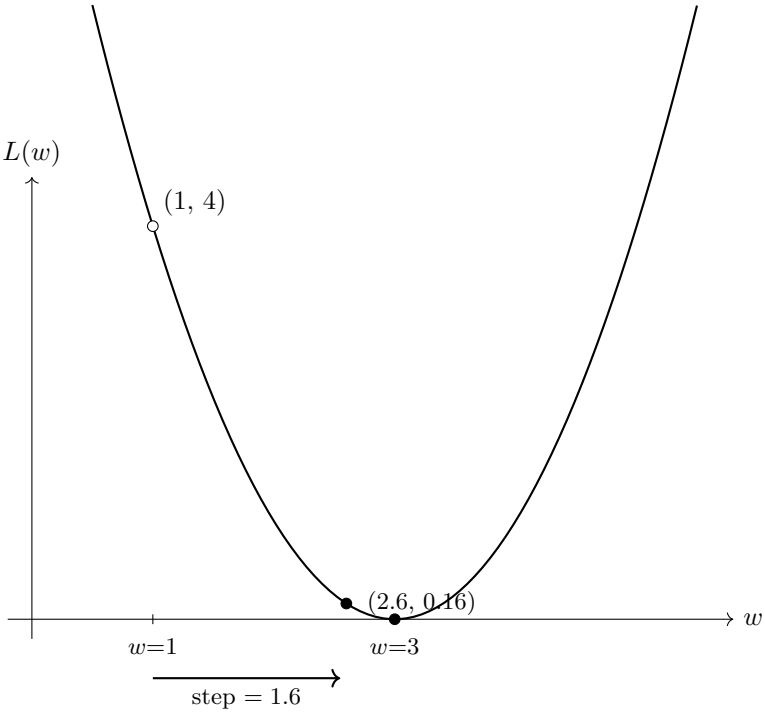
Step 1: Compute the gradient.

$$\frac{dL}{dw} = 2(w-3) = 2(1-3) = -4.$$

Step 2: Apply the update rule.

$$w_{\text{new}} = w - \eta \cdot \frac{dL}{dw} = 1 - 0.4 \times (-4) = 1 + 1.6 = 2.6.$$

The weight moved from 1 toward 3 (the minimum). At $w = 1$, the gradient points uphill in the negative direction; subtracting it moves w downhill toward the minimum, closing the gap to the optimum.



The open circle marks the starting point $w = 1$; the filled circles mark the updated weight $w = 2.6$ and the minimum $w = 3$. One gradient step more than halved the distance to the optimum. With a smaller learning rate the step would be shorter; with a larger one it might overshoot.

Most of the practical difficulty in training comes from scale, instability, architecture choice, and noisy data, not from the basic update rule itself.

Chain Rule As Credit Assignment

The chain rule matters because many models are compositions of simpler computations. If

$$u = az + b, \qquad z = cx, \qquad \mathcal{L} = (u - y)^2,$$

then the loss depends on c indirectly through z and u. The chain rule lets the model pass responsibility backward through

the computation.

> **Math Lens.** The derivative with respect to the early parameter c is
>
> $$\frac{\partial \mathcal{L}}{\partial c} = \frac{\partial \mathcal{L}}{\partial u} \cdot \frac{\partial u}{\partial z} \cdot \frac{\partial z}{\partial c} = 2(u - y) \cdot a \cdot x.$$
>
> So if $a = 2$, $x = 3$, and $u - y = 1$, then the gradient is 12: a small change in c has a clear local effect on the loss.

This is the useful reading:

> how much did an earlier quantity contribute to the final error, and therefore how much credit or blame should move backward?

That is why backpropagation is not just calculus drill. It is structured credit assignment through a composed function.

Chapter Artifact: Learning-Signal Card

Learning-Signal Card

Field	What to write
Model output	Score, probability, ranking signal, class label, or numeric prediction
Pointwise loss	The penalty on one example and what behavior it rewards
Dataset objective	The average or aggregate quantity minimized during training
Reported metric	The number used to justify the final claim
Decision rule	Threshold, ranking cutoff, abstention rule, or review policy
Main mismatch risk	What the loss may optimize that the deployment goal does not actually want

This card should make it hard to say "the model improved" without stating what improved, under which loss, and in service of which decision.

Common Mistakes

Mistake Checklist.

- **Treating a score as a calibrated probability.** Check what the number really means before interpreting it as a probability.
- **Treating lower training loss as deployment success.** The training objective can improve while the real task stays the same or gets worse.
- **Choosing a loss without asking what wrong behavior it still permits.** Loss choice is part of the task design, not a cosmetic detail.
- **Forgetting that a model is a function used on inputs.** The predictor lives on examples, not only as a parameter vector stored in memory.
- **Treating gradients as mysterious symbols instead of local direction information.** Gradients tell us how to move parameters to reduce loss.

Looking Ahead

The two bridge chapters were not meant as isolated mathematics review. Their job was to make later modeling language readable: representation in the first bridge, and score, probability, decision rule, loss, and update signal in this one. Chapter 1 now begins the main argument of the book by asking what decision, proxy target, data source, and held-out evidence make a learning problem worth modeling at all.

Chapter Summary

- A score, a probability, a decision rule, a loss, and a dataset-level objective are different objects with different jobs.
- Conditional probability expresses uncertainty given observed evidence.
- Expectation is the language of average consequence.
- Logarithms and exponentials appear because they make probability-based modeling and optimization easier to express.
- A predictor can be viewed as a function, while empirical risk is a rule that assigns one number to the whole predictor.
- Gradients and the chain rule turn a loss into a usable update signal.
- The main deliverable of the chapter is a learning-signal card written before claiming that a training objective and a deployment goal are aligned.

Study Questions

1. Why is a model score not automatically the same thing as a probability?
2. What is the practical difference between a pointwise loss and empirical risk?
3. In plain language, what is the functional-analysis viewpoint on a loss function that this book actually needs?
4. Why is the chain rule better understood as credit assignment than as symbol pushing?

Exercises

1. Give one example of a score that should be used for ranking but not treated as a calibrated probability.

2. Compute the expected value of an action that gives $+4$ utility when a positive case is correctly caught and -2 utility when a false alarm occurs, if the predicted probability of a true positive case is 0.7.
3. Write down one pointwise squared-error loss and one pointwise log-loss expression, then explain in words what each one punishes.
4. Explain the difference between "choose parameters" and "choose a predictor from a hypothesis class" in one paragraph.
5. Fill out a learning-signal card for spam filtering, student support, or binary image classification. If you plan to carry one running case through the book, keep the same case and add one sentence describing what downstream decision or review action the score should eventually support.

Challenge Exercises

1. Construct a tiny example in which two models have similar accuracy but very different log loss, and explain what that says about confidence.
2. Give a plain-language explanation of least squares as projection into an allowed function class.
3. Design a toy two-stage computation and describe how the chain rule would pass credit backward through it without computing every derivative explicitly.

Part I

Foundations

Chapter 1

Framing Learning Problems

Learning Goals

- Distinguish only briefly between the broader field of AI and the narrower practice of machine learning so the scope of the book is clear.
- Explain why a model score is not yet a decision or an intervention.
- Use the Chapter 1 framing protocol, organized through the Five Commitments, to turn a vague AI idea into a defensible learning problem.
- Audit proxy targets, data, and acceptable inputs before choosing a model.
- Treat datasets as evidence rather than truth, and explain why later evaluation must use unseen evidence.
- Produce a framing memo before any model training begins.

Many AI projects begin with a sentence that sounds ambitious and helpful but is too vague to support technical work. "Use AI for student success." "Detect bad content." "Find fake news."

None of these is yet a defensible learning problem. They are intentions, not specifications. If a team starts coding from that level of vagueness, it often builds a system whose output is mathematically polished and practically confused.

This chapter is the first workshop of the book. Its job is not to survey all of AI. Its job is to teach how a messy request becomes a task, how a score differs from an action, how collected data stands in for the world, and how to write down enough of the problem that model choice becomes meaningful rather than premature. By the end, the reader should be able to produce a framing memo before touching a model.

The chapter moves in a deliberate order. First, we sharpen the distinction between prediction and decision. Then we introduce the Chapter 1 framing protocol through the Five Commitments, repair vague goals through proxy audit, treat data as evidence rather than truth, preview why evaluation must use unseen evidence, and finally turn the whole discussion into one framing memo. Read the chapter as a sequence of justifications for that memo: each major section makes one later row of the artifact more precise.

1.1 Problem-Solving Technique: Decision-First Framing

Problem-Solving Lens. Before choosing a model, ask what decision is being supported, what measurable target can stand in for that decision, what data will make the task visible, what form of the input the model will actually receive, and how the later claim will be judged.

Problem-Solving Technique. Decision-first framing. State the action, the constraint, and the cost of mistakes before selecting a candidate model. A model becomes meaningful only inside that decision context.

> **Practical Note.** A simple workflow is enough to start: write the decision, name the user, choose a proxy target, list acceptable and excluded inputs, and then draft a memo that makes the action and review path explicit. If that sequence feels fuzzy, the problem is still not ready for model choice.

1.2 Opening Dilemma: A Messy Real Problem

Suppose a university asks: *Can we identify students who may need help early?* That sounds like a sensible use of AI. It is also badly underframed, meaning that the goal has not yet been translated into a usable technical specification.

The first weak version of the request is usually something like "predict weak students." That phrase hides nearly every important technical and ethical decision. Who is the user of the system? What action will follow? What counts as "weak"? Are we predicting missed deadlines, low quiz scores, course failure, or disengagement from the learning platform? Which inputs are acceptable, and which would be inappropriate or unavailable at prediction time? What is the most harmful mistake: missing a student who needs help, or bothering a student who does not?

Already we can see the real educational point. The opening task is not to choose a model. It is to turn an institutional concern into a problem that can be specified, evaluated, and defended.

That sentence is worth treating as the chapter's first checkpoint. If the problem statement is still vague, later mathematical sophistication will mostly hide confusion rather than solve it.

Keep one broader institutional setting in view from the start of the book: a course-support organization trying to help students earlier and more reliably. In Chapter 1, that setting appears first as early-outreach risk scoring. Later it will widen into message routing, policy retrieval, grounded answering, and escalation. The workflow changes, but the framing discipline does not.

Field	Rough first-pass entry for the opening dilemma
Decision and user	Adviser or instructor decides whether to invite a student to a check-in or tutoring conversation
Proxy target	Risk of missing early major deadlines rather than a vague idea such as "weak student"
Acceptable inputs	Early quizzes, assignment submissions, attendance, and learning-platform activity related to the course
Costliest mistake	Missing a student who needed timely human support
Downstream action	Outreach and support, not punishment or automated judgment
Review path	A human adviser interprets the score in context

Table 1.1: A rough memo is better than a vague ambition. The chapter will refine this into a fuller framing artifact later, but students should see the target shape early.

> **Practical Note.** Before the chapter develops the full protocol, it is worth seeing what a rough first-pass framing memo already looks like.

This table is intentionally incomplete. Its job is only to make the chapter's destination visible early. The fuller framing memo later in the chapter adds excluded inputs, evaluation preview, and cleaner justification for the task. The rest of Chapter 1 can be read as the work needed to justify each row.

1.3 Brief Orientation: Where Machine Learning Fits

Artificial intelligence is a broad label. It includes search, planning, reasoning, robotics, knowledge representation, and machine learning. In this book we focus mainly on machine learning because it provides the dominant framework for modern AI systems in vision, language, speech, recommendation, forecasting,

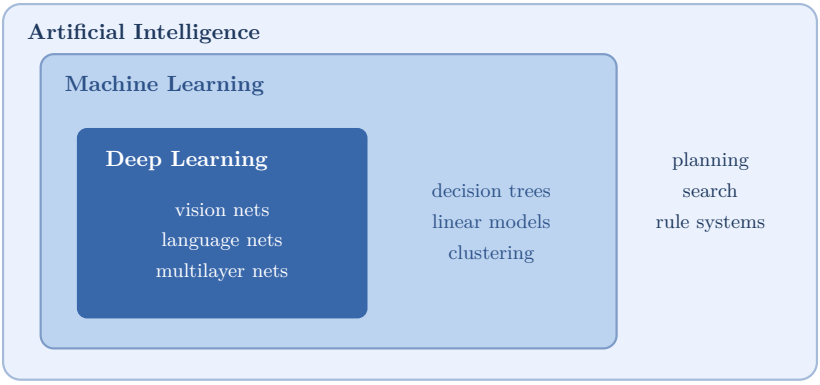


Figure 1.1: Deep learning is a subset of machine learning, and machine learning is a subset of the broader AI field.

and many decision-support applications.

This distinction matters only to orient the scope. The chapter's main work is still problem framing rather than field taxonomy.

Machine learning is the part of AI concerned with improving performance through data or experience rather than through fully hand-written rules. Sometimes that experience comes with labels, meaning desired outputs attached to examples. Sometimes it comes through rewards, raw data, or targets created from the data itself. The common structure is that we define a set of candidate models and use data to select or adapt one that behaves usefully.

It is helpful to introduce a small amount of notation early because it clarifies what is being learned. Suppose each example consists of an input x and a desired output y, often called a label. A dataset can be written as

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}.$$

The model is a function f_θ with parameters θ, the adjustable numbers that training will tune, and its prediction for an input x is

$$\hat{y} = f_\theta(x).$$

System style	Where the behavior mostly comes from	Typical Chapter 1 example
Rule-based system	Hand-written rules, search, or symbolic logic designed in advance	A student-support system that fires fixed alerts when two assignments are missed
Machine learning system	A model fitted from examples, often using hand-designed summaries of the input	A spam detector trained on labeled email using counts of which words appear
Deep learning system	A multi-layer model that learns internal features from large data rather than relying mostly on hand-designed ones	A neural vision model for counting objects from aerial images

Table 1.2: The boundary is not absolute, because real systems often combine learned components with rules and search. The table is meant only to orient the reader before the framing method begins.

> **Prerequisite Bridge.** At this stage, read the notation informally: x is the input, y is the desired output, θ is the adjustable part of the model, and $\hat{y}$ is the model's prediction.

That notation is enough for now. Chapter 1 is not about manipulating formulas. It is about deciding what the symbols should refer to in the first place.

1.4 Prediction Is Not the Decision

The book's first major distinction is simple and easy to forget: a prediction is not the same thing as a decision.

Spam filtering is a clean contrast case. Public resources such as the Enron Corpus and Apache SpamAssassin make it concrete rather than fictional [3, 36]. The model may output a score, for example an estimated probability $P(y = 1 \mid x)$, meaning the model's estimated chance that a message is spam given the email x. But the product still has to decide what action follows. One

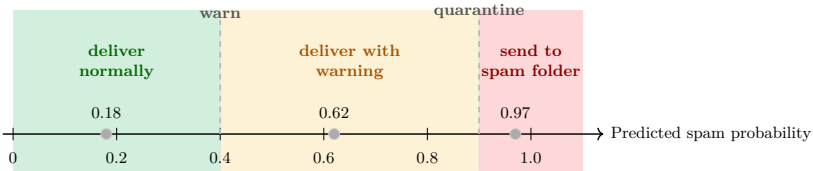


Figure 1.2: Prediction is not the same as decision. The same model score can feed several product actions through thresholds chosen by humans.

Mistake type	What happens	Why the cost matters
False positive	A legitimate email is treated as spam	The user may miss bills, job offers, travel updates, or personal messages
False negative	Spam reaches the inbox	The user faces annoyance, wasted attention, fraud attempts, or malware risk

Table 1.3: Prediction quality depends on the consequences of mistakes, not only on whether the model makes an error in the abstract.

cutoff, also called a threshold, may trigger a warning. A higher cutoff may quarantine the message. The same underlying score can therefore support several different policies.

The same distinction returns immediately to the student-support opening. A risk score might trigger an adviser review, a tutoring invitation, or no action at all. Those are different policies built on the same predictive output.

When a system marks a legitimate email as spam, that mistake is called a *false positive*. When spam gets through untouched, that mistake is called a *false negative*. Those names matter because the two mistakes usually have different costs.

If the threshold moved to 0.50, student D would also be reviewed. If it moved to 0.80, only student F would be reviewed. The prediction itself does not change; only the decision rule does.

Student	Risk score	Action at threshold 0.70
A	0.12	No outreach
B	0.29	No outreach
C	0.48	No outreach
D	0.62	No outreach
E	0.74	Invite to adviser review
F	0.91	Invite to adviser review

Table 1.4: A score becomes a decision only after a threshold is chosen. The scores stay fixed when the policy changes.

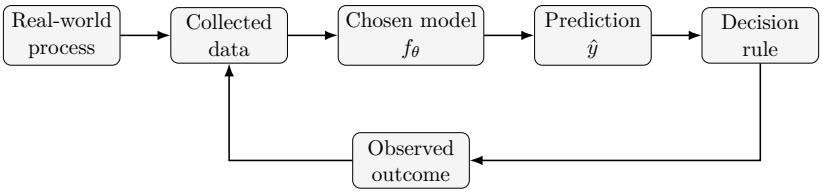


Figure 1.3: A learning problem does not end at the model. Real systems move from world to data to prediction to decision and then back to new outcomes.

This is why the book begins with framing rather than algorithms. A score without a downstream action is incomplete, and an action without a stated cost of mistakes is hard to defend.

1.5 Objective Hierarchy: Goal, Proxy, Label, Metric

One reason beginner projects drift so quickly is that several levels of description get collapsed into one vague sentence. A team says it wants to "improve retention" or "support students better," then jumps directly to labels and metrics as if they were the same thing as the institutional goal. They are not.

Keep four levels separate. The *goal* is the real-world outcome that matters. The *proxy target* is the narrower learning task that stands in for that goal. The *label rule* turns that target into observed examples. The *metric* is the number reported after development. A fifth quantity enters as soon as training begins:

Level	Core question	Student-support example	Common failure
Goal	What real-world outcome matters?	Help more students receive timely human support before early disengagement compounds	Treating an institutional hope as if it were already measurable
Proxy target	What narrower learning task stands in for that goal?	Predict risk of missing the first two major deadlines early enough to intervene	Choosing a proxy that is easy to log but poorly aligned with the real need
Label	How will the proxy be turned into observed targets?	Mark a case positive if the student misses one of the first two major deadlines or omits core work by the cutoff	Treating administratively produced labels as ground truth rather than as measurements
Metric	What number will summarize success?	Recall among students who later disengaged, measured under a fixed advising-review budget	Optimizing a convenient number that ignores the decision constraint

Table 1.5: Keep goal, proxy target, label construction, and reporting metric separate rather than pretending they are all the same thing.

the *objective*, the penalty minimized during fitting. In student support, the proxy target might be early disengagement risk, the objective might be log loss on the labeled examples, and the metric might be recall under a fixed review budget. A project can talk about one quantity, optimize another, and report a third, so the names are not interchangeable.

The difference becomes clearer in one concrete student-support story. The institutional goal is not "predict failure" in the abstract. The goal is to help students receive timely human support. The proxy target might therefore be early disengagement risk rather than final-course failure, because waiting for final out-

comes would make the intervention too late. The label then depends on a measurable rule, such as missed core work by a documented cutoff. The reported metric may finally be recall at a fixed review budget because the advising team can inspect only a limited number of cases each week.

Notice what happened. The project became more precise at every step, but it also became more honest about where mismatch can enter. A weak project often fails not because the model is poor, but because these four levels were never separated cleanly enough to inspect.

> **Problem-Solving Technique. Objective hierarchy.** Before writing code, write four short sentences: the real-world goal, the proxy target, the label rule, and the reporting metric. If two of the sentences sound identical, the problem is probably still underframed.

1.6 The Five Commitments

Many beginner mistakes come from starting with a favorite model rather than with the structure of the problem. A more reliable approach is to identify five commitments before training begins and to answer them in order.

1. **Task.** What exactly is the system supposed to predict, estimate, rank, or decide?
2. **Data.** What evidence will stand in for the world?
3. **Representation.** What form of the input will the model actually see?
4. **Objective.** What behavior is rewarded during training?
5. **Evaluation.** What later claim will count as success?

Read these commitments as operational checkpoints rather than as vocabulary words. Each one makes the next one more concrete. Task says what must be learned. Data and representation

make that task visible. Objective says what training will reward. Evaluation states what later evidence must support.

These five commitments are Chapter 1's local framing protocol inside the book's broader seven-question framework. They explain how a vague request becomes a learnable task before baseline, reliability, and systems questions are added later.

The book's broader seven-question framework can be stated compactly:

1. What decision or action is being supported?
2. What target will stand in for that decision?
3. What evidence about the world is actually available?
4. What representation will the model really see?
5. What baseline must be beaten?
6. What evidence would justify the claim?
7. What system will carry the prediction into action?

The Five Commitments are a local workshop version of that larger framework. Task mainly answers Questions 1 and 2. Data answers Question 3. Representation answers Question 4. Evaluation previews Question 6. Questions 5 and 7 are delayed because a framing chapter should first make the problem legible before it asks what baseline is credible or what deployment system will be responsible for the output. Objective is added here because once a task is framed, training still needs a concrete signal about what the model should learn to do.

This is also the right place to connect the two organizing frameworks already introduced. The objective hierarchy from the previous section asks for goal, proxy target, label rule, and reporting metric. The Five Commitments widen that picture: goal and proxy target shape the task, the label rule constrains data collection, representation says what the model actually receives, objective says what training rewards, and evaluation asks what later claim the evidence can support. The hierarchy separates

levels of meaning; the commitments turn those levels into a full project frame.

> **Practical Note.** Table 1.6 expands the rough memo from Table 1.1 into the chapter's first reusable protocol. If you cannot fill it out clearly, the project is usually not ready for model development.

The order of use matters. First write the task in operational terms. Then ask what data and representation would make that task visible. Only after that should the training objective and later evaluation claim be written down. In the broader seven-question framework, this table mainly settles Questions 1 through 4 and previews Question 6.

1.7 Proxy Audit: From Vague Goal to Defensible Task

The next move is to repair vague openings. This is the chapter's most reusable habit. A proxy audit forces us to separate the real-world goal from the measurable target we can actually train on. That measurable target is called a *proxy target* because it stands in for the broader goal rather than matching it perfectly.

> **Problem-Solving Technique. Proxy audit.** Write down the real-world goal and the label you can actually measure. Then list how sampling, timing, annotation, or historical process could make the measured target diverge from the real goal.

> **Decision Box. Proxy stress test.** Is the proxy measurable, timely, available at prediction time, action-relevant, and hard to game? If any answer is no, revise the proxy before training.

Notice what changes in each rewrite. The user becomes clearer. The proxy target becomes narrower. The downstream action becomes more defensible. The model has not improved yet. The *problem statement* has.

Problem-Solving Technique. Tempting mistake → corrected reasoning. Tempting mistake: "The project sounds helpful, so we can start modeling." Corrected reasoning: "Until the user, downstream action, proxy target, and most costly mistake are explicit, we still have an intention rather than a defensible learning problem."

1.8 What Data Actually Represents

Students often say that "the data speaks for itself." It does not. Data is produced by instruments, institutions, interfaces, and human choices. A medical image depends on scanner settings and clinical workflow. A text corpus depends on which languages, websites, or books were collected. A recommendation dataset depends on what users were previously shown. A dataset is never the world itself. It is a measurement process applied to the world.

This matters for at least three reasons. First, sampling matters. If the data covers only some users or conditions, the model may generalize poorly elsewhere. Second, labeling matters. If labels are noisy, inconsistent, or based on weak proxies, the model may learn the proxy instead of the intended target. Third, timing matters. Data gathered in one year or one market may not represent the next.

Example 1.1 (Recommendation Logs Reflect Exposure)**.** Recommendation data is a useful warning because the labels are entangled with the system that produced them. A movie platform only observes clicks and ratings for items users were actually shown. AutoDebias describes this as a combination of selection bias and exposure bias: observed ratings are not a representative sample of all possible ratings, and a missing interaction may mean non-exposure rather than dislike [15]. That is why a recommender dataset is not a neutral reading of user preference. It is a record of preference filtered through previous ranking decisions.

Example 1.2 (Toxicity Labels Are Social Measurements)**.** Content moderation provides a different kind of warning. Sap et al.

show that annotator identities and beliefs can systematically affect toxicity judgments [56]. Disagreement is not always random noise that disappears if we average enough labels. Sometimes it reflects real differences in how people interpret harm, offense, and context. A model trained on such labels is therefore learning a socially situated target, not an uncontested ground truth.

> **Failure Box.** A student-risk model can look strong if one of its inputs is a support flag created only after the first missed deadline. That feature may be predictive, but it is not available at prediction time; it leaks a later process back into the model. The same mistake appears in moderation if a post-hoc moderator outcome is reused as a feature.

For these reasons, it is better to think of a dataset as evidence rather than truth. Evidence can be strong or weak, representative or narrow, stable or outdated. The quality of the learning problem depends on the quality of that evidence.

> **Practical Note.** When a new dataset appears, ask three simple questions before trusting it: who or what is missing, how were the labels or measurements produced, and when was the data collected relative to the task you care about?

1.9 Data-Centric ML: Improve the Evidence, Not Only the Model

Once a task has been framed, beginners often assume that the next improvement must come from a more sophisticated model. In many projects, the faster and more defensible improvement comes from the data instead: clearer label definitions, fewer duplicates, broader coverage, cleaner measurements, or better documentation. Data-centric machine learning means treating data design, collection, annotation, and maintenance as first-class engineering work rather than as a fixed backdrop.

This does not mean “more data” in the abstract. It means better

evidence for the task actually being asked. If annotators disagree often, the question is not only who made the mistake. The real issue may be that the task definition is ambiguous, that important context was hidden, or that the label space is too coarse for the problem. In that situation, changing the model may help less than changing the data specification.

Documentation matters for the same reason. Gebru et al. argue that datasets should come with structured documentation about motivation, composition, collection process, and recommended uses [21]. If a team cannot answer basic questions about who was included, how labels were produced, what preprocessing was applied, and which uses are out of scope, then later claims about model quality become difficult to interpret.

In high-stakes settings, weak data practice compounds downstream. Sambasivan et al. call these failures *data cascades*: upstream problems in collection, labeling, or coordination can reappear later as model failure, monitoring surprises, or unsafe deployment behavior [55]. A stronger architecture does not automatically repair a weak dataset. It may instead fit the dataset's accidental structure more efficiently.

> **Problem-Solving Technique. Data iteration loop.** Before replacing the model, inspect failures and ask whether the problem is coverage, label definition, annotation consistency, preprocessing, or version control. Then change one data-side element deliberately and record a new dataset version.

Versioning is the practical habit that makes data-centric work scientific rather than anecdotal. If a team removes duplicates, relabels borderline cases, adds examples from an underrepresented setting, or changes preprocessing, then the dataset has changed. The result should be treated as a new version with its own documentation. Chapter 11 returns to this experimental discipline, but the habit starts here, while the problem is still being framed.

> **Practical Note.** When a model fails on a recurring slice, do not ask only "which model should we try next?" Ask whether the slice is underrepresented,

mislabeled, poorly defined, or missing key context. In many real projects, the first serious improvement is a better labeling guide and a versioned dataset release rather than a deeper network.

1.10 Evaluation Preview: Unseen Data and Hidden Targets

Chapters 2 and 3 are the real judgment chapters, so evaluation appears here only as a preview. The essential preview is simple: a learning problem is not defined only by its inputs and targets. It is also defined by what counts as unseen evidence. That is why the framing memo already needs an evaluation preview. If you do not know what evidence will stay protected until the end, you do not yet know what claim the project is trying to defend.

In practice, this is why machine learning separates training, validation, and test roles. Some data is used to fit the model. Some may be used to compare settings. Some must remain protected until the end so the reported result means something. Chapter 3 turns that principle into a full evaluation workflow.

The same discipline appears in public benchmark tasks, where some data is released for development and some answers are kept hidden until judging. One useful supporting example is aerial wildlife counting. The input is an aerial image. The output is a density map, meaning a grid of small values spread across the image so that adding them gives the final animal count. The visible data lets the student build a model, but the real claim is judged on hidden targets.

At Chapter 1 level, the detailed metrics can wait. The habit cannot. The habit is to protect some evidence so the final claim is judged on data that development did not already see.

Example 1.3 (Why the Counting Example Fits Chapter 1)**.** Aerial wildlife counting is not just a vision task. It is a framing task. The input is clear, but the target is richer than a single class label because it involves a spatial density map and a derived

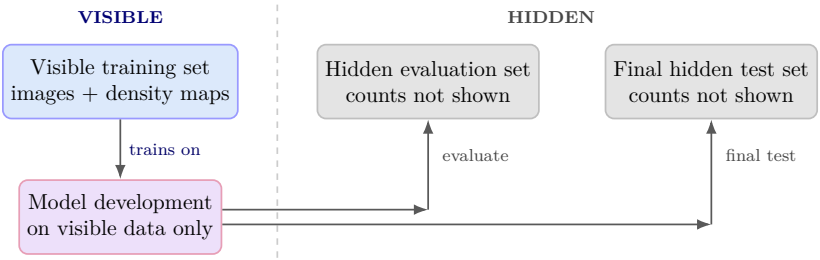


Figure 1.4: The logic of hidden evaluation is the same as sound machine learning practice: develop on visible data, then judge the claim on targets that were not available during development.

count. The evaluation is fixed in advance and hidden during development. This makes it a clean way to teach that a learning problem is defined by its structure, not only by its architecture.

> **Evidence Box.** Zech et al. studied pneumonia detection from chest radiographs and found that performance changed substantially across hospital systems; the model could rely on hospital-specific cues that were not actually signs of disease [63]. The Chapter 1 lesson is simple: a model can look strong on familiar data and still fail when the measurement process shifts.

1.11 A Bad Framing Memo Rewritten

Before reading the strong memo, it helps to see the weak version that many real projects begin with.

This memo is weak in several different ways. The decision is unclear because no user or workflow is named. The proxy target is late and blunt: by the time final failure is observed, it may be too late to help. The input policy is dangerous because "all available data" quietly invites irrelevant or sensitive variables. The evaluation plan is weak because accuracy alone says nothing about review burden, intervention usefulness, or unfair alerts. The downstream action is also underdefined. "Intervene" could mean an email, an adviser call, a tutoring offer, or a punitive

step, and those actions demand different standards of evidence.

The repair is not to make the memo longer for its own sake. The repair is to make each field operational. Replace "predict failure" with an earlier support target. Replace "all available data" with course-related signals that exist before the decision moment and that the workflow can justify. Replace "accuracy" with an evaluation preview tied to human review and useful intervention. Replace vague action with a stated review path and a bounded support response. Each repaired field should force a later design choice rather than merely sounding more polished.

> **Decision Box.** If a framing memo sounds impressive but leaves the user, action, time horizon, sensitive-input boundary, and most costly mistake unclear, the project is not yet ready for model choice. Rewrite the memo before writing the model code.

The next table is therefore not just a more polished version of the same idea. It is a different kind of object: one that could actually guide data choice, evaluation, and system design.

1.12 Worked Framing Memo: Student Support

Public datasets make this example more than a classroom invention. The Open University Learning Analytics Dataset (OULAD) contains demographic information, assessment data, and virtual-learning-environment interactions across several university courses [41]. Policy work on predictive analytics in higher education makes the institutional side equally clear: early-alert systems can support advising, but they also raise concerns about profiling, opacity, and privacy if they are treated as automated judgments rather than support tools [19].

The system here is a triage tool: it surfaces cases for human support, but it does not explain why a student is struggling and it does not make the final judgment about the student.

Now return to the opening dilemma. A university does *not* want

an automated system that decides who will pass or fail. It wants a tool that helps identify students who may benefit from timely human support. Once that shift is made, the framing memo becomes much clearer.

> **Problem-Solving Technique. Framing memo.** Before training, write down the decision, proxy target, inputs, excluded inputs, costliest mistake, evaluation preview, downstream action, and review path. If those fields remain vague, the project is still underframed.

This is the test for Chapter 1. If the reader can take a messy idea and produce a memo like Table 1.9, the chapter has done its real job. Everything earlier in the chapter should cash out into this document. In Chapter 2, the evaluation preview inside the memo will become explicit metrics, action rules, and probability checks.

1.13 Framing Is Iterative, Not One-Shot

Students sometimes read a memo like Table 1.9 and assume framing happens once, before the "real" modeling work begins. Real projects are usually messier. The first memo is a disciplined starting point, not a sacred object that survives every later piece of evidence unchanged.

Suppose the advising team trains an early model and sees three surprises:

- many flagged students recover quickly without any support action, so the proxy target may be too noisy,
- one subgroup is systematically under-flagged because early platform activity is a weak signal for their course format,
- the current threshold would send far more students to review than the advising team can actually contact.

None of these findings means the framing work was wasted. They mean the framing must now be revised with evidence rather than

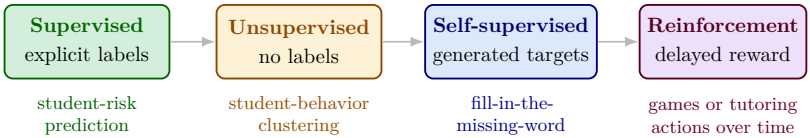


Figure 1.5: Learning settings differ mainly in the kind of feedback they provide. The distinction matters later when model families and data regimes become more specific.

defended out of pride.

This is the more mature habit. Framing begins before model training, but it does not end there. If early experiments expose label noise, unrealistic inputs, missing slices, or impossible review load, the right response is often to rewrite the memo rather than to keep optimizing the original task definition.

> **Decision Box.** Treat the first framing memo as version 1, not as final truth. When the first experiments reveal a mismatch between proxy, data, action, and capacity, revise the memo before escalating model complexity.

1.14 Brief Orientation: Learning Settings as Kinds of Feedback

The chapter's main deliverable is now on the page, so only brief orientation remains. Machine learning is often divided into supervised, unsupervised, self-supervised, and reinforcement learning. The most useful way to think about those categories here is not as separate museum rooms, but as different *feedback regimes*.

In supervised learning, the feedback is explicit labels. In unsupervised learning, there are no explicit targets and the system must discover structure on its own. In self-supervised learning, the system creates training targets from the data itself. In reinforcement learning, the feedback arrives through rewards and consequences of actions over time.

This orientation matters because later modeling choices depend

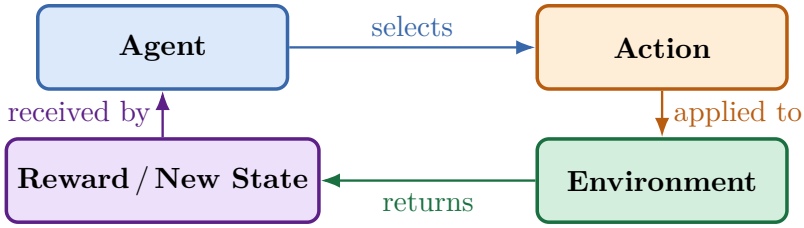


Figure 1.6: In reinforcement learning, the feedback signal is a scalar reward earned by taking actions, not a correct label provided in advance. The agent must discover which actions are valuable through repeated interaction.

heavily on what kind of feedback is available, how expensive it is, and how directly it answers the task we care about.

If that distinction still feels abstract, keep one sentence only: the kind of feedback available constrains what the system can honestly learn.

1.14.1 Reinforcement Learning: A Different Feedback Regime

In reinforcement learning, the system learns by taking actions and receiving scalar feedback — a reward — from the environment. There is no fixed dataset of (input, correct-output) pairs. Instead, the agent must discover which actions lead to good outcomes through repeated interaction. The agent observes a state, selects an action, observes the resulting new state and a scalar reward, and uses that signal to adjust future behavior. The loop repeats many times, with no human supplying a correct answer at each step.

> **Practical Note.** This book's center of gravity is supervised and self-supervised machine learning. Reinforcement learning appears here only for orientation. Students who want to go deeper should know that RL is the framework behind game-playing agents, robotic control, and some preference-alignment stages of modern language model training. In this book, only that last connection appears later in the language chapter.

Decision Box. Use supervised learning when labeled examples are available and the task is clearly defined. Consider self-supervised approaches when labels are scarce but raw data is abundant. Reinforcement learning applies when the right action can only be discovered through interaction with an environment and cannot be defined in advance as a labeled target.

1.15 Common Mistakes in First Machine Learning Projects

Several mistakes appear so often that they are worth naming early.

Mistake Checklist.

- **Starting with the model instead of the problem.** Begin with the task, the data, the consequences of mistakes, and the evaluation plan; choose the model only after those are clear.
- **Treating labels as perfect truth.** Labels are noisy measurements, not ground truth.
- **Ignoring the decision context.** A prediction matters only inside a decision process, so identify who acts and what happens next.
- **Confusing correlation with explanation.** Predictive features may be proxies or shortcuts, and correlation alone is not causation.

1.16 Looking Ahead

This chapter framed the problem: what is the task, what is the proxy, and what counts as a defensible learning problem? The next two chapters make that memo judgeable. Chapter 2 asks what predictive behavior counts as good for the decision in front of us. Chapter 3 then asks how to protect that judgment from

leakage, weak baselines, and accidental misuse of held-out evidence. In other words, Chapter 1 writes the claim in operational form; Chapters 2 and 3 test whether the later evidence deserves to support it.

> **Decision Box.** Ready to model only if you can name the user, the action, the proxy target, the available-at-prediction-time inputs, the unacceptable inputs, the costliest mistake, the held-out evidence, and the review path. If any piece is still vague, stop and reframe the problem before training. This framing discipline will resurface throughout the book, from evaluation design (Chapters 2 and 3) to reliability assessment (Chapter 12) to system deployment (Chapter 13).

Chapter Summary

- The first job in AI is not model choice. It is turning a vague request into a defensible learning problem.
- A prediction is not the same as a decision. Scores become meaningful only inside a downstream policy or intervention.
- The Five Commitments are Chapter 1's framing protocol inside the broader seven-question framework: task, data, representation, objective, and evaluation.
- Proxy targets must be audited because real-world goals and measurable labels are rarely identical.
- Datasets are evidence, not truth; they are shaped by sampling, annotation, timing, and interface decisions.
- Data-centric ML asks whether coverage, labeling, documentation, and versioning should be improved before reaching for a more complex model.
- Supervised, unsupervised, self-supervised, and reinforcement learning are best understood as different feedback regimes.

- Evaluation enters Chapter 1 only as a preview: a claim must eventually survive hidden or held-out evidence.
- The main deliverable of the chapter is a framing memo written before any model training begins.

Study Questions

1. Why is "predict weak students" a poor opening statement for an AI project?
2. What is the difference between a prediction and a decision?
3. Why are the Five Commitments better understood as Chapter 1's local protocol inside the broader seven-question framework rather than as a list of vocabulary words?
4. In what sense is a dataset evidence rather than truth?
5. What kinds of project improvements count as data-centric rather than model-centric?
6. Why does a hidden-evaluation task such as aerial wildlife counting belong in Chapter 1 even though the full evaluation chapter comes later?

Exercises

1. Prediction or decision? For each scenario, say whether it describes a prediction, a decision, or both: a spam score of 0.91, automatically quarantining an email, estimating whether a student will miss a deadline, inviting a student to office hours, and ranking comments by toxicity risk for a moderator.
2. Rewrite each vague opening into a more defensible task: "predict weak students," "detect bad content," and "find fake news." For each, name a likely user, proxy target, and downstream action.
3. Explain why the same spam-probability model could support different actions at different thresholds without the model itself changing.

4. Give one example each of supervised, unsupervised, self-supervised, and reinforcement learning, but explain them as different feedback regimes rather than as isolated categories.

5. Consider a university dataset containing attendance, quiz scores, and assignment submissions. List three ways in which the data collection process might bias the resulting model.

6. A moderation dataset shows frequent annotator disagreement on sarcasm and reclaimed slurs. Propose one data-centric intervention and explain why it may help more than simply swapping in a larger model.

7. Suppose a moderation model predicts whether an online comment should be flagged for review. Explain why the predicted probability and the final moderation decision should be treated as separate components.

8. Write down a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ for a problem of your choice. State what x_i, y_i, f_θ, and $\hat{y}_i$ mean in that context.

9. Use Table 1.9 to write a one-page framing memo for either student support, spam filtering, or a problem from your own field. If you carry one running case through the book, keep this memo and use the same case again in Chapter 2 when you add metrics and action policy.

Challenge Exercises

1. Take a problem from your own domain and argue for two different proxy targets. Explain which one you would choose first and what failure mode the rejected proxy invites.

2. Design a counting task analogous to aerial wildlife counting for another domain. State the input, target, representation, objective, and what an evaluation with hidden answers would need to protect.

3. Give an example of a predictive feature that could be useful for a model but unacceptable in deployment. Explain why the framing memo should reject it before training begins.

4. Write a short critique of a published or public AI system description by identifying which part of the framing memo is missing, underdefined, or ethically unstable.

Commitment	Guiding question	Student support	Spam filtering	Counting task
Task	What must be predicted?	Risk of missing early deadlines	Probability an email is spam	Animal count from an aerial image
Data	What evidence stands in for the world?	Assessments, attendance, and platform activity	Labeled email text and message details	Images paired with local count targets
Input form	What form does the model actually see?	Early-course signals in a student table	Words, links, sender cues, and email metadata	Pixels, local patches, and count cues
Objective	What behavior is rewarded in training?	Minimize a training loss that raises scores on students needing timely support	Minimize a training loss that separates spam from wanted mail	Minimize error between predicted and observed local count targets
Evaluation	What claim is being tested?	Does it support advising without unfair profiling or useless alarms?	Does it protect inboxes without hiding wanted mail?	Does it perform well on evaluation images whose answers stayed hidden during development?

Table 1.6: A framing canvas turns the five commitments into concrete questions before model choice begins.

Weak opening	Better framed learning problem
"Use AI to support students"	Predict which students are likely to miss the first two major deadlines early enough to trigger human support.
"Detect bad content"	Estimate the probability that a comment should be queued for moderator review under a documented moderation policy.
"Find fake news"	Classify whether an article is likely to violate a specific fact-checking standard, with human review before public action.

Table 1.7: The main improvement is not sophistication. It is moving from slogans to tasks with users, targets, and downstream actions.

Memo field	A weak student-support memo
Decision and user	"Use AI to predict which students will fail"
Proxy target	"Course failure" with no time window or intervention logic
Acceptable inputs	"All available student data"
Excluded or sensitive inputs	not stated
Most costly mistake	not stated
Evaluation preview	"High accuracy"
Downstream action	"Intervene"
Review path	not stated

Table 1.8: A bad memo is useful because it shows where vagueness hides. The fields are present, but they do not yet support a defensible learning problem.

Memo field	One defensible student-support entry
Seven-question focus	Questions 1–4 are written directly here; question 6 appears as an evaluation preview, and question 7 appears through downstream action and review path
Decision and user	An adviser or instructor decides whether to invite a student to a check-in, tutoring session, or office-hour conversation
Proxy target	Risk of missing the first two major deadlines or failing to submit early core work
Representation	One student per row, encoded with early-course features such as quizzes, submissions, attendance, and platform activity
Acceptable inputs	Early quizzes, assignment submissions, attendance, and learning-platform activity directly related to the course
Available at prediction time	Only early-course signals collected before the support decision; no later grades, no post-deadline outcomes, and no flags created after outreach
Excluded or sensitive inputs	Sensitive personal attributes, disability information, or private data unrelated to learning support
Training objective	Minimize a supervised risk loss that helps surface likely disengagement cases early enough for human support
Most costly mistake	Failing to flag a student who needed timely human help
Evaluation preview	Later evidence should show that the system surfaces useful cases without creating unfair or noisy alerts
Downstream action	Outreach, tutoring offers, office-hour invitations, or scheduling support rather than punishment
Review path	An instructor, adviser, or student-support professional interprets the output in context

Table 1.9: A framing memo turns a vague request into a concrete learning problem before any model training begins.

Memo field	What early evidence revealed	How the framing should change
Proxy target	missed deadlines were too coarse and mixed routine recovery with serious disengagement	refine the proxy toward repeated non-submission or combine it with review notes
Inputs	one course format produced weak platform signals	add course-format awareness or restrict the claim to settings where the inputs are meaningful
Downstream action	current threshold overloads the advising queue	rewrite the policy around fixed review capacity or ranking under budget
Evaluation preview	aggregate score hid subgroup undercoverage	require slice checks and workload-aware evaluation before any deployment claim

Table 1.10: A framing memo should tighten when the first experiments expose mismatch. Revision is not a failure of framing; it is the point of framing under evidence.

Feedback regime	Source of the learning signal	When it applies
Supervised learning	Correct labels provided by humans or other processes	When labeled examples are available and the task is well-defined
Unsupervised learning	Structure in the data itself	When labels are absent and the goal is to discover clusters, low-dimensional views, or other latent structure
Self-supervised learning	Targets generated from the data itself	When labels are scarce but the goal is to learn reusable representations from raw data
Reinforcement learning	Scalar rewards from environmental interaction	When the system must learn through trial and error rather than from fixed labeled examples

Table 1.11: The four main feedback regimes differ in where the learning signal comes from and in the kind of task they suit best.

Chapter 2

Prediction, Loss, and Decision Rules

Learning Goals

- Explain what would count as a useful prediction for a real decision rather than for an abstract benchmark alone.
- Distinguish clearly between the numbers reported after training, the penalties used during training, the rules that turn scores into actions, and the checks on probability quality.
- Use common regression and classification losses as training objectives rather than treating them as final judgments.
- Interpret accuracy (overall correctness), precision (how often predicted positives are right), recall (how many true positives are caught), F1 score (a combined precision-recall score), calibration (whether probabilities match frequencies), and decision-cutoff choice in terms of intervention cost and review capacity.
- Explain when a threshold, a top-k ranking policy, or abstention-to-human-review is the right way to turn scores into action.

- Produce a one-page metric-and-action worksheet before reporting a serious model output.

Chapter 1 answered the question *what is the problem?* Chapter 2 answers a different question: *what would count as a good prediction for the decision in front of us?* A model is not good in the abstract. It is only good relative to a decision, a reported measure of behavior, a rule for turning scores into actions, and a check that predicted probabilities mean what they seem to mean.

This chapter is therefore the book's first judgment protocol. Its job is not merely to list formulas. Its job is to show how a model output becomes a defensible operational signal.

Four terms appear throughout the chapter. A *metric* is the quantity reported after training, such as accuracy or recall. A *loss* is the penalty minimized during training. A *threshold* or *ranking rule* turns scores into actions. A *calibration check* asks whether predicted probabilities match observed frequencies.

The chapter moves in a deliberate order. First, we ask what behavior should be judged. Then we separate reported metrics from training losses, turn scores into actions through thresholds or rankings, and inspect whether probabilities are trustworthy enough to support action or abstention. Read the chapter as a sequence of answers to one worksheet: what should be reported, how will that score become action, and which evidence is allowed to justify the policy.

How to Read This Chapter

Chapter 2 stays on one question: once a model produces scores, what behavior should count as good? Read the chapter in three passes. First separate metric from loss. Then separate score from action. Finally ask whether the probabilities are trustworthy enough for thresholds, rankings, or abstention-to-review. Each major section fills in one part of the metric-and-action worksheet that appears near the end of the chapter.

Decision Box. For a first pass, focus on four dis-

tinctions: reported metric, training loss, action rule, and calibration. The next chapter will ask the different question of how to protect those judgments from overfitting, leakage, weak baselines, and accidental test-set reuse.

Problem-Solving Lens. This chapter turns framed tasks into judged behavior. The core move is to separate the penalty minimized during training, the behavior reported after training, and the rule that turns scores into actions.

Problem-Solving Technique. Losses are for learning; metrics are for claims. Training needs smooth, stable penalties to optimize. Reporting needs quantities that match the decision and can be defended to another human being.

Problem-Solving Technique. Wrong question → better question. Wrong: "Which metric is best?" Better: "Which error pattern matters for this decision, what rule will turn scores into action, and what evidence would justify the claim afterward?"

Problem-Solving Technique. Common failure mode → recovery move. Failure mode: reporting one attractive score without saying what threshold, base rate, review capacity, or confidence interpretation made that score meaningful. Recovery move: write the metric-and-action worksheet first, including the reported metric, action rule, calibration need, and abstention logic, then treat every later number as evidence inside that frame.

Problem-Solving Technique. Signal to trust → signal to doubt. Trust: the reported metric matches the decision, the threshold or ranking rule is explicit, and calibration is checked when probabilities matter. Doubt: a single score is reported without the action rule, class balance, or probability interpretation that would make that score usable.

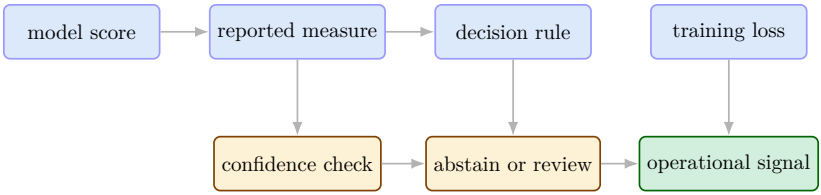


Figure 2.1: Chapter 2 is a judgment chain. A score does not become an operational signal until the chapter has answered how it will be judged, how it becomes a decision, and whether its probabilities can be trusted.

2.1 Opening Question: What Would Count as Good Here?

Return to the broader course-support setting from Chapter 1. Suppose the same organization wants an early-warning score for an introductory programming course. The system outputs a risk score for each student, and staff can contact at most five students this week.

That constraint changes the meaning of "good." A useful system is not just one with a low average training penalty. It is one whose scores place the most relevant cases near the top, whose probabilities are trustworthy enough to support action, and whose decision rule respects the real capacity of the support team. Later chapters will widen this same institutional setting into message routing, retrieval, grounded answering, and escalation. For now, one scalar risk score is enough to surface the judgment problem cleanly.

This opening question determines the rest of the chapter. We first need to know what behavior matters. Only after that do training losses make sense. The rest of the chapter keeps unpacking that sentence: report the right behavior, choose the right action rule, then decide whether the score is trustworthy enough to automate.

> **Practical Note.** Keep the chapter's workflow in mind as you read: separate the metric from the loss, choose the action rule that matches the decision, and only then check whether the probabilities are trust-

		Predicted label	
		Positive	**Negative**
Actual label	Positive	**TP** caught true case	**FN** missed true case
	Negative	**FP** false alarm	**TN** correct rejection

Figure 2.2: The confusion matrix is the bookkeeping layer between predicted scores and thresholded claims. Once a decision rule turns scores into positive or negative decisions, these counts determine what the metric rewards.

worthy enough to support that rule. The worksheet later in the chapter is the written version of that same sequence.

2.2 Reported Metric Before Training Loss

The first judgment question is not "what loss do we minimize?" It is "what behavior matters enough to report?" In some problems the key behavior is keeping false alarms low. In others it is catching as many rare positive cases as possible. In still others it is ordering cases well near the top, using probabilities honestly, or making good decisions under a limited intervention budget.

Definition 2.1 (Evaluation Metric)**.** An evaluation metric is a summary statistic used to judge model behavior on a dataset after training. It should express what success means for the real task rather than what is merely convenient for optimization.

For a yes-or-no classification problem, most threshold-based metrics begin with the same bookkeeping device: the confusion matrix. It counts true positives (real positive cases caught by the system), false negatives (real positive cases missed), false positives (false alarms), and true negatives (correct rejections).

From these counts we obtain common metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN},$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Accuracy measures overall correctness, but it can hide serious failures when one class is much more common than the other or when false positives and false negatives have very different costs. Precision asks, "When the system says positive, how often is it right?" Recall asks, "Of the truly positive cases, how many did the system catch?" F1 combines precision and recall into one number, but even that combination is only sensible when those two concerns matter in roughly balanced ways.

Example 2.1 (Spam Filtering and Precision)**.** Spam filtering is naturally precision-sensitive. False positives are painful because legitimate messages get hidden or delayed. At Gmail scale, even a small false-positive rate can affect a large number of users [40]. This is why spam systems are rarely judged by accuracy alone.

Example 2.2 (Medical Screening and Recall)**.** The metric emphasis changes when false negatives are costly. The CDC notes that rapid influenza diagnostic tests often have sensitivities around 50–70% and specificities around 90–95%; here sensitivity means how often true cases are caught, and specificity means how often true negative cases are correctly rejected, that is $\text{TN}/(\text{TN} + \text{FP})$ [14]. The false positive rate is $\text{FP}/(\text{FP} + \text{TN})$, so it is just $1 - \text{specificity}$. A stricter threshold may reduce false alarms but miss too many real cases.

Example 2.3 (Why Accuracy Can Fail Under Imbalance)**.** Imagine a screening dataset with 950 negative cases and 50 positive

cases. A useless classifier that predicts "negative" for every person achieves

$$\text{Accuracy} = \frac{950}{1000} = 95\%.$$

That number sounds strong until we inspect recall. Because the classifier never predicts positive, it has

$$TP = 0, \quad FN = 50, \quad \text{Recall} = 0.$$

The model is accurate only because the majority class dominates the dataset. This kind of skew, called class imbalance, makes accuracy an unsafe default.

2.3 Training Loss as the Quantity Optimized During Learning

Once the reported behavior is clear, training losses make more sense. A learning algorithm needs a rule for preferring one prediction over another during optimization, meaning during the process of adjusting the model to reduce error on the training data. That role is played by a loss function. This is the chapter's first separation: the number that teaches the model is not automatically the number that justifies the system.

Definition 2.2 (Loss Function)**.** A loss function $\ell(y, \hat{y})$ measures how bad a prediction $\hat{y}$ is when the desired output is y. Smaller values are better, and a value of zero usually represents a perfect prediction.

If a dataset contains n training examples, a standard training objective is the average loss:

$$\mathcal{L}_{\text{train}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f_\theta(x_i)).$$

This quantity is often called the *empirical risk*, meaning the average observed loss on the training sample.

Definition 2.3 (Empirical Risk)**.** Empirical risk is the average loss measured on the observed training sample. It is called "empirical" because it is computed from available data rather than from the full unknown distribution of future examples.

Task	Training objective	Reporting metric	Why not identical?
Spam filtering	Log loss on spam probabilities	Precision, recall, and false-positive behavior	Training needs a smooth probability loss; deployment cares about protecting inboxes without hiding wanted mail
Screening or triage	Probability loss or extra penalty for missed cases	Recall or sensitivity plus false-positive burden	The real question is often "how many true cases are caught?" rather than "what is the average training loss?"
Probability forecasts	Probability loss rewarding honest confidence	Calibration and predicted-versus-observed plots	The model is used for its probabilities, so honest confidence matters more than one thresholded label
Ranking or retrieval	A training loss over item scores	Success among the top k outputs or user success rate	The loss is chosen for optimization, but the user experiences only the ordered outputs

Table 2.1: Losses are for learning. Metrics are for claims. A good metric-and-action worksheet should explain why the optimized quantity and the reported quantity are not identical.

The wording matters. Empirical risk lives on the sample we have, not on the world we care about. This is why a low training loss is necessary but not sufficient.

> **Prerequisite Bridge.** If the summation notation looks dense, read the formula for empirical risk as "compute the loss on every training example, add the losses together, and divide by the number of examples."

In the next table, *log loss* means a probability-based penalty that grows when the model is confidently wrong.

2.3.1 Regression Losses

When the target is numeric, two common losses are mean squared error and mean absolute error:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \qquad \mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|.$$

Mean squared error punishes large mistakes more strongly because the error is squared. Mean absolute error treats error more linearly. Neither is universally better. The choice should reflect what kinds of misses the application can tolerate.

Practical Note. If a model were forced to predict one constant value for every example, squared error would pull that constant toward the average target, while absolute error would pull it toward a median or middle case. Students should read this as a modeling judgment first: squared loss reacts more strongly to outliers, while absolute loss is more resistant.

Math Lens. The previous sentence can be stated sharply when the model is forced to predict one constant value for every example.

Theorem 2.1 (Best Constant Predictor)**.** *Let $y_1, \ldots, y_n$ be observed target values, and suppose the prediction is restricted to a single constant c.*

1. *The value of c that minimizes $\frac{1}{n}\sum_{i=1}^{n}(y_i - c)^2$ is the sample mean $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$.*
2. *The value of c that minimizes $\frac{1}{n}\sum_{i=1}^{n}|y_i - c|$ is any sample median: when the targets are sorted, any middle observed value for odd n, or any value between the two middle order statistics for even n.*

Proof Sketch. For squared error, expand $(y_i - c) = (y_i - \bar{y}) + (\bar{y} - c)$. After summing, the cross-term disappears and we get

$$\sum_{i=1}^{n}(y_i - c)^2 = \sum_{i=1}^{n}(y_i - \bar{y})^2 + n(\bar{y} - c)^2,$$

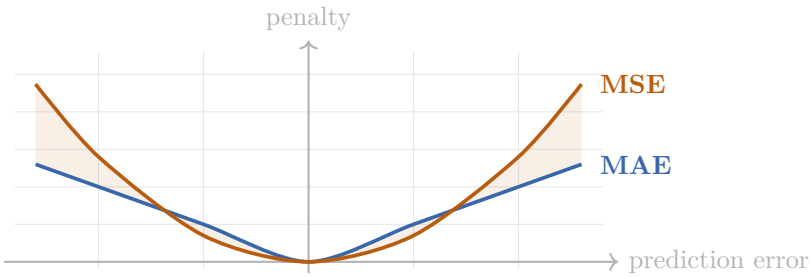


Figure 2.3: MAE grows linearly with the error size, while MSE punishes larger misses much more aggressively. That shape difference explains much of the practical tradeoff.

so the minimum occurs at $c = \bar{y}$. For absolute error, if c sits to the left of the median interval, more than half of the sorted targets lie to its right, so moving c slightly right decreases more distances than it increases. The symmetric argument applies when c lies to the right of that interval. When n is even, this leaves a flat interval between the two middle order statistics on which the total absolute deviation is constant. That is why any median minimizes total absolute deviation. □

Practical Note. This theorem is one reason squared error reacts strongly to outliers while absolute error is more resistant. They are not just different penalties. They prefer different notions of "center."

Example 2.4 (House Price Prediction)**.** If a model predicts house prices, an error of \$200,000 is often more serious than two errors of \$100,000. Squared error reflects that intuition by penalizing the larger miss more aggressively. In other settings, such as predicting waiting times or delivery delays, a more linear penalty may be easier to justify.

2.3.2 Classification Losses

In classification, the output is a class label or a probability distribution over classes. A conceptually simple loss for a yes-or-no task is the zero-one loss:

$$\ell(y, \hat{y}) = \begin{cases} 0 & \text{if } y = \hat{y}, \\ 1 & \text{if } y \neq \hat{y}. \end{cases}$$

This definition is conceptually simple, but it is usually not optimized directly because many common training algorithms work best with losses that change smoothly as the prediction changes.

Instead, classification models are often trained with cross-entropy, also called log loss. For a classifier in a yes-or-no task that predicts a probability p for the positive class, the loss on one example is

$$\ell(y, p) = -\big(y \log p + (1 - y) \log(1 - p)\big).$$

This loss becomes small when the model assigns high probability to the correct class and large when it is confident and wrong.

> **Math Lens.** If $y = 1$, then $\ell = -\log p$; if $y = 0$, then $\ell = -\log(1 - p)$. So as the model becomes more confident in the wrong class, the penalty rises fast because $\log z \to -\infty$ as $z \to 0^+$. For a concrete anchor, predicting $p = 0.99$ for a negative case gives $\ell = -\log(0.01) \approx 4.61$, while an uncertain $p = 0.5$ gives only $\ell \approx 0.69$.

Example 2.5 (Why Confidence Matters at Scale)**.** Google reports that Gmail's AI-powered defenses block nearly 15 billion unwanted messages per day and stop more than 99.9% of spam, phishing, and malware from reaching inboxes [40]. At that scale, confidence matters because downstream systems use those probabilities for ranking, quarantine decisions, and human-review queues.

> **Problem-Solving Technique. Counterexample first.** Before trusting one headline metric, try to construct two models that tie on that metric but differ

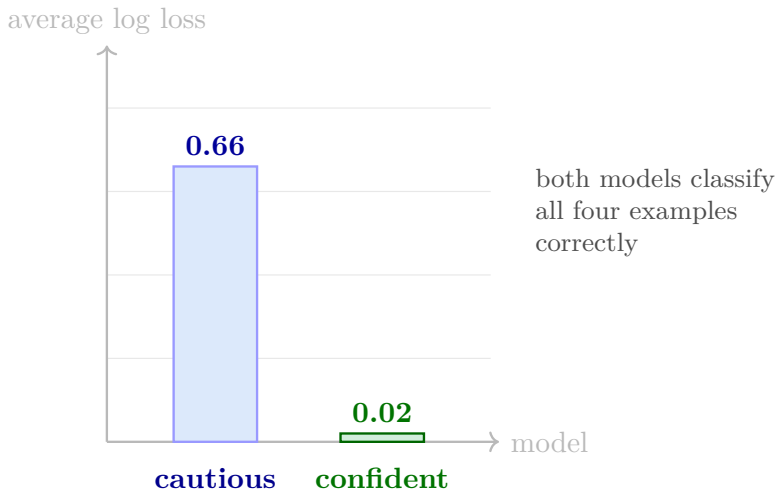


Figure 2.4: Accuracy can hide major differences in confidence. A cautious model and a confident model may both be perfectly accurate while having very different log loss.

in decision value. If you can build such a pair, the metric is not sufficient by itself.

Practical Note. When classes are imbalanced, the average loss can be dominated by the majority class. One common remedy is to weight the loss: multiply the per-example loss by a factor that is larger for minority-class examples. If the positive class appears in only 5% of training examples, an inverse-frequency weight would be $w_+ = 20$. Class-weighted cross-entropy is the standard version:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n} w_{y_i}\left[y_i \log p_i + (1 - y_i)\log(1 - p_i)\right],$$

where $w_1 > w_0$ reflects the higher cost of ignoring the rarer class. This is a training-time choice that complements the deployment-time threshold and cost-matrix choices discussed in Section 2.4. Adjusting the loss and adjusting the threshold are two different levers; in practice, teams often use both.

Example	True label	Cautious $p(y{=}1\|x)$	Confident $p(y{=}1\|x)$	Correct?
1	1	0.51	0.99	both correct
2	1	0.52	0.98	both correct
3	0	0.49	0.02	both correct
4	0	0.48	0.01	both correct

Table 2.2: The final labels are identical, but the probabilistic quality is not. Log loss rewards the model that is confidently correct, not merely barely correct.

> **Warning.** If you report only the optimized training loss, you may hide the behavior that matters for the real decision. A system can improve its loss while leaving the practically important metric unchanged or even worse.

2.4 From Score to Action: Threshold, Ranking, and Abstention

Many models produce scores or probabilities, not final actions. A decision still has to be made. That action may involve a threshold, a top-k ranking, meaning "take the highest k scores," or an abstention rule, meaning a rule that withholds automatic action on uncertain cases and sends them to human review. This is the chapter's second separation: even a well-chosen metric is incomplete until the score is attached to an operational policy.

> **Problem-Solving Technique. Threshold as policy.** A threshold is not a cleanup detail after training. It encodes a decision about missed cases, false alarms, review capacity, and intervention cost.

Message	True label	Predicted spam probability	Positive at 0.3 / 0.5 / 0.7
1	1	0.95	yes / yes / yes
2	1	0.81	yes / yes / yes
3	1	0.64	yes / yes / no
4	0	0.58	yes / yes / no
5	1	0.41	yes / no / no
6	0	0.35	yes / no / no
7	0	0.18	no / no / no
8	0	0.07	no / no / no

Table 2.3: A tiny thresholding example for binary classification metrics.

2.4.1 Threshold Tradeoffs

Suppose a spam detector assigns the following probabilities to eight messages.

At threshold 0.3, Messages 1 through 6 are predicted positive. The confusion counts are

$$TP = 4, \quad FP = 2, \quad FN = 0, \quad TN = 2,$$

so the metrics become

$$\text{Accuracy} = \frac{4+2}{8} = 0.75, \qquad \text{Precision} = \frac{4}{4+2} \approx 0.67,$$

$$\text{Recall} = \frac{4}{4+0} = 1.00, \qquad \text{F1} \approx 0.80.$$

At threshold 0.5, Messages 1 through 4 are predicted positive:

$$TP = 3, \quad FP = 1, \quad FN = 1, \quad TN = 3,$$

which gives

$$\text{Accuracy} = 0.75, \qquad \text{Precision} = 0.75,$$
$$\text{Recall} = 0.75, \qquad \text{F1} = 0.75.$$

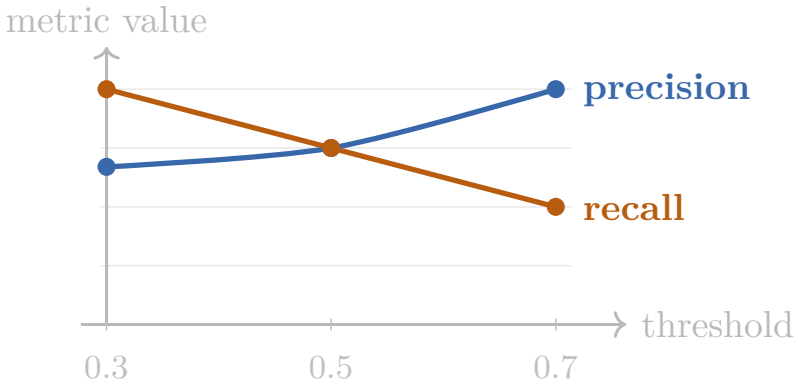


Figure 2.5: The threshold sweep makes the tradeoff visible: stricter thresholds usually raise precision and lower recall. The underlying model stays the same. Only the policy changes.

At threshold 0.7, only Messages 1 and 2 remain positive:

$$TP = 2, \quad FP = 0, \quad FN = 2, \quad TN = 4,$$

so

$$\text{Accuracy} = 0.75, \qquad \text{Precision} = 1.00,$$
$$\text{Recall} = 0.50, \qquad \text{F1} \approx 0.67.$$

This is why metric choice and threshold choice cannot be separated. Accuracy stayed the same across all three thresholds, but the operating behavior changed substantially.

The arithmetic is simple enough to reproduce with basic Python only.

Listing 2.1: A pure-Python threshold sweep

```
scores = [0.95, 0.81, 0.64, 0.58, 0.41, 0.35, 0.18, 0.07]
labels = [1, 1, 1, 0, 1, 0, 0, 0]

def metrics_at_threshold(scores, labels, threshold):
    tp = fp = fn = tn = 0

    for score, label in zip(scores, labels):
        prediction = 1 if score >= threshold else 0
```

```
        if prediction == 1 and label == 1:
            tp += 1
        elif prediction == 1 and label == 0:
            fp += 1
        elif prediction == 0 and label == 1:
            fn += 1
        else:
            tn += 1

    accuracy = (tp + tn) / len(labels)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 0.0 if precision + recall == 0 else 2 * precision
         * recall / (precision + recall)
    return tp, fp, fn, tn, accuracy, precision, recall, f1

for threshold in [0.3, 0.5, 0.7]:
    tp, fp, fn, tn, accuracy, precision, recall, f1 =
        metrics_at_threshold(
        scores, labels, threshold
    )
    print(
        threshold,
        (tp, fp, fn, tn),
        round(accuracy, 2),
        round(precision, 2),
        round(recall, 2),
        round(f1, 2),
    )
```

Practical Note. This kind of tiny script belongs in the book because it exposes the mechanism. The labs can use libraries and larger datasets; the chapter code should make the evaluation arithmetic impossible to hide.

2.4.2 Cost Matrices and Threshold Selection

Threshold choice becomes much clearer when we stop asking which threshold has the nicest generic metric and start asking what the downstream action costs.

Threshold	FP	FN	Cost if FN is severe ($FP = 1,\ FN = 6$)	Cost if FP is severe ($FP = 5,\ FN = 1$)
0.3	2	0	2	10
0.5	1	1	7	6
0.7	0	2	12	2

Table 2.4: The same score distribution can imply different thresholds once the action costs change. Threshold choice is therefore a policy choice, not a universal mathematical truth.

Suppose first that the system is a medical triage screen where a false negative is much worse than a false positive because a missed urgent case may delay treatment. If we assign cost 6 to each false negative and cost 1 to each false positive, then the total decision cost on the running eight-message threshold example would be:

$$\begin{aligned}
\text{cost}_{0.3} &= 2(1) + 0(6) = 2, \\
\text{cost}_{0.5} &= 1(1) + 1(6) = 7, \\
\text{cost}_{0.7} &= 0(1) + 2(6) = 12.
\end{aligned}$$

Under that policy, the lowest threshold is best because misses are expensive.

Now reverse the deployment story. Suppose the system is a strict inbox-protection tool where quarantining wanted mail is much more harmful than letting one nuisance message through. If a false positive costs 5 and a false negative costs 1, then

$$\begin{aligned}
\text{cost}_{0.3} &= 2(5) + 0(1) = 10, \\
\text{cost}_{0.5} &= 1(5) + 1(1) = 6, \\
\text{cost}_{0.7} &= 0(5) + 2(1) = 2.
\end{aligned}$$

Under that policy, the highest threshold is best because false alarms are expensive.

This is the operational meaning of "threshold as policy." The score has not changed. The ranking has not changed. Only the consequence model changed, and that alone was enough to reverse the preferred threshold.

Math Lens. Let $q = P(y = 1 \mid x)$ be a calibrated probability for a single case. If we predict positive, the expected cost is $C_{FP}(1-q)$; if we predict negative, it is $C_{FN}q$. Choose positive when

$$C_{FP}(1-q) \leq C_{FN}q \quad \Rightarrow \quad q \geq \frac{C_{FP}}{C_{FP}+C_{FN}}.$$

So the reusable threshold rule is $t^* = \frac{C_{FP}}{C_{FP}+C_{FN}}$ for calibrated probabilities, and the threshold moves directly with the cost ratio.

Decision Box. Do not ask which threshold maximizes one generic metric until you have written down what false positives, false negatives, abstentions, or review load actually cost in the deployment workflow. If misses are costlier, emphasize recall; if false alarms are costlier, emphasize precision.

2.4.3 Ranking Under a Fixed Budget

Suppose a university builds a model that predicts the probability that a student will miss the first two major deadlines in a programming course. The output is a probability, not a final judgment. Staff can contact at most five students this week.

This is a more realistic version of threshold selection than "pick the threshold with the highest score." A threshold of 0.30 catches every at-risk student in this toy set, but it flags nine students when the staff can contact only five. A threshold of 0.70 is very precise, but it misses half of the struggling students and leaves scarce support capacity unused. The top-five ranking policy uses the full support budget exactly and keeps four of the six at-risk students in scope.

The operational conclusion is not that one threshold is mathematically correct. It is that policy choice must be made together with capacity planning. In a real system, the university might use the top five scores rather than a fixed threshold, or it might add a review step for students near the boundary. Ranking policies become natural when intervention budget is fixed in advance.

Student	Predicted risk	Missed deadlines?	Flagged at 0.3 / 0.5 / 0.7
S1	0.92	1	yes / yes / yes
S2	0.84	1	yes / yes / yes
S3	0.78	1	yes / yes / yes
S4	0.66	0	yes / yes / no
S5	0.61	1	yes / yes / no
S6	0.54	0	yes / yes / no
S7	0.47	1	yes / no / no
S8	0.39	0	yes / no / no
S9	0.33	1	yes / no / no
S10	0.22	0	no / no / no
S11	0.14	0	no / no / no
S12	0.08	0	no / no / no

Table 2.5: A small threshold-setting example for student support. The staffing constraint forces the evaluation to include operational capacity, not only abstract metrics.

2.4.4 Top-k and Retrieval Metrics

When the system acts on a short ranked list, the top of the list matters more than average behavior over the whole candidate set. This is common in search, recommendation, document retrieval, limited-capacity triage, and review queues. In these settings, threshold metrics alone can miss the central question: how useful is the short list that users or staff will actually see?

Two common quantities are

$$\text{Precision@}k = \frac{\text{relevant items in the top } k}{k},$$
$$\text{Recall@}k = \frac{\text{relevant items in the top } k}{\text{all relevant items}}.$$

Precision@k asks how clean the exposed list is. Recall@k asks how much of the useful material the list managed to capture. If there is usually one best item, such as one authoritative policy passage or one best search result, then the position of the

Policy	Flagged	Precision	Recall	F1	Within budget?
0.30	9	$6/9 \approx 0.67$	$6/6 = 1.00$	0.80	no
0.50	6	$4/6 \approx 0.67$	$4/6 \approx 0.67$	0.67	almost, but still too many
0.70	3	$3/3 = 1.00$	$3/6 = 0.50$	0.67	yes, but leaves capacity unused
Top 5 scores	5	$4/5 = 0.80$	$4/6 \approx 0.67$	0.73	yes, exactly

Table 2.6: Threshold or ranking choice is partly a resource-allocation choice. The metric story changes once the operational budget is included.

first relevant result matters too. Information-retrieval texts often summarize that idea with reciprocal rank or mean reciprocal rank [44].

Example 2.6 (A Tiny Retrieval Ranking)**.** Suppose a policy-search system returns five passages, the first relevant passage appears at rank 2, and two of the top five passages are relevant. If the collection contains three relevant passages in total, then Precision@5 $= 2/5 = 0.40$ and Recall@5 $= 2/3 \approx 0.67$, while the reciprocal rank is $1/2 = 0.5$. The system did not put the best evidence first, but it still surfaced useful material near the top of the list.

The practical lesson is that thresholds answer "act or not?" while ranking metrics answer "how good is the short list we expose?" Retrieval systems and recommender lists are both top-of-list problems in this sense, even if their downstream actions differ.

2.4.5 Abstention and Review

Some systems should not force every score into an automatic yes-or-no action. A model may abstain on borderline cases and route them to a human reviewer. This is especially sensible when the cost of a wrong automatic action is high or when the model's probabilities are not trustworthy enough for full automation. Chapter 13 returns to this systems view in detail, but the judgment lesson

already belongs here: sometimes the best policy is "predict, but do not act automatically."

2.4.6 ROC and Precision-Recall Curves

Thresholds are often explored across a whole range rather than at one point. A receiver operating characteristic (ROC) curve plots true positive rate against false positive rate as the threshold changes. Here true positive rate is the same as recall, and false positive rate is the fraction of negative cases incorrectly flagged positive. A precision-recall (PR) curve plots precision against recall across thresholds. The ROC view is often useful when false positive rate is central. The PR view is often more informative when positive cases are rare and precision matters directly.

When positives are rare, ROC can still look deceptively healthy because the negative class dominates the denominator, while PR shows the burden of false positives directly through precision.

At Chapter 2 level, the main lesson is simple: these curves summarize *families* of operating points rather than one operating point. Confusion matrices and ROC curves are standard tools in introductory AI and machine-learning courses, so the main job here is to interpret them in context rather than to re-derive them from scratch.

> **Advanced Note.** When there are more than two classes, the same logic still applies but the bookkeeping table gets larger. A multiclass confusion matrix has one row and one column per class. Macro-averaged metrics compute the quantity separately for each class and then average across classes, which gives rare classes equal weight. Micro-averaged metrics pool the class decisions first and then compute one global quantity, which gives common classes more influence. A news-topic classifier makes this distinction practical rather than theoretical.

2.5 Confidence Quality: Calibration

Thresholds and rankings are only part of the story. If the probabilities themselves are used for downstream decisions, the next judgment question is whether those probabilities mean what they appear to mean. A model can discriminate well enough to order cases correctly and still be unreliable as a source of numerical confidence.

Calibration matters when a model's score is supposed to behave like confidence. A model is well calibrated if events predicted with probability 0.8 occur about 80% of the time. In this book's worldview, calibration is not optional decoration. Thresholds, abstention rules, review queues, and risk-based interventions behave better when the predicted probabilities can be interpreted honestly.

One standard scalar summary is the Brier score, the mean squared difference between predicted probability and observed outcome:

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2.$$

Lower is better, though a reliability diagram still shows where the probability errors live.

Accuracy tells us whether labels were often right. Calibration tells us whether confidence itself can be trusted.

This is not just a theoretical concern. Modern neural networks can achieve strong classification accuracy while still being systematically overconfident [25]. A model that looks accurate at one threshold may still be unsafe to use as a probability estimator for intervention, triage, or automated action.

Example 2.7 (Weather Forecasting and Calibration)**.** The NOAA National Severe Storms Laboratory explains calibration in exactly this way: a forecaster who predicts a 10% chance of rain should see rain on about 10% of those occasions [47]. The point is broader than weather. If the probabilities do not match the observed frequencies, the score is misleading even when some thresholded decision rule still looks acceptable.

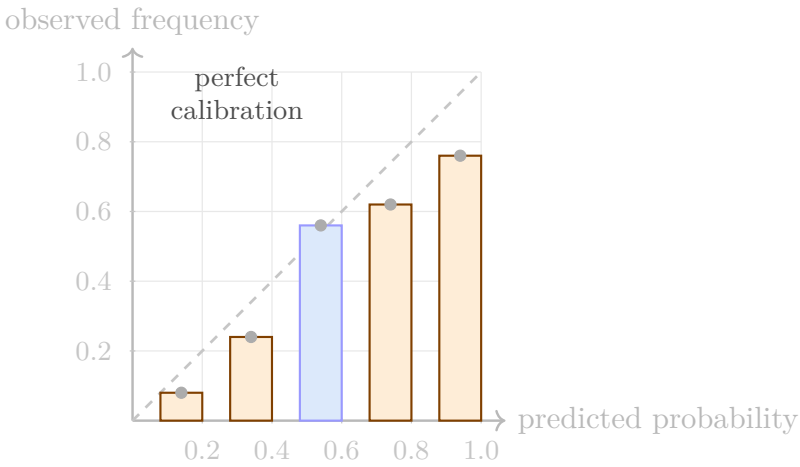


Figure 2.6: A reliability diagram compares predicted probability with observed frequency. The dashed diagonal is ideal. Bars below the line indicate overconfident predictions.

Example 2.8 (Ranking Versus Calibration)**.** A model can rank cases well while still being poorly calibrated. Suppose Model A orders the riskiest students above the least risky students perfectly, but when it says 0.9, only about 60% of such cases are actually positive. Model B may swap one borderline pair and therefore rank a little worse, but its 0.7 and 0.3 scores line up more closely with observed frequencies. Ranking quality and calibration quality are related, but they are not the same claim.

2.5.1 Base-Rate Shift and Why Precision Moves

Students often treat precision as if it were a fixed property of a model. It is not. Precision depends not only on the model's ranking quality, but also on how common the positive cases are in the population where the model is deployed.

Suppose a student-support model flags 20 students and, under the current semester conditions, 10 of those flags are truly students who need intervention. Then

$$\text{Precision} = \frac{10}{20} = 0.50.$$

Deployment setting	Students flagged	True positives among flagged	Precision
Higher-prevalence semester	20	10	$10/20 = 0.50$
Lower-prevalence semester	20	5	$5/20 = 0.25$

Table 2.7: Precision moves when prevalence moves. A fixed threshold can therefore create very different review burden quality under different base rates.

Now imagine the model is deployed in a calmer semester where truly at-risk students are less common, but the score behavior is otherwise similar. The model still flags 20 students, yet only 5 are truly positive:

$$\text{Precision} = \frac{5}{20} = 0.25.$$

Nothing in this toy example required the model to become worse at ranking. The environment changed because the base rate, also called prevalence, changed.

> **Math Lens.** Let $\pi = P(y = 1)$ be the base rate, $r = P(\hat{y} = 1 \mid y = 1)$ the true positive rate, and $f = P(\hat{y} = 1 \mid y = 0)$ the false positive rate. Among N cases, the expected true positives are $N\pi r$ and the expected false positives are $N(1 - \pi)f$, so
>
> $$\text{Precision} = \frac{N\pi r}{N\pi r + N(1-\pi)f} = \frac{\pi r}{\pi r + (1-\pi)f}.$$
>
> That is the key lens: even if r and f stay fixed, precision drops when prevalence π drops.

This is why threshold policies cannot be chosen once and forgotten. If the positive class becomes rarer, the same threshold may still produce the same queue size while delivering a much less useful queue. The staff will experience that shift as lower review value even if an offline ranking metric looked stable.

The broader lesson is important for deployment. Accuracy, recall, and ranking quality do not tell the whole operational story unless the team also asks what prevalence was assumed and whether that prevalence is likely to move.

> **Practical Note.** When precision changes unexpectedly after deployment, do not assume the model weights changed or the system suddenly "forgot" the task. First ask whether the base rate changed: new semester, new market, new device mix, new patient population, or a different triage policy upstream.

> **Practical Note.** The companion code for this chapter includes a threshold-scan script that prints same-accuracy-but-different-log-loss examples, threshold-dependent metrics, and summaries that group predictions into probability bins. Calibration is easier to understand when the probabilities and outcomes are visible side by side.

2.5.2 Uncertainty and Abstention

Calibration becomes operational when a system must decide not only *what* to predict, but also *whether* to act automatically. A score of 0.51 and a score of 0.99 may fall on the same side of a threshold, but they should not create the same level of confidence in the decision. If the model's probabilities are poorly calibrated, then a review policy built on those scores may be misleading as well.

Three ideas should be separated clearly. *Discrimination* asks whether higher-risk cases tend to receive higher scores than lower-risk cases. *Calibration* asks whether a stated probability behaves like an honest probability. *Abstention* asks whether the system should defer some cases rather than force an automatic decision. A model can rank cases well enough for prioritization while still being too overconfident for full automation.

This distinction matters in practice. A hospital triage model might be useful for ranking patients by risk even if its raw prob-

abilities need post-hoc calibration. A student-support model might correctly flag the riskiest cases, yet still need a human review queue for cases near the intervention boundary. Selective prediction formalizes this idea: the system is allowed to say "I am not confident enough to decide automatically," which can improve reliability when abstention is cheaper than a wrong action [22].

The judgment lesson is straightforward. Do not ask only whether the model is accurate enough. Ask which scores are trustworthy enough for automatic action, which scores should trigger review, and whether the uncertainty signal matches the real risk.

> **Advanced Note.** One modern way to express uncertainty is to output a prediction set or interval rather than a single label. Conformal prediction gives a general recipe for building such sets with finite-sample coverage guarantees under an exchangeability assumption [58]. The key lesson for this book is conceptual rather than technical: in some applications, the honest prediction is not one label but a small set of plausible labels, or a decision to abstain until more evidence arrives.

2.6 Validation, Cross-Validation, and Test Discipline

By this point the chapter has separated metric from loss, score from action, and calibration from ranking quality. One more distinction is needed before any of those choices deserve trust: *which data is allowed to choose them?* This is the bridge from Chapter 2's judgment logic to Chapter 3's evaluation discipline.

Thresholds, calibration methods, regularization strengths, stopping points, and even model families are not innocent details added after the score. They are part of model selection. If those choices are guided by the final test results, the test set stops being external evidence and becomes another development tool.

For this chapter, the core logic can be said plainly. The training data teaches the model parameters. The validation data chooses among serious alternatives: which threshold or ranking cutoff is usable, whether calibration is needed, and which tuned settings make the decision policy work best. The test data is then touched once, after those choices are frozen, to check whether the chosen policy still looks defensible on new cases.

Example 2.9 (Threshold Choice Is Part of Model Selection)**.** Suppose a student-support team considers thresholds 0.40, 0.55, and 0.70 on the validation set. Under the team's weekly review constraint, threshold 0.40 catches more true positives but creates too many flags to review. Threshold 0.70 keeps the queue small but misses too many students in need. Threshold 0.55 gives the best recall while staying within review capacity, so it becomes the chosen policy. If the team then checks the test set, dislikes the result, and moves the threshold again, the test set has quietly become part of development. The number is still computed correctly, but it is no longer external evidence.

2.6.1 Cross-Validation When Data Is Limited

With abundant data, one protected validation split is often enough to choose among candidate settings. With limited data, a single split can be too noisy. Cross-validation repairs that weakness by rotating the validation role across several folds of the development data.

The main idea is simple. Partition the development data into several folds. Hold one fold out as validation, train on the remaining folds, record the decision-relevant metric, and rotate. At the end, average the fold-level validation results and choose the setting that behaves best on average.

The important point is conceptual rather than procedural. Cross-validation is not a replacement for external evidence. It is a stronger way to use the development data when one validation split would be unstable. If a final test set is available, it should still remain outside the cross-validation loop.

Listing 2.2: Selection discipline when the decision policy is part of the model choice

```
development_data, test_data = split_once(raw_data)

candidate_settings = propose_settings()
best_setting = None
best_value = -inf

for setting in candidate_settings:
    fold_values = []
    for train_fold, validation_fold in make_folds(
        development_data):
        model = fit_model(train_fold, setting)
        scores = model.predict_scores(validation_fold)
        value = evaluate_policy(scores, validation_fold.
            labels, setting)
        fold_values.append(value)
    mean_value = average(fold_values)
    if mean_value > best_value:
        best_value = mean_value
        best_setting = setting

final_model = fit_model(development_data, best_setting)
final_value = evaluate_policy(
    final_model.predict_scores(test_data),
    test_data.labels,
    best_setting
)
print(final_value)
```

In this skeleton, `setting` may include more than one kind of choice: a regularization strength, a tree depth, a threshold, a ranking cutoff, or a calibration rule. That is the deeper lesson. The decision policy is not outside the experiment. It is part of what is being selected.

2.6.2 Hyperparameter Search Without Test Leakage

A *hyperparameter* is a setting chosen by the experimenter rather than learned directly by the basic fitting rule. Examples include the number of neighbors in k-nearest neighbors, the depth of

a tree, the strength of regularization, the learning rate of an optimizer, or the threshold that turns scores into action. These choices are often necessary. The mistake is not searching over them. The mistake is letting the final test result guide the search.

Once this is stated clearly, the right workflow becomes straightforward. Search over alternatives on training and validation data, or inside cross-validation on the development set. Freeze one configuration. Then evaluate once on the test set. The test set should answer "did this chosen system survive new data?" rather than "which of these many systems should we keep?"

> **Practical Note.** This chapter introduces the logic because metric choice, threshold choice, calibration, and selection protocol belong to one judgment chain. The next chapter makes the workflow fully procedural by showing how protected splits, leakage, and baseline discipline keep that chain honest.
>
> **Problem-Solving Technique. Selection protocol.** Before reporting a score, write down which data role chooses the model family, tuned settings, threshold, ranking cutoff, and calibration method. If the test set helped choose any of them, the reported score is no longer an external check.

2.7 Metric Failure Gallery

Three cases where a standard metric looks good while the real situation is not. Read them as recurring failure patterns rather than as isolated exceptions: each one shows how a real operational question can disappear behind a respectable number.

Case one: high accuracy under class imbalance. A student-support model predicts at-risk status on a cohort where only 5% of students are truly at risk. A model that predicts "not at risk" for every student achieves 95% accuracy. That number looks strong at first glance. But the model has never flagged a student in need. Accuracy rewards agreement with the label dis-

tribution. When the distribution is heavily skewed, agreement with the majority is cheap. Practitioners who report only accuracy on imbalanced problems invite exactly this confusion. The right questions are: how many true positive cases did the system surface, and at what review cost?

> **Failure Box.** High accuracy on an imbalanced dataset is often a sign that the model has learned to ignore the minority class entirely. Always report per-class recall alongside accuracy when prevalence is low.

Case two: good ROC-AUC with poor precision at the deployment budget. Suppose a classifier achieves ROC-AUC of 0.88 on a test set. That looks like a well-discriminating model. Now suppose the operations team can review at most 30 flagged cases per week. The score cutoff required to capture 80% of true positives would flag 120 cases per week, which is far above the team's capacity. If the team keeps only the top 30 scores instead, recall drops sharply and many truly important cases are missed. ROC-AUC summarizes performance across all possible thresholds and says nothing about which threshold is actually usable under the team's capacity constraint. A good ROC curve does not guarantee operational usefulness at any specific threshold.

> **Practical Note.** When reporting ROC-AUC, also report precision, recall, and queue size at the threshold the team would actually use. The gap between "aggregate discriminative quality" and "operational precision at the deployment budget" is where many model evaluations mislead.

Case three: low training loss with bad calibration at the action threshold. A model achieves very low cross-entropy loss on the training set and moderate cross-entropy on the validation set. The loss curve looks like a textbook training success. But inspection of the reliability diagram shows that cases the model scores between 0.6 and 0.8 have an observed positive rate near 0.35, not 0.7. The model is confidently overestimating risk

Metric reported	What it hides	What to check instead
Accuracy	Per-class recall on minority class	Recall, precision, and F1 broken out by class
ROC-AUC	Precision at the deployment threshold	Precision-recall curve at the review-budget cutoff
Training loss	Probability calibration in the action range	Reliability diagram binned around the operating threshold

Table 2.8: A metric gallery of misleading but common reporting choices. Each metric is genuine but incomplete when used alone in high-stakes or imbalanced settings.

in exactly the range where most of its actionable flags fall. Downstream, the intervention team is applying the same urgency to a 0.65-scored case as to a 0.95-scored case, while in reality those two cases have very different observed risk. Cross-entropy can be low while probability estimates remain poorly calibrated in the decision-relevant range. Low loss is not the same as useful probabilities.

The shared lesson is that evaluation claims are always implicitly relative to a context: the class distribution, the review capacity, and the decision-relevant probability range. A metric that ignores context can look good while concealing a serious operational problem. The right habit is to ask not only "what is the number?" but also "what could this number be hiding?"

2.8 Chapter Artifact: A Metric and Action Worksheet

By the end of Chapter 2, the reader should be able to write down not only which number to report, but how that number will become an action and which held-out role is allowed to choose the policy details. The point is to make metric choice, threshold policy, calibration expectations, and selection discipline explicit before the workflow quietly hardens around a vague score. This worksheet is the Chapter 2 counterpart to Chapter 1's framing

Worksheet field	What must be written down
Decision context	What real action or review workflow the prediction supports
Primary metric	The main quantity that captures success for that decision
Supporting metrics	Additional quantities needed to expose trade-offs such as imbalance or calibration
Action type	Thresholded decision, top-k ranking, probability forecast, or abstain-to-review workflow
Threshold or ranking policy	Exactly how scores become action under real cost or capacity constraints
Selection protocol	Which held-out role chooses tuned settings, threshold, ranking cutoff, or calibration method, and when the test set is touched
Calibration need	Whether the score must behave like an honest probability downstream
Abstention or review rule	Which cases should be deferred rather than forced into automatic action

Table 2.9: A metric-and-action worksheet forces the reader to specify not only what number matters, but what downstream policy the number is supposed to support.

memo: Chapter 1 states the problem clearly; Chapter 2 states how success will be judged and operationalized.

> **Problem-Solving Technique. Metric and action worksheet.** Before reporting a model output, write down the decision context, primary metric, supporting metrics, threshold or ranking policy, selection protocol, calibration need, and abstention rule. If the action rule or the held-out selection rule is still fuzzy, the score is not yet ready to defend.

2.9 Looking Ahead

This chapter made predictive judgment explicit: metrics, losses, thresholds, rankings, calibration, and abstention. The next chapter asks a different question: how do we protect those judgments

from overfitting, leakage, weak baselines, and accidental test-set reuse? Chapter 2 chose what would count as good behavior; Chapter 3 asks whether the evidence for that choice deserves to be trusted.

Chapter Summary

- Chapter 2 begins from decision behavior, not from formulas. A model is good only relative to the action it is meant to support.
- Reported metrics express what behavior matters for the task; training losses are optimization penalties chosen so the model can be trained.
- Accuracy, precision, recall, F1 score, top-k ranking metrics, and calibration answer different questions and should be selected from the decision context rather than from habit.
- Thresholds and top-k policies are part of system design because they turn scores into actions under real capacity and error costs.
- Calibration matters when a score must behave like a probability rather than only a ranking signal.
- Abstention and review are often better policies than forced automation when uncertainty is operationally meaningful.
- Validation or cross-validation chooses thresholds, tuned settings, and calibration methods; the test set stays external until those choices are frozen.
- The main deliverable of the chapter is a metric-and-action worksheet written before a serious model output is reported.

Study Questions

1. Why is "what would count as a good prediction here?" a better opening question than "what loss should we minimize?"

2. Why might a model be trained with cross-entropy but reported using precision and recall?
3. Give one setting in which ranking quality matters more than a fixed threshold.
4. Why might precision@k or recall@k be more informative than accuracy for search or recommendation?
5. Why is calibration different from accuracy?
6. Why is abstention different from simply choosing a stricter threshold?
7. Why is repeated test-set peeking a problem even when every reported score was computed correctly?

Exercises

1. For the course-support example, explain why a model with lower training loss could still be less useful than a model with better top-five ranking quality.
2. Draw a tiny four-example binary classification problem in which two models have the same accuracy but different log loss. Explain what the probability assignments say about the models.
3. Give a real-world example where mean absolute error would be more appropriate than mean squared error, and justify your answer.
4. Using the theorem on best constant predictors, explain why squared error prefers the mean while absolute error prefers a median. Then give one example where that difference would matter in practice.
5. Using Table 2.3, compute the confusion matrix at threshold 0.3 and verify the precision, recall, and F1 values by hand.
6. Reproduce the threshold sweep from the chapter in plain Python without external libraries. For each threshold, print the confusion counts, accuracy, precision, recall, and F1 score.

7. A ranked list of six candidate passages has relevance labels $[0, 1, 1, 0, 1, 0]$ from rank 1 to rank 6, and there are four relevant passages in the collection overall. Compute precision@3, recall@3, and reciprocal rank.

8. Explain why the weather-forecasting example is a calibration question rather than a threshold question.

9. A team compares three regularization strengths and two thresholds for the same classifier. Describe a protected workflow for choosing among the six settings, and explain where cross-validation could help if the dataset is small.

10. Use Table 2.9 to sketch a one-page metric-and-action worksheet for either spam filtering, course support, or another application of your choice. If you are carrying forward a framing memo from Chapter 1, use the same case and state which parts of the original memo stay fixed while the decision rule becomes more explicit.

Challenge Exercises

1. Construct a tiny binary classification example with six data points for which two different thresholds produce the same accuracy but very different precision and recall. Show the confusion matrix for both thresholds.

2. Suppose a hospital has resources to review only the highest-risk 10% of cases each day. Explain why ranking quality and calibration may matter more than raw accuracy in that setting.

3. Extend the course-support example in Table 2.5. Propose a rule for handling the six students above threshold 0.5 when staff can contact only five of them, and justify the rule using the chapter's evaluation vocabulary.

4. A model is well ranked overall but slightly miscalibrated near the review boundary. Explain when that is an acceptable limitation and when it becomes a deployment problem.

5. A team tunes thresholds by checking the test set after each change and keeps the best final operating point. Explain precisely why the final precision and recall are no longer external evidence, even if the confusion counts were computed honestly.

Chapter 3

Generalization, Leakage, and Evaluation Discipline

Learning Goals

- Explain generalization as the ability of a result to survive new data rather than only the observed sample.
- Distinguish underfitting and overfitting as different failure modes of the same training process.
- Use protected train, validation, and test roles as procedural boundaries rather than as vocabulary words.
- Recognize common leakage patterns before they invalidate an experiment.
- Use a baseline ladder and simple sanity checks to judge whether a reported win means anything.
- Produce a one-page evaluation plan before reporting a serious model claim.

Chapter 2 answered one judgment question: once a model outputs scores, what predictive behavior should count as good?

Chapter 3 answers the next one: when can we trust the evidence for that behavior?

This chapter is therefore about claim protection. A result can look excellent because the model learned a real pattern. It can also look excellent because the split leaked, the baseline was weak, the test set quietly became part of development, or the task changed between semesters. The chapter's job is to keep those possibilities separate. Read the chapter as the construction of one evaluation plan: what counts as new evidence, what workflow keeps it external, what traps can fake success, and what baseline makes the final claim legible.

> **Problem-Solving Lens.** Protect the claim by keeping development, external evidence, and baseline comparison separate. A strong-looking score is not enough if the workflow that produced it made the result too easy to win.

> **Problem-Solving Technique. Claim protection.** Before trusting a win, ask what part of the pipeline could have let future information, duplicate cases, excessive tuning, or a weak baseline make the score look better than it really is.

3.1 Opening Dilemma: The Validation Score Looks Great. Can We Trust It?

Keep the same course-support setting in view from the first two chapters. Suppose a support team trains several early-warning models on historical course data and one of them produces excellent validation recall and a strong final test score. Before celebrating, the team still has to ask harder questions. Were the preprocessing steps learned only from the training data? Was the test set kept external until the end? Did the model beat a serious baseline or only a convenient straw man? Would the result still look strong next semester? Each later section of the chapter turns one of those questions into a procedural safeguard.

Those questions are not paperwork added after the model. They are what make the number mean anything.

3.2 Can the Result Survive New Data? Generalization and Overfitting

The training loss tells us how well the model matches data it has already seen. The next judgment question is whether the result survives new examples.

Definition 3.1 (Generalization)**.** Generalization is the ability of a model to maintain good performance on unseen examples drawn from the same underlying problem as the training data.

Two classic failure modes define the landscape:

- *Underfitting*: the model is too simple, too restricted, or too poorly trained to capture even the main patterns in the training data.
- *Overfitting*: the model matches the training data too closely, including noise or accidental details that do not hold for new data.

One way to see this is through the *generalization gap*, the difference between training performance and performance on held-out data. A small gap does not guarantee excellence, but a large gap is a warning sign that the model has learned something too specific to the observed sample.

Model capacity, meaning how complex a pattern the model can represent, plays a central role here. A more flexible model can represent more complicated functions. That flexibility is useful when the data contains real structure, but dangerous when the data is limited or noisy. Overfitting is not evidence of intelligence. It is evidence that the experimental setup allowed memorization or unstable pattern matching to masquerade as learning.

> **Evidence Box.** Zech et al. studied pneumonia detection from chest radiographs across multiple hospital systems and found that generalization changed

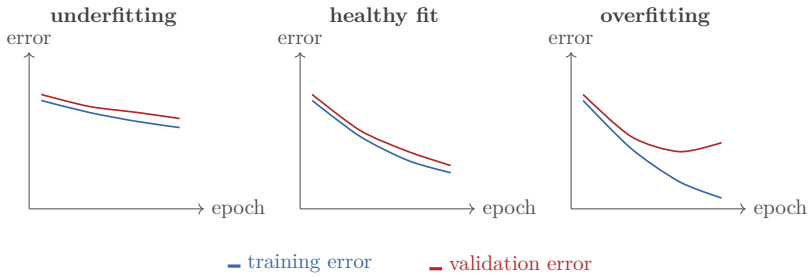


Figure 3.1: The same training loop can fail in three different ways. Underfitting keeps both errors high. Healthy fitting lowers both while keeping them close. Overfitting continues improving training error while validation error turns upward.

substantially across institutions; they also analyzed how hospital-specific cues could act as confounders [63]. That is the chapter's lesson in real form: success on familiar held-out data is not the same as robust generalization after deployment.

3.3 Protected Evaluation Workflow

The next question is procedural rather than mathematical: how do we protect evaluation from development? The answer is the disciplined split between training, validation, and test data. This is the chapter's workflow spine, because every later safeguard depends on these roles staying distinct.

- The *training set* is used to fit parameters.
- The *validation set* is used to choose tuning settings, model structures, thresholds, or stopping points.
- The *test set* is reserved for the final report after the important decisions are already made.

Practical Note. The workflow is simple enough to say plainly: split first, fit preprocessing on train only, train candidate models on train, choose among them on validation, and touch the test set once at the end.

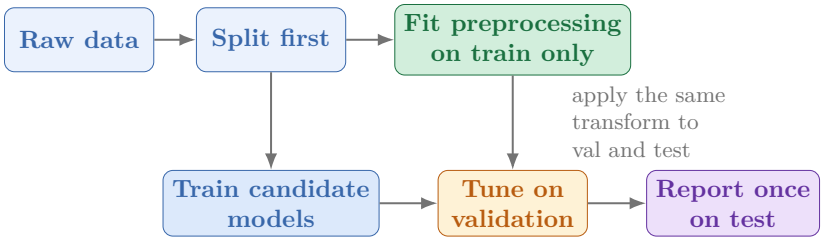


Figure 3.2: A correct evaluation workflow is procedural, not only conceptual. The split comes before the learned preprocessing, and the test set is touched only after the model-selection decisions are finished.

Students often understand those definitions and still run the workflow incorrectly. The key is not only what the sets are called. The key is the order in which they are used. The names matter less than the boundary: once information crosses from held-out evidence back into development, the claim is weaker even if the code still runs cleanly.

Here *preprocessing* means data transformations such as scaling, normalization, or feature extraction that are learned from the training data and then applied unchanged to validation and test data.

> **Problem-Solving Technique. Protected test set.** The test set is for one final report, not for development. Once test results influence model choice, the test set stops being an external check.

This logic can be expressed as simple pseudocode:

Listing 3.1: A disciplined evaluation workflow

```
train_data, validation_data, test_data = split_data(
    raw_data)

preprocessor = fit_preprocessor(train_data)
train_ready = preprocessor.transform(train_data)
validation_ready = preprocessor.transform(validation_data)
test_ready = preprocessor.transform(test_data)

model_candidates = train_many_models(train_ready)
chosen_model = select_using_validation(model_candidates,
```

```
    validation_ready)
final_score = evaluate_once(chosen_model, test_ready)
print(final_score)
```

Cross-validation is a useful extension when data is limited. Instead of relying on one validation split, the training data is partitioned into several folds, meaning several train-validation partitions, and the validation role is rotated across them. Even then, a final untouched test set remains valuable when possible.

> **Practical Note.** Chapter 3 establishes minimal experimental discipline for one credible result. Chapter 11 returns to the same logic at larger scale with repeated runs, ablations, slice analysis, and stronger empirical claims.

3.4 How Experiments Lie: Leakage

Data leakage occurs when information from outside the legitimate training process reaches the model or the experiment design. Leakage is dangerous because it often produces excellent-looking results for the wrong reason. The cleanest way to think about it is as a broken boundary: some information that should have stayed outside development was allowed back in.

Common forms of leakage include:

- normalizing or scaling the full dataset before the train-test split
- selecting features using all examples before separating the test set
- using duplicate or near-duplicate examples across training and test data
- allowing future information into a prediction task that should use only past information
- tuning repeatedly on the test set until the score looks good

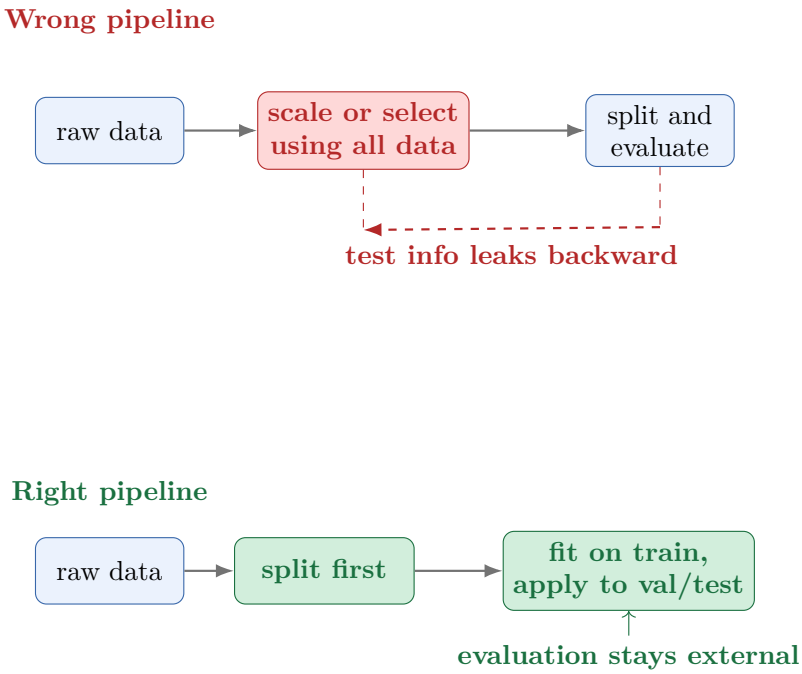


Figure 3.3: Leakage is usually procedural. The wrong pipeline lets information from validation or test examples influence preprocessing or feature design. The right pipeline keeps those steps inside the training boundary.

Leakage is not a small classroom mistake. Bernett et al. present it as a recurring threat to validity in machine learning for biology and argue that guarding against it is central to trustworthy evaluation and reproducibility [5]. The setting in their paper is biological ML, but the lesson is completely general.

Listing 3.2: Leakage in one line of pseudocode

```
# Wrong: statistics use all examples, including future
    test cases.
scaled_all = scale(raw_data)
train_data, test_data = split_data(scaled_all)

# Right: split first, then learn the scaling rule from
    train only.
train_data, test_data = split_data(raw_data)
```

Baseline rung	What it tells you
Majority class or constant prediction	Whether the headline score is beating trivial class imbalance or central tendency
Simple weighted model	Whether the task already has a strong low-complexity solution
Small tree, simple word-count model, or nearest-neighbor copy rule	Whether modest nonlinear structure or simple representation tricks already explain most of the gain
Only then a complex model	Whether additional complexity is buying something real rather than hiding weak experimental design

Table 3.1: A baseline ladder keeps evaluation honest. If a sophisticated model is not clearly above a simple rung, the experiment is not yet ready for a strong claim.

```
scaler = fit_scaler(train_data)
train_data = scaler.transform(train_data)
test_data = scaler.transform(test_data)
```

> **Warning.** Leakage does not merely make a result slightly noisy. It can destroy the meaning of the experiment. A leaked benchmark score is not a weaker scientific claim; it is often no scientific claim at all.

3.5 Baseline Ladder and Sanity Checks

Before trusting a sophisticated model, ask how well simple methods perform. Always predicting the majority class, using a simple weighted linear model, fitting a small decision tree, or even comparing against a random baseline can reveal whether the task is easy, imbalanced, or poorly specified. The ladder is not a ritual of humility for its own sake. It is a way to make sure the final claim is about real added value rather than about avoiding an obvious comparison.

Baselines are not only practical convenience. They are part of the philosophy of evidence. They show whether the new method actually improves on something simple, expose hidden issues such as class imbalance or weak labels, and anchor interpretation. A score of 85% may be excellent or trivial depending on the baseline.

The same habit appears in good course design and good benchmark practice. Starter baselines, public datasets, and example solutions do not make the work trivial. They make the claim legible.

> **Practical Note.** Two sanity checks should become automatic:
>
> - **Shuffled labels.** If labels are randomized and the model still looks strong, the pipeline is broken.
> - **Split stability.** If validation performance swings wildly under small changes in the train-validation split, the setup may be too fragile for a strong conclusion.

3.6 Metric Failure Gallery

By this point in the chapter, the reader has seen that no single number settles evaluation. It helps to make that lesson explicit through a small gallery of common metric traps. A correct metric can still be the wrong summary if it leaves the operational question untouched.

Example 3.1 (High Accuracy, No Useful Screening)**.** A screening model evaluated on a heavily imbalanced dataset may report very high accuracy by predicting the majority class almost all the time. The metric is not numerically wrong. It is simply answering the wrong practical question.

Example 3.2 (Strong Ranking, Weak Top-of-Queue Policy)**.** A triage model can achieve a strong ranking score overall and still place too few true urgent cases in the top twenty reviewed items if prevalence is low or the score distribution is compressed near

Headline metric	Why it looks reassuring	What it can still hide
High accuracy	Most predictions are correct overall	Class imbalance may make the system useless on the rare class that matters most
Strong ROC or ranking score	The model orders cases well in aggregate	The chosen operating point may still produce too many false alarms or too few true positives for the actual review budget
Lower loss or better log loss	Average prediction quality improved	The action threshold, calibration in the high-risk region, or severe-error slice may still be unacceptable

Table 3.2: A metric can be locally correct and still globally misleading if it does not match the deployment question.

the operational cutoff. In that setting, the aggregate ranking summary is not enough. The top-of-queue behavior is the real test.

Example 3.3 (Better Loss, Worse Operational Region)**.** A probability model may reduce average log loss while becoming slightly worse calibrated exactly in the score range that triggers human escalation. If decisions are made in that region, the lower loss does not automatically imply a better system.

The lesson is not that metrics are untrustworthy. The lesson is that every metric answers a narrower question than students first assume. Good evaluation therefore asks not only "what is the score?" but also "what behavior is this score summarizing, and what important behavior is still outside it?"

> **Problem-Solving Technique. Metric failure gallery.** Whenever a headline number looks persuasive, name one failure mode it cannot rule out. If no such failure mode comes to mind, the evaluation may still be too shallowly understood.

Plan field	What must be written down
Seven-question focus	Questions 1, 5, and 6 directly: what decision matters, what baseline counts, and what evidence supports the claim. The plan also fixes the threshold or ranking policy that later systems work must implement responsibly
Decision context	What real decision or policy the evaluation is meant to support
Primary metric	The main quantity that expresses success for that decision
Supporting metrics	Additional metrics needed to expose tradeoffs such as imbalance or calibration
Threshold or ranking policy	How scores become actions under real capacity or cost constraints
Calibration check	How confidence quality will be inspected if probabilities matter downstream
Split protocol	How training, validation, and test roles are separated
Leakage controls	What preprocessing or feature steps must stay inside the training boundary
Baseline ladder	Which simpler baselines the system must beat before a strong claim is made
Claim statement	The exact sentence the final result is supposed to justify

Table 3.3: If this plan cannot be written, the evaluation claim is usually still underdesigned.

3.7 Chapter Artifact: A Minimal Evaluation Plan

By the end of Chapter 3, the reader should be able to write a short evaluation plan before reporting any result. The goal is not paperwork. The goal is to make the claim explicit enough that weak evidence and procedural mistakes become visible early. This is the point where decision context, threshold as policy, leakage control, and the baseline ladder become one reusable document. Chapter 2 said what good behavior would look like; Chapter 3 says what workflow and evidence would justify believing that behavior is real.

Problem-Solving Technique. Evaluation plan. Before reporting a score, write down the decision context, primary and supporting metrics, threshold or ranking policy, calibration check, split protocol, leakage controls, baseline ladder, and final claim sentence.

3.8 Common Mistakes in Evaluation

Mistake Checklist.

- **Optimizing the wrong quantity.** A model can improve its training loss and still fail at the deployment metric.
- **Letting the test set become a teacher.** Once test results influence model choice, the final score stops being an external check.
- **Reporting one number and hiding the tradeoff.** One metric is rarely enough because accuracy, threshold policy, calibration, and rare-case behavior can differ.
- **Skipping leakage and baseline checks.** A contaminated pipeline or a weak baseline does not support a strong claim.
- **Comparing complex models without a simple baseline.** You still cannot tell whether the task is genuinely hard or the gain is meaningful.

3.9 Looking Ahead

This chapter made evaluation discipline explicit: generalization, protected evidence, leakage control, baselines, and claim design. The next chapter asks whether a simple linear model can already solve the task once the representation is chosen well. In other words, the book now moves from protecting the claim to choosing the first serious model family that deserves to make one.

Chapter Summary

- Chapter 3 asks whether the result survives new evidence rather than only the sample already seen during development.
- Underfitting and overfitting are different ways a training process can fail to generalize.
- Protected train, validation, and test roles exist to keep evaluation external to model development.
- Leakage, weak baselines, and test-set reuse are among the most common ways experiments become self-deceptive.
- A serious empirical claim needs both a defended metric and a defended workflow.
- The main deliverable of the chapter is a one-page evaluation plan written before a serious model claim is made.

Study Questions

1. Why is generalization a better judgment question than training loss alone?
2. How does a validation set differ from a test set in purpose?
3. Why must learned preprocessing stay inside the training boundary?
4. Why is a weak baseline a threat to interpretation rather than only to competitiveness?
5. Why can one strong-looking metric still support a weak claim?
6. Why is an evaluation plan more than paperwork?

Exercises

1. Design a train-validation-test workflow for a small tabular dataset. State clearly which choices may use the validation set and which choices must not use the test set.

2. A researcher scales all input features using the mean and variance of the entire dataset before creating train and test splits. Explain why this is leakage.

3. Give one example of a weak baseline and one example of a serious baseline for the same task. Explain what each one would and would not tell you.

4. Propose two sanity checks for a model-development workflow in a domain of your choice, and explain what failure on each check would mean.

5. Use a public task such as sports-shot prediction or news topic classification and state one sensible primary metric, one possible secondary metric, and one failure mode the headline metric cannot rule out.

6. Write a one-page evaluation plan using Table 3.3 for either spam filtering, course support, or another application of your choice. If you are carrying one case through the book, write this for the same case as your framing memo and metric-and-action worksheet so the three documents can be read together.

Challenge Exercises

1. A team tunes ten different model families by checking their performance on the test set and reports the best one. Explain precisely why this invalidates the final claim even if the reported score is numerically correct for that one run.

2. A pipeline beats a strong baseline by a small margin, but the shuffled-label check still reports high performance. Diagnose what kinds of bug or contamination could create that result.

3. A course-support team trains on one semester and evaluates on a random split from the same semester. Explain why that may still exaggerate generalization and propose a better split protocol.

4. Write the strongest honest claim for a model that wins on the main metric, loses on a critical slice, and was compared under

unequal tuning effort.

Chapter 4

Linear Models and Representation

Learning Goals

- Explain why linear modeling begins with a representation question rather than with a model formula.
- Treat feature design as part of the scientific content of a machine learning problem rather than as clerical preprocessing.
- Understand linear regression and logistic regression as two versions of the same weighted-sum pipeline.
- Read coefficients with language discipline, especially under scaling, correlation, and omitted-variable problems.
- Distinguish predictive association from intervention claims, especially when coefficients look tempting to overinterpret.
- Use L1 and L2 regularization to keep a linear baseline controlled rather than merely simple.
- Produce a first baseline design sheet before moving to more expressive model families.

The previous two chapters established how model claims should be judged and protected. The next modeling question is now

operational: once a task and evaluation plan are in place, what is the simplest serious model family that deserves a real test?

Very often, the answer is a linear model. But that answer is only meaningful if the representation makes the relevant structure visible. A linear model can only add weighted pieces of what it is given. If the important structure has not been encoded, the weighted sum cannot rescue the problem. If the representation is strong, however, one weighted sum can go surprisingly far. The model family is therefore not only a choice of formula. It is a test of whether the representation already exposes the signal we care about.

This chapter is therefore not mainly a tour of linear regression and logistic regression. It is the book's representation chapter and its first real baseline chapter. The real question is:

Can a good representation plus one weighted sum already solve the task? If yes, why? If not, what missing structure does the failure reveal?

The chapter's reusable deliverable is a first baseline design sheet. Keep that target in mind while reading: the goal is to make the first serious linear model explicit enough that later model changes become justified rather than impulsive. By the end of the chapter, the reader should be able to say what the model sees, what it predicts, how its coefficients may be read safely, how it will be regularized, and what specific failure would justify leaving linearity.

> **Problem-Solving Technique. Artifact preview: first baseline design sheet.** Keep five fields active from the beginning of the chapter: the raw input and chosen feature map, the target type, the first linear family to try, the regularization choices to validate, and the specific failure pattern that would justify leaving linearity.
>
> **Problem-Solving Lens.** This chapter tackles the first major modeling dilemma in the book: once the task is framed and the judgment protocol is clear, is a

transparent linear baseline already enough? The answer depends less on the fitting rule than on whether the representation makes the relevant structure visible.

4.1 Opening Dilemma: Is One Weighted Sum Enough?

Suppose we have already framed an early student-support task, chosen an evaluation plan, and agreed on what counts as a useful prediction. A support team wants to use early quizzes, submissions, and platform activity to estimate which students may need timely human outreach. We still need a first model family. A deep model may eventually win, but it is rarely the right first move. The right first move is usually the simplest serious model that can reveal whether the representation already contains most of the signal.

That is why linear models appear early in this book. They are not here because they are historically old or mathematically easy. They are here because they make the route from representation to prediction unusually visible. A linear model says, in effect: encode the example as numbers, attach a weight to each one, add the contributions, and transform the result if needed. The simplicity is pedagogically valuable because it makes failure diagnostic instead of mysterious.

Keep that same early student-support setting in view through this chapter. Later chapters will widen the institutional workflow into message routing, retrieval, and grounded answers, but the first baseline question is simpler: can a transparent weighted-sum model already surface the right cases for human review? Each major section of the chapter turns that one question into a modeling discipline: choose the representation, understand the score, read the coefficients carefully, control complexity, and diagnose failure structurally.

Evidence Box. CDC guidance for Youth Risk Behavior Survey trend analysis still uses logistic regres-

Problem shape	What linear models are usually good for
Structured tabular data	Fast baselines, interpretable coefficients, and surprisingly strong performance when the feature design is careful
Sparse text features	Reliable bag-of-words or TF–IDF baselines where weighted word evidence already carries most of the signal
High-dimensional raw images, audio, or long sequences	Usually only baseline value unless a strong representation has already been learned elsewhere

Table 4.1: Linear models are not universal, but they are often the right first serious baseline for structured and sparse problems.

sion for dichotomous risk behaviors and linear regression for continuous ones [12]. These models are not just teaching artifacts. They remain standard analytical tools in serious public-health workflows.

4.2 Representation Is the Model

The heart of this chapter is not a fitting rule. It is a representation question.

Definition 4.1 (Feature)**.** A feature is a measurable attribute used to represent an example for a model. Features may be numeric, binary, categorical after encoding, counts, ratios, embeddings, or other derived quantities.

This definition sounds technical, but its consequences are conceptual. A feature is not just a column in a table. It is a modeling decision. When we choose floor area, age, and distance to transit as features for house-price prediction, we are asserting that these properties may help explain price. When we encode words or character patterns for spam detection, we are asserting that certain patterns in language or formatting may carry useful signal. Feature design is therefore not bookkeeping after the interesting part. It is where a large share of the interesting part already happens.

This is why representation is already part of the model. The model does not see "a house in a good neighborhood" or "a student who is disengaging." It sees the encoded representation. Someone must decide which numbers or indicators could plausibly carry the relevant information.

Problem-Solving Technique. Representation audit. Before fitting a model, ask:

- What is the raw input?
- What is the encoded representation?
- What information is lost by the encoding?
- What assumptions are being inserted artificially?

4.2.1 Raw Inputs and Feature Maps

Sometimes the raw input is already numeric and usable. Age, temperature, income, and elapsed time fit that description. In many problems, however, raw inputs must be transformed before a linear model can use them effectively. Text must be turned into counts, indicator variables, or embeddings. Categories must be encoded. Dates may need to become cyclic or calendar-aware features. Interactions may need to be added explicitly if they matter.

Mathematically, this is often written as a feature map $\phi(x)$, which transforms a raw example x into a feature vector:

$$\phi(x) = [\phi_1(x), \phi_2(x), \ldots, \phi_d(x)]^\top.$$

The model then operates on $\phi(x)$ rather than directly on the raw input. This notation is useful because it separates two sources of modeling power:

- the *representation* chosen by the feature map
- the *fitted weights* learned by the model

If the representation is weak, fitting better weights may not be enough. If the representation is strong, even a simple weighted sum can work well.

Practical Note. The linear-model workflow is compact: encode the raw input, fit a baseline, inspect the residuals or errors, then ask whether the representation needs interaction terms, scaling, or a different feature map before changing model family.

Prerequisite Bridge. If the notation $\phi(x)$ is new, read it as "the version of the raw input that the model actually uses." In many practical problems, this is where most of the modeling judgment lives.

From this point forward, the notation will be slightly compressed: unless a distinction matters, x will usually mean the *encoded feature vector* rather than the raw object. When the distinction matters, it is safer to think in two layers:

$$x_{\text{raw}} \longrightarrow \phi(x_{\text{raw}}) \longrightarrow w^\top \phi(x_{\text{raw}}) + b.$$

That small discipline prevents one of the most common beginner confusions in linear modeling. The coefficients do not multiply "the email" or "the apartment" in the abstract. They multiply the engineered features that someone chose to expose.

Example 4.1 (From Raw Input to Feature Vector)**.** Suppose the raw email is "FREE cash offer today," sent from an unfamiliar domain and containing two links. A simple feature map might turn that raw message into

$$\phi(x_{\text{raw}}) = \begin{bmatrix} 1 \\ 2 \\ 0.8 \\ 0 \end{bmatrix},$$

where the four entries mean urgent-language indicator, number of links, sender-risk score, and whether the sender matches a trusted course domain. The linear model never sees the sentence directly. It sees only these four numbers. If the feature map throws away evidence that later turns out to matter, the weighted sum cannot reconstruct it after the fact.

4.2.2 Interaction Features and Basis Expansion

A linear model applied to raw features can only express additive effects: the contribution of each feature is independent of every other. But many real problems have structure that is inherently interactive. The effect of floor area on apartment price may depend on the neighborhood. The effect of missing assignments on student risk may depend on whether the student is in the first or last week of term.

Interaction features make these joint effects explicit. An interaction term for two features x_1 and x_2 is simply their product $x_1 x_2$. When this product is added as a new coordinate, the linear model can now assign a separate coefficient to the joint effect.

Basis expansion generalizes the idea. Any function of the raw inputs can be added as a new feature: squares, square roots, logarithms, indicator functions for threshold conditions, products of three or more features, or Fourier-style sinusoidal terms. The resulting feature map $\phi(x)$ may be much richer than the original input, but the model on top of it remains linear in the expanded features.

Example 4.2 (Interaction Features in Practice)**.** Suppose we predict student risk from two features: fraction of assignments missed (x_1) and week of semester (x_2). Independently, missing a moderate fraction of assignments may be less alarming in week two than in week ten. Adding the product $x_1 x_2$ gives the model a way to express that the effect of missed assignments grows with time. The coefficient on $x_1 x_2$ will be positive if late-semester missing work is especially predictive, regardless of what the individual coefficients on x_1 and x_2 say.

> **Math Lens.** If the feature map is $\phi(x) = [x_1, x_2, x_1^2, x_2^2, x_1 x_2]^\top$, then the model $w^\top \phi(x) + b$ is nonlinear in the raw input x but still linear in the expanded coordinates $\phi(x)$. Training with gradient descent or ordinary least squares still applies without modification. The representational power comes from the engineer's choice of ϕ, not from a more complex model family.

Why does this matter? Many students conclude too early that linear models are only useful for obviously linear worlds. Basis expansion shows that a thoughtful feature map can extend a linear model's expressiveness considerably. The cost is interpretability: once features interact, individual coefficients become harder to read as standalone effects. The judgment call is whether the gain in fit justifies the loss in narrative clarity.

> **Practical Note.** Add interaction features deliberately rather than exhaustively. Generating all pairwise products of d features produces $d(d-1)/2$ new coordinates. On a dataset with hundreds of features, that explosion can create severe overfitting and slow training. Add interactions when domain knowledge suggests a specific joint effect, and test whether each addition improves held-out performance.

4.2.3 Common Feature Types

Numeric features are the simplest case. If we predict house prices, a feature like floor area may enter directly as a number. But even then, choices remain. Should we use total area or the logarithm of area? Should we center and scale it? Should we include an interaction between area and neighborhood?

Categorical features require more care. A feature like neighborhood, major, product category, or browser type cannot be inserted directly as a single integer without creating an artificial ordering. A common solution is one-hot encoding, in which each category receives its own indicator variable. In models with an explicit intercept, practitioners usually drop one indicator as the reference category or use regularization carefully so the representation is not perfectly collinear.

Text features are often sparse. A bag-of-words representation may include one feature per word, with each feature recording whether the word appears or how often it appears. This is one reason linear models remain strong in text classification: even though the representation is high-dimensional, the modeling assumption is still a weighted combination of informative signals.

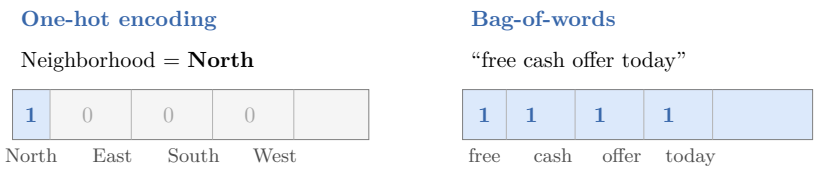


Figure 4.1: Representation choices make assumptions visible. One-hot vectors avoid fake category ordering, while bag-of-words vectors make text usable through sparse counts.

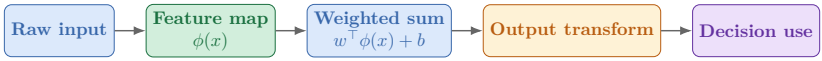


Figure 4.2: Linear-model pipeline.

Example 4.3 (Feature Design for Housing)**.** Suppose we want to predict apartment prices. Raw inputs might include address, floor area, year built, floor number, and text descriptions. A reasonable first feature set could contain area, number of bedrooms, age of the building, distance to the city center, and one-hot indicators for neighborhood. This feature set is already making several claims: location matters, age matters, and neighborhood effects should be modeled separately rather than forced into a single numeric code.

Example 4.4 (Feature Design Across Public Tasks)**.** The same logic appears across public tasks. A sports-shot prediction problem suggests structured tabular features such as player position, shot distance, and remaining time. A news topic classification problem suggests sparse lexical features or TF–IDF vectors. The tasks are different, but the chapter's question is the same: what representation would let a weighted sum detect the relevant signal?

4.3 The Generic Linear Pipeline

Before splitting into regression and classification, it helps to establish the reusable pipeline that both share.

Listing 4.1: A generic feature pipeline for a linear model

```
raw_example = collect_input()
```

```
features = feature_map(raw_example)
score = dot(weights, features) + bias
prediction = output_transform(score)
```

This small program is worth remembering. Linear regression and logistic regression are two specific instances of this general pattern. The main difference is what the output transform does. The score-producing core stays the same, which is why the chapter can teach both models as one reusable pipeline rather than as two unrelated formulas.

Problem-Solving Technique. Structural rescue test. If a linear baseline fails, ask two questions in order:

- Could one additional transformed feature make the missing pattern visible, such as area-squared or attendance times homework score?
- If not, what kind of structure is missing entirely: local neighborhoods, branching rules, sequence order, or a learned representation?

Do not jump too quickly from "the line failed" to "the whole model family failed." First test whether the representation, not the weighted sum, is the real bottleneck.

Decision Box. Which linear family first?

- Numeric target such as price, time, or demand: start with linear regression.
- Binary target or event probability such as spam versus not spam: start with logistic regression.
- Need multiclass probabilities: the same score-then-transform logic extends, but the chapter's conceptual split remains numeric output versus probability output.
- Need an action rather than only a score: keep the fitted score, the probability, and the thresholding

rule conceptually separate.

4.4 Linear Regression as the First Concrete Case

Definition 4.2 (Linear Regression)**.** Linear regression predicts a numeric target as an affine function of the features:

$$\hat{y} = w^\top x + b,$$

where x is the feature vector, w is the vector of coefficients, and b is the intercept.

Here again, x means the representation after encoding. If the notation begins to feel too compressed, mentally expand the same model back to $\hat{y} = w^\top \phi(x_{\text{raw}}) + b$. That is the cleaner way to remember that linear modeling and feature design are one pipeline, not two disconnected steps.

The word *linear* here refers to linearity in the parameters and the feature representation. This is important. A linear model can still express nonlinear behavior in the original raw input if the features themselves encode nonlinear transformations.

4.4.1 Residuals and Squared Error

For a training example (x_i, y_i), the prediction error is often written as a residual:

$$r_i = y_i - \hat{y}_i.$$

Linear regression is commonly fit by minimizing the mean squared residual:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2.$$

Chapter 2 already explained why squared error is a useful training loss. Here the important skill is more concrete: move comfortably from feature values to prediction to residual, and understand what the residual says about the baseline. A positive residual means the model predicted too low; a negative residual means it predicted too high.

4.4.2 Minimizing the Loss: Gradient Descent

The standard procedure for finding weights that minimize the MSE is gradient descent. Starting from some initial weight vector w, we repeatedly move the weights in the direction that decreases the loss the fastest. The gradient $\nabla_w \mathcal{L}$ points uphill, so we subtract a small multiple of it:

$$w \leftarrow w - \eta \, \nabla_w \mathcal{L}(w),$$

where $\eta > 0$ is the learning rate, which controls the step size. For MSE, the gradient with respect to weight w_j is

$$\frac{\partial \, \mathrm{MSE}}{\partial w_j} = \frac{2}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) \, x_{ij}.$$

Each component of the gradient tells one weight which direction to move and by how much. One application of this update is one gradient descent step.

Listing 4.2: From scratch: one gradient descent step for linear regression

```
# Student support task: features are (attendance_rate,
    late_submission_rate),
# target is risk_score (higher means more at-risk)
data = [
    ([0.9, 0.1], 0.2), # high attendance, few lates -> low
         risk
    ([0.7, 0.3], 0.4),
    ([0.5, 0.5], 0.6),
    ([0.3, 0.7], 0.7),
    ([0.2, 0.8], 0.9), # low attendance, many lates ->
        high risk
]

learning_rate = 0.1
weights = [0.0, 0.0] # one weight per feature; no explicit
     bias here
n = len(data)

def dot(a, b):
    return sum(wi * xi for wi, xi in zip(a, b))
```

```
# --- compute MSE loss before the step ---
loss_before = sum((dot(weights, x) - y) ** 2 for x, y in
    data) / n

# --- compute gradient ---
gradient = [0.0] * len(weights)
for x, y in data:
    error = dot(weights, x) - y # (pred - true)
    for j in range(len(weights)):
        gradient[j] += (2 / n) * error * x[j]

# --- update weights ---
weights = [weights[j] - learning_rate * gradient[j]
          for j in range(len(weights))]

# --- compute MSE loss after the step ---
loss_after = sum((dot(weights, x) - y) ** 2 for x, y in
    data) / n

print(round(loss_before, 4)) # loss before the step
print(round(loss_after, 4)) # loss after one correction
```

Running this program produces a smaller loss after a single step. The code makes two things visible at once: the loss goes down because the weights moved in the direction that locally reduced the squared errors, and each component of the gradient tells its corresponding weight how far to move. In this tiny no-bias setup, both weights increase on the first step because the model starts by underpredicting every positive target from an all-zero score. That first-step sign should not be overinterpreted as the final semantic effect of each feature; later updates, or a model with an explicit bias term, separate those effects more clearly.

4.4.3 Worked Example: Predicting Apartment Prices

Imagine a small apartment dataset with features for floor area, number of bedrooms, and age of the building.

A fitted model might look like

$$\hat{y} = 48 + 3.1 \cdot \text{area} + 11 \cdot \text{bedrooms} - 1.4 \cdot \text{age},$$

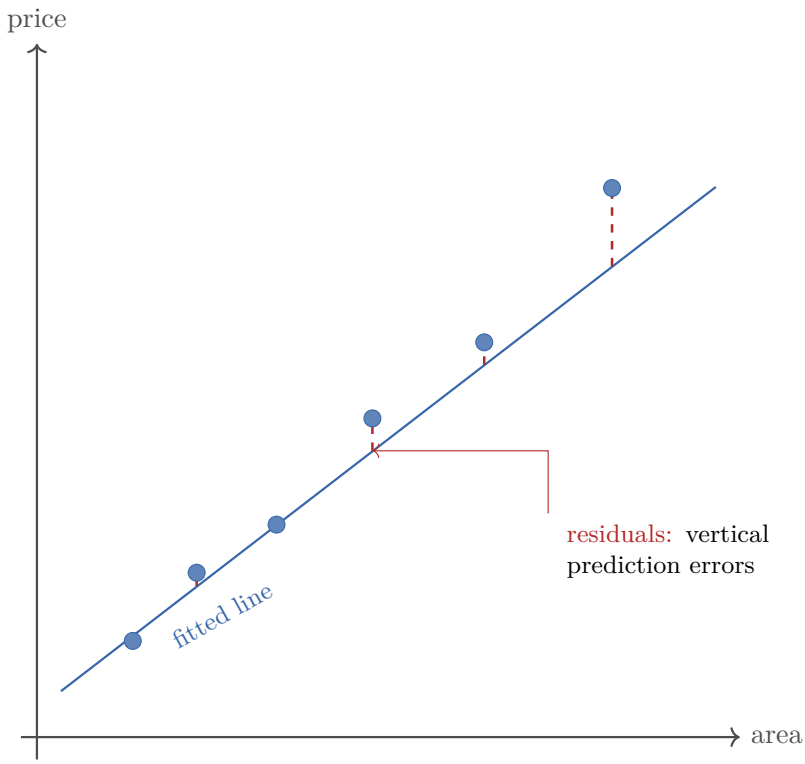


Figure 4.3: Linear regression predicts with a line in feature space. Residuals measure how far the observed targets fall above or below the fitted prediction.

where the target is measured in thousands of currency units. This model says that, all else equal, each additional square meter adds about 3.1 thousand to the predicted price, each extra bedroom adds 11 thousand, and each additional year of age subtracts about 1.4 thousand.

Now take an apartment with 82 square meters, 3 bedrooms, and age 6:

$$\hat{y} = 48 + 3.1(82) + 11(3) - 1.4(6) = 326.8.$$

If the actual sale price turned out to be 338, the residual would be

$$r = 338 - 326.8 = 11.2.$$

Apt	Area (m^2)	Beds	Age (yr)	Price (k)
A	50	1	25	175
B	60	2	18	220
C	70	2	12	260
D	80	3	8	310
E	90	3	5	350
F	105	4	3	405

Table 4.2: A tiny housing table is enough to make the regression story concrete.

That is the first concrete skill the chapter wants to automate in the reader's head: move from encoded features to a predicted numeric value, then inspect the residual rather than admiring the formula.

> **Warning.** An interpretable model is not the same thing as a correct interpretation. Coefficients are only interpretable relative to the features included, their scales, and the correlations among them.

4.4.4 Residual Diagnostics

Residuals are not just bookkeeping after the fit. If residuals curve with the fitted values, the representation is probably missing nonlinear structure. If their spread grows with the prediction, the error variance may not be constant. If one slice produces extreme residuals, that slice may need a separate feature, a missingness indicator, or a different interaction term. In a linear baseline, residual plots are often the fastest way to learn what structure the representation left out.

4.4.5 Linear in the Parameters, Not Necessarily in the World

Students often hear "linear model" and imagine "straight line in reality." That is too narrow. The model is linear in the *parame-*

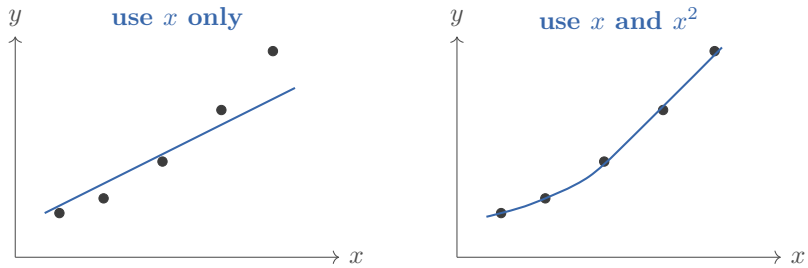


Figure 4.4: Both panels can come from linear models. The right panel is still linear in the parameters because the nonlinear behavior is carried by the feature map, not by a nonlinear weighting rule.

ters and representation, not necessarily in the raw input.

Suppose Model A uses only one feature, area. Model B uses both area and area-squared. Model B can represent curvature in the relationship between area and price even though it is still linear in the parameters. This is one of the highest-value lessons in the chapter: the phrase "linear model" names the weighting rule, not the shape of the world.

4.5 Logistic Regression as Score-to-Probability Modeling

Linear regression predicts unrestricted numeric outputs, which makes it unsuitable for binary classification. If the target is whether an email is spam or whether a student needs support, we usually want a probability or a class label, not an arbitrary real number such as 1.7 or -0.8.

Logistic regression keeps the same weighted-sum backbone and changes only the output transform. The model first computes a score

$$z = w^\top x + b,$$

and then maps that score to a probability through the sigmoid

function:

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$
$$P(y = 1 \mid x) = \sigma(w^\top x + b).$$

In predictive language, this is the model's estimated conditional probability, not automatically a calibrated real-world frequency. This is why the generic pipeline matters. Linear regression uses the identity transform. Logistic regression uses the sigmoid transform. But the change is not only cosmetic. Logistic regression is trained against a classification objective such as log loss, and its output is meant to support probability estimation and threshold policy rather than unrestricted numeric prediction. The reader should keep three layers separate all the way through: score, probability, and decision rule.

Logistic regression is linear in the score, the log-odds, and the decision boundary, not in the final probability output itself.

4.5.1 From Score to Probability

Why use the sigmoid? It maps any real-valued score into the interval $(0, 1)$, so the output can be interpreted as a probability, and it preserves ordering: larger scores correspond to larger predicted probabilities.

Another useful viewpoint is through the log-odds:

$$\log \frac{P(y = 1 \mid x)}{1 - P(y = 1 \mid x)} = w^\top x + b.$$

This follows directly from the sigmoid. If $p = \sigma(z) = \frac{1}{1+e^{-z}}$, then $1 - p = \frac{e^{-z}}{1+e^{-z}}$, so

$$\frac{p}{1-p} = e^z \quad \Longrightarrow \quad \log \frac{p}{1-p} = z = w^\top x + b.$$

Students do not need to become obsessed with this form, but it clarifies why logistic regression is mathematically clean: the classification problem is still built from a linear scoring rule.

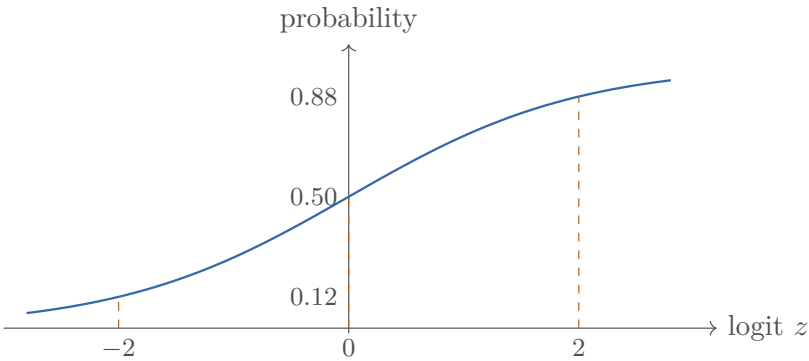


Figure 4.5: Logistic regression turns a linear score into a probability. The key distinction is score → probability → thresholded decision. Those are three different objects.

> **Warning.** Probability and odds are not the same thing. A probability of 0.8 means "80% chance." Odds of 4:1 mean four positive cases for every one negative case. Logistic regression becomes easier to interpret once that distinction is kept explicit.
>
> **Practical Note.** A one-unit increase in feature x_j changes the log-odds by w_j and multiplies the odds by e^{w_j}, holding the encoded features fixed. That is often the cleanest coefficient interpretation in logistic regression.

Theorem 4.1 (A 0.5 logistic threshold is a score threshold)**.** *If logistic regression predicts the positive class whenever $P(y = 1 \mid x) \geq 0.5$, then this rule is exactly the same as predicting the positive class whenever*

$$w^\top x + b \geq 0.$$

The boundary between the two decisions is therefore the set of inputs for which

$$w^\top x + b = 0.$$

Proof. Let $z = w^\top x + b$. The sigmoid sends $z = 0$ to $\sigma(0) = 0.5$,

and larger scores produce larger probabilities. Therefore

$$P(y = 1 \mid x) \geq 0.5 \quad \Longleftrightarrow \quad \sigma(z) \geq 0.5 \quad \Longleftrightarrow \quad z \geq 0.$$

Substituting back gives the result. □

> **Math Lens.** For any probability threshold $\tau \in (0, 1)$, the equivalent score cutoff is
>
> $$w^\top x + b \geq \log \frac{\tau}{1 - \tau}.$$
>
> The theorem's 0.5 case is the special case where this cutoff is zero.

4.5.2 Worked Example: A Spam Probability

Suppose a mail system wants to estimate the probability that a message is spam. Features might include suspicious-link signal, urgent-language signal, and sender-risk score. A logistic model might learn that stronger values on these features push the score upward.

To see the mechanics directly, suppose the feature vector for one message is

$$x = \begin{bmatrix} 0.6 \\ 0.8 \\ 0.4 \end{bmatrix},$$

where the three components represent scaled signals for suspicious links, urgent sales language, and sender-risk score. Let the fitted logistic-regression parameters be

$$w = \begin{bmatrix} 1.4 \\ 1.8 \\ 1.1 \end{bmatrix}, \qquad b = -2.0.$$

The linear score is therefore

$$z = w^\top x + b = 1.4(0.6) + 1.8(0.8) + 1.1(0.4) - 2.0 = 0.72.$$

Applying the sigmoid gives

$$P(y = 1 \mid x) = \sigma(0.72) \approx 0.673.$$

Feature	Value	Coefficient	Contribution to z
Suspicious-link signal	0.6	1.4	0.84
Urgent-language signal	0.8	1.8	1.44
Sender-risk score	0.4	1.1	0.44
Bias	—	—	−2.00
Total logit	—	—	0.72

Table 4.3: A by-hand logistic-regression score decomposition.

Listing 4.3: Algorithm sketch: logistic score and probability

```
Input: feature vector x, weight vector w, bias b
score = dot(w, x) + b
probability = 1 / (1 + exp(-score))
if probability >= 0.5:
    label = 1
else:
    label = 0
return score, probability, label
```

This tiny program mirrors the worked example exactly. It is worth typing once. The purpose is not software engineering. The purpose is to make the pipeline concrete enough that a student can move from feature values to score to probability to label without hiding behind a library call.

Here "all else equal" means holding the encoded features fixed inside the fitted model, not holding the whole world fixed.

At a spam threshold of 0.5, this message would be flagged. At a threshold of 0.7, the same message would not. Chapter 2 already explained threshold policy in detail. Here the example serves one narrower purpose: keep the boundary clear between the model's score and probability, and the larger system's decision about how to act.

Example 4.5 (Why Marginal Effects Help)**.** CDC teaching material on marginal effects emphasizes a point that matters here:

Logit z	Probability $\sigma(z)$	If threshold is 0.5
-2.0	0.12	negative
-1.0	0.27	negative
0.0	0.50	boundary
1.0	0.73	positive
2.0	0.88	positive

Table 4.4: A small lookup table helps connect raw scores to probabilities before students worry about coefficient interpretation.

changes in predicted probability are often easier to communicate than raw changes in log-odds [13]. Logistic coefficients are mathematically elegant, but many users understand "the predicted probability rises by about five points" more naturally than "the log-odds increase by 0.2."

For a single feature x_j, the corresponding marginal effect comes from the chain rule:

$$\frac{dP(y = 1 \mid x)}{dx_j} = \frac{d\sigma(z)}{dz}\frac{dz}{dx_j} = \sigma(z)\big(1 - \sigma(z)\big)w_j = p(1 - p)w_j.$$

So the same coefficient can have a larger or smaller probability impact depending on where the example sits on the sigmoid curve.

4.6 Reading Coefficients Without Fooling Yourself

One of the selling points of linear models is interpretability. That selling point is real, but it needs language discipline. Coefficients are not self-explaining truths. They are statements about the fitted relationship *within a particular representation.* The next three subsections give the minimum guardrails: scale, correlation, and omitted variables all change what a coefficient can honestly be said to mean.

4.6.1 Scale Matters

If one feature is measured in square meters and another in kilometers, their coefficients will naturally be on different scales. A larger coefficient does not automatically mean a more important feature. Standardizing features can help when we want to compare relative effect sizes more meaningfully, especially in regularized models.

Even standardized coefficients are not robust feature-importance measures when features are strongly collinear or when regularization is active.

This point is easy to underestimate. A coefficient of -1.4 per year of building age and a coefficient of -0.117 per month describe the same practical effect under different units. Without understanding the units, the raw numbers invite the wrong comparison.

4.6.2 Correlation Matters

If two features are strongly correlated, such as floor area and number of bedrooms, the model may distribute their effects somewhat arbitrarily. Both features may matter, but the exact coefficient values can become unstable. This does not always ruin prediction, but it weakens naive coefficient stories.

In the extreme case of near-perfect collinearity, coefficients can become non-unique or wildly unstable even when predictions barely change. Ridge-style regularization can stabilize the allocation, but it does not restore a unique interpretation by itself.

In practice, this means two models trained on slightly different samples can make similar predictions while assigning noticeably different weights to correlated inputs. Prediction can remain stable while interpretation becomes fragile.

Resampling reveals instability. One diagnostic for coefficient fragility is to train the same model on bootstrap resamples of the training set and inspect how much each coefficient varies across those resamples. A coefficient that regularly switches sign,

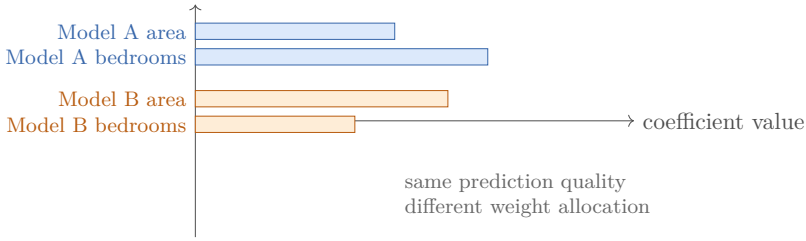


Figure 4.6: Correlated features can make coefficient stories unstable. Two training runs may produce similar predictions while shifting weight between overlapping signals.

or that varies by more than its mean value, should not be interpreted as evidence of a stable directional effect. It may be absorbing the variance of a correlated neighbor. A stable, interpretable coefficient stays in the same direction and roughly the same magnitude across resamples.

Adding or removing a feature changes the story. Suppose a wage model is trained first with both years of experience and job tenure, then with only job tenure removed. If the coefficient on years of experience changes substantially when tenure is dropped, that is evidence of collinearity. The original coefficient was not measuring the "pure effect" of experience; it was absorbing part of the tenure effect as well. This is why coefficient narratives built on a single model fit can be misleading even when the model predicts well.

> **Warning.** Before reporting a coefficient as evidence of a meaningful relationship, check: does the coefficient stay stable when correlated predictors are added or removed? Does it stay stable across resamples of the data? If the answer to either question is no, the narrative is fragile. The model may still predict well, but the specific coefficient value should not carry the story.

4.6.3 Missing Variables Matter

Suppose a wage model includes education but omits years of experience or industry. The education coefficient may absorb some of those missing effects. The model may still have predictive value, but the interpretation of that coefficient becomes less trustworthy.

Example 4.6 (A Misleading Coefficient Story)**.** Imagine a customer-churn model in which the number of refund requests has a positive coefficient. A naive interpretation would be that requesting refunds causes customers to leave. A more plausible explanation is that dissatisfied customers are precisely the ones who request refunds. Predictive association is not causal explanation.

Warning. What you are usually allowed to say:

- "This feature is associated with higher predicted risk in this model."
- "Holding the encoded features fixed, increasing this variable raises the score."

What you are usually not allowed to say from the model alone:

- "This feature causes the outcome."
- "Changing this feature in the world will definitely change the target by this amount."

Practical Note. These assumptions matter for inference, not for prediction alone. Linearity in the encoded features, exogeneity or zero conditional mean of the errors, independent or appropriately modeled errors, roughly constant variance, and no perfect multicollinearity are the classical conditions behind standard errors and hypothesis tests. A linear baseline can still be useful even when some of those assumptions are only approximate.

4.7 Prediction Is Not Intervention

Linear models tempt people into causal language because the coefficients look clean. This is the point where linear transparency becomes dangerous if the reader stops being precise. A predictive model answers an associational question: given the observed features, what outcome should we expect? A causal question is different: what would happen if we actively changed one part of the world? Pearl's causal framework makes this distinction central by separating ordinary observation from intervention and counterfactual reasoning [48].

This distinction matters even in simple tabular problems. A model may learn that students who attend more office hours tend to earn higher grades. That makes office-hour attendance a potentially useful predictive feature. It does *not* by itself prove that forcing attendance to increase would raise grades by the same amount. Students who seek help may also be more motivated, have different schedules, or receive more support elsewhere. Those hidden differences can create confounding.

4.7.1 Confounding Changes the Story

A *confounder* is a variable that influences both the feature of interest and the outcome. When confounders are omitted or measured poorly, the fitted coefficient may mix together several stories: direct effect, indirect effect, selection effect, and historical artifact. The model can still predict well, but its coefficients become unsafe to read as intervention advice.

Example 4.7 (Tutoring Is Predictive, but the Policy Claim Is Harder)**.** Suppose a model finds that students who sign up for tutoring are less likely to fail. That may mean tutoring helps. It may also mean that students who sign up early are the same students who are more organized, more supported, or more engaged in the course. A tutoring coefficient may therefore be useful for prediction while remaining insufficient for the policy claim "increasing tutoring sign-ups will reduce failure by this amount."

4.7.2 What Stronger Causal Language Requires

To move from predictive association to a causal claim, the evidence standard changes. Stronger causal language usually requires a design that supports intervention reasoning: a randomized experiment, a natural experiment with defensible assumptions, or a domain model that makes the required causal assumptions explicit and testable. A fitted linear model on observational data is rarely enough on its own.

The practical rule for this book is conservative. If the task is prediction, say "associated with," "useful for prediction," or "raises the model score when other encoded features are held fixed." Reserve words such as "causes," "effect," or "would improve outcomes if changed" for settings where the causal design has actually been defended.

4.8 Controlled Simplicity: Regularization

Even linear models can overfit, especially when the number of features is large or when features are noisy. Regularization addresses this by penalizing overly large or overly complex coefficient vectors. It is the chapter's answer to a practical baseline question: how do we keep the first serious model controlled without making it uselessly weak?

Definition 4.3 (L2 Regularization)**.** L2 regularization adds a penalty proportional to the squared magnitude of the coefficients:

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, \hat{y}_i) + \lambda \|w\|_2^2.$$

This penalty discourages extreme weights and tends to spread influence more smoothly across correlated features. In regression, the resulting model is often called *ridge regression*. In classification, the same idea is commonly used with logistic regression.

Definition 4.4 (L1 Regularization)**.** L1 regularization adds a

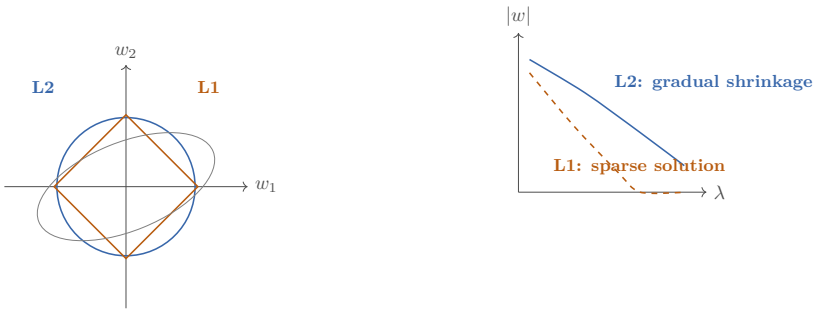


Figure 4.7: L2 regularization prefers smaller coefficients smoothly. L1 regularization can drive some coefficients all the way to zero. Validation, not taste, should decide how much regularization is appropriate.

penalty proportional to the sum of absolute coefficient values:

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, \hat{y}_i) + \lambda \|w\|_1.$$

L1 regularization often drives some coefficients exactly to zero. This can be useful when we suspect that only a subset of features truly matters, or when we want a sparse model that is easier to inspect.

In most implementations, the intercept b is left unpenalized; the regularizer is meant to control feature weights, not to force the global baseline toward zero.

Conceptually, both penalties express a preference for restrained explanations. They do not ban complexity entirely. They require the data to justify it. That is why regularization belongs to baseline design rather than to after-the-fact cleanup.

The geometry gives a useful picture: the L2 penalty has round contour lines, so optimization usually prefers to shrink many weights smoothly rather than eliminate them. The L1 penalty has diamond-shaped contours with corners on the axes, so solutions are more likely to land exactly on an axis, which is why zeros appear naturally.

Practical Note. Regularization strength λ is not a matter of aesthetic preference. It is a model-selection choice and should be chosen by validation performance. This is also why standardization matters more once regularization is active: if one feature is measured on a much larger scale than another, the penalty no longer treats them comparably.

Listing 4.4: From scratch: L2 versus L1 gradient penalty

```
# Both penalties are added to the data-loss gradient
    before each weight update.
# lam is the regularization strength; weights is the
    current weight vector.

# L2 penalty gradient: proportional to the weight itself (
    smooth shrinkage)
l2_penalty_gradient = [2 * lam * w for w in weights]

# L1 penalty gradient: proportional to the sign of the
    weight (sparse shrinkage)
def sign(v):
    return 1 if v > 0 else (-1 if v < 0 else 0)

l1_penalty_gradient = [lam * sign(w) for w in weights]

# Combined update step (choose one penalty):
# weights[j] -= learning_rate * (data_gradient[j] +
    l2_penalty_gradient[j])
# weights[j] -= learning_rate * (data_gradient[j] +
    l1_penalty_gradient[j])
```

The L2 penalty nudges every weight toward zero by an amount proportional to the weight's current value, so large weights shrink faster than small ones but none ever reaches exactly zero. The L1 penalty moves every non-zero weight by the same fixed step regardless of magnitude, which is why it can drive weights all the way to zero and produce sparse models.

4.9 Why Linear Models Are the Right First Baseline

A strong baseline should be cheap to run, understandable to debug, and hard to beat for the right reasons. Linear models satisfy all three criteria. A good baseline is supposed to be beatable eventually, but not trivially. If a richer model wins, it should win by capturing a named piece of structure that the linear baseline could not.

> **Problem-Solving Technique. Baseline ladder.** Start with the simplest serious model that matches the task. Climb only when you can name the missing structure.

On structured tabular data, much of the useful signal may already be present in reasonably chosen features. A linear model can exploit that signal immediately without needing enormous datasets. On sparse text classification, the additive effect of informative words is often enough to perform well. In such cases, if a larger model wins, it should win by a clear and justified margin.

Linear baselines are also diagnostic. If a linear model already performs well, the problem may not require extreme expressiveness. If it performs badly, the failure teaches us something. Perhaps nonlinear interactions matter. Perhaps spatial or sequential structure matters. Perhaps the feature representation is too weak. In each case, the linear model gives us a clean reference point for deciding what kind of additional structure is needed.

Example 4.8 (Baseline Logic on Public Tasks)**.** For a tabular task such as sports-shot prediction, a standardized linear or logistic baseline is often the right first move. For a text task such as news topic classification, bag-of-words or TF–IDF plus logistic regression is still a meaningful reference point. The baseline is useful even when it loses, because the manner of the failure tells us what structure may be missing.

4.10 Sparse Linear Models Stay Strong in Text and Tabular Work

A persistent misconception is that linear models are only a warm-up exercise before the real work with deep models begins. In several important problem families, sparse linear models remain highly competitive not as compromises but as genuine strong baselines.

Text classification. A bag-of-words or TF–IDF representation with logistic regression can achieve classification accuracy on tasks like news topic classification, spam filtering, or sentiment classification that deep models beat by only a few percentage points, at a fraction of the computational cost. The reason is straightforward: in text classification, the vocabulary already exposes the relevant signal. A word like "inheritance" is informative about legal topics. A word like "free" is informative about spam. A linear model on a well-engineered bag-of-words can identify those associations efficiently.

High-dimensional tabular prediction. In fraud detection, credit scoring, and clinical risk modeling, datasets often have hundreds or thousands of engineered features over millions of examples. Many of those features are individually informative, and their combination is largely additive. A regularized linear model can handle this well. The L1 penalty selects a sparse subset of informative features, making the model both competitive and interpretable. Tree ensembles often win eventually, but rarely by a margin that justifies a tenfold increase in computational cost for the initial prototyping.

Example 4.9 (Linear Baseline on a Text Task)**.** For a news topic classifier over four categories, a logistic regression with TF–IDF features might achieve 93% accuracy. A fine-tuned transformer might reach 96%. That 3-point gap may matter for a production system, but it does not mean the linear baseline was uninformative. It established that the task is solvable, identified which categories confuse the model, and provided a reference point against which the expensive model must justify its cost.

> **Problem-Solving Technique. Baseline ladder in text and tabular work.** Before training any deep model, ask: does a regularized logistic or linear regression on engineered features already work well? If it does, the deep model must provide either a meaningful accuracy gain on the evaluation set or a well-justified argument for why it will perform better in production. Neither argument is hard to make, but both should be made explicitly.

The broader point is that linear models expose the relationship between representation and prediction transparently. When they succeed, they tell you something about the structure of the problem: it was largely additive in the feature space you chose. When they fail by a large margin, they tell you something even more useful: the problem contains interactions, local geometry, or sequential structure that the feature set could not capture. Either result is informative. The deep model that follows should be chosen in response to that specific structural diagnosis.

4.11 Failure Diagnosis, Not Method Shopping

No model family is universal. Linear models struggle when the decision boundary or regression surface cannot be captured well by a weighted sum of the available features. Images, audio, and raw language sequences often contain local, hierarchical, or contextual structure that a simple linear score cannot exploit directly unless those structures have already been encoded by other means.

They also struggle when interactions are essential but omitted. If the effect of one feature depends strongly on another, and that interaction is not encoded explicitly, a linear model may be systematically wrong even when each feature is individually informative.

This is the right way to think about moving beyond linear models. Do not abandon them because a richer family is fashionable.

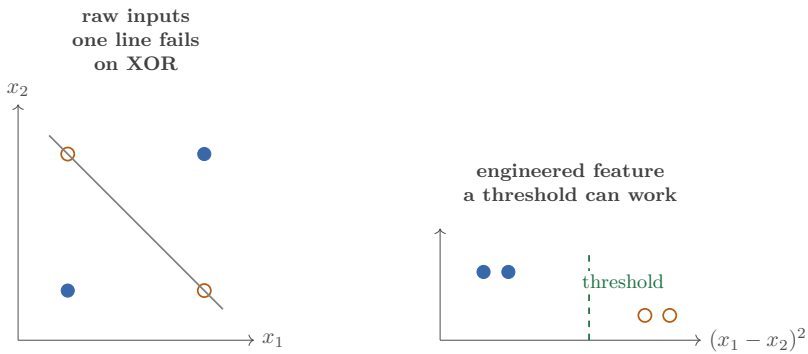


Figure 4.8: Failure should be diagnosed structurally. Sometimes one weighted sum on raw inputs is not enough, but a better feature map can rescue the linear model. Other times a different model family is needed.

Abandon them only when the problem or the evidence shows that the needed structure cannot be captured adequately in the current representation.

The diagnostic questions are often more useful than the answer:

- If interactions are missing, perhaps trees or engineered interaction features matter.
- If local neighborhoods matter, perhaps nearest-neighbor ideas matter.
- If unlabeled structure matters, clustering or dimensionality reduction may matter.
- If raw images or sequences matter, learned representations may matter later.

That is why the next chapter is the natural move. It does not replace linear thinking. It asks a sharper structural question: is the missing piece mainly conditional thresholds and interactions, local similarity, or unlabeled organization? The weighted-sum baseline is what makes that diagnosis possible.

Design field	What must be stated clearly
Seven-question focus	Questions 4, 5, and 6: choose the representation, justify the first serious baseline, and state what evidence would show that linearity is no longer enough
Task and target type	Regression or classification, and what the model is being asked to predict
Raw input	What the original data looks like before encoding
Feature map	Which engineered or encoded features the weighted sum will actually see
Baseline model	Linear regression, logistic regression, or another linear baseline appropriate to the target
Regularization choice to validate	Which penalty strengths or types will be compared using validation data
Interpretation boundary	What coefficients would and would not justify saying about the world
Failure pattern to watch for	What kind of error would indicate that one weighted sum is missing important structure
Next structural hypothesis	What broader model family would be justified if the baseline fails for the named reason

Table 4.5: If this sheet cannot be written, the first baseline is usually underdesigned rather than underpowered.

4.12 Chapter Artifact: A First Baseline Design Sheet

By the end of this chapter, the reader should be able to write down the first defensible baseline before trying more expressive methods. The point is not to freeze creativity. The point is to make the first modeling move explicit enough that later changes become justified rather than impulsive. The framing memo named the decision context, the metric worksheet named good behavior, and the evaluation plan protected the claim. This sheet names the first serious model that deserves to make that claim. It is where representation audit and baseline ladder turn into one written modeling commitment.

Problem-Solving Technique. Baseline design

sheet. Before training the first serious model, write down the representation, baseline family, regularization choices, interpretation boundary, and what kind of failure would justify moving beyond linearity.

Example 4.10 (Filled Baseline Design Sheet)**.** For early student-support prediction, the raw input might be course activity logs, attendance, and the first assignment timeline. The feature map could expose missed-deadline count, recent platform inactivity, quiz completion rate, and whether the student has already triggered a manual outreach flag. The first baseline would then be logistic regression with validation over a small range of regularization strengths. The interpretation boundary would say that the coefficients are being used to understand the baseline's evidence pattern, not to make causal claims about why a student disengages. The named failure pattern might be weak performance on students whose problems appear only through timing or sequence order, which would justify moving next to a sequential model rather than jumping blindly to a more complex classifier.

4.13 Common Mistakes with Linear Models

Mistake Checklist.

- **Treating feature engineering as mechanical.** For linear models, representation is part of the model.
- **Overinterpreting coefficients.** Coefficients describe the fitted model, not the world's causal structure.
- **Encoding categories as ordered integers.** Use one-hot encoding for nominal categories unless order is real.
- **Ignoring missingness as information.** Missing values may be signal, so impute on the training split and add a missingness indicator when needed.

- **Ignoring standardization.** Scale affects optimization and coefficient comparison, especially with regularization.
- **Skipping the linear baseline.** A simple linear baseline is a useful diagnostic, even when you expect a richer model.
- **Forgetting the evaluation boundary.** Preprocessing must respect train, validation, and test splits.
- **Assuming linearity means simplicity of the world.** A linear model can fit a complex problem only if the representation makes it approximately linear.

4.14 Looking Ahead

This chapter showed that representation is already part of the model. The next chapter studies what kind of structure is missing when one global weighted sum is not enough: conditional thresholds and interactions, local similarity, or unlabeled organization.

Chapter Summary

- This chapter begins with a representation question: can a good encoding plus one weighted sum already solve the task?
- The model does not see raw reality. It sees an encoded representation, so feature choices are already scientific claims about what signal matters.
- Linear regression and logistic regression share one pipeline: encode the input, compute one weighted sum, then choose the output transform that matches the target.
- Coefficients become useful only under discipline about scale, correlation, omitted variables, and the boundary between predictive language and causal claims.

- Regularization keeps the first baseline controlled: L2 shrinks weights smoothly, while L1 can also make the model sparse.
- Linear models are strong first baselines because they are cheap, legible, and diagnostically useful.
- Failure should be interpreted structurally: first ask whether a better feature map rescues the weighted sum, and only then ask what richer family is justified.
- The main deliverable of the chapter is a first baseline design sheet written before method shopping begins.

Study Questions

1. Why is "can one weighted sum already solve the task?" a better opening question than "which regression formula comes first?"
2. In what sense is representation already part of the model?
3. Why can a model be linear even when the relationship between the raw input and target is not?
4. Why can correlated features make coefficient interpretation unstable even when prediction stays good?
5. Why is a predictive association not the same as evidence for an intervention?
6. What kind of failure would justify leaving a linear baseline for a different model family?

Exercises

1. One-hot encoding drill: encode the categorical variable *browser* with possible values {Chrome, Firefox, Safari} for an example in which the browser is Safari.
2. Sparse bag-of-words drill: suppose the vocabulary is {free, cash, meeting, project} and the email text is "free project

cash." Write the bag-of-words vector and compute its dot product with weights $[1.2, 0.8, -0.5, -0.3]$.

3. Representation audit: for a task of your choice, state the raw input, the encoded representation, one thing the representation loses, and one assumption it inserts.

4. Logit-to-probability drill: convert logits -1.5, 0, and 1.5 into probabilities using the sigmoid function.

5. Score-to-boundary drill: explain why a logistic-regression threshold of 0.5 is equivalent to testing whether the linear score is nonnegative. Then, for $w = [2, -1]$ and $b = -0.5$, write the boundary equation.

6. Which feature map makes this relation more linear? A target grows slowly for small x and much faster for large x. Suggest one transformed feature set that could help a linear model fit that pattern better.

7. Same prediction, different coefficients: explain how two models could make similar predictions while assigning different weights to area and bedrooms in a housing model.

8. L1 or L2? Give one situation where sparse feature selection is especially useful and one situation where smooth shrinkage across correlated features is more appropriate.

9. A model finds that students who attend more office hours tend to earn higher grades. Write one predictive interpretation and one intervention claim that the model does *not* justify by itself.

10. Use a public tabular task such as sports-shot prediction and propose a first linear baseline with 4–6 features, a target type, and one failure pattern you would watch for.

11. Write language-neutral pseudocode that takes a feature vector, weights, and a bias, then returns the logistic score, probability, and thresholded label.

12. Use Table 4.5 to write a one-page baseline design sheet for either spam filtering, student support, or another application of your choice. If you are carrying one case through the book,

inherit the same task from the earlier framing, metric, and evaluation artifacts, then end by naming the next structural hypothesis you would test if the linear baseline fails.

Challenge Exercises

1. A logistic model has coefficients $w = [1.2, -0.8, 0.5]$, bias $b = -0.4$, and feature vector $x = [2, 1, 3]$. Compute the linear score, the predicted probability, and the final predicted label if the threshold is 0.5.

2. Propose a nonlinear real-world relationship that can still be modeled by a linear model after feature engineering. State the transformed features explicitly and explain why the model remains linear in the parameters.

3. Construct a small binary-classification task in which one weighted sum on the raw inputs fails, but one additional transformed feature rescues the linear model. State the transformed feature and explain why the rescue works.

4. Give an example in which a coefficient is highly interpretable for prediction but misleading for causal inference. Explain what additional assumptions or data would be needed before making a causal claim.

5. Take one failed linear baseline result from your own experience or invent one. Diagnose the failure structurally: what missing interaction, locality, sequence effect, or learned representation would justify the next modeling step?

Chapter 5

Structure Beyond Linearity

Learning Goals

- Explain how a failed linear baseline can reveal different kinds of missing structure rather than merely "insufficient complexity."
- Diagnose whether the next useful model family should come from recursive partitioning, local similarity, or unlabeled organization.
- Understand decision trees and ensembles as models for conditional logic and region-specific rules.
- Recognize when a margin-based classifier is the better answer than a tree or neighbor method.
- Understand nearest-neighbor methods as models whose behavior is determined by representation and distance.
- Distinguish clustering and dimensionality reduction as exploratory tools for unlabeled structure rather than as automatic discovery devices.
- Produce a structure diagnosis sheet before moving beyond the

linear baseline.

The previous chapter ended with a question rather than a conclusion. If a careful linear baseline fails, what kind of structure is missing? This chapter answers that question.

The answer is not always "use a more complicated model." The better question is structural. Sometimes the missing structure is conditional logic: different rules apply in different regions, and a single weighted sum is too rigid. Sometimes the missing structure is local geometry: nearby cases should inform one another, so distance is more meaningful than one global score. Sometimes the problem is unlabeled organization: there is no target yet, and the right question is how the data groups or compresses itself. The chapter's job is not to rank methods by prestige. It is to classify failure patterns well enough that the next move becomes principled.

This chapter is therefore the book's structure-diagnosis chapter. It should not be read as a bundle of classical methods. It should be read as one diagnostic with three answers.

The chapter's reusable deliverable is a structure diagnosis sheet. Keep that target in view: the point is to make the failure pattern, the structural hypothesis, and the next justified family explicit before changing models. By the end of the chapter, the reader should be able to say not just what method comes next, but what evidence made that choice defensible.

> **Problem-Solving Lens.** When a linear baseline fails, ask what kind of missing structure the failure suggests. Does the task need conditional thresholds and interactions, local similarity, or unlabeled organization? The methods in this chapter are direct answers to that question.
>
> **Practical Note.** A compact workflow is enough to run the diagnosis: state what the linear model is getting wrong, name the likely structural hypothesis, choose the next candidate family, and write down what evidence would justify moving forward.

5.1 Opening Question: What Kind of Structure Is Missing?

Return to that same early student-support setting. Suppose a university wants to identify students who may need extra support. One group of at-risk students may have very low attendance but acceptable quiz scores. Another may attend regularly but miss many deadlines. A single weighted sum may blur those patterns together. A branching rule can describe them more naturally:

- if attendance is very low, flag the student;
- otherwise, if late submissions are frequent, flag the student;
- otherwise, do not flag the student.

That rule is not just "nonlinear." It is region-specific. Different logic applies in different parts of the feature space. That is one possible answer to a failed baseline.

But it is not the only answer. Sometimes the right story is that similar examples should borrow judgment from one another. Sometimes it is that labels are absent and the job is exploratory organization rather than supervised prediction.

The opening question of the chapter should therefore be stated brutally clearly:

When the linear baseline fails, is the missing structure mainly conditional thresholds and interactions, local similarity, or unlabeled organization?

Everything that follows should be read as one answer to that question.

5.2 A Structural Map Before the Methods

Before naming model families, it helps to name the diagnostic.

> **Problem-Solving Technique. Structure diagnosis.** Before changing model families, state what structural mismatch the baseline has revealed.

Structural mismatch	Natural next family	What the family contributes
Conditional thresholds and interactions	Trees and tree ensembles	Different rules in different regions of feature space
Local similarity	Nearest neighbors and related geometry-based methods	Borrow prediction from nearby examples under a chosen similarity notion
Unlabeled organization	Clustering and dimensionality reduction	Summarize groupings, directions of variation, or lower-dimensional views without labels

Table 5.1: This chapter is one diagnostic with three answers. The point is not to memorize method names, but to connect model families to structural mismatch.

- If the task sounds like branching logic, think trees.
- If prediction should come from nearby similar cases, think neighbors.
- If labels are absent and you are looking for organization or compression, think clustering or dimensionality reduction.

Decision Box. Which family next?

- Different rules in different regions? Choose a tree.
- Nearby cases should borrow labels? Choose k-NN, but defend the distance metric and scaling first.
- No labels and you want grouping or compression? Choose clustering or PCA.
- The splits feel brittle or overfit quickly? Prefer an ensemble over a single tree.
- The distance metric feels arbitrary? Fix the representation before trusting neighbors.

Problem-Solving Technique. Shared cloud. Keep one tiny student-profile cloud in mind throughout the chapter:

- A: 50,000 clicks, 0 late submissions, safe
- B: 52,000 clicks, 1 late submission, safe
- C: 20,000 clicks, 4 late submissions, risk
- D: 22,000 clicks, 5 late submissions, risk

Trees read that cloud as threshold logic, k-NN reads it as distance, k-means reads it as grouping, and PCA reads it as a dominant direction of variation.

Problem-Solving Technique. Wrong question → better question. Wrong: "Which stronger model should we try next?" Better: "What specific structure did the baseline miss: branching logic, local similarity, or unlabeled organization?"

5.3 Method Selection by Missing Structure

Moving beyond the linear baseline is not a choice to make at random. It is a diagnostic response to a specific structural deficit. When a linear model falls short, it usually does so in one of three identifiable ways, and each way points toward a different family of methods.

Conditional thresholds and interactions. If the decision rule sounds like branching logic—"if this condition holds *and* that other condition holds, then the outcome changes"—then a linear model will fail because it cannot express regions. Trees are the natural response. They partition the input space recursively, and each leaf can express a different rule. This is the right structure for problems like loan approval, customer segmentation, or academic risk scoring, where the relevant factors interact nonlinearly across subgroups.

Structural deficit observed	Method family	What it contributes
Threshold interactions, region-specific rules	Trees and ensembles	Different rules in different parts of feature space
Local similarity should drive prediction	Nearest neighbors	Borrow labels from nearby examples under a chosen metric
No labels; seek grouping or compression	Clustering, PCA	Organize or compress without a target label

Table 5.2: Method selection should follow structural diagnosis. The question is not "which stronger model shall we try?" but "what specific structure did the baseline miss?"

Local similarity. If prediction should come from nearby examples—because similar students, similar products, or similar sensor readings tend to have similar outcomes—then the relevant structure is geometric rather than partitioned. Nearest-neighbor methods are the natural response. They delegate intelligence to the geometry of the feature space rather than building a global rule. This is the right structure when the decision boundary is irregular but local smoothness is plausible.

No labels: organization or compression. If labels are absent and the goal is to discover natural groups, reduce dimensions for visualization, or build a compact intermediate representation, then clustering or dimensionality reduction is the right response. These methods ask different questions than supervised methods do, and they should be chosen when the task is genuinely exploratory rather than predictive.

The wrong version of this reasoning is method shopping: trying several model families in sequence and reporting the one with the best number, with no explanation of why that family should work. The right version names the structural mismatch first and then selects a method specifically because it addresses that mismatch. That discipline makes model choice defensible and keeps

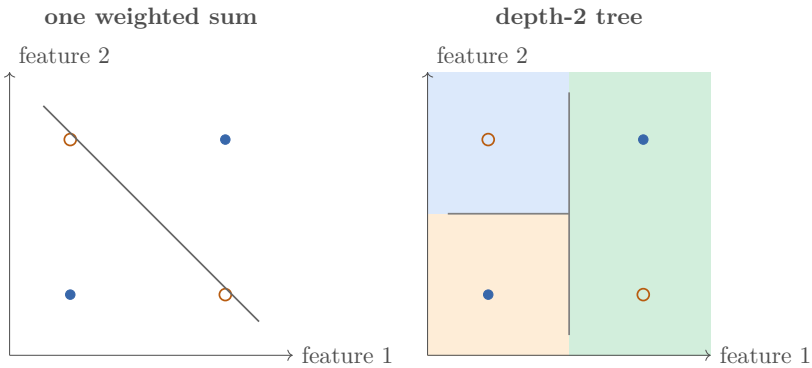


Figure 5.1: The same 2D dataset can defeat a single linear separator and still invite shallow, region-specific tree rules. The structural difference is global score versus local branching; additional splits can refine the partition further.

experiments interpretable. It also helps rule methods out for good reasons instead of trying everything indiscriminately.

5.4 Trees: Conditional Logic and Region-Specific Rules

Definition 5.1 (Decision Tree)**.** A decision tree is a model that predicts by recursively partitioning the feature space. Each internal node asks a question about the input, each branch corresponds to an answer, and each leaf produces a prediction.

The right headline for trees is not merely "nonlinear model." It is this: a tree is a model for conditional logic. The reader should picture conditions and regions, not coefficients with a more complicated curve wrapped around them.

The simplest mental model is a flowchart. A tree starts at the root and repeatedly asks questions such as:

- Is attendance less than 70%?
- Is the number of late assignments greater than 2?
- Is the neighborhood in the set {A, C, D}?

Depending on the answer, the example travels down one branch or another until it reaches a leaf. That leaf contains either a class label, a class-probability estimate, or a numeric prediction.

Trees are useful because they capture nonlinear behavior without forcing us to hand-engineer every interaction explicitly. A feature may matter in one branch and become irrelevant in another. This makes trees especially natural when the task contains threshold-like behavior or region-specific rules.

The same cloud that will appear later in the k-NN and unsupervised sections can already be read this way: the tree is looking for a threshold on lateness, not a single line that explains everything at once.

5.4.1 Choosing a Split

A tree is built by choosing splits that make the resulting child nodes "purer" than the parent node. For classification, one common measure of impurity is the Gini impurity:

$$G = 1 - \sum_k p_k^2,$$

where p_k is the fraction of examples in the node that belong to class k. A pure node containing only one class has impurity zero. Mixed nodes have higher impurity.

> **Prerequisite Bridge.** If the impurity formula looks unfamiliar, the main idea is simple: a good split creates children that are more homogeneous than the parent. The formula is just a scoring rule for that intuition.
>
> **Math Lens.** For binary classification, if the fraction of positive examples in a node is p, then the Gini impurity becomes
>
> $$G = 1 - p^2 - (1-p)^2 = 2p(1-p).$$
>
> This is 0 when $p = 0$ or $p = 1$, and it is largest at $p = 0.5$. That is why pure nodes feel "finished" and evenly mixed nodes feel maximally impure.

Student	Attendance rate	Late submissions	At risk?
1	0.52	5	yes
2	0.58	4	yes
3	0.61	4	yes
4	0.68	3	no
5	0.72	1	no
6	0.78	2	no
7	0.81	3	yes
8	0.88	1	no

Table 5.3: A tiny tree-building dataset is enough to show what "choosing a split" means numerically.

For regression trees, the objective is analogous but numeric: the split is chosen to reduce variance or squared error in the target values within each child node. The recursive structure stays the same even though the leaf predictions differ.

In Table 5.3, the parent node contains four risky students and four non-risky students, so the parent Gini impurity is

$$G_{\text{parent}} = 1 - \left(\frac{4}{8}\right)^2 - \left(\frac{4}{8}\right)^2 = 0.5.$$

Now compare two candidate root splits:

- `attendance rate` ≤ 0.65
- `late submissions` ≤ 1.5

For the attendance split, the left child contains three risky students and no safe ones, so its impurity is 0. The right child contains one risky student and four safe ones, so its impurity is

$$1 - \left(\frac{1}{5}\right)^2 - \left(\frac{4}{5}\right)^2 = 0.32.$$

The weighted child impurity is therefore

$$\frac{3}{8}(0) + \frac{5}{8}(0.32) = 0.20,$$

so the impurity reduction is

$$0.50 - 0.20 = 0.30.$$

For the late-submission split at 1.5, the left child contains two safe students and no risky ones, so its impurity is again 0. The right child contains four risky students and two safe students, so its impurity is

$$1 - \left(\frac{4}{6}\right)^2 - \left(\frac{2}{6}\right)^2 \approx 0.44.$$

The weighted child impurity is

$$\frac{2}{8}(0) + \frac{6}{8}(0.44) \approx 0.33,$$

so the impurity reduction is only about

$$0.50 - 0.33 = 0.17.$$

The attendance split is therefore better at the root.

Listing 5.1: From scratch: computing Gini impurity reduction for one candidate split

```
def gini(labels):
    n = len(labels)
    if n == 0:
        return 0.0
    counts = {}
    for label in labels:
        counts[label] = counts.get(label, 0) + 1
    return 1.0 - sum((c / n) ** 2 for c in counts.values())


# Student at-risk labels from the chapter dataset
left_labels = ["yes", "yes", "yes"] # attendance <= 0.65
right_labels = ["no", "no", "no", "no", "yes"] #
    attendance > 0.65
parent_labels = left_labels + right_labels

n_total = len(parent_labels)
n_left = len(left_labels)
n_right = len(right_labels)
```

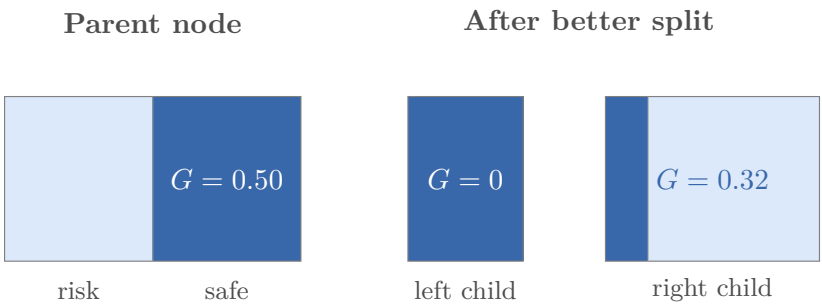


Figure 5.2: Impurity reduction is not mystical. A useful split creates children that are more class-homogeneous than the parent node.

```
parent_impurity = gini(parent_labels)
left_impurity = gini(left_labels)
right_impurity = gini(right_labels)
weighted_child = (n_left / n_total) * left_impurity \
               + (n_right / n_total) * right_impurity
reduction = parent_impurity - weighted_child

print("parent impurity:", round(parent_impurity, 4))
print("left impurity:", round(left_impurity, 4))
print("right impurity:", round(right_impurity, 4))
print("weighted reduction:", round(reduction, 4))
```

5.4.2 A Tree and Its Regions

The split calculation tells us which question to ask. The tree itself tells us how the space is partitioned.

Example 5.1 (Why Trees Can Feel Natural)**.** Suppose two students both missed two assignments. A linear model may treat those missed assignments similarly regardless of other context. A tree can make a different judgment if one student also has very low attendance while the other attends consistently and performs well on quizzes. The branching structure lets the model represent conditional logic directly.

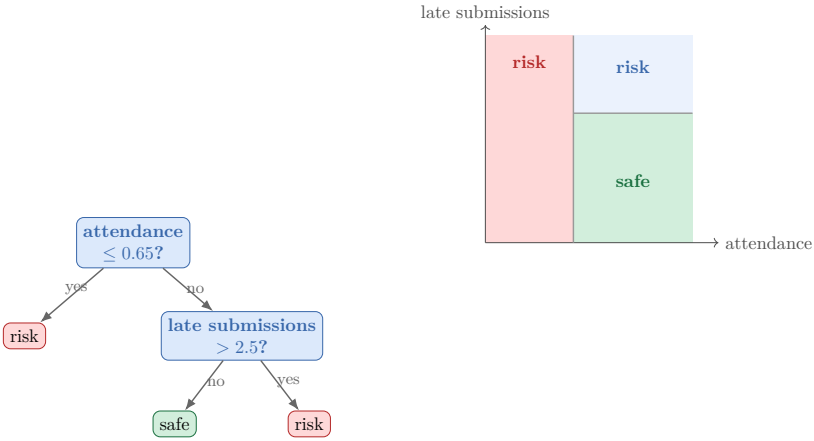


Figure 5.3: A tree can be read both as a flowchart and as a partition of feature space into rectangular regions. That dual picture is what makes the method legible.

5.4.3 Depth, Overfitting, and Pruning

The danger of trees is that they can keep splitting until they fit noise or accidental patterns in the training data. A very deep tree can memorize small quirks of the dataset while appearing perfectly confident.

Several controls are common:

- limiting the maximum depth of the tree
- requiring a minimum number of examples in a node before splitting
- pruning branches that do not improve validation performance

This is where Chapter 3 returns. A tree that keeps improving training accuracy is not necessarily improving the model we want. Validation performance determines whether additional branching is helping or merely memorizing.

> **Failure Box.** A single decision tree is often unstable. Small changes in the data can produce different root splits, different branches, and therefore different stories about what matters. This is one reason tree

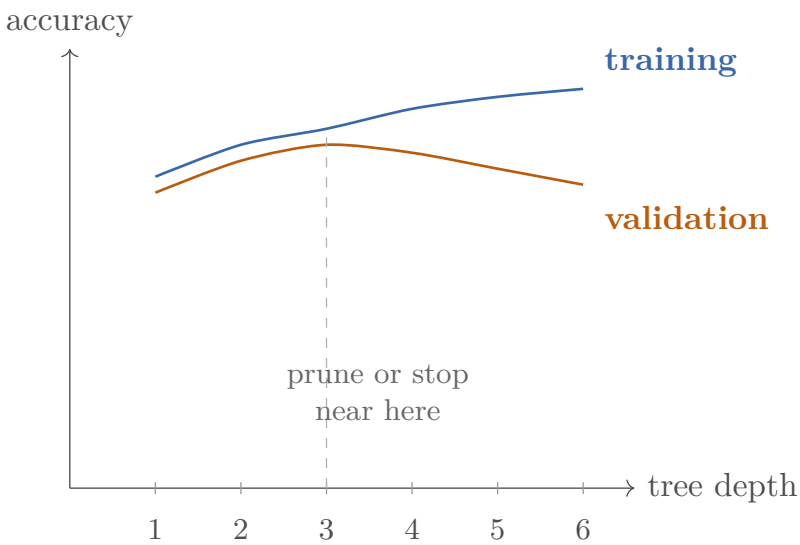


Figure 5.4: Single trees make overfitting visible quickly. Depth increases flexibility, but after a point the training gain is mostly memorization.

interpretation should be treated as useful but not absolute.

5.5 From Single Trees to Ensembles: Same Structure, More Stability

Single trees are easy to understand, but they are often noisy and high-variance models. Once the tree story is clear, the next question is not new structure but stability. Ensemble methods do not replace tree logic. They stabilize or strengthen it.

5.5.1 Why Averaging Helps: Variance Without Bias

A single decision tree is a low-bias model: given enough depth it can represent almost any boundary in its training data. But it is also high-variance. A small change in the training sample, perhaps a few swapped examples, can produce an entirely different

Model family	Main idea	Typical strength	Typical weakness
Single tree	One recursive partition of the feature space	Interpretable rules and threshold logic	Unstable and easy to overfit
Random forest	Many diverse trees trained in parallel and averaged	Strong tabular performance with lower variance	Less interpretable than a single tree
Boosted trees	Trees trained sequentially to correct earlier errors	Often very strong accuracy on structured data	More tuning-sensitive and easier to overfit if pushed too far

Table 5.4: Students often blur all tree-based methods together. The point of this comparison is to keep the workflows distinct.

root split, different branches, and a different story about what matters. The tree reacts strongly to the particular accidents of the data it saw.

Averaging many independent trees does not change each tree's bias—each one is still fitting the same underlying problem. But it does reduce variance. Predictions that fluctuate differently across trees tend to cancel out in the average, leaving behind the stable signal that most trees agree on.

Think of it this way: one panelist's opinion about whether a student is at risk is noisy and idiosyncratic. The average of fifty panelists who independently reviewed the same evidence is far more reliable. Their individual errors are uncorrelated enough to wash out, and the shared signal survives.

> **Math Lens.** Suppose T trees each produce an independent prediction with variance σ^2. The variance of their average is
>
> $$\mathrm{Var}\left(\frac{1}{T}\sum_{t=1}^{T}\hat{y}_t\right) = \frac{\sigma^2}{T}.$$

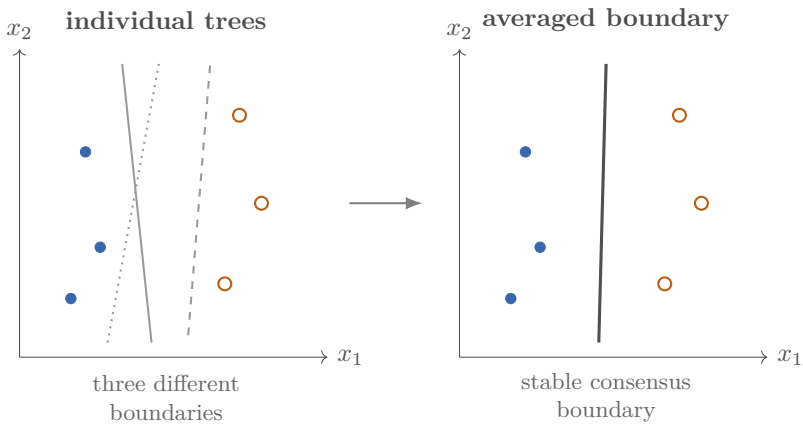


Figure 5.5: Individual trees produce noisy, jittery boundaries that vary with the training sample. Averaging many diverse trees smooths those fluctuations while preserving the underlying signal.

> Averaging T uncorrelated estimators reduces variance by a factor of T while leaving the expected prediction unchanged. In practice trees are not perfectly uncorrelated, so the gain is smaller than $1/T$, but the direction is the same: more diverse trees, lower variance.

Diversity is what makes averaging work. The variance-reduction argument above rests on one critical assumption: the trees must not all be making the same mistakes. If every tree in the ensemble produces the same error on the same examples, averaging changes nothing. The average of ten identical wrong answers is still a wrong answer.

This is why random forests introduce two sources of randomness deliberately. First, each tree is trained on a bootstrap resample of the data, so trees see overlapping but different subsets of examples. Second, each split considers only a random subset of features, so trees cannot all latch onto the same dominant feature and must find different evidence.

Boosted trees achieve diversity differently and more carefully: each new tree is trained to correct the specific errors of the trees already in the ensemble. Early trees handle the easy cases;

later trees focus on the residual. This sequential structure means boosted trees do not rely on random diversity. They build complementarity by design.

Example 5.2 (When Averaging Fails)**.** Suppose a training set has a systematic labeling error: all examples from a particular sensor type are mislabeled. Every tree in the ensemble trains on that same biased data. The average of their predictions inherits the same bias. Ensemble methods reduce variance arising from random sample fluctuations. They do not reduce bias arising from structural problems in the data itself. This is one reason error analysis on the full ensemble is still essential even when the ensemble is strong on average.

Decision Box. Random forest vs. boosted trees: which to prefer?

- Prefer a **random forest** when you want a robust default on tabular data with minimal tuning. It is harder to badly overfit and easier to parallelize.
- Prefer **boosted trees** when you need the strongest possible accuracy on structured data and are willing to tune carefully. Boosting can squeeze out more performance, but it is more sensitive to the number of rounds, tree depth, and learning rate.
- If the dataset is small or noisy, lean toward random forests; boosting can amplify noise when pushed too far.

5.5.2 Random Forests

A random forest trains many trees on different resampled subsets of the data and different subsets of features. Each tree sees a slightly different view of the problem. Their predictions are then averaged or voted together.

The main idea is variance reduction. A single tree may overreact to quirks of the sample. Many diverse trees, averaged together, tend to be more stable. Random forests often perform very well

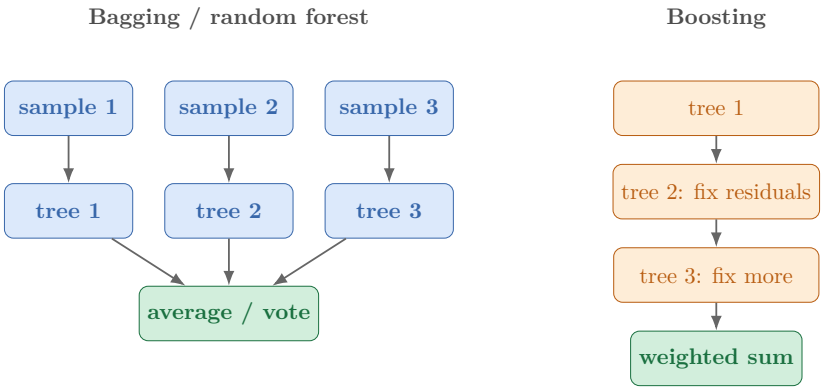


Figure 5.6: Random forests and boosting both use trees, but they organize them differently. One reduces variance through diverse parallel trees; the other adds trees sequentially to correct mistakes.

on tabular data for this reason.

> **Advanced Note.** Random forests also come with an appealing internal-validation intuition through out-of-bag examples. Because each tree is trained on a bootstrap-style resample, some training examples are left out of that tree's fit. Those left-out examples can be used as a rough internal check without a separate validation pass. It is not a replacement for careful evaluation, but it is a useful diagnostic idea.

Listing 5.2: From scratch: the prediction step of a random forest

```
def majority_vote(votes):
    counts = {}
    for v in votes:
        counts[v] = counts.get(v, 0) + 1
    return max(counts, key=counts.get)

def forest_predict(trees, example):
    # Each tree is an object with a .predict() method
    votes = [tree.predict(example) for tree in trees]
    return majority_vote(votes)

# Suppose we have a list of already-trained tree objects
# and one new student profile to classify
```

```
new_student = {"attendance": 0.55, "late_submissions": 4}

prediction = forest_predict(trained_trees, new_student)
print("ensemble prediction:", prediction)

# The ensemble vote might look like:
# tree 1 -> "yes"
# tree 2 -> "yes"
# tree 3 -> "no"
# tree 4 -> "yes"
# tree 5 -> "yes"
# majority_vote returns "yes"
```

5.5.3 Boosted Trees

Boosting takes a different approach. Instead of averaging many independent trees, it trains a sequence of trees, where each new tree focuses on the mistakes of the current ensemble. The result is often a highly accurate model that builds complexity gradually.

Boosting is powerful, but it also requires careful tuning. If the trees are too strong or the sequence is pushed too far, the ensemble can still overfit. The educational point is that ensembles improve trees not by abandoning recursive partitioning, but by controlling how many trees are combined and how they share the work.

Example 5.3 (Structured Ensembles in Weather Forecasting)**.** NOAA-supported work on storm-scale severe-weather guidance has compared tree ensembles with logistic baselines in the Warn-on-Forecast setting [20]. That is a good structured-ensemble example because the data is structured, the interactions are real, and the ensemble story is not hype-driven. The model family is chosen because the problem structure rewards it.

5.6 Margin-Based Classification: Support Vector Machines

Trees handle region-specific logic. Nearest neighbors handle local similarity. Margin-based classification handles a different ge-

ometric story: the classes already look roughly linearly separable in the current representation, and the best separator is the one that stays as far as possible from the nearest training points. In that sense, SVMs are the geometric continuation of the global-separator idea from the linear baseline, but with a stricter robustness preference.

Definition 5.2 (Support Vector Machine)**.** A support vector machine is a classifier that chooses a separating hyperplane with the largest margin between classes. When perfect separation is not realistic, the soft-margin version allows some violations and penalizes them.

The geometric idea is simple. Many separating lines may correctly classify the same data. An SVM prefers the line that is most robust to small perturbations. The closest points to that line are the *support vectors*; they are the points that actually define the margin.

Math Lens. For binary labels $y_i \in \{-1, +1\}$ and a linear score $f(x) = w^\top x + b$, the hard-margin SVM solves

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|_2^2 \\ \text{subject to} \quad & y_i f(x_i) \geq 1 \text{ for all } i. \end{aligned}$$

The soft-margin form adds slack variables ξ_i and a penalty:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|_2^2 + C\sum_i \xi_i \\ \text{subject to} \quad & y_i f(x_i) \geq 1 - \xi_i,\ \xi_i \geq 0. \end{aligned}$$

This is equivalent in spirit to minimizing a regularized hinge-loss objective:

$$\frac{1}{2}\|w\|_2^2 + C\sum_i \max(0, 1 - y_i f(x_i)).$$

The first term prefers a wide margin. The second term charges a cost when an example falls inside the margin or on the wrong side of the boundary.

Example 5.4 (A One-Dimensional Margin Choice)**.** Suppose the negative class lies at $x = 0$ and $x = 1$, while the positive class lies at $x = 3$ and $x = 4$. Any threshold between 1 and 3 separates the classes. A threshold at $x = 1.5$ is valid, but the nearest points are only 0.5 away. A threshold at $x = 2$ is also valid, and the nearest points are 1 unit away. The second separator has the larger margin, so it is the better SVM-style choice.

Listing 5.3: A hinge-loss style training sketch for a linear SVM

```
initialize w and b
for each pass over the data:
    for each example (x_i, y_i):
        margin = y_i * (dot(w, x_i) + b)
        if margin < 1:
            # inside the margin or on the wrong side
            w = (1 - eta * lambda) * w + eta * C * y_i *
                x_i
            b = b + eta * C * y_i
        else:
            # only the margin term applies
            w = (1 - eta * lambda) * w

# The sketch shows the core tradeoff:
# widen the margin, but pay for violations.
```

Decision Box. When does an SVM fit?

- Use it when the current representation already makes the classes look close to linearly separable and you want a robust, wide-margin boundary.
- Use it when small perturbations should not flip the decision easily.
- Prefer a tree or tree ensemble when the task is really region-specific threshold logic or different subgroups obey different rules.
- Prefer learned representations first when the geometry itself is poor; an SVM cannot invent missing features or repair a bad embedding.

Failure Box. An SVM is not a replacement for representation learning. If the problem needs many local rules, structured interactions, or a better geometry before any separator makes sense, a tree-based model or a learned representation can be the more honest next step.

5.7 Local Similarity: When Geometry Is the Model

Definition 5.3 (Nearest-Neighbor Method)**.** A nearest-neighbor method predicts for a new example by comparing it to nearby examples in the training set and borrowing information from those neighbors.

This is a very different philosophy from linear models or trees. A nearest-neighbor model does not summarize the whole dataset in coefficients or branching rules. Instead, it stores the examples and relies on the geometry of the feature space.

That is why one sentence in this section matters more than any formula:

Distance is the real model.

Defending a nearest-neighbor result therefore means defending the representation and similarity rule together.

For k-nearest neighbors, a new point is compared to the training examples, the k closest examples are identified, and the prediction is based on those neighbors. In classification, this is often a majority vote. In regression, it is often an average of target values.

5.7.1 Distance Defines the Model

The crucial choice is not only the number k, but the meaning of distance. Euclidean distance between two vectors x and x' is

$$\|x - x'\|_2 = \sqrt{\sum_{j=1}^{d} (x_j - x'_j)^2}.$$

This formula is easy to write and dangerous to use blindly. If one feature is measured on a much larger scale than the others, it can dominate the distance. If irrelevant features are added, they can drown out the useful ones. If the representation does not place truly similar examples near one another, the nearest-neighbor method has little chance to succeed.

> **Prerequisite Bridge.** For nearest-neighbor methods, the distance function is part of the model, not an afterthought. Asking "what counts as near?" is as important as asking "what are the labels?"

5.7.2 Worked Example: Local Similarity in Student Profiles

Suppose we want to classify a new student profile using past examples described by two features: total course-platform clicks and late submissions. The query student has profile $(51{,}000, 5)$, meaning many clicks but also many late submissions. This is the shared student-profile cloud introduced earlier in the chapter. Consider the following training examples:

Under the raw Euclidean distance, the nearest examples are A and B because the click count dominates the geometry. The query therefore looks safe. But if clicks are rescaled into units of tens of thousands before measuring distance, the late-submission difference matters much more, and the nearest neighbors become the risky students C and D. The predicted label flips even though the labels and raw data did not change.

> **Math Lens.** Using the query $(51{,}000, 5)$, the raw

Profile	Clicks	Late submissions	Outcome
A	50,000	0	safe
B	52,000	1	safe
C	20,000	4	risk
D	22,000	5	risk

Table 5.5: A tiny k-NN dataset is enough to show why scale changes the model itself.

Representation	Nearest neighbors	k=1 pred.	k=3 pred.
Raw (clicks, late)	A, B, D	safe	safe
Scaled (clicks/10,000, late)	D, C, B	risk	risk

Table 5.6: When a distance-based model changes after scaling, that is not an implementation detail. It is the geometry of the model changing.

distances are approximately

$$d(A) \approx 1000.0, \qquad d(B) \approx 1000.0,$$
$$d(C) \approx 31000.0, \qquad d(D) \approx 29000.0.$$

After rescaling clicks by 10,000, the same query becomes $(5.1, 5)$, while the points become $A = (5, 0)$, $B = (5.2, 1)$, $C = (2, 4)$, $D = (2.2, 5)$. The scaled distances are approximately

$$d(A) \approx 5.00, \qquad d(B) \approx 4.00,$$
$$d(C) \approx 3.26, \qquad d(D) \approx 2.90.$$

The nearest-neighbor order changes because the geometry changed. In k-NN, scaling is part of the model, not a cosmetic preprocessing choice.

Listing 5.4: Algorithm sketch: 1-nearest neighbor on the scaled representation

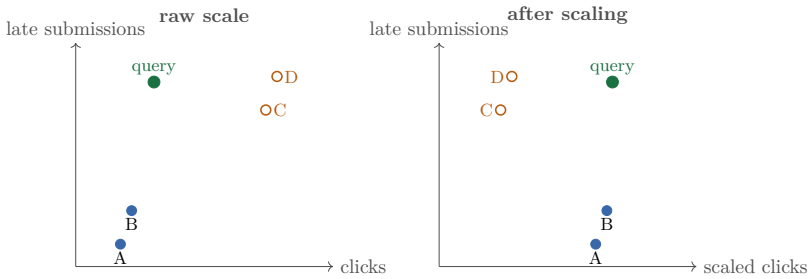


Figure 5.7: The same points can produce different nearest neighbors after sensible scaling. For k-NN, geometry is not a visualization choice. It is the model itself.

```
Input: query q, labeled dataset D
best_distance = infinity
best_label = none
for each (x, y) in D:
    d = EuclideanDistance(q, x)
    if d < best_distance:
        best_distance = d
        best_label = y
return best_label, best_distance
```

For comparison, the raw representation keeps clicks in their original units:

Listing 5.5: The same 1-NN sketch on the raw representation

```
Input: raw query q_raw, raw labeled dataset D_raw
run the same nearest-neighbor loop without rescaling
    features
compare the returned label and distance with the scaled
    version
```

This snippet is intentionally small. It exposes the whole method: store the examples, measure distance to each one, and borrow the label of the nearest case. Later libraries may optimize this idea, but they should not hide it.

> **Failure Pattern.** A nearest-neighbor result can flip after a harmless-looking unit conversion or after

> adding one large-scale irrelevant feature. That is not random behavior. The geometry has changed, so the model has changed. If the notion of similarity is not defended, the prediction is not defended either.

Euclidean distance is not sacred either. Manhattan distance, cosine similarity, learned metrics, and domain-specific similarity functions all define different notions of neighborhood. Changing the similarity rule changes the model.

5.8 Scale, Similarity, and the Curse of Dimensionality

The previous section gives the positive story: geometry can be the right inductive bias. This section gives the warning: geometry is fragile if the representation is poor.

As the number of features grows, distances often become less informative. Many points begin to look similarly far away, and local neighborhoods become harder to define meaningfully. This is one form of the curse of dimensionality.

> **Advanced Note.** Nearest-neighbor methods are sometimes called *nonparametric* because they do not compress the training data into a small fixed set of learned parameters the way linear models do. That flexibility is attractive, but it shifts the burden onto representation quality and data density. In high dimensions, good neighbors are harder to find unless the representation has already learned a useful geometry.

> **Complexity Lens.** Nearest-neighbor methods are cheap to "train" because they mostly store the data, but they can be expensive at prediction time because each query may need to be compared with many stored examples. Trees often have the opposite profile: more work during fitting, faster prediction once the structure is built.

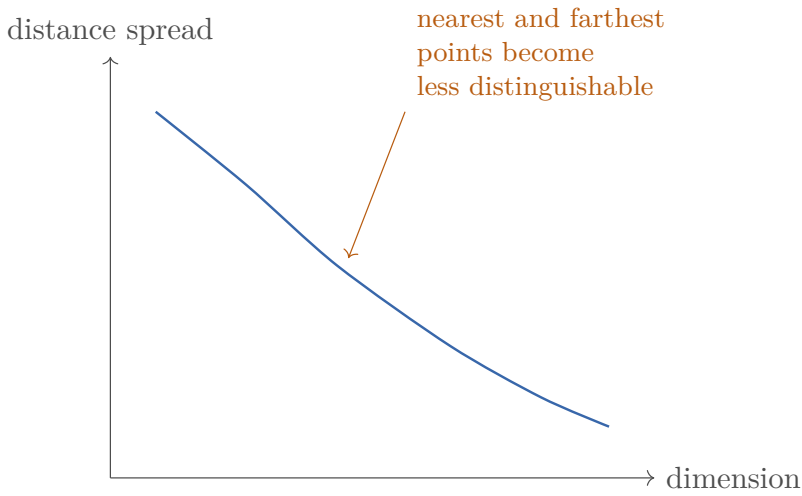


Figure 5.8: The curse-of-dimensionality intuition is not that all distances become large, but that they become less discriminative. "Nearest" stops meaning much if every point is similarly far away.

This warning also creates a natural bridge forward. Later chapters on learned representations matter partly because they improve the geometry itself. A poor representation makes neighborhood methods brittle. A better representation can make geometry useful again.

5.9 Unsupervised Structure: Organization Without Labels

So far the chapter has still assumed labeled tasks. Trees and nearest neighbors answer supervised prediction problems. The unsupervised part of the chapter changes the question itself.

Now the question is no longer "what label should I predict?" It is closer to "what organization or lower-dimensional summary is worth inspecting in the data I have?" That shift matters. It is one of the chapter's biggest intellectual moves. The model is no longer trying to predict a labeled outcome. It is trying to produce a useful summary of structure.

Sanity Check. When labels are absent, ask what counts as evidence. For clustering, check stability under resampling, initialization, and small scaling changes. For PCA, check whether the compressed view preserves enough variation or reduces reconstruction error enough to justify the loss of detail. If the structure is unstable or unhelpful for the downstream question, treat it as exploratory rather than factual.

5.10 Clustering as Hypothesis, Not Discovery

Definition 5.4 (Clustering)**.** Clustering is the task of grouping examples so that members of the same cluster are more similar to one another than to members of other clusters, according to some chosen notion of similarity.

The shared student-profile cloud from the opening of the chapter is useful here too: k-means is asking whether the same points can be summarized as compact groups, but the answer still depends on the representation and metric.

One of the simplest clustering methods is k-means. It alternates between two steps:

- assign each point to the nearest cluster center,
- recompute each center as the mean of the points assigned to it.

This process repeats until the assignments stabilize.

Listing 5.6: The basic loop of k-means clustering

```
initialize cluster_centers
repeat:
    assign each point to its nearest center
    recompute each center as the mean of its assigned
        points
until assignments stop changing
```

Math Lens. Why is the recomputed center the mean? Suppose one cluster contains points $x_1, \ldots, x_m$, and we want one center c that minimizes the within-cluster squared error

$$J(c) = \sum_{i=1}^{m} \|x_i - c\|_2^2.$$

Differentiate with respect to c:

$$\nabla_c J(c) = 2m\,c - 2\sum_{i=1}^{m} x_i.$$

Setting the gradient to zero gives

$$c = \frac{1}{m}\sum_{i=1}^{m} x_i.$$

So the mean is not an arbitrary convention. It is the point that minimizes squared distance to the assigned cluster members.

K-means is appealing because it is simple and fast. But its assumptions are strong. It works best when clusters are roughly compact and separable under the chosen distance metric. If the structure is elongated, nested, imbalanced, or highly nonconvex, k-means can mislead badly.

Example 5.5 (One K-Means Iteration by Hand)**.** Suppose four points are

$$(1,1),\ (1,2),\ (4,4),\ (5,4),$$

and the initial centers are

$$c_1 = (1,1), \qquad c_2 = (5,4).$$

The first two points are much closer to c_1, while the last two are much closer to c_2. After the assignment step, cluster 1 contains $(1,1)$ and $(1,2)$, so its updated mean is

$$c_1' = \left(\frac{1+1}{2}, \frac{1+2}{2}\right) = (1, 1.5).$$

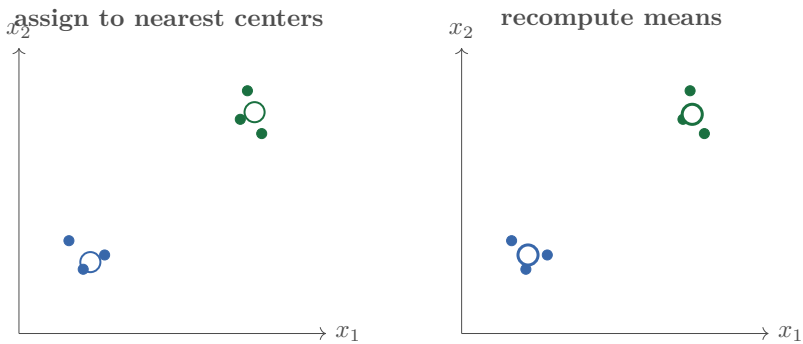


Figure 5.9: One full k-means iteration consists of assignment and recomputation. It is a simple loop, which is why it is worth learning by hand once.

Cluster 2 contains $(4, 4)$ and $(5, 4)$, so its updated mean is

$$c_2' = \left(\frac{4+5}{2}, \frac{4+4}{2}\right) = (4.5, 4).$$

That is the whole loop in miniature: assign to the nearest center, then move each center to the mean of its assigned points.

Example 5.6 (Study Patterns Without Labels)**.** Suppose a university has only behavioral data such as attendance, quiz attempts, and time spent on the learning platform, but no final labels yet. A clustering method might reveal several behavioral groups: consistent high-engagement students, low-engagement students, and a group that studies heavily but only near deadlines.

This can be useful for exploration or for designing support interventions. But the clusters are not automatically "real types of students." They are patterns produced by a particular representation, a particular metric, and a particular clustering procedure.

Example 5.7 (A Public Clustering Caution)**.** A sports-analytics clustering exercise is a good reminder that the hard part is not only producing clusters, but justifying what they mean. A cluster label such as "defensive specialist" or "high-risk learner" is an interpretation layered on top of a mathematical grouping. It needs domain evidence, not just a pleasing scatter plot.

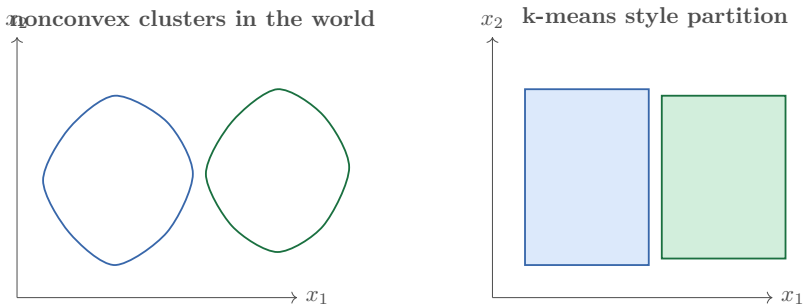


Figure 5.10: K-means is not a universal clustering truth machine. When cluster shape matters, the method's geometric bias can force a misleading partition.

Failure Box. Clusters are model-dependent summaries, not discovered natural kinds. Treating them as objective human categories is one of the most common mistakes in applied machine learning.

Claim Defense. From clustering alone, the safe claim is that a particular representation, metric, and clustering rule produce a grouping pattern that may be analytically useful. The unsafe claim is that the groups are natural human types or policy categories.

5.10.1 Choosing K

There is rarely one objectively "true" number of clusters. Methods such as elbow plots, silhouette scores, and stability checks can help compare choices of K, but they do not eliminate judgment. A useful clustering is one that remains stable enough to be informative and supports the actual analytic purpose.

Sanity Check. Rerun clustering after changing feature scaling, random initialization, or a small resample of the data. If the clusters drift badly, treat them as fragile exploratory summaries rather than stable structure.

5.11 Clustering Validity and the Danger of Naming Clusters Too Fast

A clustering algorithm produces a partition. That partition is geometrically defined by the algorithm's notion of similarity, the number of clusters chosen, the initialization, and the features provided. None of those choices are neutral, and none of them guarantee that the resulting groups correspond to anything meaningful in the world.

Three levels of interpretation should be kept distinct. Each level makes a stronger claim than the one before it.

Geometric grouping. A clustering is valid in a narrow sense if the groups are internally coherent under the chosen metric. This is testable: intra-cluster distances should be small relative to inter-cluster distances under the metric used for clustering. Silhouette scores and related diagnostics measure exactly this. But geometric validity says nothing about whether the groups mean anything.

Operational usefulness. A clustering is operationally useful if acting on it produces better outcomes than ignoring it. A segmentation of students into three groups may be useful if the three groups genuinely benefit from different kinds of support, regardless of whether those groups correspond to any natural social or psychological categories. Usefulness is measured by downstream outcomes, not by the cluster algorithm's internal objective.

Causal or social interpretation. Naming a cluster is a much stronger act than finding it. A label like "students who lack intrinsic motivation" or "customers who are price-insensitive" asserts a causal story or a social category. That label cannot be read off the clustering. It requires external validation: interviews, longitudinal follow-up, or theory that connects the geometric group to the label.

> **Failure Box.** A common pattern: researchers run k-means on a student dataset, find three clusters, label them "engaged," "struggling," and "disengaged,"

and then report policy recommendations as if those labels are real categories. The algorithm found a geometric partition. The labels are a narrative imposed afterward. If the partition is sensitive to initialization or feature scaling, the narrative is doubly fragile.

Example 5.8 (Clustering of Customer Data)**.** An e-commerce team clusters purchasing histories into five groups and labels them "deal hunters," "brand loyalists," and "impulse buyers." These labels feel intuitive. But stability analysis reveals that three of the five clusters change substantially when the clustering is rerun with a different random seed. The geometric groups were artifacts of initialization, not stable population structure. The labels were named before the groups were tested.

The right habit is to treat any cluster as provisional until three questions are answered: Is the partition stable under small perturbations of the data and initialization? Does the partition support the operational decisions it will be used to justify? And what external evidence would be needed before a causal or social interpretation could be defended?

Problem-Solving Technique. Cluster naming discipline. Describe clusters by their measurable properties first: average feature values, distinguishing coordinates, and sample size. Name them by social or causal labels only after external validation. A cluster labeled "high-risk" that is defined only geometrically is not a validated risk group. It is a starting hypothesis.

5.12 Dimensionality Reduction as a Lens

Another unsupervised goal is dimensionality reduction. Rather than grouping examples, we try to represent them with fewer coordinates while preserving important structure. Principal component analysis (PCA) is a standard example. It finds directions in feature space along which the data varies most and projects the data onto those directions.

PCA is best taught here not as a competitor to predictive methods, but as a lens:

- a geometric summary,
- a useful preprocessing or inspection tool,
- and a reminder that simplified views are still model choices.

PCA is also scale-sensitive. If one feature is measured in a much larger numerical unit than another, it can dominate the first principal component. Standardize features first when the units are not already comparable. On the shared student-profile cloud, the first component mainly tracks raw clicks unless the features are standardized; after standardization, it can reflect the click-versus-lateness tradeoff.

> **Math Lens.** For centered data $x_1, \ldots, x_n$, a one-dimensional PCA direction u with $\|u\|_2 = 1$ can be found by maximizing the projected variance
>
> $$\frac{1}{n}\sum_{i=1}^{n}(x_i^\top u)^2.$$
>
> Equivalently, it minimizes the total squared reconstruction error after projection:
>
> $$\sum_{i=1}^{n}\left\|x_i - (x_i^\top u)u\right\|_2^2.$$
>
> These are the same objective up to the constant $\sum_i \|x_i\|_2^2$, since
>
> $$\left\|x_i - (x_i^\top u)u\right\|_2^2 = \|x_i\|_2^2 - (x_i^\top u)^2.$$
>
> So PCA is choosing the direction that keeps the largest amount of variation after compression.

For a tiny numeric intuition, imagine two centered features that mostly vary together. If increasing one feature almost always means increasing the other, then most of the variation lies along a diagonal direction. PCA captures that shared direction first. A two-coordinate description can then be reduced to one dominant coordinate plus one smaller residual coordinate.

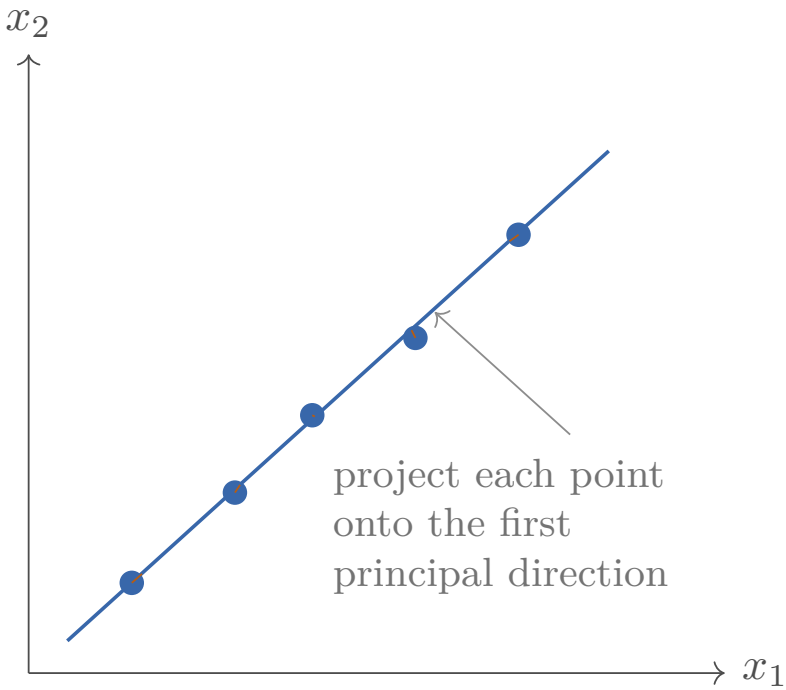


Figure 5.11: PCA chooses a direction of large variation and projects the data onto it. It is a geometric summary, not "compression magic."

Example 5.9 (Projecting One Point onto a Principal Direction)**.** Suppose the first principal direction is

$$u = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

and one centered data point is

$$x = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

The one-dimensional PCA coordinate is the dot product

$$x^\top u = \frac{3+1}{\sqrt{2}} = 2\sqrt{2}.$$

Projecting back onto the principal direction gives

$$(x^\top u)u = 2\sqrt{2} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

So PCA keeps the dominant diagonal component of the point and discards the residual orthogonal variation. That is the precise sense in which PCA compresses: it preserves some directions and throws away others.

> **Advanced Note.** The unsupervised landscape is broader than k-means and PCA. Alternatives such as DBSCAN, hierarchical clustering, t-SNE, and UMAP answer different needs. The conceptual lesson is enough for this chapter:
>
> - DBSCAN and hierarchical clustering are useful when cluster shape or nested structure matters.
> - t-SNE and UMAP are mainly visualization tools; they can be very informative, but they are not truth machines.

5.13 Chapter Artifact: A Structure Diagnosis Sheet

By the end of this chapter, the reader should be able to explain not only which method they want to try next, but why that method matches the failure pattern of the linear baseline. Chapter 3 named the first serious model to try. This chapter names the reason to leave it. The chapter should end with one diagnosis, not with a shopping list of unrelated methods.

> **Problem-Solving Technique. Structure diagnosis sheet.** Before changing model families, write down what the baseline missed, what structural hypothesis now seems plausible, why a tree, neighbor, or unsupervised method matches that hypothesis, and what evidence would justify the move.

5.14 Common Mistakes with Structural Methods

Mistake Checklist.

- **Treating trees as automatically interpretable.** Deep trees and boosted ensembles are not automatically transparent.
- **Treating feature importance as causation.** Importance summarizes predictive usefulness, not causal effect.
- **Ignoring scale in distance-based models.** Geometry depends on feature units.
- **Believing clusters too quickly.** A cluster can be mathematically convenient without being substantively meaningful.
- **Treating a low-dimensional plot as the true data structure.** PCA, t-SNE, and UMAP are views, not reality.
- **Skipping the baseline logic.** Explain what structure the tree or neighbor model captures that a linear baseline misses.
- **Mistaking better fit for better judgment.** Better fit does not automatically mean better decisions.

5.15 Looking Ahead

This chapter broadened structure beyond one weighted sum to branches, neighborhoods, and low-dimensional views. The next chapter asks when structure should be learned rather than engineered, and how neural networks learn internal representations and decision rules jointly through optimization.

Chapter Summary

- This chapter answers one question: when the linear baseline fails, what kind of missing structure does that failure reveal?

- Trees are models for conditional logic and region-specific rules rather than just "nonlinear alternatives."
- Ensembles do not replace tree logic; they stabilize or strengthen it.
- Support vector machines are the geometric companion to that story: they prefer the widest robust separator when the current representation already looks linearly organized.
- Nearest-neighbor methods make geometry the model, which means representation and distance choices become central.
- Clustering and dimensionality reduction change the question from supervised prediction to exploratory organization and compression.
- Clusters are model-dependent summaries, not discovered natural kinds, and low-dimensional plots are useful views rather than truth machines.
- The main deliverable of the chapter is a structure diagnosis sheet that justifies the move beyond the linear baseline.

Study Questions

1. Why is "what kind of structure is missing?" a better opening question than "which classical method comes next?"
2. Why should trees be understood as models for conditional logic rather than merely as nonlinear models?
3. Why is the notion of distance the real modeling choice in nearest-neighbor methods?
4. Why does maximizing margin make an SVM more robust than a separator that merely classifies correctly?
5. Why should clusters be interpreted more cautiously than labeled classes?
6. What would count as evidence that a tree, neighbor, or unsupervised method is justified beyond the linear baseline?

Exercises

1. Give a real-world example of a problem that naturally sounds like a branching rule rather than a single weighted score. Explain why.

2. Using Table 5.3, verify the impurity reduction for the two candidate root splits by hand.

3. Explain why classification trees and regression trees share the same recursive structure even though their leaf predictions are different.

4. Classify the query student in Table 5.5 with $k = 1$ and $k = 3$ before and after scaling. Explain why the prediction changes.

5. For the one-dimensional SVM example in this chapter, explain why the threshold at $x = 2$ has a larger margin than a threshold at $x = 1.5$.

6. Using the binary-case identity $G = 2p(1 - p)$, compute the Gini impurity of nodes with positive-class fractions 0.1, 0.5, and 0.9. Explain which node is hardest to split cleanly.

7. Run one k-means iteration by hand for the two-cluster example in Figure 5.9. State the assignments and the recomputed centers.

8. Sketch a small 2D dataset for which a shallow tree should beat a linear model for the right reason. State clearly what threshold or interaction the tree captures.

9. Why might a random forest outperform a single decision tree even though both are built from the same basic building block?

10. Give one example in which clustering could be useful for exploration and one example in which clustering could be dangerously overinterpreted.

11. Write language-neutral pseudocode for 1-nearest-neighbor classification using Euclidean distance, then explain how the returned label can change after rescaling one feature.

12. Use Table 5.8 to write a one-page diagnosis for a failed linear baseline in either student support, spam filtering, or another application of your choice. If you are carrying one case through the book, extend the baseline design sheet from the previous chapter instead of inventing a fresh failure.

Challenge Exercises

1. Construct a tiny binary classification dataset in two dimensions for which a linear separator performs poorly but a depth-2 decision tree performs perfectly. Explain the split sequence.
2. Suppose a 1-nearest-neighbor classifier gives unstable predictions when one feature is added to the dataset. Give two different explanations for why this could happen.
3. A team clusters user behavior data into four groups and labels them “leaders,” “followers,” “at risk,” and “disengaged.” Explain why this move from mathematical grouping to substantive naming requires additional evidence, and describe what evidence would help.
4. Give a clustering result that looks convincing on one 2D plot but should still fail a stability sanity check. Explain what change in scaling, initialization, or sampling would reveal the weakness.
5. DBSCAN, hierarchical clustering, t-SNE, and UMAP sit alongside k-means and PCA for different reasons. Give one sentence each on what problem characteristic might make you look beyond k-means.

Method family	Best when	Main warning
Single tree	You need explicit branching logic and readable rules	A single tree can be unstable and overly sensitive to the sample
Random forest	Tabular data is strong and you want a safer default than one tree	Interpretation becomes weaker because the vote is spread across many trees
Boosted trees	You want strong accuracy on structured data and can tune carefully	More tuning-sensitive; easy to overfit if pushed too far
SVM	A wide-margin linear boundary is plausible in the current feature space	It depends on the representation; it will not repair missing structure by itself
k-NN	Similar cases should borrow labels through a defended distance rule	Scaling and feature geometry are the model itself
Clustering/PCA	You have no labels and want organization, grouping, or compression	The result is a summary, not a discovered truth

Table 5.7: A compact synthesis helps students choose the next family without treating method names as interchangeable.

Diagnosis field	What must be written clearly
Seven-question focus	Questions 4, 5, and 6: identify the missing structure, choose the next justified baseline family, and state what evidence would support the move
Baseline failure	What the linear model is getting wrong and on what kind of examples
Structural hypothesis	Conditional logic, local similarity, or unlabeled organization
Candidate family	Tree, ensemble, nearest-neighbor, clustering, or dimensionality reduction method
Representation dependency	Which feature choices or scaling rules matter most for this family
Main risk	Instability, weak geometry, cluster overinterpretation, or another failure mode
Evidence needed	What validation or inspection result would justify moving beyond the baseline
Interpretation boundary	What the method's output does and does not justify claiming

Table 5.8: The chapter should end with a structural diagnosis, not with method shopping.

Part II

Deep Learning

Chapter 6

Optimization and Neural Networks

Learning Goals

- Explain when learned internal structure becomes more appropriate than hand-designed features alone.
- Understand a neural network as a learned feature pipeline rather than as a bag of layers and tricks.
- Compute a simple forward pass by hand and explain what each hidden unit contributes.
- Treat losses as teaching signals, optimization as repeated correction, and backpropagation as credit assignment.
- Explain how regularization tries to make learned representations less brittle on new data.
- Produce a minimum credible neural training report before making a serious model claim.

The previous chapter broadened our idea of structure by introducing branching rules, local geometry, and unlabeled organization. Neural networks broaden it again, but in a different way. Instead of depending so heavily on manually chosen splits, distances, or

handcrafted interactions, neural models learn internal representations directly through optimization. Standard references such as LeCun et al. [42] and Goodfellow et al. [23] treat this layered view as the central idea of modern deep learning. This chapter should therefore be read as the point where structure diagnosis turns into learned structure rather than into one more hand-designed model family.

The chapter should therefore answer one question clearly:

When should structure be learned rather than engineered, and how does optimization actually teach a network to build that structure?

> **Problem-Solving Lens.** This chapter addresses a different modeling dilemma: when hand-engineered structure stops being enough, how can a model learn the intermediate features it needs? Neural networks matter here not as a catalog of architectures, but as a mechanism for learned representation through optimization.

By the end of the chapter, the reader should be able to produce a minimum credible neural training report before making a serious model claim. The chapter's job is to make that report feel like the natural endpoint of one coherent training story rather than a checklist added afterward.

> **Problem-Solving Technique. Artifact preview: minimum credible neural training report.** Keep six reporting commitments visible while reading: the baseline to beat, the input representation and output type, the loss, the optimization setup, the stopping rule, and the evidence that separates optimization failure from generalization failure.

6.1 Opening Dilemma: When Engineered Structure Stops Being Enough

The earlier modeling chapters already established a core logic. We frame a task, design an evaluation plan, try a serious baseline, and inspect what kind of structure that baseline is missing. Sometimes the answer is still manageable with trees, engineered interactions, or geometry-based methods. Sometimes it is not.

Images, language, and audio often contain structure that is difficult to specify manually at the right level of abstraction. Even in smaller problems, the useful intermediate signals may not be obvious in advance. In such cases, the main challenge is not only drawing the final decision boundary. It is discovering the space in which that boundary should be drawn.

That is when learned structure becomes the right move. The issue is not merely that deep learning is powerful. The issue is that the representation itself is part of what must be learned.

Decision Box.

- Choose a neural network when the task likely depends on multiple layers of representation that are difficult to specify manually.
- Prefer a simpler baseline if a linear model, tree ensemble, or hand-designed feature pipeline already solves the problem well enough.
- Use the added flexibility only when you have enough data, compute, and evaluation discipline to justify it.
- Decide early whether the output will be used as a score, a thresholded decision, or a calibrated probability.

Practical Note. Workflow sketch: start with the strongest simple baseline, decide what structure it is missing, build the smallest network that could plausibly learn that structure, train with a validation split

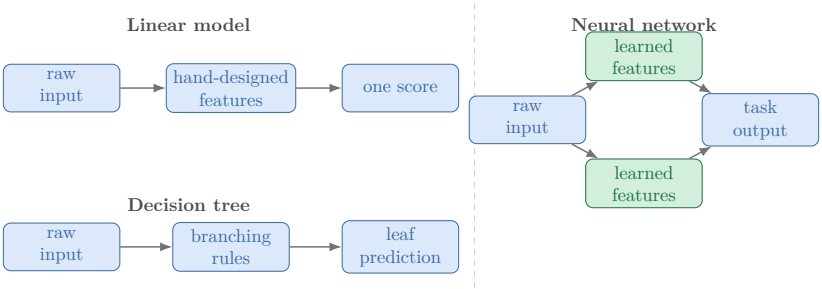


Figure 6.1: These model families all map inputs to predictions, but they place structure in different places. Neural networks are distinctive because they can learn intermediate representations rather than depending entirely on fixed features or fixed rules.

> while watching for optimization versus generalization failure, and then report the result in a minimum credible training report. The rest of the chapter follows that order: representation, forward pass, loss, optimization, debugging, generalization, and then reporting.

This does not mean neural networks are always the best choice. On small tabular datasets, a linear model or tree ensemble may still be stronger, easier to debug, and easier to justify. The important shift is conceptual: neural networks are most valuable when the main modeling problem is that the useful internal features are unknown.

6.2 A Neural Network as a Learned Feature Pipeline

The conceptual heart of the chapter is simple:

$$\textit{raw input} \rightarrow \textit{learned intermediate features} \rightarrow \textit{final score or probability}$$

A hidden layer is not extra decoration. It is a learned representation stage. Everything else in the chapter explains how that pipeline is computed, corrected, stabilized, and judged.

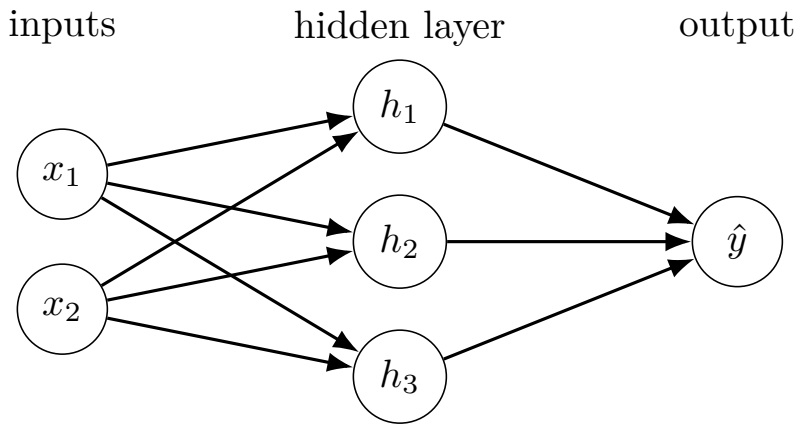


Figure 6.2: A one-hidden-layer network is structured computation. Inputs produce pre-activations, activations become learned intermediate features, and those features feed the final output.

Definition 6.1 (Neural Network)**.** A neural network is a parameterized function built by composing layers. Each layer typically applies an affine transformation followed by a nonlinear activation function.

At the level of one unit, the calculation still looks familiar:

$$z = w^\top x + b, \qquad a = \phi(z).$$

Here x is the input vector, w is a vector of weights, b is a bias term, z is the pre-activation score, and ϕ is the activation function. The output a then becomes part of the input to the next layer.

This should look like an extension of Chapter 4 rather than a complete break from it. The affine part $w^\top x + b$ is still a weighted sum plus bias. What changes is that the network repeats this process across layers and inserts nonlinearities between them. Those nonlinearities allow the model to reshape the representation itself.

> **Prerequisite Bridge.** An affine transformation is a linear transformation followed by a shift. In practice, students can think of $w^\top x + b$ as "score the input, then move the score up or down with a bias."

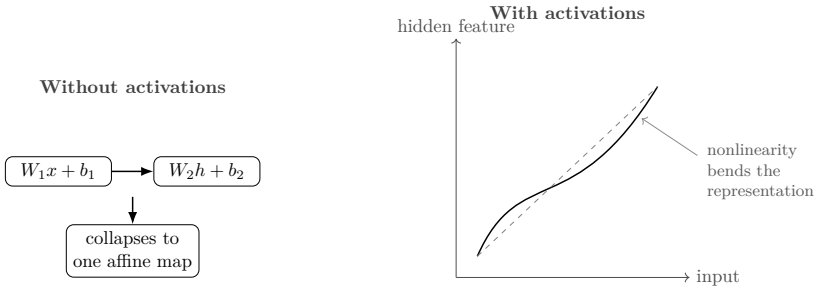


Figure 6.3: Depth alone is not enough. Stacking affine maps just rewrites one affine map; inserting nonlinearities changes the space itself.

6.3 Why Nonlinearity Matters

Students do not need a long catalog of activation functions at this stage. They need one main idea: nonlinearity is what allows the learned feature space to bend at all.

Suppose a network had two layers but no activation function:

$$h = W_1x + b_1, \qquad y = W_2h + b_2.$$

Substituting the first expression into the second gives

$$y = W_2W_1x + (W_2b_1 + b_2).$$

This is still just another affine transformation of x. No matter how many such layers are stacked, without nonlinearity the whole network is equivalent to one larger affine model.

This is why activation functions matter pedagogically. Their job is not to create trivia to memorize. Their job is to make learned internal features more expressive than the weighted-sum models of Chapter 4.

> **Warning.** Older saturating activations can produce very small gradients when their outputs are near the flat ends of the curve. ReLU avoids that kind of saturation for positive inputs, but it can create a different failure mode: if a unit stays in the negative region, it may output zero repeatedly and stop updating in a

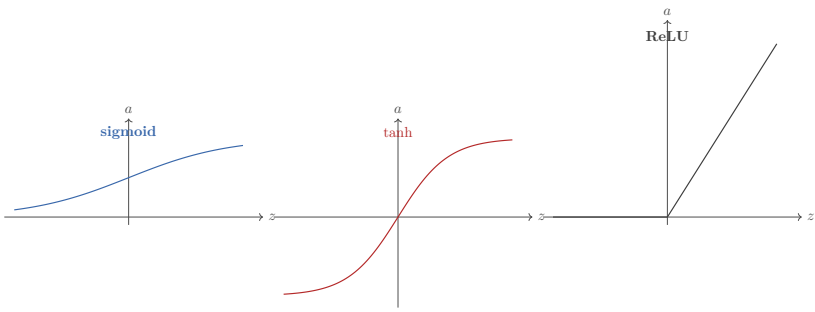


Figure 6.4: Students should notice three different behaviors immediately: sigmoid squashes into $(0, 1)$, tanh is zero-centered, and ReLU is piecewise linear with a flat region for negative inputs.

useful way. This is the intuition behind "dead ReLUs."

6.4 Worked Anchor Example: One Forward Pass, Fully Legible

Keep one small network in view long enough that the rest of the chapter feels like one story rather than a collection of topics. Consider a tiny network for predicting whether an email message is likely to be spam. Let the input features be:

- x_1 = suspicious-link signal
- x_2 = urgent-language signal

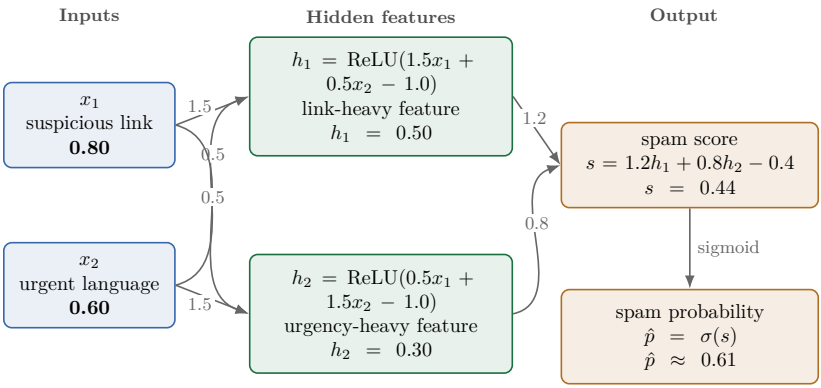
Assume both features have already been scaled into a rough range between 0 and 1. For one message, let

$$x = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix}.$$

Suppose the first hidden layer has two ReLU units:

$$h_1 = \mathrm{ReLU}(1.5x_1 + 0.5x_2 - 1.0),$$

$$h_2 = \mathrm{ReLU}(0.5x_1 + 1.5x_2 - 1.0).$$



All displayed values match the hand-worked forward pass above.

Figure 6.5: The worked spam-classification network with the actual numbers from the forward pass. Inputs, hidden-feature formulas, output score, and final sigmoid probability are kept visually separate so the computation is easier to follow.

Now compute them:

$$h_1 = \text{ReLU}(1.5(0.8) + 0.5(0.6) - 1.0) = \text{ReLU}(0.5) = 0.5,$$

$$h_2 = \text{ReLU}(0.5(0.8) + 1.5(0.6) - 1.0) = \text{ReLU}(0.3) = 0.3.$$

The output layer produces a spam score

$$s = 1.2h_1 + 0.8h_2 - 0.4 = 1.2(0.5) + 0.8(0.3) - 0.4 = 0.44.$$

If the final layer uses a sigmoid to convert that score to a probability, then

$$\hat{p} = \sigma(0.44) \approx 0.61.$$

> **Warning.** A sigmoid or softmax output is a model score, not yet an action. If the model will make decisions, choose the threshold on validation data. If the output will be interpreted as a probability, check calibration separately instead of assuming the network did that for free.

Quantity	Expression	Before act.	After act.
Hidden unit h_1	$1.5x_1 + 0.5x_2 - 1.0$	0.50	0.50
Hidden unit h_2	$0.5x_1 + 1.5x_2 - 1.0$	0.30	0.30
Output score s	$1.2h_1 + 0.8h_2 - 0.4$	0.44	—
Final probability $\hat{p}$	$\sigma(s)$	0.44	0.61

Table 6.1: A forward pass becomes much less mysterious when every stage is named explicitly. Pre-activations, activations, score, and probability are different objects and should not be conflated.

Listing 6.1: From scratch: a complete forward pass with numerical values

```
# Network: 2 inputs, 2 hidden units (ReLU), 1 output (
    sigmoid)
# Input: suspicious-link signal and urgent-language signal
x1 = 0.8 # suspicious-link signal
x2 = 0.6 # urgent-language signal

# Hidden layer weights and biases (hardcoded for
    illustration)
w11, w12, b1 = 1.5, 0.5, -1.0 # weights into hidden unit 1
w21, w22, b2 = 0.5, 1.5, -1.0 # weights into hidden unit 2

# Forward pass: hidden unit 1
z1 = w11 * x1 + w12 * x2 + b1 # linear combination
print("hidden unit 1 pre-activation:", round(z1, 4))
h1 = max(0.0, z1) # ReLU: pass positive values, zero out
    negative
print("hidden unit 1 post-ReLU: ", round(h1, 4))

# Forward pass: hidden unit 2
z2 = w21 * x1 + w22 * x2 + b2 # linear combination
print("hidden unit 2 pre-activation:", round(z2, 4))
h2 = max(0.0, z2) # ReLU activation
print("hidden unit 2 post-ReLU: ", round(h2, 4))

# Output layer: combine hidden activations into a single
    score
```

```
v1, v2, b_out = 1.2, 0.8, -0.4 # output-layer weights and
    bias
s = v1 * h1 + v2 * h2 + b_out # pre-sigmoid output score
print("output score (pre-sigmoid): ", round(s, 4))

# Sigmoid squashes the score into a probability
import math
p = 1.0 / (1.0 + math.exp(-s)) # sigmoid probability for
    class 1
print("predicted probability: ", round(p, 4))
```

Every library that trains a neural network executes exactly this sequence of multiplications, additions, and activation calls. Seeing the numbers makes the pipeline concrete: the hidden units are not magic—they are weighted sums with a nonlinearity applied, and their outputs become the inputs to the next layer.

This small example already shows the logic of a network. The hidden units do not directly output the final prediction. They create intermediate signals. One hidden unit responds more strongly to suspicious-link evidence; the other responds more strongly to urgent-language evidence. The output layer then combines those intermediate signals into a final decision.

Example 6.1 (What the Hidden Layer Is Doing)**.** In a linear model, the final score would be computed directly from x_1 and x_2. In this network, the model first builds two internal features, h_1 and h_2. Those features are not handed to the model by the user. They are learned during training. This is the conceptual heart of neural networks.

Now change the message slightly and let $x = (0.2, 0.7)$. Then

$$h_1 = \mathrm{ReLU}(1.5(0.2) + 0.5(0.7) - 1.0) = \mathrm{ReLU}(-0.35) = 0,$$

while

$$h_2 = \mathrm{ReLU}(0.5(0.2) + 1.5(0.7) - 1.0) = \mathrm{ReLU}(0.15) = 0.15.$$

One hidden unit is now shut off entirely. This will matter again when we discuss backpropagation. Keeping one small network alive across several sections is deliberate: the goal is to make the later training story feel like continuation, not topic-switching.

6.5 Loss as the Teaching Signal

Once the network outputs a score or probability, what tells it how to improve? The answer is the loss. The same worked network now stops being only a predictor and becomes an object that can be corrected.

> **Problem-Solving Technique. Metrics judge; losses teach.** Evaluation metrics tell us what claim we can make. Training losses tell the optimizer how to correct the model.

Definition 6.2 (Loss Function)**.** A loss function measures how bad a model's prediction is on a single example. Training seeks parameter values that make the average loss over the training data small.

For regression, a standard choice is mean squared error:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2.$$

For binary classification, a common choice is binary cross-entropy:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right].$$

> **Math Lens.** For one example with logit s, probability $\hat{p} = \sigma(s)$, and label $y \in \{0,1\}$, the loss is
>
> $$\mathcal{L}(s) = -\big[y \log \hat{p} + (1-y)\log(1-\hat{p})\big].$$
>
> Differentiate with respect to s:
>
> $$\frac{d\mathcal{L}}{ds} = \frac{d\mathcal{L}}{d\hat{p}} \cdot \frac{d\hat{p}}{ds},$$
>
> with
>
> $$\frac{d\mathcal{L}}{d\hat{p}} = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}}, \qquad \frac{d\hat{p}}{ds} = \hat{p}(1-\hat{p}).$$

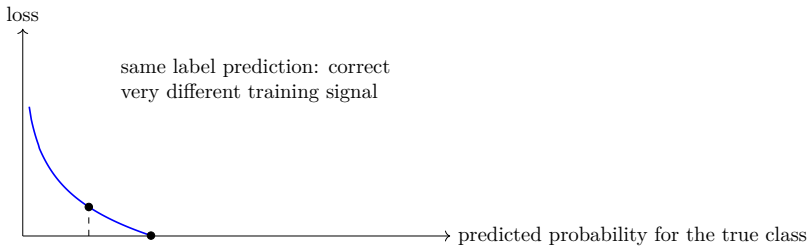


Figure 6.6: Training loss distinguishes weakly correct predictions from confidently correct ones. That is why cross-entropy is useful even when accuracy is the metric reported to an audience.

Multiplying and simplifying gives

$$\frac{d\mathcal{L}}{ds} = \left(-\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}}\right)\hat{p}(1-\hat{p}) = \hat{p} - y.$$

That is the reusable training signal for a sigmoid output layer: prediction minus truth.

These formulas matter, but the deeper point is that the loss function defines what kinds of mistakes should create the strongest correction pressure during training.

6.5.1 Why Not Optimize Accuracy Directly?

Accuracy is often the evaluation metric students care about first, but it is a poor training objective for gradient-based learning. It changes only when a prediction crosses a threshold. That makes it too coarse for smooth optimization.

Loss functions such as cross-entropy provide richer feedback. If the true label is 1, then a predicted probability of 0.51 and a predicted probability of 0.99 are both counted as correct by accuracy, but they should not be treated as equally good. Cross-entropy captures that difference:

$$-\log(0.51) \approx 0.673, \qquad -\log(0.99) \approx 0.010.$$

The second prediction is not only correct but confidently correct, so it receives much lower loss.

Class	Logit	Softmax prob.	If this is true class
Class 1	1.4	0.654	loss contribution $-\log(0.654) \approx 0.425$
Class 2	0.5	0.266	larger loss if true
Class 3	-0.7	0.080	much larger loss if true

Table 6.2: For multiclass problems, logits are not yet probabilities. Softmax turns them into a normalized distribution, and cross-entropy trains the model to place mass on the correct class.

6.5.2 Multiclass Outputs

Later chapters will repeatedly use multiclass classification, so it is worth seeing the standard pattern now. A multiclass output layer usually produces one score per class, called a *logit*. Softmax converts those logits into probabilities:

$$\mathrm{softmax}(s)_k = \frac{e^{s_k}}{\sum_j e^{s_j}}.$$

If the true class is $k^\star$, multiclass cross-entropy penalizes the model according to

$$\mathcal{L} = -\log \mathrm{softmax}(s)_{k^\star}.$$

Math Lens. Softmax is invariant to adding the same constant to every logit. For any scalar c,

$$\begin{aligned}\mathrm{softmax}(s + c\mathbf{1})_k &= \frac{e^{s_k + c}}{\sum_j e^{s_j + c}} = \frac{e^c e^{s_k}}{e^c \sum_j e^{s_j}} \\ &= \mathrm{softmax}(s)_k.\end{aligned}$$

So only differences between logits matter, not their absolute level. This is why logits are best read as relative evidence before normalization.

6.6 Optimization as Repeated Correction

Once a loss has been defined, training becomes an optimization problem: find parameter values that reduce that loss. Gradient-

based optimization works by asking how the loss would change if each parameter were nudged slightly.

> **Prerequisite Bridge.** A derivative measures local slope. If changing a parameter a little causes the loss to rise sharply, the derivative is large. If changing it causes the loss to fall, the derivative has the opposite sign. A gradient is the collection of these slopes across many parameters.

Suppose a one-parameter model predicts $\hat{y} = wx$ and uses squared error:

$$L = (wx - y)^2.$$

Take one example with $x = 2$, $y = 6$, and current parameter $w = 1$. Then

$$\hat{y} = 2, \qquad L = (2 - 6)^2 = 16.$$

The derivative with respect to w is

$$\frac{dL}{dw} = 2(wx - y)x.$$

Substituting the current values gives

$$\frac{dL}{dw} = 2(2 - 6)(2) = -16.$$

The negative sign says that increasing w will reduce the loss. If the learning rate is $\eta = 0.1$, one gradient descent step gives

$$w_{\text{new}} = w - \eta \frac{dL}{dw} = 1 - 0.1(-16) = 2.6.$$

The operational pipeline of training is then:

- compute a forward pass,
- compute a loss,
- push error information backward,
- update parameters,
- check validation behavior.

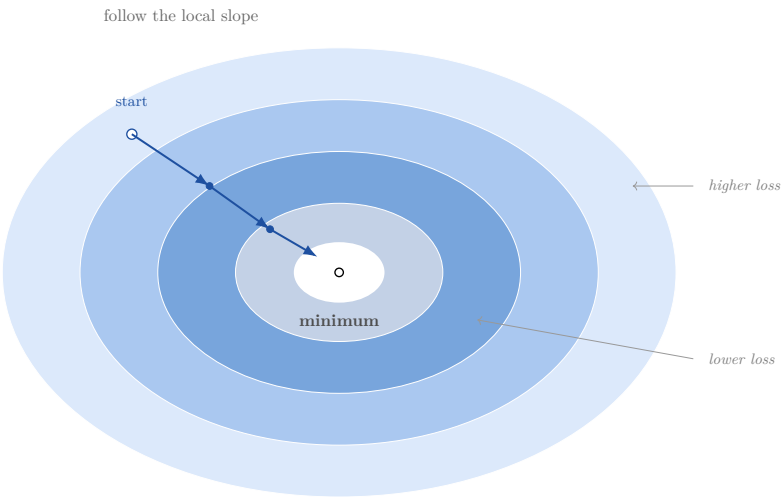


Figure 6.7: Gradient descent is repeated local correction, not global magic. Each update uses nearby slope information to move toward lower loss.

Every later training detail in the chapter—learning rate, AdamW, mini-batches, batch normalization, and regularization—modifies this loop rather than replacing it.

Listing 6.2: A generic neural training loop

```
initialize parameters
for epoch in range(num_epochs):
    for batch in training_data:
        predictions = model(batch.inputs)
        loss = objective(predictions, batch.targets)
        compute gradients of loss with respect to
            parameters
        update parameters using optimizer
    evaluate on validation data
```

6.6.1 Learning Rate

The learning rate controls step size. If it is too small, training can become painfully slow. If it is too large, optimization can overshoot good regions and become unstable. This is one reason neural training is partly a modeling problem and partly an op-

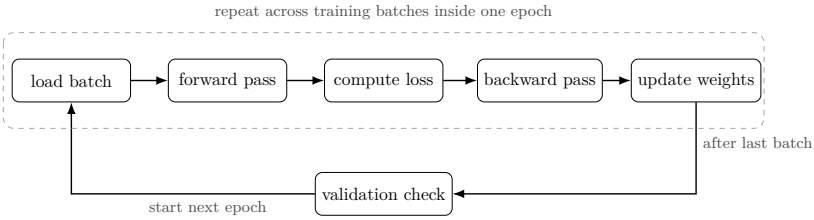


Figure 6.8: The training loop is short, but each stage has a distinct role. Batch updates repeat inside an epoch, and validation is checked after the training batches for that epoch. Students should be able to explain what changes during the forward pass, backward pass, and update step.

timization problem. The architecture may be sound while the training procedure is poor.

6.6.2 Adaptive Optimization: Momentum, Adam, and AdamW

The learning rate tells us how large a step to take, but it does not say how to shape the step when the loss surface is uneven. Adaptive optimizers try to make optimization less brittle by using recent gradient information to smooth motion or scale updates per parameter.

Momentum accumulates a running direction of travel. If successive gradients point roughly the same way, momentum keeps the update moving that way instead of restarting from scratch at every batch. If gradients alternate direction, momentum damps the zig-zag. The idea is simple: do not treat the latest gradient as the entire story.

Adam adds a second idea: keep running statistics of both the gradient and its squared magnitude. The first statistic tracks direction, and the second tracks scale. Parameters with consistently large gradients get smaller effective steps; parameters with consistently small or noisy gradients get more usable steps. In practice, this often makes early training easier and less sensitive to a single global step size.

AdamW keeps the adaptive step logic but separates weight decay

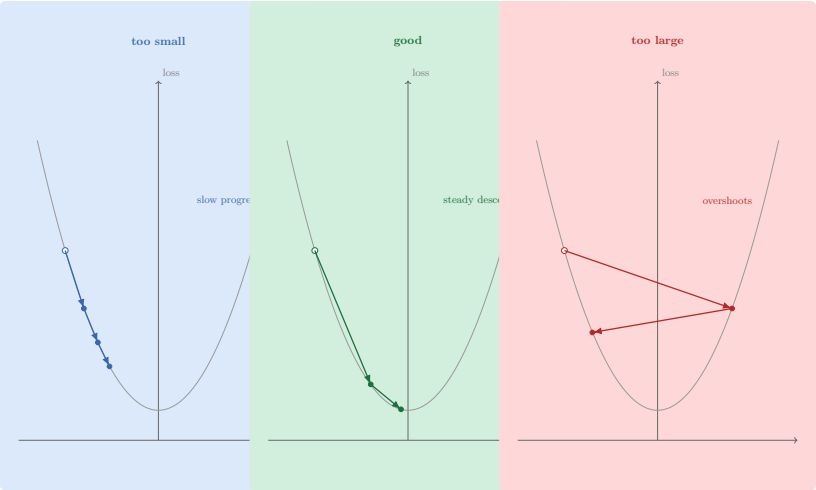


Figure 6.9: Step size is part of the training design. Too small wastes computation, while too large can bounce across good regions instead of settling into them.

from the gradient update itself. That separation matters because weight decay is a regularization choice, not a disguised learning-rate trick. Students should think of AdamW as saying: "adapt the step, but still regularize the weights in the intended way."

Example 6.2 (A One-Coordinate Optimizer Intuition)**.** Suppose one weight sees successive gradients $8, 7, 9$, while another sees $0.2, 0.1, 0.3$. Plain gradient descent uses the same global learning rate on both coordinates. Momentum treats the first sequence as persistent downhill evidence and keeps velocity in that direction instead of restarting every step. Adam goes further and rescales the coordinates differently: the large-gradient weight receives a more conservative effective step, while the small-gradient weight is not drowned out entirely by the same global rate. The point is not that Adam knows the right answer. The point is that different coordinates can require different step behavior.

Listing 6.3: A compact adaptive-update skeleton

```
initialize parameter vector $\theta$
initialize first moment $m$ and second moment $v$ to zero
for each minibatch gradient $g$:
```

```
    m = beta1 * m + (1 - beta1) * g
    v = beta2 * v + (1 - beta2) * (g * g)
    m_hat = m / (1 - beta1^t)
    v_hat = v / (1 - beta2^t)
    theta = theta - learning_rate * m_hat / (sqrt(v_hat) +
         epsilon)
    if using AdamW:
        theta = theta - learning_rate * weight_decay *
            theta
```

Practical Note. Momentum and Adam-like methods help most when optimization is noisy, poorly scaled, or sensitive to a single global step size. They do not fix label noise, leakage, a broken loss-output match, or a model that is simply too weak to represent the task.

Example 6.3 (When Adam Helps and When It Does Not)**.** Suppose a model trains erratically because different parameters receive gradients on very different scales. Adam often stabilizes that situation quickly. But if the labels are wrong or the output layer does not match the task, Adam can still produce a confident failure faster than SGD. A better optimizer is not a substitute for a correct problem formulation.

6.6.3 Optimization Failure Versus Generalization Failure

When a neural network performs poorly, the cause is usually one of two fundamentally different problems. Conflating them leads to wasted effort because the remedies are different.

Optimization failure means the model cannot fit the training signal at all. Training loss stays high, barely decreases, or fluctuates without settling. The model is not learning the mapping the data contains. Common causes include a learning rate that is too large or too small, a weight initialization that puts the network in a poor region, a mismatch between the output layer and the loss function, or label corruption that prevents any consistent rule from forming.

Failure type	Training loss	Validation loss	Where to look
Optimization failure	High	High	Learning rate, initialization, output layer, label noise
Generalization failure	Low	High	Capacity, regularization, data size, augmentation
Both	High	High	Fix optimization first, then assess generalization

Table 6.3: Diagnosing the failure type before applying remedies saves considerable effort. Regularization does not help when the model cannot fit its training data.

Generalization failure means the model fits the training data well but fails on new examples. Training loss is low; validation loss is high or diverges from training loss as training continues. The model is memorizing rather than learning a pattern that transfers. Common causes include a model with too much capacity relative to the dataset size, insufficient regularization, insufficient data augmentation, or distribution shift between training and deployment.

This distinction is foundational for debugging. Students who do not draw it clearly will add dropout when they should be fixing the output layer, or increase model capacity when they should be adding regularization. Both changes are reasonable in isolation; neither is helpful when the wrong diagnosis drives it. The first debugging reflex in this chapter should therefore be diagnostic, not architectural.

The cleanest diagnostic procedure is to first establish whether the model can overfit a very small dataset. Take a batch of ten to twenty training examples and train the model until training loss is very low. If the model cannot overfit even ten examples, the problem is optimization. If it can overfit ten examples but not generalize on the full dataset, the problem is generalization. The separation is not always perfect, but the small-batch overfit

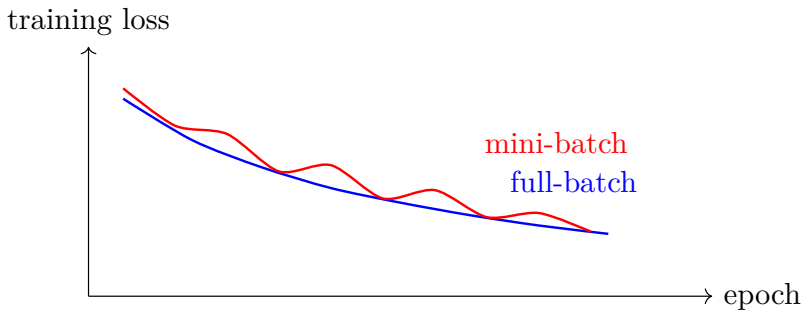


Figure 6.10: Mini-batch updates usually make optimization noisier than full-batch updates. That noise is often acceptable and sometimes beneficial, but it changes how training curves should be read.

test is the fastest way to decide which problem to solve first.

6.6.4 Mini-Batches, Initialization, and Stability

In principle, gradients could be computed using the entire training set before every update. In practice, that is often too expensive. Instead, we use mini-batches: small subsets of examples that produce noisy but useful gradient estimates. This gives rise to stochastic gradient descent and its many variants.

Training also does not begin from nowhere. Parameters must be initialized, inputs must be scaled sensibly, and layers must receive signals of a manageable size. Bad initialization can produce activations that are too small, too large, or too similar, making optimization slow or unstable from the first step.

> **Math Lens.** If two hidden units start with exactly the same weights and bias, then they compute the same pre-activation on every example, produce the same activation, and receive the same gradient update. Gradient descent preserves that equality, so the two units never specialize; the network learns one feature twice. Random initialization breaks that symmetry by giving each unit a slightly different starting point, which lets different hidden units move toward different useful features.

Example 6.4 (Three Bad Starts)**.** If every hidden unit starts at zero, the layer stays symmetric and learns duplicate features. If the initial weights are extremely small, signals shrink as they pass through many layers and learning becomes sluggish. If the initial weights are extremely large, activations or gradients can explode or saturate. Good initialization is therefore about scale and diversity at the same time.

> **Warning.** When students say "the network did not work," the failure may be in the learning rate, initialization, batch setup, or data preprocessing rather than in the idea of the model itself.

> **Practical Note.** If a neural model fails early, check simple stability questions before redesigning the architecture: Were the inputs normalized? Is the learning rate reasonable? Are the initial weights making activations explode or vanish? Is the batch size so small that the gradients are mostly noise?

6.6.5 Batch Normalization as Signal Stabilization

Batch normalization is another way to make optimization less brittle. It normalizes intermediate activations using statistics computed from the current mini-batch, then restores flexibility with learned scale and shift parameters. The intuition is that if the hidden signals stay in a manageable range, the next layers see a more stable training problem.

That description is enough for this book. Students do not need to memorize the exact implementation before they understand the purpose: batch normalization reduces internal scale drift, which can make optimization smoother and allow somewhat larger learning rates. It is a training aid, not a cure-all.

> **Math Lens.** For mini-batch activations $a_1, \ldots, a_m$, batch normalization computes

$$\hat{a}_i = \frac{a_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y_i = \gamma \hat{a}_i + \beta,$$

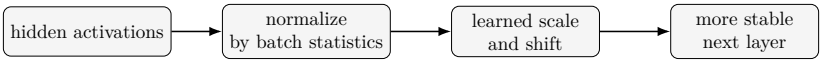


Figure 6.11: Batch normalization aims to stabilize the signal passing through the network so later layers do not have to adapt to wildly changing activation scales.

where μ_B and σ_B^2 are the batch mean and variance, and γ, β are learned scale and shift parameters. The normalization stabilizes the signal; the learned affine step keeps the layer expressive.

Example 6.5 (What Batch Normalization Fixes, and What It Does Not)**.** If hidden activations are drifting in scale from batch to batch, batch normalization can make training smoother and less sensitive to the exact learning rate. For a mini-batch with activations $2, 4, 6$, the layer first recenters them around zero before the learned rescaling step. The next layer therefore reacts to relative variation rather than raw drift in magnitude. But batch normalization still does not fix a wrong objective, a poor label set, or a model that lacks the capacity to learn the task in the first place.

Practical Note. Batch normalization stabilizes the signal within a layer. A complementary technique called a *residual connection* (or skip connection) helps the signal flow between layers. The idea is simple: instead of computing $h = f(x)$, the layer computes $h = f(x) + x$, adding the input directly to the output. If the transformation f is unhelpful, the gradient can flow through the identity branch unchanged, so deep networks become much easier to train. Residual connections are a key design principle behind ResNets in vision and many transformer blocks in language.

6.6.6 Debugging a Broken Training Run

Students often respond to a bad neural result by changing the architecture first. That is usually too early. The more disciplined first question is: what kind of failure are we seeing?

Observed pattern	First interpretation to test	Better first response than "change the architecture"
Training loss hardly moves	optimization setup may be broken	check loss-output alignment, learning rate, normalization, and whether the model can overfit one tiny batch
Training loss falls but validation is poor	optimization is working but generalization is weak	inspect data quality, regularization, augmentation, label noise, and evaluation split logic
Loss becomes NaN or oscillates wildly	updates are unstable	lower the learning rate, inspect initialization, check activation scale, and verify preprocessing
Training and validation both look strangely strong immediately	leakage or target mismatch may exist	audit splits, labels, duplicated examples, and feature construction before celebrating

Table 6.4: Neural debugging improves when students distinguish optimization failure from generalization failure instead of treating every weak result as an architecture problem.

This distinction is foundational. If the network cannot even reduce the training objective, then there is no point debating whether it overfits. If it fits training well but collapses on validation, then the training loop is not the first suspect. Different failures demand different questions.

Listing 6.4: A minimal debugging sequence before redesigning the network

```
pick one tiny batch of examples
verify that targets, output layer, and loss function match
try to overfit that tiny batch nearly perfectly
if this fails:
```

```
    inspect learning rate, normalization, initialization,
        and gradient scale
if this succeeds but validation is weak:
    inspect data quality, slice coverage, regularization,
        and leakage
change one training choice at a time and record the effect
```

The tiny-batch test is especially valuable because it removes excuses. A reasonably implemented network with aligned outputs and targets should usually be able to memorize a very small batch. If it cannot, the problem is often in the training setup rather than in the grand modeling idea.

> **Decision Box.** Before increasing model depth or width, ask whether the run failed because the model could not fit the training signal at all, or because it fit the training signal and failed to generalize. Those are different problems and should trigger different interventions.

> **Problem-Solving Technique. Tempting mistake → corrected reasoning.** Tempting mistake: "Validation is bad, so the architecture must be wrong." Corrected reasoning: "First separate optimization failure from generalization failure. If the network cannot overfit a tiny batch, debug the training setup. If it can, inspect data quality, split logic, regularization, and evaluation before redesigning the model family."

6.7 Backpropagation as Credit Assignment

Definition 6.3 (Backpropagation)**.** Backpropagation is the efficient procedure for computing how the loss changes with respect to every parameter in a layered network by repeatedly applying the chain rule from the output layer backward.

Conceptually, backpropagation solves one clear problem: when the network makes an error, which earlier computations deserve credit or blame, and by how much?

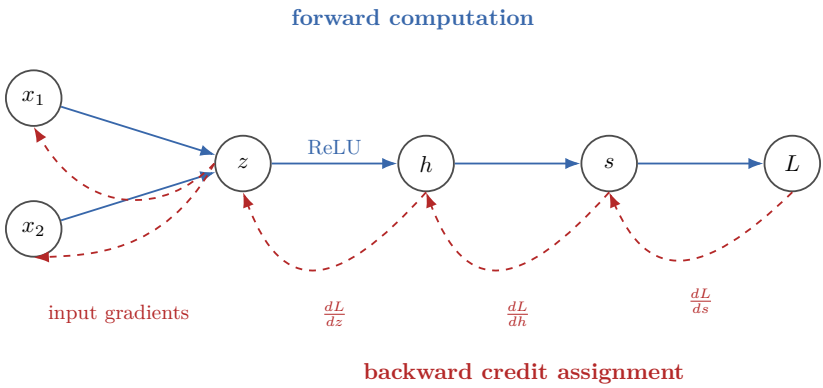


Figure 6.12: Backpropagation is easiest to understand as a computational graph. The forward pass creates values; the backward pass distributes the loss signal through those same dependencies in reverse.

6.7.1 A Small Worked Gradient Example

Consider a network with one ReLU hidden unit:

$$z = w_1 x_1 + w_2 x_2 + b_h, \qquad h = \mathrm{ReLU}(z),$$

$$s = vh + b_o, \qquad L = \frac{1}{2}(s - y)^2.$$

Take one example with input $(x_1, x_2) = (2, 1)$ and target $y = 1$. Let the current parameter values be

$$w_1 = 0.5, \qquad w_2 = 1.0, \qquad b_h = -1.0,$$
$$v = 1.2, \qquad b_o = -0.3.$$

First perform the forward pass:

$$z = 0.5(2) + 1.0(1) - 1.0 = 1, \qquad h = \mathrm{ReLU}(1) = 1,$$

$$s = 1.2(1) - 0.3 = 0.9, \qquad L = \frac{1}{2}(0.9 - 1)^2 = 0.005.$$

Now move backward through the computation. Start with the loss derivative with respect to the score:

$$\frac{dL}{ds} = s - y = 0.9 - 1 = -0.1.$$

This says increasing the score slightly would reduce the loss.

The output weight v multiplies h, so

$$\frac{dL}{dv} = \frac{dL}{ds} \cdot h = (-0.1)(1) = -0.1.$$

The hidden activation affects the loss through the output layer, so

$$\frac{dL}{dh} = \frac{dL}{ds} \cdot v = (-0.1)(1.2) = -0.12.$$

Because $z = 1 > 0$, the derivative of ReLU at this point is 1, so

$$\frac{dL}{dz} = \frac{dL}{dh} \cdot 1 = -0.12.$$

Now the hidden-layer parameters follow directly:

$$\frac{dL}{dw_1} = \frac{dL}{dz} \cdot x_1 = (-0.12)(2) = -0.24,$$

$$\frac{dL}{dw_2} = \frac{dL}{dz} \cdot x_2 = (-0.12)(1) = -0.12,$$

$$\frac{dL}{db_h} = \frac{dL}{dz} = -0.12.$$

The output bias behaves similarly:

$$\frac{dL}{db_o} = \frac{dL}{ds} = -0.1.$$

This is backpropagation in miniature. The error signal begins at the loss, flows backward through the output computation, then through the activation, and finally into the earlier weights and bias.

To make that chain feel like learning rather than bookkeeping, take one gradient-descent step with learning rate $\eta = 0.05$:

$$w_1' = 0.512, \quad w_2' = 1.006,$$

$$b_h' = -0.994, \quad v' = 1.205,$$

$$b_o' = -0.3 - 0.05(-0.1) = -0.295.$$

Now rerun the same example with the updated parameters:

$$z' = 0.512(2) + 1.006(1) - 0.994 = 1.036,$$
$$h' = \mathrm{ReLU}(1.036) = 1.036,$$

$$s' = 1.205(1.036) - 0.295 \approx 0.953,$$
$$L' = \tfrac{1}{2}(0.953 - 1)^2 \approx 0.0011.$$

The loss fell from 0.005 to about 0.0011 after one step. That does not prove the network will generalize, but it does show the operational meaning of the gradients we just computed: they point toward a local parameter change that improves this example.

6.7.2 What Changes If ReLU Is Off?

Now imagine the same network but with a hidden pre-activation $z = -0.5$. Then $h = \mathrm{ReLU}(-0.5) = 0$, and the local derivative of ReLU is 0 at that point in the usual teaching approximation. The backward chain therefore gives

$$\frac{dL}{dz} = \frac{dL}{dh} \cdot 0 = 0.$$

The gradients into the earlier hidden-layer weights also become zero for that example. This is a useful practical fact: whether a ReLU unit is active changes not just the forward value, but whether gradient signal can flow through that pathway on that example.

> **Failure Box.** Backpropagation does not guarantee good learning. It only tells us the local direction of improvement. If the data is weak, the model is badly chosen, the learning rate is poor, or the objective is mismatched to the task, the network can still fail badly while all gradients are computed correctly.

> **Advanced Note.** In real implementations, these gradient calculations are handled by automatic differentiation systems. That convenience is useful, but students should still understand the manual logic. Otherwise training becomes a black box: loss goes in, gradients come out, and no one can explain why.

Listing 6.5: From scratch: one backpropagation step through the output layer

```
# Continue from the forward pass above.
# Restate the values computed there for clarity.
# h1, h2: hidden unit activations (post-ReLU)
# v1, v2, b_out: output-layer weights and bias
# s: output score (pre-sigmoid); p: predicted probability
# y: true label (1 = positive class)
y = 1

# --- Binary cross-entropy loss (single example) ---
# L = -(y * log(p) + (1-y) * log(1-p))
import math
loss = -(y * math.log(p) + (1 - y) * math.log(1 - p))
print("binary cross-entropy loss: ", round(loss, 4))

# --- Gradient at the output layer ---
# For sigmoid + cross-entropy the combined gradient
    simplifies to (p - y).
dL_ds = p - y # dL/ds: how the loss changes with the pre-
    sigmoid score
print("dL/ds (output gradient): ", round(dL_ds, 4))

# Gradient w.r.t. output weight v1 (chain rule: dL/dv1 =
    dL/ds * ds/dv1 = dL/ds * h1)
dL_dv1 = dL_ds * h1 # credit assigned to v1
print("dL/dv1: ", round(dL_dv1, 4))

# --- Gradient flowing back to hidden unit 1 ---
# dL/dh1 = dL/ds * ds/dh1 = dL/ds * v1
dL_dh1 = dL_ds * v1 # error signal arriving at hidden unit
     1
print("dL/dh1: ", round(dL_dh1, 4))

# Pass through ReLU: derivative is 1 if h1 > 0, else 0
relu_gate = 1.0 if h1 > 0 else 0.0 # local gradient of
    ReLU
dL_dz1 = dL_dh1 * relu_gate # dL/dz1
print("dL/dz1 (through ReLU gate): ", round(dL_dz1, 4))

# Gradient w.r.t. the hidden weight w11 (dL/dw11 = dL/dz1
    * dz1/dw11 = dL/dz1 * x1)
dL_dw11 = dL_dz1 * x1
```

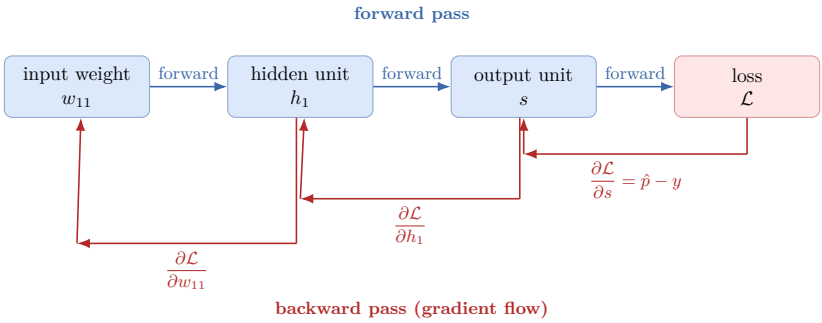


Figure 6.13: Backpropagation is credit assignment flowing backward through the network. Each arrow carries the gradient signal that tells one weight how much to adjust.

```
print("dL/dw11: ", round(dL_dw11, 4))

# --- One gradient-descent update (learning rate = 0.1)
    ---
learning_rate = 0.1
new_v1 = v1 - learning_rate * dL_dv1 # output weight
    update
new_w11 = w11 - learning_rate * dL_dw11 # hidden weight
    update
print("updated v1: ", round(new_v1, 4))
print("updated w11: ", round(new_w11, 4))
```

6.8 Generalization Returns: Regularization as Bias Toward Stable Patterns

Neural networks are flexible enough to fit very complicated patterns. That flexibility is powerful and dangerous. A large network can capture real structure, but it can also memorize accidents of the training set.

The Chapter 3 question returns in sharper form: are we reducing training loss in a way that helps on unseen data, or are we merely fitting noise more effectively?

The best way to understand regularization in this chapter is not as a list of tricks. It is as a bias toward less brittle learned

representations. These tools matter after the model can learn; they are not substitutes for fixing a broken optimization setup.

6.8.1 Weight Decay, Early Stopping, Dropout, and Augmentation

Several common tools are used to control overfitting:

- **weight decay** discourages large parameter values by penalizing them during optimization
- **early stopping** halts training when validation performance stops improving
- **dropout** randomly suppresses some activations during training so the network does not rely too heavily on any one pathway
- **data augmentation** expands effective data variety by transforming inputs in label-preserving ways

These techniques are different in mechanism but similar in purpose. They push the network toward patterns that are stable enough to survive outside the training sample.

Dropout deserves one extra caution. It is useful because it prevents a network from depending too heavily on one hidden pathway, but it does not magically create information that is absent from the data. If the core representation is wrong, dropout only regularizes a bad model.

Example 6.6 (What Dropout Actually Changes)**.** Suppose a hidden layer currently produces activations $(1.2, 0.9, 0.0, 1.1)$. With dropout rate 0.5, one training step might keep only the first and fourth activations, while another might keep the second and fourth. The next layer therefore cannot rely on one fragile path always being present. Over many updates, the network is pushed to spread evidence across several features. At inference time, all units are available again, usually with a compensating rescaling so the expected signal size stays comparable.

Example 6.7 (A Familiar Tradeoff)**.** Suppose an augmented im-

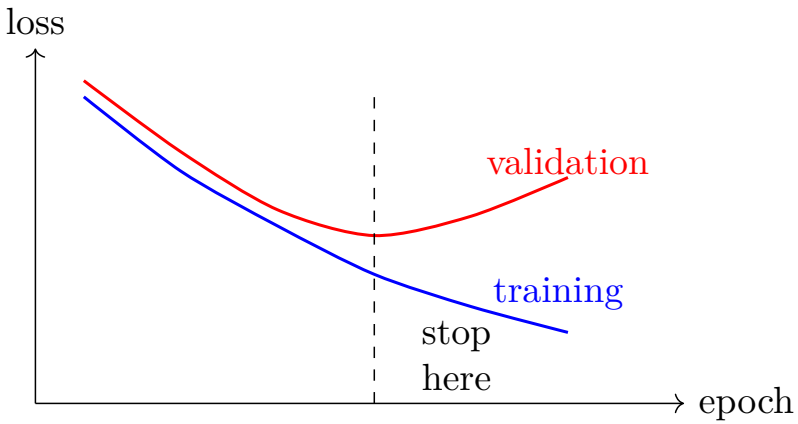


Figure 6.14: Early stopping marks the point where additional training mainly improves fit to the seen data rather than generalization to new data.

Figure 6.15: These regularization tools work differently, but they all try to bias learning toward patterns that remain useful outside the training sample.

age model scores slightly lower on clean validation images than a non-augmented model, but performs much better on shifted or noisy images. That is not a contradiction. It means the model traded some specialization for robustness. Honest reporting should describe both sides of that tradeoff.

6.9 Capacity, Data Regime, and Label Noise

"Use a bigger model" is not a universal recommendation. Model capacity interacts with dataset size, label quality, and augmentation quality in ways that make the same capacity increase beneficial in one regime and harmful in another.

Large capacity with sufficient clean data. When the dataset is large and labels are reliable, a higher-capacity model can find

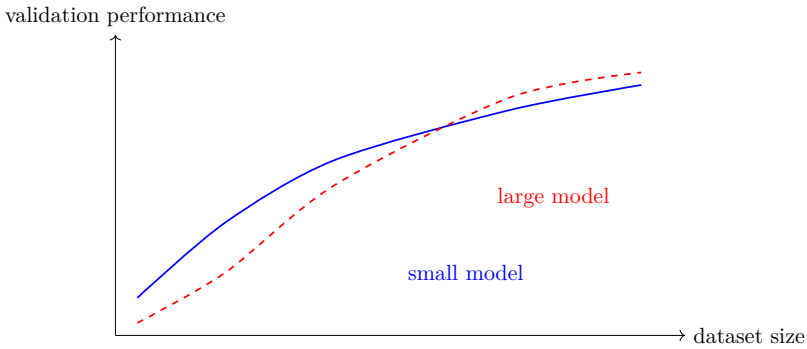


Figure 6.16: The benefit of increased model capacity depends on data scale. A larger model does not uniformly outperform a smaller one; it does so reliably only when the training set is large enough to prevent memorization.

finer structure and use its additional parameters productively. Adding layers, widening the network, or removing regularization may all improve performance in this regime because the training signal is abundant and trustworthy.

Large capacity with limited clean data. When the dataset is small or labels are sparse, additional capacity often leads to memorization rather than generalization. The model learns idiosyncratic patterns of the training examples instead of the underlying structure. In this regime, regularization, early stopping, data augmentation, and smaller architectures tend to help more than increasing capacity.

High label noise. When labels contain substantial errors—from careless annotation, ambiguous task definitions, or label leakage—a higher-capacity model is more likely to memorize those errors than a lower-capacity one. The large model has the flexibility to overfit the noise; the small model may not. Label quality should be assessed and improved before architectural decisions are made, because adding layers to fix a label-noise problem usually makes the problem worse.

Example 6.8 (Capacity Decision on a Small Medical Dataset)**.** A team has a labeled dataset of 800 patient records for a readmission prediction task. They train a six-layer transformer and

achieve very low training loss but poor validation performance. Adding dropout and reducing to two layers closes the gap substantially. The larger model had the capacity to overfit the 800 examples; the smaller model was forced to learn a more general pattern. The improvement had nothing to do with the learning rate or optimizer.

The operational summary is that model size should be chosen in conversation with data size and label quality. The right question before adding capacity is: does this training set contain enough reliable signal to fill these additional parameters? If yes, larger may help. If no, regularization and data quality improvements come first.

> **Problem-Solving Technique. Capacity audit.** Before increasing model size, ask three questions: How large is the labeled training set? How clean are the labels? How much useful augmentation is available? If the answers are "small," "noisy," and "limited," adding capacity is unlikely to help and may harm. Fix the data first.

6.10 Chapter Artifact: A Minimum Credible Neural Training Report

Real neural training involves more than equations on a board. Several practical choices strongly affect outcomes. By the end of this chapter, a reader should be able to produce a minimum credible training report before making a serious neural-network claim. This is the document that keeps flexible neural models tied back to baselines and evidence instead of hype. The framing memo named the decision, the metric worksheet named good behavior, the evaluation plan protected the claim, the baseline sheet justified linearity, and the structure diagnosis sheet justified leaving it. This report justifies the learned-model claim itself.

> **Problem-Solving Technique. Minimum credible neural training report.** Before reporting a neural result, write down the data split, preprocess-

Item	What should be reported
Seven-question focus	Questions 4, 5, and 6: what representation the network sees, what baseline it must beat, and what training evidence makes the neural claim credible
Data split	train / validation / test sizes and how they were chosen
Preprocessing	normalization, tokenization, resizing, augmentation, or other transforms
Architecture	model family, hidden sizes, depth, and output layer
Loss and metric	training objective plus the main evaluation metric
Decision rule	the threshold or ranking rule used to turn scores into actions
Calibration	whether probabilities were calibrated before interpretation, and by what method
Optimizer	SGD, Adam, AdamW, or another method, with justification if possible
Learning rate and batch size	the main optimization controls
Stopping rule	fixed epochs, early stopping, or another criterion
Randomness	number of seeds or repeated runs
Baseline	the simpler model used for comparison

Table 6.5: This is a minimum credible training report, not an idealized one. Bachelor-level project work improves immediately when these items are stated clearly instead of being left as library defaults.

ing, architecture, loss, optimizer, learning rate, batch size, stopping rule, randomness, and baseline comparison.

Popular optimizers such as Adam and AdamW often make training easier in practice than plain stochastic gradient descent, especially for smaller research prototypes and teaching examples. The educational lesson is not that one optimizer has permanently won. It is that optimizer choice is part of the experiment and must be justified empirically.

Optimizer	Main intuition	What students should remember
SGD	follow noisy gradients directly	simple, still important, often benefits from careful tuning and schedules
Adam / AdamW	adapt step sizes using running gradient statistics	often easier to train with at first, but still requires validation and does not replace good data practice

Table 6.6: Optimizer choice is part of the training setup, not a cosmetic library setting. The right comparison is empirical, not tribal.

6.11 Representation Learning as the Payoff

The most important reason neural networks changed AI is not simply that they fit nonlinear functions. Trees and kernels already offered nonlinearity. The deeper change is that hidden layers can learn representations that make downstream tasks easier.

An image classifier does not begin with hand-coded edge counters written by the user. A neural system can learn early filters, then combine them into motifs, parts, and larger semantic patterns. A language model can learn embeddings in which some relationships become geometrically meaningful. An audio model can learn internal features that capture frequency patterns, rhythm, and temporal context. In each case, the network is learning the coordinates in which the task becomes more manageable.

This is the culmination of the chapter. Optimization is not only fitting a final boundary. It is teaching the model which internal coordinates make the task easier.

This is also the bridge to the next chapter. This chapter learned features for one task through one training loop. The next chapter asks when those learned spaces become reusable assets across tasks, domains, and label budgets. Once representation learning becomes strong enough, we can pretrain models on broad data and reuse those learned spaces across many downstream tasks.

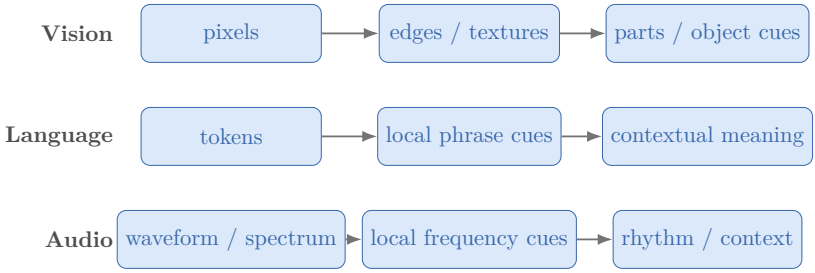


Figure 6.17: Representation learning is the real bridge to the next chapter. The model is not merely drawing a final boundary; it is building the space in which the final boundary becomes easier to draw.

That is the foundation-model story in one sentence.

6.12 Common Mistakes with Neural Networks

The mistakes in this section are not merely "deep-learning mistakes." They are failures of scientific judgment under flexible models.

Mistake Checklist.

- **Treating architecture as the whole story.** Data quality, preprocessing, metrics, and split design matter just as much.
- **Ignoring simpler baselines.** If a neural network beats no baseline, the result is uninterpretable.
- **Overreading training curves.** A single clean-looking training run proves little.
- **Changing too many training choices at once.** Change a manageable number of factors at a time.
- **Confusing confidence with understanding.** High confidence is not robustness, interpretability, or causal insight.

- **Thinking bigger automatically means better.** Larger models are justified only when they produce better evidence.

6.13 Looking Ahead

This chapter explained how neural learning works: hidden features, loss, backpropagation, and regularization. The next chapter asks when those learned spaces are reusable across tasks and how to tell whether transfer is really helping.

Chapter Summary

- This chapter answers one question: when should structure be learned rather than engineered, and how does optimization teach a network to build it?
- A neural network is best understood as a learned feature pipeline rather than as a bag of layers and tricks.
- Nonlinearity matters because without it a deep network collapses back into one larger affine map, so depth alone does not create learned structure.
- Losses provide the teaching signal for optimization, while evaluation metrics remain the tools for judgment.
- Gradient descent is repeated local correction, and backpropagation is the mechanism that assigns credit or blame through the network.
- Optimization choices and regularization are part of the experiment, which is why the minimum credible neural training report is part of the chapter's core discipline rather than an appendix.
- Weight decay, early stopping, dropout, and augmentation all try to bias learning toward patterns that remain stable on new data.

- The payoff of the chapter is representation learning: optimization is teaching the model which internal coordinates make the task easier.

Study Questions

1. Why is "the useful internal representation is unknown" a better opening problem than "deep learning is powerful"?
2. Why does a deep network without nonlinear activations still behave like a linear model?
3. Why is cross-entropy usually a better teaching signal than accuracy for binary classification?
4. What role does the learning rate play in repeated correction?
5. What problem does backpropagation solve conceptually?
6. Why is representation learning more fundamental than merely saying that neural networks are nonlinear?

Exercises

1. Compute the forward pass of a one-hidden-layer network with input $x = [1, 2]$, hidden unit $h = \mathrm{ReLU}(2x_1 - x_2 - 1)$, and output score $s = 1.5h - 0.5$. Give the final score and the sigmoid probability.
2. Evaluate sigmoid, tanh, and ReLU at $z = -2$, $z = 0$, and $z = 2$. What qualitative differences do you observe?
3. Suppose the true label is 1 and two models predict probabilities 0.55 and 0.95. Compute the binary cross-entropy loss for both and explain why the difference matters even though both predictions are correct at a 0.5 threshold.
4. A multiclass classifier outputs logits $(1.0, 0.0, -1.0)$. Compute the softmax probabilities approximately and state the cross-entropy loss if class 1 is the true class.

5. For the scalar model $\hat{y} = wx$, let $x = 3$, $y = 9$, and current $w = 1$. Using squared error loss, compute the gradient with respect to w and perform one gradient descent update with learning rate 0.05.

6. Explain in words why backpropagation is a credit-assignment procedure rather than just a formula-manipulation trick.

7. Using Table 6.5, sketch a minimum credible training report for a small binary classifier and name the simpler baseline it must beat. If you are carrying one case through the book, use the same case as your structure diagnosis sheet and state which failure pattern justified moving to a neural model.

8. Give one example of a task for which a neural network may be justified because representation learning matters, and one example of a task where a simpler model may still be preferable. State what evidence would decide between them.

9. A training run shows steadily falling training loss but worsening validation performance after epoch 12. Name the failure type, one sensible intervention, and whether the next move should be better regularization, a better baseline comparison, or a transfer-learning workflow from Chapter 7.

Challenge Exercises

1. Consider a two-layer network with no activation between layers. Write the full expression for the output and show algebraically why it can be rewritten as one affine transformation.

2. In the worked backpropagation example, recompute the gradients if the hidden pre-activation had been $z = -0.5$ instead of $z = 1$. What changes, and what does that say about ReLU units?

3. A model family with many parameters performs worse than logistic regression on a small text-classification dataset. Give three technically plausible explanations that do not reduce to "neural networks are bad."

4. Pick one public learning task of your choice. Propose a minimum neural training plan: output layer, loss, optimizer, batch size, validation strategy, and baseline.

5. Two neural runs differ in architecture, optimizer, batch size, and augmentation policy, and one of them scores higher by one percentage point on the validation set. Explain why this is not yet convincing evidence of a better model.

Chapter 7

Representation Learning and Foundation Models

Learning Goals

- Explain when it is worth learning a reusable representation instead of solving only one task directly.
- Use the "change of coordinates" mental model to explain why a learned space can make downstream tasks easier.
- Understand embeddings, cosine similarity, and linear probes as tools for studying reusable representations rather than as isolated techniques.
- Distinguish supervised, self-supervised, multimodal, and transfer-learning pressure in terms of what kind of reusable structure they encourage.
- Choose among frozen probes, partial tuning, and full fine-tuning based on label budget, domain shift, and evidence needs.
- Produce a reuse and adaptation plan before making a strong transfer or foundation-model claim.

The previous chapter explained how neural networks learn inter-

nal features through layered optimization. That naturally raises the next question: when are those learned features valuable *beyond one task*? This chapter answers that question.

The shift is larger than it first appears. In classical machine learning, a model is often trained for one task on one dataset with one objective. Representation learning changes the unit of value. What matters is not only whether the model solves today's task, but whether it learns a space that remains useful for tomorrow's tasks as well. Foundation-model workflows push this further by pretraining for reuse first and adapting later. The modern form of this idea draws on work on representation learning as exposed structure, contextual word representations, latent prediction, attention-based sequence modeling, bidirectional pretraining, and foundation-model analysis [4, 7, 17, 45, 49, 60, 61]. The chapter follows that same path: first what a reusable space is, then what kinds of training pressure create it, then how probes and adaptation test it, and finally how attention and foundation-model workflows scale the same logic.

> **Problem-Solving Lens.** This chapter asks when it is worth learning a reusable representation instead of solving one task directly, and how much downstream adaptation that reusable representation really justifies. The central discipline is to separate reusable representation, adaptation strategy, and evidence for the source of improvement.

7.1 Opening Dilemma: Solve One Task, or Learn a Reusable Space?

The previous chapter ended by showing that neural networks can learn internal features instead of relying only on hand-built ones. This chapter begins where that idea becomes strategically important. Sometimes the real value of a model is not that it solves one task well, but that it learns a space in which many related tasks become easier.

This is the opening dilemma of the chapter. Should we solve

a task directly from scratch, or should we invest in learning a reusable representation that may help across tasks, domains, or label budgets?

That question matters whenever the useful structure is expensive to relearn from scratch. A model trained only for one classifier may work well today and still throw away value that could have helped retrieval, clustering, transfer, or later fine-tuning. A reusable representation changes what we mean by progress. The rest of the chapter turns that claim into a workflow: define the space, ask what pressure created it, test it with the cleanest downstream diagnostic, and only then decide how much adaptation is justified.

> **Decision Box.** Invest in reusable representation learning when the raw input is too entangled for simple task-specific modeling and when the learned space is likely to be reused across tasks, domains, or limited-label settings.

By the end of the chapter, the reader should be able to write a reuse and adaptation plan before claiming that transfer helped.

> **Practical Note.** Workflow sketch: identify the source representation, test it first with a frozen-feature or linear-probe diagnostic, compare against a retrieval-only or prompting-only baseline when relevant, choose the shallowest adaptation that still addresses label budget and domain shift, and write down alternative explanations for any gain.

> **Problem-Solving Technique. Artifact preview: reuse and adaptation plan.** Keep five fields active while reading: what pretrained object is being reused, what first clean diagnostic isolates reuse, what adaptation depth is the smallest justified move, what external evidence the workflow still needs, and what alternative explanation could fake the gain.

> **Decision Box. What should change first in a reuse workflow?**

- To test whether the pretrained space already exposes the signal, start with a linear probe or frozen features.
- If correctness depends on current external evidence, add retrieval before blaming the representation.
- If the representation seems useful but the task is moderately shifted, prefer the shallowest tuning that can fix the mismatch.
- Reserve full fine-tuning for large domain shift or strong labeled evidence.

The chapter's key question can therefore be stated plainly:

When is it worth learning a reusable representation instead of solving only one task directly, and how do we know the reusable representation is actually what is helping?

7.2 Representation as a Change of Coordinates

The best mental model for a representation is a change of coordinates. It is also the mental model that holds the rest of the chapter together: if the coordinates improve, simple downstream readouts, transfer diagnostics, and later adaptation should all become easier for a reason we can inspect.

Definition 7.1 (Representation Learning)**.** Representation learning is the process of learning transformations of data that make useful structure easier to detect, compare, transfer, or predict from.

In one coordinate system, classes may overlap, neighborhoods may be meaningless, and prediction may require many labels. In another coordinate system, examples that "mean the same thing" may cluster naturally and simple downstream decisions may become effective.

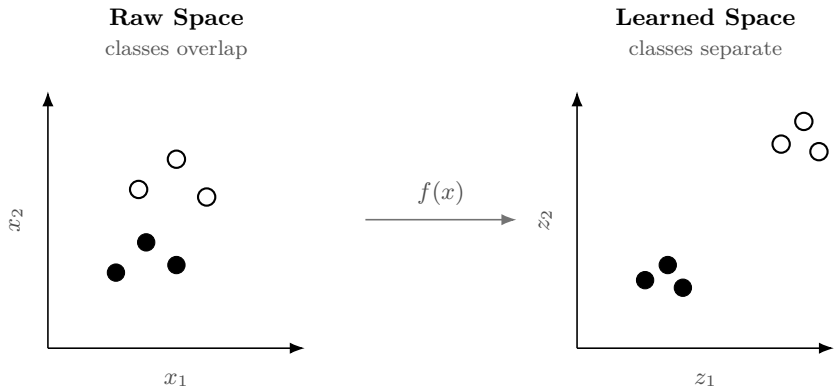


Figure 7.1: The central promise of representation learning is not magical. It is the possibility of mapping a hard raw space into a more useful space where simpler downstream decisions work better.

This is why a weak downstream model on top of a strong representation can outperform a strong downstream model built on weak inputs. The representation may already have done most of the conceptual work.

7.3 What Makes a Representation Reusable?

Not every learned representation transfers to new tasks. A representation is *reusable* in a meaningful sense only when it satisfies several conditions at once. Naming these conditions makes the concept durable beyond any specific model family. These conditions also become the chapter's checklist for deciding whether downstream success is evidence of reuse or only evidence that a later adaptation stage compensated for a weak source space.

Invariances captured. A reusable visual representation should be roughly invariant to the specific lighting conditions, camera angle, and background of the training images, since those factors do not define the content. A reusable language representation should be roughly invariant to paraphrase: "the student submitted late" and "the assignment was turned in after the deadline" should be close in a representation meant to capture meaning. A

Raw input	Useful intermediate structure	Why downstream tasks get easier
Pixels	edges, textures, parts, object cues	classification or detection can operate on visual concepts instead of raw intensities
Tokens	phrase patterns, context, semantics	language tasks can reason about meaning rather than only symbol identity
Spectrograms / sensor traces	local frequency cues, rhythm, temporal context	forecasting or recognition can use stable temporal structure instead of raw measurements

Table 7.1: A good representation does not magically solve the task. It rearranges the space so that simpler downstream decisions become possible.

representation that has learned the right invariances can transfer to new examples that share the same semantic content under different surface forms.

Information preserved. A representation that compresses too aggressively may throw away information that a new task needs. A representation designed for topic classification that discards fine-grained syntax may not be reusable for grammatical error detection. Reuse requires that the relevant information is still present in the space, even if it was not the primary target during pretraining.

Task mismatch tolerated. The pretraining task and the new task are never exactly the same. A representation pretrained on predicting masked words may not encode visual layout, even if it was pretrained on captioned images. Reuse is easier when the pretraining task was broad enough to touch the relevant signals and when the mismatch between source and target objectives is modest.

Adaptation cost contained. A representation that requires full fine-tuning on thousands of labeled examples to be useful

Reusability condition	What to check
Invariances captured	Does the space stay stable under transformations that should not change the label?
Information preserved	Does a probe on frozen features show that the target signal is accessible?
Task mismatch tolerated	How different is the pretraining objective from the new task semantically?
Adaptation cost contained	Can a linear probe or small adapter achieve competitive performance?

Table 7.2: Four conditions for meaningful representation reuse. Meeting all four is not always necessary, but checking all four is useful before committing to a reuse-first workflow.

for a new task is reusable in a narrow sense, but the practical barrier is high. Reuse is most valuable when a small labeled set, a linear probe, or a lightweight adapter is sufficient. This is the operational meaning of the transfer learning promise: the pretrained space should reduce the labeled-data burden for new tasks.

This framing is more durable than naming current model families because it applies equally to word embeddings, visual encoders, audio representations, and tabular feature extractors. Whatever the source architecture, the reuse question is always: which invariances did it learn, what information does it still carry, how far is the source task from the target, and how much new labeled data does adaptation require?

7.4 Embeddings and Geometry: Useful Space, Still a Model Choice

Definition 7.2 (Embedding)**.** An embedding is a learned vector representation of an object such as a word, image patch, document, audio segment, or user event, placed in a continuous space where geometric relationships are intended to capture useful structure.

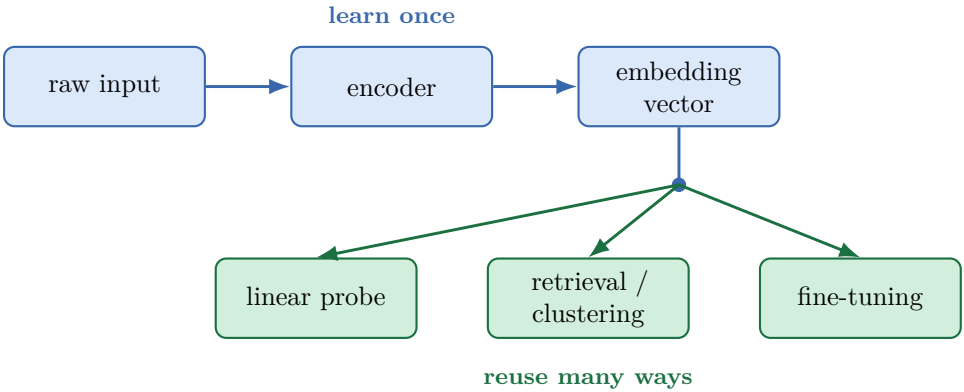


Figure 7.2: A reusable representation is valuable because the same encoder and embedding space can support several downstream operations: simple probes, retrieval, clustering, or more task-specific adaptation.

Embeddings are one of the clearest examples of reusable representation learning. Instead of treating each object as an isolated identifier, we map it to a vector. That geometric view supports retrieval, clustering, similarity search, and transfer. It is the first place where the change-of-coordinates idea becomes directly inspectable.

7.4.1 A Tiny Embedding Example

Suppose a system has learned the following two-dimensional embeddings:

$$e_{\text{exam}} = \begin{bmatrix} 0.9 \\ 0.7 \end{bmatrix}, \qquad e_{\text{quiz}} = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix}, \qquad e_{\text{guitar}} = \begin{bmatrix} 0.1 \\ 0.95 \end{bmatrix}.$$

The dot product between *exam* and *quiz* is

$$0.9(0.8) + 0.7(0.6) = 1.14,$$

while the dot product between *exam* and *guitar* is

$$0.9(0.1) + 0.7(0.95) = 0.755.$$

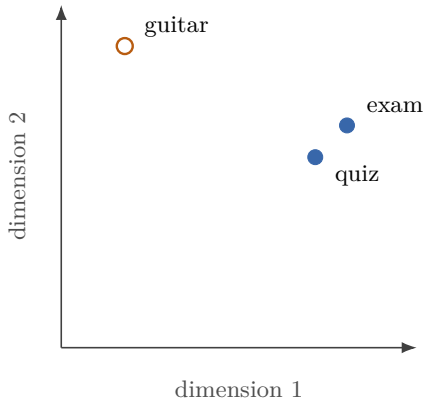


Figure 7.3: Even a toy embedding plot makes the chapter's geometric point visible. Related terms occupy nearby directions; unrelated terms sit elsewhere in the learned space.

After normalization by vector lengths, *exam* and *quiz* remain more similar than *exam* and *guitar*. Even this tiny example captures the intuition: related academic terms occupy closer directions in the learned space than an unrelated music term.

This is the right moment to keep one distinction explicit. The raw dot products 1.14 and 0.755 already suggest stronger alignment for *exam* with *quiz*, but they are not yet scale-free similarities. Dot product mixes direction and length. Cosine similarity is what removes the length effect and answers the cleaner question, "are these vectors pointing in similar directions?"

7.4.2 Cosine Similarity

Once objects are represented as vectors, we need a way to compare them. A standard choice is cosine similarity:

$$\cos(e_i, e_j) = \frac{e_i^\top e_j}{\|e_i\|_2 \|e_j\|_2}.$$

This measures the angle between two vectors rather than their raw length. It is especially useful when direction matters more than magnitude.

Prerequisite Bridge. The dot product $e_i^\top e_j$ is

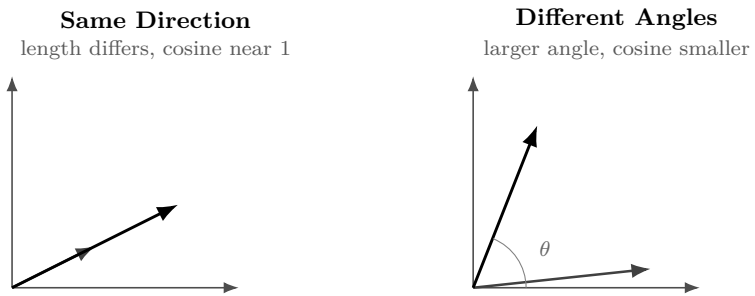


Figure 7.4: Cosine similarity cares about angle, not just length. That makes it useful when vector magnitude reflects scale or confidence but direction reflects role.

large when two vectors point in similar directions. Dividing by the vector lengths turns that raw alignment into a scale-free similarity score.

Advanced Note. Cosine and Euclidean similarity answer different questions. Euclidean distance cares about absolute position and scale. Cosine cares about direction. In some applications the distinction is minor; in others it changes the neighbor ranking substantially.

Warning. Embedding neighborhoods are evidence-bearing summaries, not ground-truth meaning. A near neighbor means the training objective and data made the model treat two objects similarly. That can be useful and still be biased, partial, or unstable.

7.5 How Representations Are Learned: Supervisory Pressure

The next question is where reusable structure comes from in the first place. The chapter is strongest when these workflows are understood as different kinds of pressure on the learned space, not as disconnected method names. Different objectives really do produce different reuse profiles. Supervised pretraining often optimizes one label space well and can transfer strongly when

Learning type	Where the target comes from	What it pressures the representation to learn
Supervised	human labels	task-specific discriminative structure
Self-supervised	the data itself	reusable structure, invariances, and context
Transfer / adaptation	downstream labels after pretraining	specialization to the target task or domain

Table 7.3: These workflows are related, but not interchangeable. The source of supervision shapes what the representation becomes good at.

the downstream task is close. Self-supervised objectives such as masked prediction, contrastive alignment, and self-distillation instead force the model to preserve context, invariance, or cross-view agreement that may transfer more broadly [4, 11, 16, 17, 24, 28]. Transfer and adaptation then test how much of that reusable structure actually survives contact with a new task.

7.5.1 Supervised Pressure

In supervised learning, the representation is shaped by one labeled task. A classifier trained to distinguish diseases from scans will learn features that help separate those labels. This can be very effective when labels are abundant and the deployment task closely matches the training task.

The limitation is narrowness. A representation learned only for one fixed target may not transfer well if the next task asks a different question.

7.5.2 Self-Supervised Pressure

Definition 7.3 (Self-Supervised Learning)**.** Self-supervised learning trains a model using targets generated from the data itself rather than relying entirely on external human labels.

> **Math Lens.** One visible example is masked-token prediction. If M is the set of masked positions in

a sequence x, and the model predicts a distribution $p_\theta(\cdot \mid \tilde{x})$ from the corrupted sequence $\tilde{x}$, then a simple objective is

$$\mathcal{L}_{\text{mask}}(\theta) = -\frac{1}{|M|} \sum_{i \in M} \log p_\theta(x_i \mid \tilde{x}).$$

The loss is small only when the model assigns high probability to the original tokens at the masked positions. That makes the training signal explicit: the model must use surrounding context to recover what was removed.

Self-supervised learning creates prediction problems from structure already present in the input. Common patterns include:

- predicting masked or missing parts of a sequence,
- reconstructing a clean example from a corrupted one,
- matching two augmented views of the same underlying object,
- predicting future segments of time-series or audio data.

The pretext task is not the final goal. It is pressure that teaches invariance, context, continuity, or structure. Later probe and transfer results are only as good as the pressure that created the representation in the first place.

The exact objective matters. BERT-style masking pressures the model to recover missing language from context, which is why it helped many downstream language tasks after adaptation [17]. Contrastive predictive coding made the same pressure explicit for sequential data by asking latent states to predict future observations [60]. SimCLR and MoCo use augmentations to define what should count as the same image, so the learned representation preserves object identity while discarding nuisance variation [16, 28]. BYOL and DINO then showed that strong visual representations can also emerge from self-distillation without explicit negative pairs, provided the training dynamics still prevent collapse [11, 24].

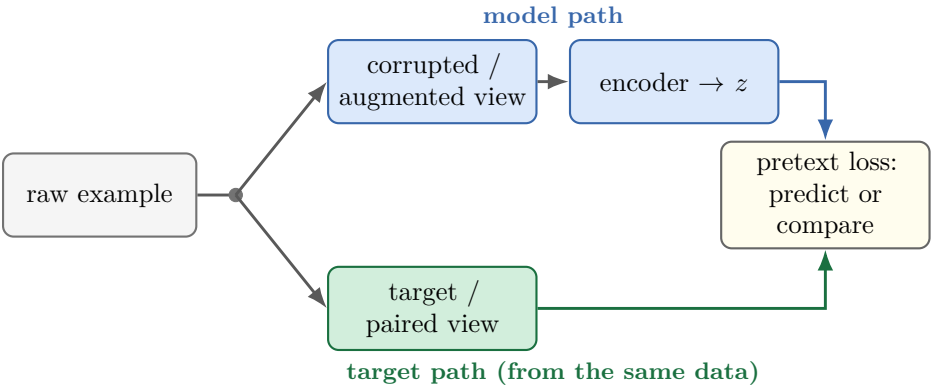


Figure 7.5: Self-supervision is not "learning without targets." It is learning from targets created out of the structure already present in the data.

Students do not need to memorize every acronym in that list. The durable lesson is simpler: each objective teaches the model what should stay stable across views, context, corruption, or time, and that choice determines what later transfer is likely to work.

7.5.3 Examples Across Modalities

Two evidence-bearing cases are more useful here than a long taxonomy. In language, masked-token pretraining teaches the model to encode what neighboring words make plausible, which is why contextual encoders can separate different uses of the same surface token [17]. In vision, SimCLR showed that representation quality moved sharply with augmentation design, projection heads, and batch size, which is exactly what we should expect if self-supervision is teaching invariance rather than merely fitting labels [16]. The point is not that every modality uses the same pretext task. The point is that each objective defines what structure should be stable enough to reuse later.

> **Practical Note.** The hard part of self-supervision is choosing an objective that cannot be solved by a cheap shortcut. If a corruption can be inverted from a trivial cue, or if paired views do not really describe the same underlying content, the model may score

well on the pretraining task while preserving little that helps later.

Listing 7.1: A minimal self-supervised training loop

```
for each raw example x:
    x_view = corrupt_or_augment(x)
    target = hidden_part_or_paired_view(x)
    z = encoder(x_view)
    prediction = prediction_head(z)
    loss = pretext_loss(prediction, target)
    update parameters
```

7.5.4 Autoencoders as Reconstruction-Based Representation Learning

Definition 7.4 (Autoencoder)**.** An autoencoder is a model that learns an encoder that maps an input into a latent representation and a decoder that tries to reconstruct the original input from that latent representation. In the standard representation-learning setup, the latent state is smaller or otherwise constrained.

The training signal comes from reconstruction. When the model must recover the original input from a smaller or constrained internal state, it cannot simply memorize every detail; it has to keep the information that is most useful for rebuilding the example. Under those conditions, reconstruction becomes a representation-learning pressure rather than just a copying exercise.

Math Lens. A simple reconstruction objective is

$$\mathcal{L}_{\text{rec}}(\theta) = \|x - \hat{x}\|_2^2, \qquad \hat{x} = g_\theta(f_\theta(x)),$$

where f_θ is the encoder and g_θ is the decoder. The loss is small only when the latent representation preserves enough structure to rebuild the input faithfully.

Listing 7.2: A minimal autoencoder loop

```
for each example x:
    z = encoder(x)
```

```
    x_hat = decoder(z)
    loss = reconstruction_distance(x, x_hat)
    update encoder and decoder
```

Example 7.1 (A Small Reconstruction Example)**.** Suppose a sensor stream has a stable daily pattern with occasional spikes. An autoencoder trained on the stream can learn the regular cycle and treat the spikes as hard-to-reconstruct irregularities. That can be useful for denoising, compression, or anomaly detection because the representation has learned what is stable enough to predict or rebuild.

If the latent space is almost as large and unconstrained as the input itself, reconstruction can collapse into copying rather than abstraction. Bottlenecks, denoising, sparsity, or related constraints are what turn reconstruction into a representation-learning pressure rather than a memorization path.

> **Decision Box.** Autoencoders help when the input contains redundancy, when a compressed latent space is useful, or when the downstream task benefits from denoising, anomaly detection, or a compact summary of the input.
>
> **Warning.** Autoencoders fail when reconstruction is too easy, when the bottleneck is too weak, or when the downstream label depends on structure that reconstruction does not force the model to preserve.

7.5.5 Multimodal Transfer as Shared Structure Across Modalities

Multimodal transfer asks a different but related question: when two or more modalities describe the same underlying object or event, can one learned representation serve all of them? The reusable signal is the overlap in meaning across aligned views. A photo and its caption, an audio clip and its transcript, or a radiology image and its report may all point to the same underlying content from different directions.

The core idea is shared structure. If training aligns those

views, the representation can learn what survives across modality boundaries and ignore what only one view happens to contain. That is why multimodal training is often a source of reusable structure rather than just a collection of separate encoders.

Listing 7.3: A simple paired-view alignment sketch

```
for each paired example (view_a, view_b):
    z_a = encoder_a(view_a)
    z_b = encoder_b(view_b)
    increase similarity(z_a, z_b)
    decrease similarity to mismatched pairs in the same
        batch
```

The algorithmic point is simple: the model is rewarded not only for what belongs together, but also for what should stay apart. That is what makes alignment pressure informative rather than merely associative.

Example 7.2 (Shared Structure Example)**.** If a model learns from product photos paired with short product descriptions, it can often transfer to later tasks such as search, retrieval, or coarse categorization because the paired modalities teach the model what the product is, not just how the image looks or how the text sounds.

> **Decision Box.** Multimodal transfer helps when the modalities are tightly aligned, when one modality supplies a cheaper or cleaner signal for the other, and when the target task benefits from concepts shared across views.
>
> **Warning.** Multimodal transfer fails when the alignment is shallow, when captions or transcripts are generic, or when the downstream task depends on modality-specific detail that the shared space never had to preserve.

Together, reconstruction and cross-modal alignment show two of the main ways a model can be pressured into reusable structure: by rebuilding what was hidden, or by agreeing across different

views of the same thing. The next step is to ask when that broader pressure actually pays off downstream.

7.6 Why Self-Supervision Scales

The main practical advantage of self-supervision is that unlabeled data is usually much more abundant than carefully labeled data. If the pretraining objective is chosen well, large amounts of raw data can be turned into representational pressure before a downstream label set even exists. This section is therefore not making a bigger-is-better claim. It is explaining why reuse-first workflows became practical at all.

This is one of the major reasons the field moved toward pretraining and adaptation. Instead of rebuilding features from scratch for every new task, we can first learn broad reusable structure and then specialize it later.

Scale helps because raw data is plentiful, but the chapter should resist a lazy conclusion here. Self-supervision scales only when the objective teaches the invariances or contextual structure the downstream task actually needs. More unlabeled data is not a substitute for a reuse audit [16, 17].

Example 7.3 (Denoising as a Representation Task)**.** Suppose an image model is trained to reconstruct a clean image from a corrupted version. The model cannot succeed by memorizing one pixel at a time. It must learn which patterns are stable enough to infer the missing content. This forces the internal representation to organize edges, textures, and shapes rather than only the raw corruption.

> **Advanced Note.** Contrastive learning is an especially influential self-supervised idea. Two views of the same underlying example are encouraged to have similar representations, while different examples are pushed apart. This teaches invariance: what changes under augmentation should often be ignored, while what remains stable should be preserved.

7.7 Transfer Begins: Pretrain, Probe, Adapt

The chapter now shifts from how reusable spaces are created to how they are tested. Once a representation has been learned, we still need to use it. This is where transfer learning begins.

Definition 7.5 (Transfer Learning)**.** Transfer learning is the reuse of knowledge learned on one task, dataset, or objective to improve performance on another task.

Transfer is not a victory lap after pretraining. It is the first serious test of whether the representation contains linearly or cheaply accessible structure for the new task. Yosinski et al. showed that lower layers tend to transfer more generally while upper layers become more task-specific, and Kornblith et al. found that stronger source models often transfer better but not uniformly across downstream tasks [39, 62].

The simplest reuse workflow is:

1. pretrain an encoder on a broad source objective,
2. freeze or partially freeze the encoder,
3. train a smaller downstream head on the target task,
4. optionally fine-tune some or all of the encoder.

Listing 7.4: A generic pretrain-and-adapt workflow

```
encoder = pretrain_on_broad_data()

frozen_features = encoder(target_inputs)
head = train_linear_probe(frozen_features, target_labels)

if task_or_domain_shift_is_large:
    fine_tune(encoder, head, target_data)
```

Problem-Solving Technique. Reuse audit. Before adapting a pretrained model, ask:

- What structure is being reused?

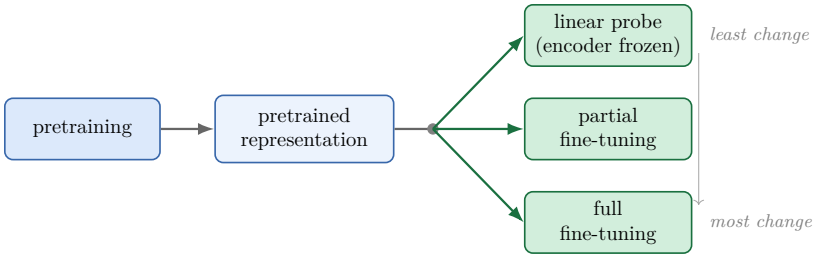


Figure 7.6: Pretraining and downstream adaptation are separate phases. The main engineering question is how much of the pretrained representation to keep fixed and how much to adapt.

- How much of it stays frozen?
- What downstream head or adaptation method is being added?
- What evidence would show that reuse, rather than downstream complexity, is doing the real work?

7.8 Linear Probe as the First Diagnostic

A linear probe is a deliberately simple downstream model, often just a linear classifier or regressor trained on frozen embeddings. Following the probe tradition used to inspect intermediate representations, it is useful precisely because it separates two questions without hiding them behind a powerful downstream head [1]:

- Did the representation capture useful information?
- Can a powerful downstream model exploit it?

This is one of the cleanest habits in the chapter:

Probe first when you want to test the representation, not hide it behind a fancy head.

If a linear probe performs well, then the representation has already organized the information in a usable way for that target. If a very complex downstream model is still required, then the representation may be less mature than it first appeared. A weak probe does not prove the representation is useless. It means the

current frozen representation does not make the target easily accessible under a simple readout, which is exactly the diagnostic question we wanted to ask.

> **Math Lens.** Let $h(x)$ be a frozen representation and let g_ϕ be a linear probe trained on top of it. If $g_\phi(h(x))$ performs well, then the target is linearly accessible in the frozen space. That is evidence of reuse, but it does not prove three stronger claims: that the representation is causally organized the way humans describe it, that the signal is robust under distribution shift, or that a nonlinear head could not rescue a weak frozen space. The probe tests accessibility, not truth.

That is the narrow claim the probe supports: linearly accessible information in a frozen space. It is not, by itself, evidence of causal structure or robustness under shift.

Example 7.4 (Few Labels, Frozen Encoder)**.** Imagine a vision encoder pretrained on a large collection of unlabeled classroom images. Later, a school wants to classify a small labeled set of whiteboard photos into categories such as clear, blurred, glare-obstructed, or partially cropped. If the encoder already captures strokes, edges, writing density, and camera artifacts, then a small linear probe may perform surprisingly well even with few labels.

7.8.1 When Reuse Fails or Is the Wrong Fit

Not every pretrained space should be reused. Negative transfer appears when the source invariances are wrong for the target, when label meanings shift, or when full fine-tuning erases useful source structure while trying to specialize.

This is the boundary-testing version of the chapter's claim: a strong frozen probe can still be the wrong coordinate system for the new task. A representation may be linearly accessible and still be brittle under shift, too narrow for the target, or too dependent on source-domain assumptions that no longer hold.

> **Warning.** Use broad reuse-first workflows only

when the source representation is plausibly aligned with the target and when the evidence justifies the added complexity. Skip them when the task is tiny and specialized, when fresh external evidence matters more than pretrained priors, or when privacy, latency, or interpretability constraints dominate.

7.9 Probe, Freeze, Partial Tune, or Full Tune?

The previous section argued for probing first. This section names the broader menu of adaptation choices that comes after that first diagnostic. The choice of adaptation strategy is not only a computational decision. It is an experimental claim about the quality and fit of the pretrained representation. Each strategy implies a different answer to the question: "how much of the pretrained representation do I trust for this new task?"

Linear probe (frozen encoder). Train only a linear head on top of the frozen pretrained features. This strategy makes the strongest claim about the representation: the task-relevant information is already linearly accessible in the frozen space. The probe is also the cleanest diagnostic: if it works well, the frozen representation is reusable for that target under a simple linear readout. If it works poorly, you learn that either the representation does not contain the right signals or those signals require a more complex readout.

Partial fine-tuning (unfreeze top layers). Allow the later layers of the pretrained network to update while keeping the early layers frozen. This strategy claims that the early-layer representations are general enough to reuse directly, but the later-layer abstractions need task-specific adjustment. The tradeoff is between compute efficiency and the risk of task-specific overfitting in the upper layers.

Full fine-tuning. Allow all layers to update. This strategy makes the weakest claim about the pretrained representation: some layers may need substantial adjustment to be useful for

Strategy	Compute	Data needed	Diagnostic clarity	Performance ceiling
Linear probe	Low	Very small	Strongest	Limited if target differs
Partial fine-tuning	Medium	Moderate	Moderate	Moderate
Full fine-tuning	High	Large	Weakest	Highest if shift is large
Adapter / low-rank	Low-medium	Small-moderate	Moderate	Good with less compute

Table 7.4: Adaptation strategies differ in what they claim about the pretrained representation and in what they require to work. The linear probe is the starting point; more expensive strategies should be justified by the probe's failure.

the new task. Full fine-tuning can achieve higher performance when the target task differs significantly from pretraining, but it requires more labeled data, more compute, and more care to avoid catastrophic forgetting of the source representation.

Adapter modules and low-rank updates. Insert small trainable modules between frozen pretrained layers, or decompose weight updates into low-rank matrices. These strategies aim to capture task-specific adjustments while touching as little of the pretrained space as possible. They are useful when compute or data is limited and when preserving most of the pretrained representation is a priority.

The judgment rule is to start with the linear probe. If the probe achieves strong performance, the pretrained representation is already fit for the task and no further adaptation is necessary. If the probe performs poorly, partial or full fine-tuning is warranted, but one should then ask *why* the probe failed. Was the pretraining task too different? Is the target task genuinely out-of-distribution? Is the labeled set too small for fine-tuning to help? The probe failure is a diagnostic, not just a prompt to try the next strategy in the table.

Choice	Cost	Overfitting risk	Diagnostic cleanliness	Performance ceiling
Frozen encoder + probe	low	low	strongest	often lower if shift is large
Parameter-efficient tuning	low to medium	low to medium	stronger than full tuning	often high when modest specialization is enough
Partial fine-tuning	medium	medium	moderate	often good compromise
Full fine-tuning	highest	highest on small data	weakest	highest when enough data and meaningful shift exist

Table 7.5: These choices are not value judgments. They are tradeoffs among compute, evidence quality, and adaptation flexibility.

7.10 Adaptation Depth: Freeze, Partial Tuning, Full Tuning

The previous section named the options. This section turns that menu into a decision rule. Once a representation is shown to contain useful information, the next question is how much of it should be allowed to move.

The modern design space is wider than freeze versus full fine-tuning. Gradual-unfreezing strategies such as ULMFiT made this visible early in NLP, and later adapter modules and low-rank updates made the same point at larger scale: teams can specialize a pretrained model while moving only part of it [31–33]. These methods can lower compute and preserve more of the original model, but they do not remove the need to show whether the observed gain came from reused representation or from extra task-specific capacity.

> **Decision Box.** Freeze first when you want a clean diagnostic of representation quality or when labeled

Scenario	Sensible first choice	Why
few labels, similar downstream domain	frozen encoder + linear probe	cheapest test of whether the representation already contains the needed signal
moderate labels, clear task shift, limited compute	parameter-efficient tuning	specializes the model without moving every parameter
moderate labels, moderate domain shift	partial fine-tuning	adapts the representation without fully relearning it
larger label budget, important task, large shift	full fine-tuning	lets the model reshape more of the representation for the new domain

Table 7.6: A useful first-pass rule is to match the amount of tuning to the amount of labeled evidence and the severity of domain shift.

data is small. Use parameter-efficient tuning when full adaptation is too expensive or too entangled for the evidence you have. Fine-tune more deeply only when the task matters enough, the domain shift is meaningful, and you have enough data and compute to justify moving more parameters.

Example 7.5 (Same Representation, Different Target Scenarios)**.** Suppose a pretrained encoder already gives a strong frozen linear probe on typed classroom notices. Fine-tuning may add little there because the domain shift is small. Now move to handwritten whiteboard photos with glare, blur, and camera perspective. The same encoder may still help, but fine-tuning becomes more plausible because the downstream domain asks the representation to adapt to new nuisance structure.

Evidence Box. Evidence that transfer learning is genuinely helping should include at least one of the

Method	Test accuracy	What the result suggests
Raw-feature logistic regression	0.565	weak baseline under low labels and domain shift
Frozen encoder + linear probe	0.683	the pretrained representation already exposes useful signal
Frozen encoder + learned classifier	0.691	a slightly richer head helps, but only modestly
Partial fine-tuning	0.625	more adaptation can overfit before it helps
Full fine-tuning	0.570	aggressive tuning can erase the advantage when labels are scarce

Table 7.7: A small pedagogical domain-shift example from the companion materials. The point is not that full fine-tuning is always worse, but that low-label adaptation often rewards restraint and good diagnostics.

following:

- similar performance with fewer labeled examples,
- better performance at the same label budget,
- faster convergence during downstream training,
- better robustness under domain shift or nuisance variation.

7.11 Attention: Building Context-Dependent Representations

Everything so far has treated reusable representation as a space we can inspect, probe, and adapt. Attention adds a second idea: the representation itself can change with context at prediction time. Representation learning accelerated further when models gained a more flexible way to build context-dependent features. That mechanism is attention, and transformer-style attention made this shift scalable across modalities [61].

Definition 7.6 (Attention)**.** Attention is a mechanism that computes context-dependent weighted combinations of representations, allowing each element to focus on the most relevant parts of the input.

The central sentence of this section is simple:

Attention matters because representation becomes context-sensitive.

In a sentence, a word can attend to previous or later words. In an image, a patch can attend to other patches. In multimodal systems, text tokens can attend to image regions or audio frames.

> **Static vs. contextual representations.** A static embedding gives an object the same vector regardless of where it appears. A contextual representation changes with surrounding elements. The word "bank" in a finance sentence and in a river sentence may start from the same token identity, but attention lets the resulting vector depend on context.

Before the notation appears, it helps to say the mechanism in plain language. Each token or patch already has a current representation. Attention lets one element ask, "which other elements are most relevant to me right now, and how much of each should I borrow?" In that reading, the query is what the current element is looking for, the keys describe what other elements offer, and the values are the information those elements can contribute.

In the transformer formulation, each element has a query vector q, key vectors k_j, and value vectors v_j. The attention weights are computed by normalizing the full list of query-key similarities:

$$\text{score}_j = \frac{q^\top k_j}{\sqrt{d}}, \qquad \alpha_j = \frac{\exp(\text{score}_j)}{\sum_\ell \exp(\text{score}_\ell)}, \qquad c = \sum_j \alpha_j v_j.$$

Here c is the context-aware representation produced by attention.

> **Prerequisite Bridge.** The softmax function converts a list of raw scores into nonnegative weights that sum to 1. It turns "how relevant is each item?" into a proper weighted average.

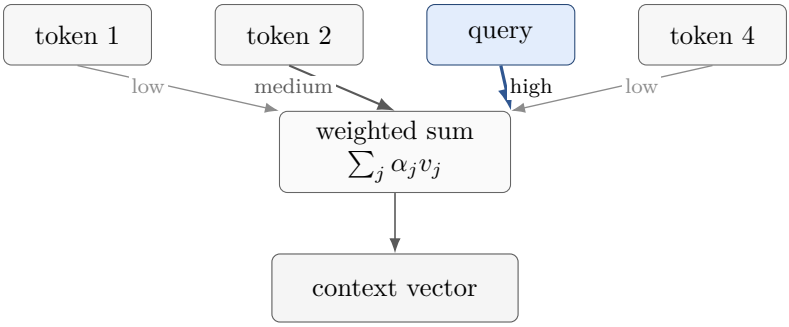


Figure 7.7: Attention can be read as weighted averaging over relevant items. The current query does not treat every token or patch equally; it builds a context-specific mixture.

> **Math Lens.** The factor $\sqrt{d}$ keeps dot-product scores from growing too large as the key/query dimension d grows. A useful intuition is that if the coordinates of q and k_j are roughly centered with unit-scale variation, then $q^\top k_j$ can grow in typical magnitude as d increases. Dividing by $\sqrt{d}$ makes the softmax input less sensitive to dimension, so no single head becomes overwhelmingly peaky just because its vectors are longer.

7.11.1 Worked Example: Attention as Weighted Averaging

Suppose a query vector is

$$q = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

and there are three keys:

$$k_1 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \qquad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad k_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

The raw dot-product scores are

$$q^\top k_1 = 2, \qquad q^\top k_2 = 0, \qquad q^\top k_3 = 1.$$

Following the formula as written, with $d = 2$, the scaled scores are approximately

$$(1.414,\ 0,\ 0.707).$$

Applying a softmax to those scores gives approximate weights

$$(0.576,\ 0.140,\ 0.284).$$

So the representation will focus mostly on the first key, somewhat on the third, and very little on the second.

If the corresponding values are

$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad v_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

then the context vector is approximately

$$c = 0.576v_1 + 0.140v_2 + 0.284v_3 = \begin{bmatrix} 0.860 \\ 0.424 \end{bmatrix}.$$

Example 7.6 (Reading a Tiny Attention Matrix)**.** Suppose a three-token sentence produces the following attention weights for one head:

$$\begin{bmatrix} 0.70 & 0.20 & 0.10 \\ 0.15 & 0.70 & 0.15 \\ 0.45 & 0.10 & 0.45 \end{bmatrix}.$$

Each row says how one token distributes its attention across the sequence. This is useful because it shows which interactions are being emphasized. It is not a full causal explanation of model behavior.

7.12 Why Transformers Matter

This is the bridge from one mechanism to the workflow shift that follows. Transformers became important not only because they achieved strong results, but because they offered a general architecture for context-aware representation learning at scale. The same broad mechanism can be used across language, vision, audio, and multimodal systems. CLIP later made the reuse point

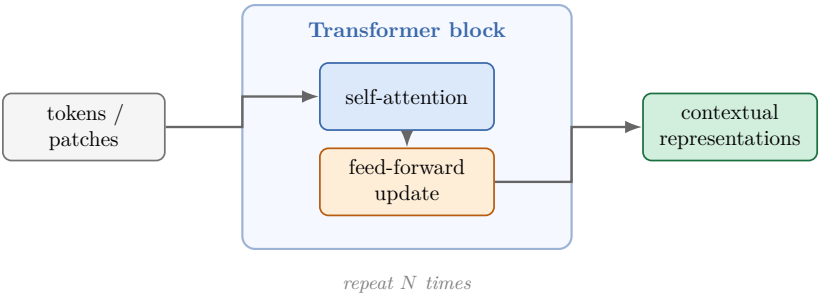


Figure 7.8: At a high level, a transformer block repeatedly does two things: mix information across elements through attention, then transform each contextualized representation further.

especially visible: language supervision produced a vision representation that supported strong zero-shot transfer across many image tasks [52].

The point of this section is not to create a buzzword parade. It is to explain why attention became the dominant vehicle for large-scale reusable representation learning.

> **Advanced Note.** Transformers matter because they combine flexible context modeling, shared architecture across modalities, and scalable pretraining. That combination made reusable representation learning easier to scale.

7.13 Foundation Models as a Workflow Shift

Definition 7.7 (Foundation Model)**.** A foundation model is a broadly pretrained model intended to support adaptation across many downstream tasks, often across domains or modalities.

The previous sections described a mechanism and an architecture. This section changes level again, from model internals to engineering workflow. The defining shift is not just model size. It is workflow. Instead of training one model for one narrow task, a foundation-model workflow usually looks like this:

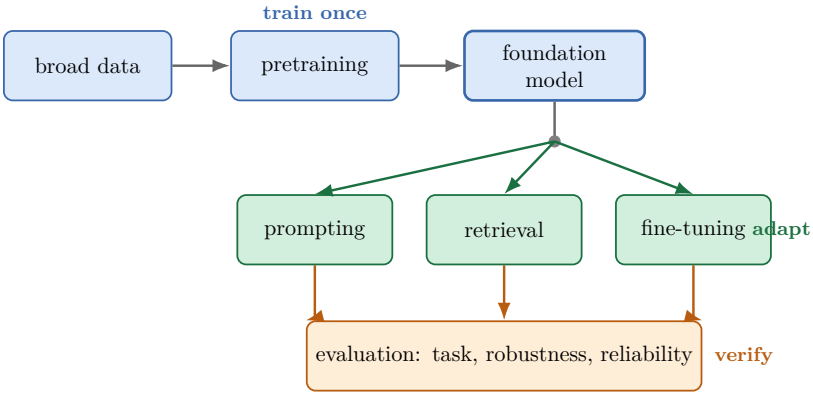


Figure 7.9: Foundation models are better understood as a workflow shift than as a single model class. Broad pretraining is only the beginning; adaptation and evaluation are where many practical failures emerge.

- collect broad data,
- choose a large-scale pretraining objective,
- train a reusable model,
- adapt or prompt the model for many narrower tasks,
- evaluate carefully under real deployment conditions.

GPT-3 made this workflow shift difficult to ignore because the same pretrained language model could change behavior substantially through prompting and few-shot examples even before full task-specific tuning [9]. CLIP made a parallel point in multimodal transfer by using language supervision to build a reusable image representation with zero-shot behavior on many downstream tasks [52]. Bommasani et al. use the term foundation model for precisely this pattern of broad pretraining plus many downstream adaptation paths [7].

That is why the cleanest sentence in this section is:

Foundation models are better understood as reusable pipelines than as prestigious checkpoints.

Workflow	Where most effort goes	What data scale matters most	Common illusion or failure mode
Task-specific model from scratch	task dataset and final objective	labeled downstream data	thinking the one-task result says anything about broader reuse
Pretrained model + adaptation	source pretraining plus downstream adaptation	unlabeled source data plus target labels	assuming transfer will work without checking domain shift
Foundation-model workflow	broad data, pretraining, adaptation, and evaluation	large-scale broad data	mistaking broad competence or fluency for grounded reliability

Table 7.8: These workflows differ not only in scale, but in where evidence and failure can enter the pipeline.

7.13.1 Why They Are Powerful

Foundation models can be powerful because they amortize representation learning across many tasks, exploit broad unlabeled or weakly labeled data, reduce downstream label requirements, and support multimodal reuse. The gain is often reuse efficiency rather than magical understanding: one expensive pretraining run can expose structure that many later tasks can read out, prompt, or adapt.

7.13.2 Why They Need More Discipline, Not Less

The risks are equally important:

- training data may be opaque or poorly documented,
- broad competence can mask brittle failure modes,
- inherited biases and harmful associations may transfer down-

stream,

- large compute cost can make experimentation uneven and less transparent,
- users may overtrust fluent output as if it were verified knowledge.

> **Failure Box.** Foundation models create a serious illusion risk. Because they can produce coherent text, plausible image descriptions, or polished outputs across many tasks, users may infer understanding, truthfulness, or reliability that has not actually been demonstrated. Breadth of behavior is not the same as grounded competence.

7.14 Same Base Model, Different Claims

This is the operational consequence of the foundation-model shift. Downstream adaptation can take many forms: prompting, instruction tuning, retrieval augmentation, frozen feature extraction, linear probing, parameter-efficient fine-tuning, or full fine-tuning [9, 31, 33, 43]. These are engineering choices, but they are also epistemic choices. They determine what part of the system is allowed to change, what evidence is available, and what claims become defensible. Retrieval augmentation is especially important because it creates a separate evidence path: part of the answer can now be justified by retrieved documents rather than by the pretrained parameters alone [43].

7.14.1 Prompting, Retrieval, and Fine-Tuning Solve Different Problems

One of the most useful distinctions for current AI work is that prompting, retrieval, and fine-tuning are not three interchangeable ways to "make the model better." They change different parts of the system and should therefore be chosen for different reasons.

This distinction becomes clear in the running support-assistant

Intervention	What it mainly changes	Best use case	Main risk if misunderstood
Prompting	the instruction and local behavior policy	when the base model already has the needed knowledge or skill but needs clearer task framing	mistaking better phrasing for new evidence or durable adaptation
Retrieval	the evidence available at inference time	when answers must depend on current, auditable external information	blaming the model when the real failure is stale, weak, or missing evidence
Fine-tuning	the predictor itself through parameter updates	when task-specific behavior must be learned consistently beyond prompting alone	attributing every gain to the base model instead of to downstream specialized training

Table 7.9: Prompting, retrieval, and fine-tuning improve different parts of the workflow. Treating them as interchangeable obscures where the gain really came from.

style case. If the model already knows how to answer politely but needs clearer format instructions, prompting may be enough. If the challenge is that the answer must follow the current semester's policy documents, retrieval is the central intervention because the evidence has to be present at prediction time. If the challenge is that the assistant must consistently adapt to a specialized institutional tone or recurring support patterns, then some form of fine-tuning may be appropriate.

The claim changes with the intervention. A prompting gain often supports a behavior-design claim. A retrieval gain supports an evidence-access claim. A fine-tuning gain supports a task-adaptation claim. Collapsing these together produces exactly the kind of blurry evaluation story the book is trying to prevent.

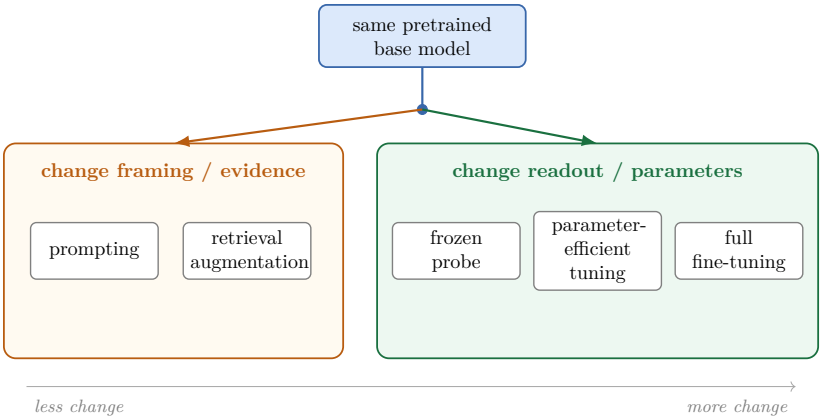


Figure 7.10: These are not rungs on one universal ladder. They intervene at different places in the workflow: prompting and retrieval change instructions or evidence, while probes and tuning choices change how much of the learned representation is read out or allowed to move.

> **Problem-Solving Technique. Intervention audit.** When a system improves, ask what changed: the instruction, the accessible evidence, or the predictor itself. Then make the claim at that same level rather than attributing the gain to vague "model quality."
>
> **Problem-Solving Technique. Common failure mode → recovery move.** Failure mode: treating prompting, retrieval, and fine-tuning as interchangeable upgrades and then making one blurry claim about "the model." Recovery move: run the smallest comparison that changes only one layer of the workflow, name whether the gain came from instruction, evidence, or parameter updates, and keep the claim at that same level.

Example 7.7 (Same Base Model, Different Claims)**.** Consider a device-support assistant built on the same pretrained language model. With prompting alone, the defensible claim is narrow: the system can often produce fluent answers, but those answers are not yet grounded in the manufacturer's current manuals and

warranty rules. After adding retrieval over the official support documents, the claim changes. Now the system can be evaluated as a document-grounded assistant whose answers should track retrieved sources. If the model is then fine-tuned on past resolved support chats, another claim becomes possible: the assistant may adapt better to local tone and recurring cases. But that gain introduces a new ambiguity. Better answers may now come from retrieval, from the pretrained model, or from the fine-tuned parameters, and each source of improvement has to be validated separately.

> **Practical Note.** Changing the adaptation method changes the claim you are allowed to make. Always ask what part of the observed gain is being attributed to reusable representation, what part to retrieval or prompting, and what part to downstream parameter updates.

7.15 Chapter Artifact: A Reuse and Adaptation Plan

By the end of this chapter, the reader should be able to write down not only which pretrained representation or foundation model they want to use, but how they plan to test what is actually being reused and what evidence would justify that claim. The point is to keep reuse claims separate from gains that really come from retrieval, prompting, or downstream tuning. The framing memo named the decision, the metric worksheet named good behavior, the evaluation plan protected the claim, the baseline sheet justified linearity, the structure diagnosis sheet justified leaving it, and the training report justified the learned-model claim. This chapter's artifact asks a new question: what part of that learned system is reusable, and what evidence would justify saying so?

> **Problem-Solving Technique. Reuse and adaptation plan.** Before adapting a pretrained model, write down what is being reused, what is being changed, the first clean diagnostic of reuse, and what

Plan field	What must be written clearly
Seven-question focus	Questions 4, 5, and 6: what representation is being reused, what cleaner baseline comes first, and what evidence separates reuse from other explanations
Source representation or model	What pretrained encoder, embedding model, or foundation model is being reused
Source objective	What pretraining task or data produced that representation
Target task	What downstream problem is being solved now
Expected domain shift	How the new setting differs from the source setting
First diagnostic	Linear probe, frozen features, retrieval-only baseline, or another clean reuse test
Chosen adaptation depth	Prompting, retrieval, probe, partial tuning, or full tuning
Alternative explanations	What else besides representation reuse could explain the gain
Evidence needed	What result would justify claiming that reuse is genuinely helping

Table 7.10: A reuse and adaptation plan turns transfer learning from a buzzword into a testable design choice.

evidence would justify a strong transfer claim.

Example 7.8 (Filled Reuse and Adaptation Plan)**.** Suppose a university wants to classify short student emails into routine, ambiguous, and high-risk support categories. The reused object might be a broadly pretrained text encoder. The source objective would be large-scale language modeling rather than local support labels. The expected domain shift would include institutional vocabulary, policy references, and higher stakes around escalation misses. The first diagnostic could be a frozen linear probe on a small labeled set, because that tests whether the representation already exposes the needed signal before new task-specific capacity is added. If retrieval over current policy documents is later added, the plan should say explicitly that any new gain might

come from better evidence rather than from the reused encoder alone.

7.16 Common Mistakes with Representation Learning

The mistakes in this section are different surface versions of one deeper error:

confusing reusable behavior with justified evidence about why that behavior exists.

Mistake Checklist.

- **Assuming a good representation is task-free magic.** A representation can be useful across tasks without being universally useful.
- **Treating embedding neighbors as ground-truth meaning.** Nearest neighbors are model-dependent summaries, not philosophical definitions.
- **Skipping the linear probe.** A simple probe is one of the cleanest tests of representation quality.
- **Believing self-supervision automatically solves label scarcity.** The pretraining objective still has to match the downstream task.
- **Confusing transfer success with the source of transfer.** Say what part of the gain came from the representation, retrieval, prompting, or fine-tuning.
- **Treating attention maps as full explanations.** Attention can be informative without being a faithful causal explanation.
- **Confusing scale with validation.** Large models still need controlled evaluation.

7.17 Looking Ahead

This chapter framed AI around reusable representations: embeddings, probes, transfer, attention, and foundation-model workflows. The next chapter grounds those ideas in vision, where the reuse question becomes concrete: which visual changes should a model ignore, which should it preserve, and what augmentations or architectures actually encode that judgment?

Chapter Summary

- This chapter answers one question: when is it worth learning a reusable representation instead of solving only one task directly, and how do we know that reusable representation is what is helping?
- A good representation rearranges the space so that simpler downstream decisions become possible, and embeddings make that geometric claim explicit.
- Supervised, self-supervised, and transfer-learning workflows differ mainly in what kind of pressure they place on the learned space.
- A linear probe is one of the cleanest diagnostics of reusable representation quality, so adaptation should usually begin there rather than with maximal tuning.
- Adaptation depth should be chosen by label budget, domain shift, and evidence needs rather than by default.
- Attention matters because representation becomes context-sensitive, and transformers matter because they scale that idea across modalities.
- Foundation models are best understood as reusable workflows: changing the adaptation method changes both the system behavior and the claim you are allowed to make.

Study Questions

1. Why can a weak downstream model perform well when built on top of a strong representation?
2. Why is "change of coordinates" a useful mental model for representation learning?
3. What does cosine similarity measure, and why is it often used with embeddings?
4. Why is a linear probe a better first diagnostic than a powerful downstream head when you want to test representation quality?
5. Why does attention help build context-dependent representations?
6. Why do foundation models increase the importance of evaluation rather than reduce it?

Exercises

1. Draw a tiny 2D picture of a "bad" space and a "good" learned space for the same binary classification task. Explain what changed.
2. Compute the cosine similarity between the vectors $a = [1, 2]^\top$ and $b = [2, 1]^\top$.
3. Suppose a model has strong training performance on its self-supervised pretraining objective but weak downstream transfer. Give two plausible explanations.
4. For each of the following scenarios, choose a sensible first adaptation strategy and justify it: few labels with small domain shift, moderate labels with moderate shift, many labels with large shift.
5. Explain why a linear probe may be a better first diagnostic than a deep downstream head when evaluating a new embedding model.

6. In your own words, explain why attention can be understood as data-dependent weighted averaging.
7. A model produces the attention row $(0.10, 0.70, 0.20)$ for one token over a three-token sequence. What does that row tell you, and what does it *not* justify you claiming?
8. Use Table 7.10 to sketch a reuse and adaptation plan for one target task of your choice. Include the first clean diagnostic and one alternative explanation you would have to rule out before claiming that reuse helped. If you are carrying one case through the book, say what remains fixed from the previous chapter's training report and what this chapter changes first.

Challenge Exercises

1. Construct a tiny embedding example with three objects for which Euclidean distance and cosine similarity rank the neighbors differently. Explain what this reveals about vector magnitude versus direction.
2. A self-supervised vision model is pretrained on natural photographs and then adapted to satellite imagery. Give three technical reasons why transfer may fail even if the pretraining dataset was very large.
3. Explain why a high-performing linear probe is evidence that the representation is useful, but not proof that the model's internal organization is causally meaningful or robust under distribution shift.
4. A foundation-model pipeline improves after adding retrieval augmentation. Explain what new evidence you would need before claiming that the pretrained representation itself became better.

Part III

Modalities

Chapter 8

Vision: Learning from Images

Learning Goals

- Explain why vision begins by deciding which visual variations should count as nuisance and which should count as task-relevant signal.
- Use an invariance audit to connect task formulation, augmentation, architecture, and evaluation.
- Understand convolution, parameter sharing, receptive fields, and pooling as architectural responses to image structure.
- Distinguish classification, detection, segmentation, and related tasks before discussing model depth.
- Evaluate vision systems in terms of shortcut learning, robustness, and deployment-camera realism rather than benchmark accuracy alone.
- Produce a vision design and stress-test card before making a serious vision-model claim.

The previous chapter treated reusable representation at a general level. Vision is one of the clearest places to see that logic be-

come operational. Images are not just arrays of pixels to classify. They are measurements shaped by lighting, viewpoint, occlusion, background, sensor choice, and task definition. Those factors determine what a model should ignore, what it must preserve, and what kinds of shortcuts the data may accidentally reward. The chapter follows that logic in order: first decide the output and invariances, then decide what labels and local structure the model must respect, then decide what augmentations and backbones are justified, and finally decide what evidence would make a visual claim trustworthy.

> **Problem-Solving Lens.** This chapter asks a vision-specific modeling question: which visual variations should count as noise, which should count as signal, and what evidence shows that the model learned the right cue rather than a shortcut? The answer drives architecture, augmentation, transfer, and evaluation choices.

By the end of the chapter, the reader should be able to write a vision design and stress-test card before making a serious claim.

8.1 Opening Dilemma: The Same Object Can Look Like Many Different Pixel Arrays

Suppose the task is to support a road-scene system that must detect pedestrians. A person standing on the roadside can appear under bright sun, at dusk, under headlight glare, partially occluded, seen from different camera heights, or blurred by motion. The semantic object may be the same, but the pixel array can change drastically.

This is why vision should not begin with architecture names. It should begin with a judgment question: what variation should preserve the label, and what variation should change it? A pedestrian shifted slightly within the image should still be a pedestrian. A change in lighting should usually not erase the label. But removing the pedestrian entirely, replacing the person with a traffic

sign, or changing the task from detection to segmentation should change what the model is supposed to output.

Keep one running case in view throughout this chapter: a dashboard-camera pedestrian detector for a low-speed campus shuttle. The detector must work through windshield glare, dusk and night conditions, light rain, partial occlusion, and camera-specific geometry. That single case is useful because it forces the chapter to connect invariance, architecture, augmentation, shortcuts, and deployment-camera realism inside one visual system rather than treating them as disconnected ideas.

That dilemma gives the whole chapter its shape. First the chapter fixes the output object and the invariance audit, then it clarifies annotation and locality, then it turns to augmentation and transfer, and finally it asks how to distrust benchmark success correctly.

8.2 Problem-Solving Technique: Invariance Audit

Problem-Solving Technique. Invariance audit. Before training a vision model, write down:

- what variation should preserve the label,
- what variation should change the label,
- what sensor or camera changes are realistic in deployment,
- what spatial detail the downstream task actually needs.

This is the chapter's most reusable habit. The audit connects task formulation, augmentation, architecture, and evaluation. If the task is image classification, the model may need broad invariance to small translations and lighting changes. If the task is segmentation, some of that same variation may be harmless, but fine spatial detail becomes far more important. If deployment

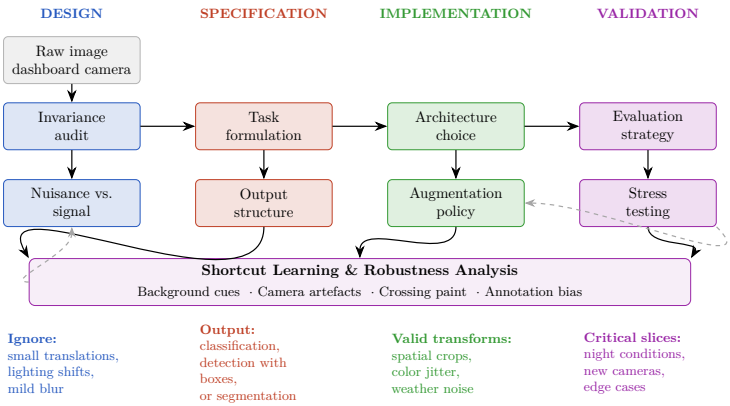


Figure 8.1: The vision development pipeline integrates invariance auditing, task formulation, architecture choices, and evaluation strategy into a coherent workflow. Each stage blocks different failure modes, with feedback loops revealing shortcut learning and robustness gaps that require pipeline redesign.

uses a different camera from the training set, the camera itself becomes part of the modeling problem.

Decision Box. Vision performance cannot be interpreted before the invariance audit is clear. A model cannot be said to work unless we know what it was supposed to ignore, what it was supposed to preserve, and what the deployment camera will actually produce.

Practical Note. Workflow sketch: start from the task and required output, run the invariance audit, make the annotation policy explicit, pick the smallest baseline that matches the output shape, add augmentations only for label-preserving variation, then compare a pretrained backbone, a frozen backbone, and fine-tuning on deployment-relevant slices.

Problem-Solving Technique. Artifact preview: vision design and stress-test card. Keep six fields visible from the start of the chapter: task type, intended signal, nuisance variation, deployment camera,

likely shortcut risks, and the slices or stress tests that would reveal failure before deployment.

8.3 Task Formulation Before Architecture Depth

One of the biggest mistakes in vision is to ask "CNN or transformer?" before asking what output is actually required. This section settles what the system must produce and therefore what kind of evidence later architecture and evaluation need to preserve.

8.3.1 Classification

Classification asks what is in the image. A road-scene classifier might answer "pedestrian present" or "no pedestrian present." This is useful, but it is not automatically enough for many safety-critical settings.

On the same scene, classification answers a coarse yes/no question about the whole image. It does not say where the pedestrian is, whether the object is partially occluded, or whether the model confused the crossing with the person. That is why the output object must be chosen before architecture depth becomes the main topic.

8.3.2 Detection

Object detection asks both *what* appears and *where*. A detector must localize relevant objects, usually with bounding boxes, while also assigning categories.

Math Lens. Suppose a predicted bounding box is B and the ground-truth box is G. A standard overlap score is intersection over union:

$$\mathrm{IoU}(B, G) = \frac{|B \cap G|}{|B \cup G|}.$$

If the class is correct and IoU is above a chosen threshold such as 0.5, the detection is often counted as a

hit. IoU matters because detection is not only about naming an object; it is about placing the box tightly enough that the localization is operationally useful.

Practical Note. A detector usually makes three linked predictions at many candidate locations or regions: is there an object here, what class is it, and how should the current box move to fit it better? That is why detection errors come in three distinct forms: missed candidates, mislabeled objects, or badly placed boxes.

8.3.3 Segmentation

Segmentation assigns labels to pixels or regions rather than to the whole image. Semantic segmentation marks categories such as road, pedestrian, and sky. Instance segmentation goes further by separating individual object instances.

Math Lens. If M is the predicted mask and G the ground-truth mask, two common overlap scores are

$$\mathrm{IoU}(M, G) = \frac{|M \cap G|}{|M \cup G|},$$
$$\mathrm{Dice}(M, G) = \frac{2|M \cap G|}{|M| + |G|}.$$

Both reward overlap, and for a fixed pair of masks Dice is a monotone transform of IoU. In practice people often report both because small foreground regions can make either score move sharply under modest boundary errors.

Example 8.1 (Why Small Masks Are Hard)**.** Missing twenty pixels on a large road region may barely change the score. Missing twenty pixels on a pedestrian silhouette can erase an arm or leg entirely. Segmentation metrics therefore need to be read together with class size and boundary importance, not as if every pixel error had the same operational meaning.

8.3.4 Tracking and Other Structured Vision Tasks

Other vision tasks include pose estimation, keypoint detection, tracking across video frames, depth estimation, and optical flow. These tasks show that vision is not one problem but a family of problems organized around spatial perception.

For video, a single-frame detector may be enough if each frame is evaluated independently, but tracking or temporal smoothing becomes part of the task the moment the system must follow the same object over time.

8.3.5 Why Structured Outputs Need Structured Evaluation

The output type changes the evidence you must collect. Classification can often be judged with image-level accuracy, precision, recall, and calibration. Detection needs those same judgment habits plus localization quality, because a box in the wrong place is not the same as a box with the wrong class. Segmentation needs overlap metrics and boundary checks because the task is about regions rather than whole images. Tracking adds temporal consistency because the system must keep identity stable across frames. In each case, the evaluation should match the output object, not just the underlying image.

> **Practical Note.** Structured outputs create structured failures. Classification can be wrong about the presence of an object, detection can be wrong about presence or placement, segmentation can be wrong about boundaries or small regions, and tracking can be wrong about identity over time. The model choice should make those failure modes legible.
>
> **Decision Box. Choose the output before the backbone.**
>
> - Need only image-level presence or absence: classification.
> - Need location, count, or a box around each object:

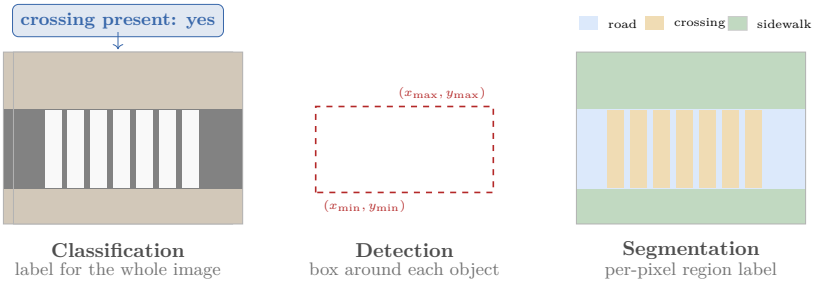


Figure 8.2: The same scene can be read three ways depending on the output required. Classification asks whether a crossing is present. Detection localises it with a bounding box. Segmentation assigns a label to every pixel. Task formulation must come first because each output demands a different loss, metric, and architecture head.

Task	Output on the same road-scene image
Classification	`marked crossing present = yes`
Detection	`crosswalk = [x_min, y_min, x_max, y_max]`
Segmentation	Pixelwise mask separating road, crossing markings, sidewalk, and background

Table 8.1: The same image can define very different output structures. This is why task formulation must come before architecture choice.

detection.

- Need region shape, area, or fine boundary support: segmentation.
- Need identity over time rather than only one frame: tracking.

Figure 8.2 makes this concrete. Even a simple street image can support several different outputs depending on what the downstream system needs.

Example 8.2 (Why Task Formulation Matters)**.** A system that can classify an image as "contains a marked crossing" is not automatically sufficient for a street-scene system. The system may instead need to localize the crossing, identify curb geometry, or

reason about nearby objects. The output requirement should be decided before architecture depth becomes the main topic.

> **Practical Note.** Model choice follows the output object. Image-level labels suggest a classification head and image-level evaluation. Box labels suggest a detector with localization-aware evaluation. Pixel masks suggest a segmentation head with overlap and boundary checks. If the failure pattern is wrong, the next question is usually whether the annotation policy or output type was underframed, not whether the network was too small.

> **Problem-Solving Technique. Task-specific evaluation.** Classification is judged with accuracy, precision, recall, and calibration if the output drives a decision. Detection must also respect localization, so IoU and false positives matter alongside class recall. Segmentation usually needs overlap quality such as IoU or Dice, plus boundary behavior when fine spatial detail matters.

8.3.6 A Running Case: What the Camera Actually Sees

Suppose the campus shuttle uses a forward-facing dashboard camera mounted low behind the windshield. For the rest of this chapter, assume the real task is *pedestrian detection*, not scene classification, because the system must know where the person is rather than only whether a person is somewhere in the frame. That one running case will now anchor the rest of the chapter: annotation policy, locality assumptions, augmentation choices, shortcut hypotheses, and deployment slices all have to answer to the same detector.

That detail matters because the camera is not neutral. A low-mounted dashboard camera sees glare from the glass, reflections from the dashboard, raindrops and smear marks on the windshield, seasonal shadows, and strong scale changes as pedestrians move from far away to very close. A benchmark image can

therefore be visually clean while still being a weak proxy for deployment. The invariance audit is already doing work here: small horizontal shifts, mild lighting changes, and modest blur should usually preserve the label, while heavy occlusion, extreme darkness, or a viewpoint change that makes the person unresolvable may change what the system can reliably output.

> **Practical Note.** In vision, the image is not the world. It is the output of a camera, lens, mounting position, exposure system, and environment. A good vision chapter should keep asking what that acquisition process adds or removes before the model ever sees the scene.

> **Warning.** For image sequences or camera data, avoid random splits that put near-duplicate frames on both sides of the evaluation boundary. Split by camera, route, day, clip, or location when those sources can leak the same visual context into train and test.

8.4 Annotation Ontology and Label Ambiguity

After the output object is fixed, the next question is what the labels actually mean. Every vision dataset rests on an annotation policy that defines, sometimes implicitly, which objects count, what counts as an occurrence, and how annotators should resolve ambiguous cases. That policy is a modeling decision. It shapes the label space, and therefore the task, before any image is fed to any model.

What counts as an object? For the pedestrian-detector running case, the annotation policy must decide whether a person seen through a store window counts. Whether a person behind a fence counts. Whether a headless torso at the image boundary counts. Whether a person with their back turned counts. Different policies on these questions produce different label sets, and a model trained under one policy may behave differently from a model trained under another, even on the same raw images.

What counts as an occurrence at a scale threshold? Objects at long range are often tiny in the image. Below some pixel area, a human annotator may not be able to resolve the object reliably. Detection datasets therefore often define a minimum bounding-box area for inclusion. That threshold is not a property of reality; it is a property of the annotation protocol. A model trained with a 32×32 pixel minimum will not have been taught to find objects smaller than that, which is not the same as saying the real deployment system is allowed to miss them.

Annotator disagreement as task ambiguity. When two annotators draw different bounding boxes around the same pedestrian, or one annotator marks an occluded person and another does not, the disagreement usually reflects genuine ambiguity in the task rather than careless labeling. The annotation protocol must resolve this ambiguity with explicit rules. But explicit rules do not eliminate the underlying fuzziness. They replace it with a conventionalized definition that some downstream uses will satisfy and others will not.

Example 8.3 (Annotation Policy Shapes Recall)**.** Suppose a pedestrian-detector dataset uses a policy that excludes heavily occluded people (visible fraction below 30%). A model trained on this dataset will learn to suppress predictions on heavily occluded cases. In a safety evaluation, engineers later discover that the model misses pedestrians who are partially hidden behind parked vehicles. The model is behaving correctly under its training policy. The training policy was wrong for the deployment context.

> **Problem-Solving Technique. Label space audit.** Before training a vision model, write down the annotation rules that govern edge cases: minimum object size, occlusion policy, boundary handling, and multiple-instance rules. Then ask whether those rules are defensible for the real deployment context. The label space is a modeling choice, not a given.

8.5 Why Raw Pixels Are Not Enough

Images are often described as arrays of pixels, but that description is too weak to explain why vision is difficult. A photograph is not merely a grid of numbers. It is the result of lighting, geometry, occlusion, texture, viewpoint, scale, and sensor properties. Different pixel arrays can represent the same object, and nearly identical arrays can correspond to different scenes.

This is the core educational point. Vision is hard because raw measurements do not come with meaning attached. A model must infer edges, parts, shapes, textures, and object relationships from those measurements. The real challenge is to build internal descriptions that remain useful under translation, clutter, lighting change, background change, and viewpoint variation.

Classical computer vision relied heavily on hand-designed features: edges, corners, texture histograms, local descriptors, and shape cues. Those pipelines remain conceptually important because they reveal what practitioners once thought stable visual structure looked like. Deep learning changed the field not by making structure irrelevant, but by making feature extraction itself learnable. The next section gives the chapter's first architectural answer to that problem: ask the same local question across the image instead of treating every pixel position as unrelated.

8.6 Convolution as a Repeated Local Question

Definition 8.1 (Convolutional Layer)**.** A convolutional layer applies a small learnable filter across an image, producing a feature map whose values indicate how strongly the filter matches local patterns at different positions.

The right headline for convolution is simple. It is also the first concrete architectural response to the invariance audit:

Convolution matters because vision is local before it is global.

Suppose an image is represented by a grid X, and a small filter

K is represented by a 3×3 matrix. A convolutional response at location (i, j) can be written as

$$S_{i,j} = \sum_{u=0}^{2} \sum_{v=0}^{2} K_{u,v} X_{i+u,j+v}.$$

The filter slides across the image and produces a new grid of responses. One filter may respond to horizontal edges, another to vertical edges, another to corners, and later filters may respond to more complex motifs built from earlier ones.

> **Prerequisite Bridge.** The formula above is just a weighted sum applied repeatedly at many image locations. Instead of learning one set of weights for the whole image at once, the model reuses the same small weights everywhere. That reuse is called parameter sharing.
>
> **Math Lens.** Let $T_\delta X$ denote an image shifted by δ. For an ideal convolution operator $f_K(X) = K * X$, shifting the input shifts the feature map:
>
> $$f_K(T_\delta X) = T_\delta f_K(X),$$
>
> ignoring boundary effects. This is translation equivariance: the same filter detects the same pattern wherever it appears, and the response moves with the pattern. Parameter sharing is what makes that possible, because the model does not need separate weights for the same edge at every location.

Deep-learning libraries often implement the unflipped-kernel form of this operation and still call it convolution. That terminology is standard in practice, but the distinction is worth noting for mathematically careful readers.

8.6.1 Why Parameter Sharing Matters

If a vertical edge matters on the left side of an image, it probably matters on the right side too. A dense model would need to learn those cases separately. A convolutional layer reuses the same filter across positions, which gives two advantages:

- fewer parameters, making learning more data-efficient,
- a built-in bias toward detecting the same local pattern at multiple locations.

This does not make a convolutional network perfectly location-invariant. Weight sharing mainly gives translation-equivariant local detectors, and later operations such as pooling can add partial invariance. Even so, the architecture is far more compatible with image structure than a fully unstructured dense network on raw pixels.

8.6.2 Worked Example: Detecting a Vertical Edge

Consider the following simple 3×3 filter:

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}.$$

This filter gives a large positive value when the right side of a local patch is brighter than the left side, which is one way to detect a vertical edge.

Now consider a local image patch:

$$P = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

The convolution response is

$$\begin{aligned} \sum_{u,v} K_{u,v} P_{u,v} = {} & (-1)(0) + 0(1) + 1(1) \\ & + (-1)(0) + 0(1) + 1(1) \\ & + (-1)(0) + 0(1) + 1(1) = 3. \end{aligned}$$

The response is strongly positive because the patch contains exactly the kind of vertical contrast the filter is designed to detect.

Example 8.4 (What a Feature Map Means)**.** If the same vertical edge appears at several positions in the image, the feature map

produced by K will contain strong responses at those positions. The feature map is therefore not the final answer; it is a spatial summary of where a certain kind of pattern was found.

8.7 Hierarchy and Approximate Invariance

Each neuron in a convolutional network does not usually see the whole image at once. It sees a local region called its receptive field. As layers stack, receptive fields grow. Early layers may respond to edges and textures, while later layers may integrate those into parts and larger shapes. This is where the chapter moves from local detectors to partial robustness: architecture can help with small shifts, but it does not settle the invariance question by itself.

This is one reason deep vision models are hierarchical:

- early layers detect local patterns,
- later layers combine them into larger structures,
- pooling and later processing help make the final decision less sensitive to small shifts,
- but perfect invariance is not automatic.

8.7.1 Pooling

Definition 8.2 (Max Pooling)**.** Max pooling replaces a small local region of activations with its largest value, reducing spatial resolution while preserving strong local evidence.

Pooling is useful because it can make representations less sensitive to tiny translations or distortions. If the same edge appears one pixel to the left or right, the pooled response may remain similar even though the raw feature map changes.

> **Advanced Note.** Convolution is often said to be *translation equivariant*: if the input shifts, the feature map tends to shift too. Pooling and augmentation

then help create a more *invariant* representation, one whose downstream decision does not change much under small shifts. Equivariance and invariance are related but not identical ideas.

Math Lens. Equivariance means the representation moves when the input moves:

$$f(T_\delta X) = T_\delta f(X).$$

Invariance means the representation stays the same:

$$g(T_\delta X) = g(X).$$

Convolution is usually equivariant, not invariant. That is fine for detection and segmentation, where location matters. Later pooling or global aggregation can move the system toward invariance when the final task is classification and position should not change the answer.

8.8 Augmentation as a Label-Preserving Claim

If convolution and pooling give an architectural bias toward useful local structure, augmentation states the same judgment at the data level. Augmentation is not just a bag of tricks for improving leaderboard performance. It is an explicit claim about which transformations should preserve the label.

Common examples include:

- random crops and translations
- horizontal flips
- small rotations
- color jitter
- blur, noise, or cutout-style masking

These transformations can help a model learn that position, lighting, or small perturbations should not override the semantic content of the image. But augmentation must match the task. A horizontal flip may be harmless for generic object recognition and harmful for text recognition or medical laterality tasks.

> **Decision Box.** Every augmentation is a claim about the world. Use it only if the transformed example should still carry the same label in the real problem.

Example 8.5 (Augmentation Must Match the Camera)**.** For the running pedestrian-detector case, small crops, modest brightness changes, mild blur, and limited weather-style corruption may be justified because the deployment camera really does see framing variation, dusk, motion blur, and windshield noise. Vertical flips, large rotations, or extremely stylized color shifts are different. They create scenes the dashboard camera will never produce and can teach the wrong invariances. Good augmentation imitates the sensor world rather than decorating the dataset.

8.9 Augmentation Claim Matrix

Each augmentation is a claim. The claim is that the transformation leaves the label intact. Making that claim explicit, rather than applying a standard list by convention, helps practitioners choose augmentations that actually match their task and deployment context.

The pedestrian-detector running case illustrates how to use the matrix. Small crops are justifiable: the system must detect pedestrians across framing variations as the camera moves. Modest brightness changes are justifiable: the deployment camera sees dusk, overcast conditions, and reflections. Large rotations are not: the camera is fixed in orientation and the system does not need to find upside-down pedestrians. Heavy stylization augmentations are not: the camera is a real lens, not an artistic filter. Each inclusion or exclusion is a deliberate claim rather than a recipe selection.

> **Practical Note.** Before finalizing an augmentation

pipeline, write down the claim for each augmentation you include. Then ask whether that claim is true for the real deployment scenario. Augmentations whose claims are false are not neutral: they actively teach the wrong invariances and can hurt performance on the deployment-relevant cases that violate the claimed invariance.

8.10 Modern Visual Backbones and Transfer

Convolutional networks were dominant in vision for many years because their inductive biases fit images well. More recently, transformers and hybrid architectures also became important, especially when pretrained at scale. The high-level lesson from the previous chapter still applies: once a model learns a strong visual representation, many downstream tasks can reuse it. But backbone choice only becomes meaningful after the task, annotation policy, and invariance claims are already clear.

Pretrained visual backbones often learn reusable low-level and mid-level features such as edges, textures, contours, shapes, and object-part relationships. This is why transfer learning became so effective in vision long before many students encountered large language models.

Practical Note. In applied vision work, training from scratch should not be the default assumption. If a reliable pretrained backbone exists and the target domain is not radically different, transfer learning is often the stronger baseline.

8.11 Shortcut Learning, Robustness, and Failure Analysis

The chapter now turns from building the visual system to distrusting it correctly. This section is the emotional center of the chapter.

A vision model may look competent because the dataset made the shortcut useful.

Data curation matters intensely in vision. Label quality, viewpoint diversity, class balance, background variation, sensor differences, and cropping policies all affect what a model learns. Vision models are especially vulnerable to shortcut learning because images contain many correlated signals besides the intended object.

> **Failure Box.** A vision model may appear to recognize the object when it is really using background, watermark, scanner artifact, camera angle, room lighting, or text overlay as a shortcut. High test accuracy on a narrow benchmark does not rule this out.

8.11.1 Shortcut Learning

Suppose a dataset of wolves often contains snow while a dataset of dogs does not. A classifier may rely on snow texture more than animal shape. A medical model may rely on a scanner marker instead of pathology. A classroom whiteboard classifier may rely on room lighting rather than handwriting clarity. These are not bizarre edge cases. They are normal consequences of optimizing on finite datasets with accidental regularities.

The running pedestrian-detector case has its own likely shortcuts. If most positive examples show pedestrians near painted crossings, then crossing stripes can become an accidental cue. If the training set is dominated by bright daytime footage from one vehicle, then camera height, windshield tint, and day-specific contrast patterns can become part of the internal decision rule. A model may then appear to recognize pedestrians while partly recognizing "pedestrian near familiar crossing under familiar camera conditions."

8.11.2 Worked Example: Slice Breakdown Reveals the Wrong Cue

Suppose the detector is evaluated on several held-out slices:

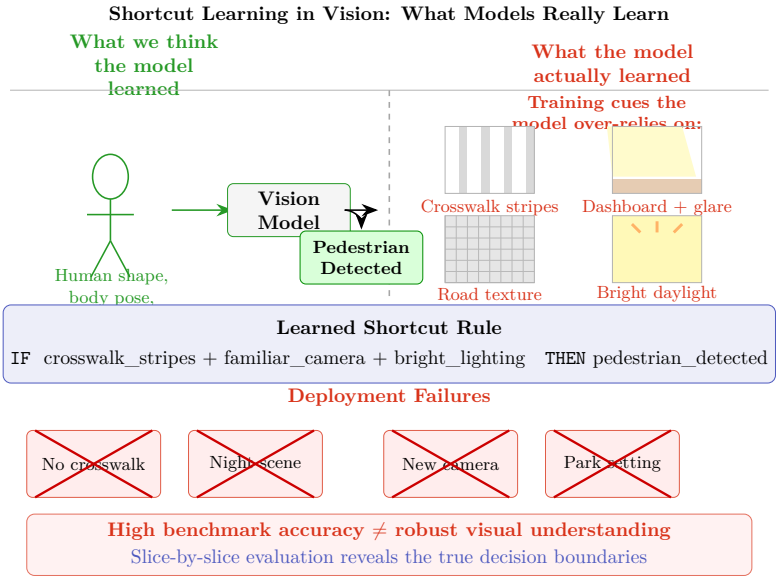


Figure 8.3: Shortcut learning occurs when models discover easily-exploitable correlations rather than learning robust visual concepts. High benchmark accuracy can mask reliance on background cues, camera artifacts, or contextual shortcuts that fail during deployment under different conditions.

The overall benchmark score might still look respectable, especially if the test distribution resembles the daytime training distribution. But Table 8.3 tells a different story. The detector loses much more performance when crossing markings disappear, illumination worsens, and the camera changes. Because several factors move together, this table does not identify the cause. It does support a shortcut hypothesis: background or acquisition cues may be carrying part of the signal instead of robust pedestrian evidence.

The right claim is therefore not "the model fails everywhere," and it is also not "crossing paint is definitely the cause." It is narrower and more useful: the detector appears competent on the familiar camera-and-crossing regime but is not yet reliable across the deployment conditions that matter. That claim leads to better next steps, such as collecting non-crossing positives, holding out

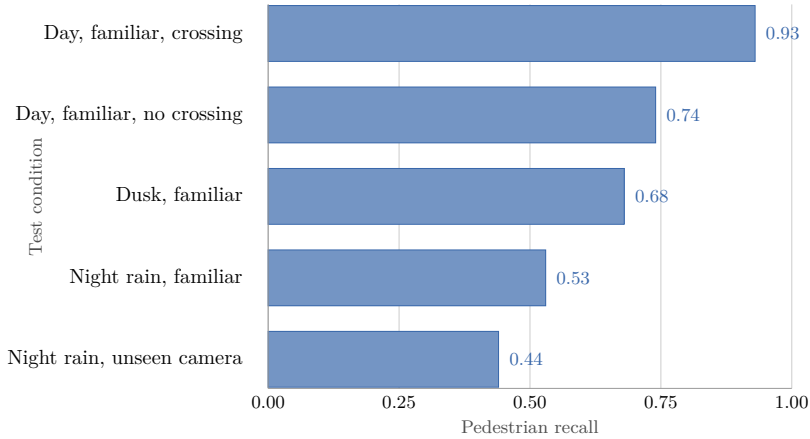


Figure 8.4: The slice pattern becomes visually obvious once recall is plotted by condition. Performance stays high in the familiar crossing regime, then falls as the detector loses crossing paint, daylight, and finally the familiar camera itself.

camera sources by design, and reviewing night failures frame by frame instead of celebrating one aggregate score.

> **Practical Note.** Shortcut diagnosis usually comes from slice design, source breakdowns, and failure review rather than from one dramatic heatmap. If a visual concept matters, evaluate it under the backgrounds, cameras, and conditions that might tempt the model to use something easier.

8.11.3 Robustness and Domain Shift

An image model can perform well on the benchmark distribution and still fail under:

- new lighting conditions,
- small geometric shifts,
- blur or motion artifacts,
- different cameras or sensors,
- new backgrounds,

- rare subgroups or unusual poses.

This is why average test accuracy is only one part of evaluation. Vision systems should also be stress-tested under plausible shifts and broken down by subgroup, condition, or source.

> **Evidence Box.** Evidence that a vision model is genuinely learning the intended visual concept should include more than clean benchmark accuracy. Useful supporting evidence includes robustness to small shifts, reasonable behavior under augmentation, inspection of failure cases, subgroup analysis, source breakdowns, and comparisons against simpler baselines.

8.11.4 Annotation Policy and Label Noise

Vision labels are often less objective than they first appear. For pedestrian detection, the dataset policy must decide how to label tiny far-away people, heavily occluded people, reflections, partial bodies, or cases where the person is visible but not clearly localizable. Annotation disagreement is not always a sign of bad labeling; sometimes it means the task boundary itself is fuzzy.

> **Warning.** Write down the annotation policy before training. In vision, label noise often comes from ambiguous visibility rules, crop choices, or object-boundary disagreement rather than from random mistakes alone.

8.12 Robustness by Nuisance-Factor Stress Tests

This section converts shortcut suspicion into a test plan. Robustness evaluation should be organized around nuisance factors: properties of the input that should not change the correct label but that vary across deployment conditions. Naming the factors first makes the stress tests concrete and the coverage gaps visible.

Naming the nuisance. For the pedestrian-detector case, the

relevant nuisance factors include lighting level (daylight, dusk, night, artificial illumination), weather and precipitation (dry, wet, fog, rain on the windshield), motion blur (from the vehicle or the pedestrian), camera-specific artifacts (compression, rolling shutter, chromatic aberration), and distance-related scale change. For a medical imaging classifier, the nuisance factors include scanner model, acquisition protocol, patient position, image compression, and contrast agent.

Predicting the failure. Before running any test, state which nuisance factors are likely to degrade performance and why. For night conditions, the model may rely on high-contrast edges that become invisible in low-light. For rain on the windshield, texture-based features may be disrupted. Making the prediction explicit before testing means the results provide information regardless of outcome: a predicted failure that materializes confirms the concern; a predicted failure that does not materialize is evidence of genuine robustness.

Testing deliberately. Collect or synthesize examples that represent each nuisance factor at several intensity levels. Report per-nuisance performance separately, not only as an aggregate. An average that combines strong daylight performance with severe night failure conceals the operational risk.

Deciding whether the failure matters. Not every nuisance failure requires a fix. If the deployment context genuinely excludes nighttime operation—the shuttle only runs during daylight hours—then night failure is not an operational concern. The stress-test output should be connected back to the deployment specification, not treated as a standalone ranking of robustness.

> **Problem-Solving Technique. Nuisance-factor stress-test habit.** Name the nuisance, predict the failure, test deliberately, report per-nuisance, and judge relevance to deployment. The habit is the structure; the specific tests will differ across domains.

8.13 Deployment-Camera Realism

Many of the chapter's earlier concerns collapse into one practical rule here: the sensor is part of the task. Vision systems inherit the acquisition process. If the benchmark images come from one camera and deployment images come from another, the task has changed even if the object categories have not. Camera height changes object geometry. Lens choice changes distortion. Compression changes texture. Exposure changes what is visible at night. Dirt, rain, glare, and rolling shutter all affect the evidence the model receives.

For the running pedestrian-detector case, a training set collected from public road datasets may still be an imperfect proxy for the actual shuttle camera. The shuttle may sit lower, use a wider lens, record at a different frame rate, and see more campus-specific lighting and pedestrian behavior. A system that looks strong on benchmark road scenes may therefore still be weak on the real sensor pipeline.

> **Decision Box.** In vision, the deployment camera is part of the task definition. If camera geometry, image quality, or capture conditions differ materially from training, the model has not yet been tested on the real problem.

Before claiming the detector is ready, a minimal stress-test matrix should check at least:

- daylight, dusk, and night,
- dry roads, rain, and glare,
- familiar and unseen camera sources,
- marked crossings and unmarked curb approaches,
- near, medium, and far pedestrian scale.

This is not bureaucratic caution. It is how the team discovers whether the system is robust to the actual image-acquisition process or only to the benchmark story.

8.14 Saliency Is Diagnostic Evidence, Not Proof

Students sometimes hope that a heatmap or saliency map fully explains a model. Such tools can be informative, but they are not definitive. Explanation methods are themselves model-dependent and can be unstable. They should be used as diagnostic evidence, not as proof that the model "looked at the right thing."

This matters because the rest of the chapter is about evidence quality. A saliency map may help localize suspicion or give a clue about model focus, but it does not by itself settle whether the model learned the intended cue or a shortcut.

8.15 Chapter Artifact: A Vision Design and Stress-Test Card

By the end of this chapter, the reader should be able to produce a one-page design and stress-test card before making a serious vision claim. The framing memo named the decision, the metric worksheet named good behavior, the evaluation plan protected the claim, the baseline sheet justified linearity, the structure diagnosis sheet justified leaving it, the training report justified the learned-model claim, and the reuse-and-adaptation plan clarified what transfer could legitimately explain. This card adds the vision-specific layer: what the camera sees, what variation should be ignored, what shortcuts are likely, and what slices must be passed before a benchmark result means anything operational.

> **Problem-Solving Technique. Vision design and stress-test card.** Before reporting performance, write down the task type, intended signal, nuisance variation, acceptable augmentations, shortcut risks, deployment camera, critical slices, and required stress tests.

Example 8.6 (Filled Vision Design Card)**.** For a pedestrian detector on dashboard-camera footage, a sensible card would read

as follows: task type is *detection*; target visual signal is pedestrian shape, motion, and local road context; nuisance variations include small translation, daylight change, and mild blur; label-changing variations include replacing the pedestrian with a sign or removing the pedestrian entirely; acceptable augmentations include small crops, brightness changes, and mild weather corruption but not arbitrary vertical flips or unrealistic rotations; shortcut risks include crossing paint, road background, camera height, and annotation crop policy; the backbone may be a pretrained road-scene detector; deployment uses a low-mounted dashboard camera at dusk and night as well as daytime; critical slices include marked versus unmarked crossings, night rain, and unseen camera sources; required stress tests include low-light slices, rain, glare, source-heldout evaluation, and manual review of missed close-range pedestrians.

8.16 Common Mistakes in Vision

Mistake Checklist.

- **Treating pixels as independent features.** Vision models should exploit locality and neighborhood relationships whenever the task allows.
- **Using augmentations without asking what they mean.** An augmentation is a modeling assumption; if the label should change, the augmentation is harmful.
- **Declaring understanding from benchmark accuracy.** Benchmark accuracy does not prove robustness, causal understanding, or safe deployment behavior.
- **Ignoring data collection bias.** A poor data pipeline can create misleading success even with a strong architecture.
- **Forgetting the deployment camera.** The camera and acquisition process are part of the prob-

lem.

8.17 Looking Ahead

This chapter grounded representation learning in vision: invariance, locality, augmentation, shortcut learning, and camera realism. The next chapter keeps the same methodological discipline but changes the source of difficulty. Instead of asking which visual variation should be ignored, it asks which linguistic context must be preserved and when external grounding is needed before fluent output can be trusted.

Chapter Summary

- This chapter answers one question: which visual variations should the model ignore, which must it preserve, and what evidence shows that it learned the right cue rather than a shortcut?
- Vision begins with an invariance audit, not with architecture shopping.
- Task formulation matters before architecture depth: classification, detection, segmentation, and tracking need different outputs, different spatial detail, and different evidence.
- Convolution matters because vision is local before it is global, and parameter sharing plus pooling create a useful but limited image-specific inductive bias.
- Every augmentation is a claim about the world and should be justified by the task.
- Vision models are especially vulnerable to shortcut learning, which is why robustness, slice analysis, and stress testing matter as much as benchmark accuracy.
- The main deliverable of the chapter is a vision design and stress-test card that makes invariance assumptions and shortcut checks explicit.

Study Questions

1. Why is it misleading to think of image understanding as only a pixel-processing problem?
2. What does an invariance audit force the modeler to state explicitly?
3. What does parameter sharing buy us in a convolutional layer?
4. Why can max pooling make a representation less sensitive to tiny image shifts?
5. Why must augmentation choices depend on the task?
6. Why does high benchmark accuracy not prove robust visual understanding?
7. Why can a pedestrian detector look strong on familiar crossings yet still be weak on the real deployment route?

Exercises

1. Use the vertical-edge filter from the chapter on a new 3×3 patch of your own design. Compute the convolution response by hand.
2. Explain in words why a dense network on raw pixels may need far more parameters than a convolutional network to detect the same local pattern at many image positions.
3. Perform a short invariance audit for a pedestrian-detection system. Name two variations that should preserve the label and two that should change what the system must output.
4. Give one example of an augmentation that is likely helpful for a generic object classifier and one example of an augmentation that could be harmful in a specialized vision task.
5. Describe a real-world setting in which object detection is necessary but classification alone is insufficient.
6. Give two examples of shortcut signals a vision model might exploit in a poorly designed dataset.

7. Suppose a model performs well on benchmark photos but poorly on low-light phone images from deployment. Give two plausible technical reasons for the gap.
8. Sketch a minimal stress-test matrix for a dashboard-camera pedestrian detector. Include at least one lighting slice, one weather slice, one source slice, and one scene-structure slice.
9. Use Table 8.5 to sketch a one-page design and stress-test card for a vision task of your choice. If you are carrying one case through the book, explicitly preserve the same task, review policy, and evaluation commitments while adding the vision-specific invariance and shortcut checks.

Challenge Exercises

1. Explain carefully why convolution supports translation equivariance rather than perfect invariance, and why later layers or augmentation are still needed.
2. A vision model trained to detect disease from scans performs extremely well, but a hospital discovers that the model relies heavily on scanner-specific marks. Explain how this can happen and propose an evaluation protocol that would reveal it.
3. Compare two deployment strategies for a small vision dataset: frozen pretrained backbone plus linear head, versus full fine-tuning. State when each would be preferable and what evidence would justify the choice.
4. Design a stress-test suite for a road-scene pedestrian detector that uses at least four shift types or subgroup checks. Explain what failure in each case would mean.

Augmentation	Implicit claim	When the claim fails
Random crop and translate	Location within the frame should not change the label	Tasks where position is semantically relevant (e.g. reading a gauge value that depends on dial position)
Horizontal flip	Left-right orientation is irrelevant to the label	Text recognition, medical laterality (left/right hand), traffic signs
Color jitter	Exact hue and saturation are not defining features	Ripeness grading by color, skin-condition classification, specific plant disease detection
Gaussian blur	Fine-grained texture detail is not required	Texture classification tasks where fine-grained surface structure is the target
Cutout or random erasing	The label is still clear even when parts are occluded	Any task where the critical feature may be the part that was erased
Rotation by large angle	The label is invariant to orientation	Text detection, aerial images where "up" carries orientation meaning

Table 8.2: Each augmentation encodes a claim about label invariance. Applying an augmentation whose claim is false for the task will teach the model a wrong invariance.

Evaluation slice	Pedestrian recall
Daytime, familiar camera, marked crossing visible	0.93
Daytime, familiar camera, no crossing markings	0.74
Dusk, familiar camera	0.68
Night rain, familiar camera	0.53
Night rain, unseen dashboard camera	0.44

Table 8.3: A slice breakdown can reveal a shortcut hypothesis, but these slices still confound markings, illumination, and camera source rather than isolating one cause cleanly.

Nuisance factor	Predicted failure mode	Test method	Deploy risk?
Night / low light	Misses pedestrians at distance	Held-out night clips	Yes, if shuttle runs at night
Rain on windshield	Spurious edge detections	Synthetic blur or real rain clips	Moderate
Unfamiliar campus zone	Background shortcut does not transfer	Out-of-area held-out set	Yes, if routes expand

Table 8.4: A structured stress-test record connects each nuisance factor to a predicted failure, a test, and a deployment-risk judgment. This prevents robustness tests from being disconnected from the deployment specification.

Card field	What must be written clearly
Seven-question focus	Questions 3, 4, 6, and 7: what image evidence exists, what visual structure matters, what tests justify the claim, and how deployment conditions affect action
Task type	Classification, detection, segmentation, tracking, or another structured vision output
Target visual signal	What visual content the model is truly supposed to use
Nuisance variations to ignore	Lighting, small translation, blur, background, viewpoint, or other variations that should preserve the label
Variations that should change the label	The changes that really do alter the class or output structure
Acceptable augmentations	Which label-preserving transformations are justified by the task
Likely shortcut risks	Background, watermark, scanner marker, room lighting, crop policy, or sensor artifact
Pretrained backbone choice	What representation source will be reused and why
Deployment camera or sensor	What image-acquisition process the real system will actually face
Critical slices	Which deployment-relevant conditions such as night, rain, unseen cameras, rare poses, or changed backgrounds must be checked separately
Required stress tests	Shift tests, subgroup checks, source breakdowns, or robustness probes that must be run before a claim is trusted

Table 8.5: A vision design and stress-test card turns the chapter's ideas into a practical protocol before benchmark numbers are reported.

Chapter 9

Language: Context and Grounding

Learning Goals

- Explain why language tasks depend on deciding what linguistic structure must be preserved rather than on choosing an architecture by habit.
- Use a context audit to decide whether token counts, local order, long-range context, or external grounding are needed.
- Understand tokenization, bag-of-words, and n-grams as serious representation choices rather than as obsolete preprocessing steps.
- Explain why contextual representations, sequence models, and transformers matter when meaning depends on surrounding language.
- Distinguish generation quality from grounded, trustworthy behavior.
- Distinguish a base language model from an instruction-following model and explain what additional training stages produce the behavioral difference.

- Use prompting strategies to choose the simplest inference-time intervention that produces reliable behavior.

- Choose among prompting, retrieval, and fine-tuning based on what kind of behavioral change is needed and what evidence is available.

- Produce an NLP context-and-grounding card before making a serious NLP system claim.

The vision chapter treated invariance, shortcuts, and deployment realism as central problems. Language creates a parallel but different challenge. The same words can mean different things in different contexts, while the same idea can be expressed through very different surface forms. A language system therefore has to decide what may be collapsed away, what must be preserved, and when external grounding is required before fluent output can be trusted. The chapter follows that route in order: first decide the task shape and the context that must survive, then decide what representation preserves it, then decide how grounding and behavior design change the claim, and finally decide what evaluation would make the claim defensible.

> **Problem-Solving Lens.** This chapter asks an NLP-specific modeling question: what linguistic structure must the task preserve, and what evidence shows that the system is using that structure rather than producing merely fluent behavior? The answer drives tokenization, baseline choice, contextual modeling, retrieval, and evaluation.

By the end of the chapter, the reader should be able to write an NLP context-and-grounding card before claiming that a language system is ready.

9.1 Opening Dilemma: Same Words, Different Meaning; Same Meaning, Different Words

Suppose a university wants to build a course-support assistant for a large introductory course. Students send messages such as:

```
I missed the lab because I was sick.
Can I still submit tonight?
```

```
I was sick and missed today's lab.
Is late submission still possible?
```

The wording is different, but the underlying request is almost the same. Now compare those with:

```
I missed the lab.
Does that affect my visa attendance record?
```

Surface overlap remains, but the operational meaning has changed sharply. One question may be answered from the course late-policy page. Another may require escalation to a human adviser because the institutional risk is different.

Language is difficult because surface form is not enough. The sentence `the bank approved the loan` uses the token `bank` differently from `the river reached the bank`. Meanwhile, two answers to the same question may use different wording while expressing the same underlying idea. If a model preserves too little structure, it collapses distinctions the task needs. If it preserves structure in the wrong way, it may sound fluent without being grounded, faithful, or useful.

This means NLP should not begin with model names. It should begin with a judgment question: what information must survive the representation? Some tasks care mainly about which words are present. Some care about local order such as negation. Some depend on long-range context, discourse, or entity reference. Some require external knowledge or retrieval before any answer can be trusted.

Throughout this chapter, keep one running case in view: a

grounded course-support assistant. The same incoming student message may need to be categorized, matched to the correct policy passage, answered conservatively, or escalated to a human. That single workflow is useful because it shows why representation, retrieval, generation, grounding, and evaluation are not separate concerns in modern NLP. It also gives the chapter its shape: preserve the right linguistic structure first, then decide what evidence must accompany the answer, then decide what the system should do when that evidence is weak.

9.2 Problem-Solving Technique: Context Audit

Problem-Solving Technique. Context audit. Before choosing an NLP representation, write down:

- whether token presence is enough,
- whether local order matters,
- whether long-range dependence matters,
- whether external knowledge or retrieval is required,
- whether dialect, register, or identity variation matters,
- what output the task requires: a label, a span, a ranked list, or generated text.

This is the chapter's most reusable habit. A bag-of-words baseline, an n-gram repair, a contextual encoder, or a retrieval-augmented generator are not interchangeable choices. They encode different claims about what linguistic structure matters. The context audit forces that claim onto the page before the model is built.

Decision Box. In NLP, model choice should follow the context audit. If the task can safely collapse order and ambiguity, a sparse lexical baseline may

be enough. If meaning depends on local or global context, the representation must preserve it. If correctness depends on outside facts, grounding becomes part of the system design rather than an optional upgrade.

Practical Note. Workflow sketch: first decide whether token presence, local order, long-range context, or grounding matters. Then start with the simplest baseline that preserves the needed structure, add n-grams or contextual encoders if order and context matter, add retrieval if current evidence is required, and only then choose prompting, fine-tuning, or a hybrid after checking how the workflow will be evaluated and when it should abstain or escalate.

Problem-Solving Technique. Artifact preview: NLP context-and-grounding card. Keep six fields visible from the start: task type, output type, the linguistic structure that must survive, whether grounding is required, the escalation rule, and the evidence that would justify the final claim.

Problem-Solving Technique. Wrong question → better question. Wrong: "Which NLP model is strongest?" Better: "What linguistic structure must survive, what evidence must be available at prediction time, and what kind of output does the workflow actually need?"

Problem-Solving Technique. Signal to trust → signal to doubt. Trust: the system retrieves the right evidence, preserves the task-relevant context, cites or exposes the source when needed, and escalates when the evidence is weak or the case is high risk. Doubt: the answer sounds fluent but the supporting passage is wrong, missing, stale, or never shown, and the workflow gives no safe way to abstain or defer.

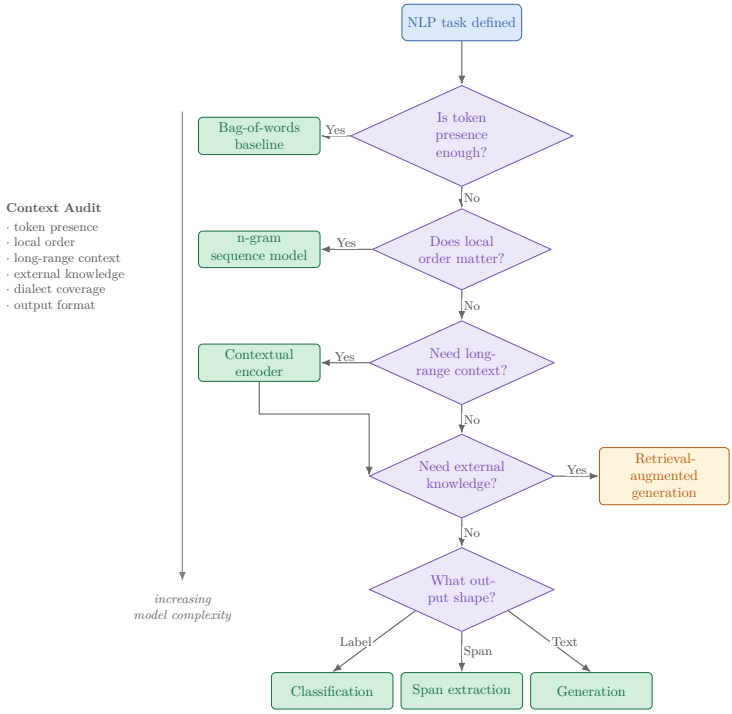


Figure 9.1: Context audit flowchart for NLP representation choice. Each decision point corresponds to a key question in the context audit framework. The path through the flowchart determines the appropriate representation and model architecture, from simple bag-of-words baselines to retrieval-augmented generation systems.

9.3 Task Shape Before Representation Depth

The first serious question in NLP is not "which model is strongest?" It is "what behavior should the system produce?" This section fixes the output object first so the later representation and evaluation choices have a clear job.

9.3.1 Classification and Sequence Labeling

Text classification assigns one label to a document or sentence: spam versus not spam, topic A versus topic B, positive versus negative. Sequence labeling assigns outputs to tokens or spans, such

as named entities, part-of-speech tags, or extracted fields. Sequence labeling usually needs finer contextual precision because token meaning depends heavily on neighboring tokens.

Example 9.1 (A Tiny Slot-Label Example)**.** Suppose the message is `I missed Lab 3 because I was sick tonight.` A sequence labeler might assign labels like this:

Token	Label
`I`	O
`missed`	O
`Lab`	B-LAB
`3`	I-LAB
`because`	O
`I`	O
`was`	O
`sick`	B-REASON
`tonight`	B-TIME
`.`	O

The exact tag set is not the point. The point is that the system must preserve token-level detail well enough to identify the lab reference, the reason, and the timing cue before it can route, retrieve, or answer safely.

9.3.2 Retrieval and Ranking

Some NLP systems are supposed to find relevant documents, passages, or answers rather than synthesize new text. In those cases, the output is a ranked list. This matters because evaluation should ask whether the right evidence was found, not only whether the final wording sounds plausible.

> **Math Lens.** Suppose a retrieval system returns four passages with relevance labels $[0, 1, 0, 1]$, and suppose there are three relevant passages in the whole collection.
>
> $$\text{precision@2} = \frac{1}{2}, \qquad \text{recall@4} = \frac{2}{3}, \qquad \text{RR} = \frac{1}{2}.$$

Precision@2 asks how clean the top of the list is, recall@4 asks how much of the relevant material the top four covered, and reciprocal rank cares only about where the first relevant passage appeared.

9.3.3 Translation, Summarization, Question Answering, and Generation

Machine translation must preserve meaning while changing surface form. Summarization must compress content without inventing unsupported claims. Question answering may require retrieval, reasoning, or synthesis across several passages. Open-ended generation asks the system to continue, rewrite, or compose text under a prompt. These tasks differ sharply in what counts as success.

Question answering itself can be extractive, templated, or generative. An extractive system copies a span from the source, a templated system fills a bounded slot, and a generative system composes a freer response. The safest output style depends on how much wording freedom the task actually allows.

Example 9.2 (Why Task Shape Comes First)**.** A course-support assistant may be built in at least three different ways. It might classify messages into categories such as *deadline*, *grading*, or *technical issue*; retrieve the most relevant syllabus or policy passage; or generate a direct response for the student. Those are not cosmetic differences. They define different outputs, different evaluation standards, and different failure modes.

Decision Box. Choose the smallest NLP workflow that preserves the needed structure.

- Stable lexical cues and label output: start with bag-of-words or n-grams.
- Token-level extraction or strong local-order dependence: sequence labeling or contextual encoders.
- Long-range context, paraphrase, or discourse dependence: contextual sequence models or trans-

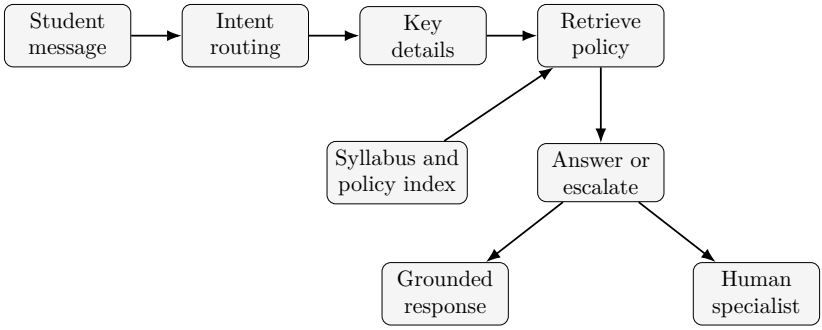


Figure 9.2: A grounded language workflow does not move directly from message to fluent reply. It routes the request, preserves task-relevant details, retrieves authoritative evidence, and only then decides between bounded automation and escalation.

formers.

- Correctness depends on current documents or policies: retrieval must be part of the system.
- Output is a controlled transformation of source text: encoder-decoder modeling is a natural fit.

9.4 Worked Workflow: One Message, Several NLP Jobs

The running case becomes clearer when the whole workflow is written down. Consider the message

```
I missed Lab 3 because I was sick.
Can I submit tonight without penalty?
```

The system should not jump directly from raw text to a fluent answer. It usually has to solve several smaller NLP problems in sequence. Figure 9.2 gives the high-level pipeline, while Table 9.1 names the main subproblems more explicitly. That workflow also previews the rest of the chapter: lexical and contextual representation support routing and slot reading, grounding supports retrieval and answer faithfulness, and behavior design determines whether the system answers, hedges, or escalates.

Stage	Typical output	Role in the workflow
Intent routing	`deadline`, `extension`	Separates routine policy requests from disputes, technical issues, visa questions, or wellbeing cases
Entity or slot reading	`Lab 3`, `illness`, `tonight`	Preserves the details that control retrieval, escalation, and answer wording
Retrieval	syllabus or illness-policy passages	Adds current institutional evidence before any answer is written
Policy decision	answer directly or escalate	Marks which cases are safe to automate and which must go to a human specialist
Grounded response generation	brief answer tied to evidence	Produces readable language that stays faithful to the retrieved text

Table 9.1: One support message often activates several NLP subproblems, not just one. This is why a single NLP system often needs classification, retrieval, generation, grounding, and escalation in the same conceptual frame.

Now compare the same workflow for the message

```
I missed the lab.  Does that affect my visa
attendance record?
```

The lexical overlap is high, but the correct system behavior may be completely different. A direct answer may be unsafe because the question is both higher stakes and more dependent on individual institutional context. The right system may still classify the message, retrieve the most relevant advising page, and then route the student to a human specialist rather than generating a confident answer on its own.

> **Practical Note.** In modern NLP systems, the model is often only one stage in a language workflow. The chapter is easier to read if students keep asking: what subproblem is being solved at this stage, and what later stage depends on it?

9.5 Tokenization as a Modeling Decision

Machine-learning models do not receive meaning directly. They receive tokenized input. Tokenization converts raw text into units such as words, subwords, or characters, and that choice changes what the model is allowed to notice and reuse. It is therefore the chapter's first representational bottleneck.

Tokenization decides what the model is allowed to notice and reuse.

9.5.1 Why Tokenization Matters

Consider the strings `unbelievable`, `un-believable`, and `un believable`. A word-level tokenizer may treat them as distinct whole tokens. A subword tokenizer may expose shared pieces such as `un` and `believe`. A character-level model may see only letters. Each choice changes what the model can generalize across and what it must memorize.

The same issue appears in support messages. Strings such as `late-day`, `late day`, `lab-3`, and `CS101` contain structure that may be split, preserved, or blurred depending on the tokenizer. If the tokenizer handles course codes, dates, or policy terms poorly, the system may fail before the main language model has had any chance to reason well.

> **Prerequisite Bridge.** For this chapter, it is enough to think of a token as a unit the model can read. A document becomes a sequence $(w_1, w_2, \ldots, w_T)$ of such units, and the sequence length T can vary from one example to another.

Tokenization is therefore not clerical preprocessing. It is part of the representation claim. A bad tokenization choice can erase useful regularity before the main model has even started learning.

9.6 Bag-of-Words as the First Serious Lexical Baseline

Definition 9.1 (Bag-of-Words Representation)**.** A bag-of-words representation maps text to token counts while discarding token order.

Suppose the vocabulary contains V tokens. A document is mapped to a vector $x \in \mathbb{R}^V$ whose component x_j records how often vocabulary item j appears. This sounds simple because it is simple. That is exactly why it remains a valuable baseline. If the task depends mainly on broad lexical content, topic markers, or keyword presence, bag-of-words can work well.

> **Math Lens.** Take the two sentences `dog bites man` and `man bites dog`. If the unigram vocabulary is ordered as `[dog, bites, man]`, then both sentences have the same count vector
>
> $$x_{\text{uni}} = (1, 1, 1).$$
>
> Now use a bigram vocabulary with four entries: `dog bites`, `bites man`, `man bites`, and `bites dog`. The first sentence has
>
> $$x_{\text{bi}} = (1, 1, 0, 0),$$
>
> while the second has
>
> $$x_{\text{bi}} = (0, 0, 1, 1).$$
>
> Bag-of-words collapses the order distinction; bigrams repair exactly that local structure.

For the running support-assistant case, bag-of-words can already be a serious routing baseline. Messages containing words such as `grade`, `rubric`, `deadline`, `extension`, `compile`, or `autograder` often reveal a lot about which queue or policy family is relevant. That does not make bag-of-words sufficient for the whole system, but it does make it a meaningful first rung on the baseline ladder.

9.6.1 Worked Example: Same Words, Different Meaning

Consider the two sentences

```
dog bites man
man bites dog
```

Their bag-of-words vectors are identical. Both contain one occurrence of `dog`, one of `bites`, and one of `man`. Yet their meanings are very different.

This example matters because it teaches the right judgment rule. A representation is not evaluated by whether it sounds sophisticated. It is evaluated by whether it preserves the distinctions the task requires. If the task is broad topic detection, bag-of-words may be enough. If the task is to identify who acted on whom, it is inadequate.

Example 9.3 (Bag-of-Words Can Still Be Strong for Routing)**.** Suppose the course-support system must first route messages into *deadline*, *grading*, *technical issue*, or *administrative request*. A message containing `deadline`, `extension`, and `submission` may be routable quite well with bag-of-words alone. That is a real success, not an embarrassment. It shows that some tasks depend mainly on lexical content even if later grounded answering requires more structure.

The point is not that every useful NLP system must build an explicit parse tree. The point is that language has compositional structure, and a bag-of-words vector cannot record which words group together or which phrases modify which others.

> **Failure Box.** Bag-of-words is not "dead." It is a strong baseline when token presence carries most of the signal. It fails when the task depends on distinctions that token counts erase.

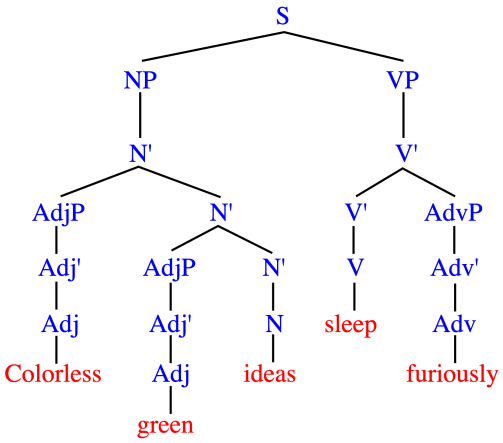


Figure 9.3: A syntax tree is one visual reminder that sentences contain internal structure beyond token counts. The example sentence is "Colorless green ideas sleep furiously." Public-domain image from Wikimedia Commons [54].

9.7 N-Grams as the Local-Order Repair

One classical way to restore some order information is to use n-grams. A unigram is a single token, a bigram is an adjacent token pair, and a trigram is a length-three token sequence. Including bigrams lets the model distinguish between `dog bites` and `bites dog`, which plain bag-of-words cannot. If this local repair is enough, the model family need not become more complex. If it is not, the chapter moves next to contextual meaning.

If the needed context is local, repair the representation before escalating the model family.

This is one of the chapter's main modeling lessons. Some problems that appear to require deep architectures can be solved by preserving a little more local structure. N-grams are therefore not just historical trivia. They are evidence that the missing structure may be local rather than global.

Example 9.4 (Local Order Matters in Policy Language)**.**

Consider the phrases `late submission allowed` and `late submission not allowed`. A unigram representation treats both as nearly the same bag of words and can understate the force of the local negation. A bigram feature such as `not allowed` repairs exactly that kind of short-range structural loss. This is why n-grams remain conceptually important: sometimes the missing structure is not broad world knowledge but one small local dependency with operational consequences.

9.8 From Static to Contextual Meaning

Bag-of-words and n-grams produce sparse count vectors. Modern NLP often uses dense embeddings instead.

Definition 9.2 (Word Embedding)**.** A word embedding is a dense vector representation of a token learned so that tokens occurring in similar contexts receive similar vectors.

In modern NLP, *word* is often shorthand for *token*: the embedding may correspond to a word, a subword piece, or even a character, depending on the tokenizer.

Embeddings allow statistical sharing across related words. If `exam`, `quiz`, and `test` occur in similar contexts, their vectors may become nearby in embedding space. This makes downstream learning easier when exact lexical overlap is limited.

9.8.1 Static Embeddings

Static embeddings assign one vector per word type. This is already much more expressive than isolated one-hot vectors, but it still has a serious limitation: a token receives the same representation regardless of where it appears.

9.8.2 Contextual Representations

Contextual models allow a token's representation to depend on the surrounding text. This is one of the deepest shifts in modern NLP.

Example 9.5 (Why Contextualization Matters)**.** In the sentence `the bank approved the loan`, surrounding words suggest a financial institution. In `the river reached the bank`, surrounding words suggest a geographic boundary. A contextual model can represent these two uses differently even though the surface token `bank` is the same.

The same issue appears in the running case. The token `extension` may refer to a deadline extension, a visa extension, or an extension number for a campus office. A good support assistant should not collapse those meanings merely because the same surface word appears. The surrounding language determines which workflow, evidence source, and risk level are appropriate.

A contextual representation does not assign meaning to a word in isolation; it computes meaning inside a sentence.

9.9 Sequence Models and Attention as Context Builders

Once text is treated as a sequence rather than a bag, the next question is how context is carried. This is the point where the chapter moves from representation choices to mechanisms that build context dynamically.

9.9.1 Recurrent Models

Recurrent neural networks and LSTMs process text one token at a time while carrying forward a hidden state. Conceptually, that hidden state summarizes what has been seen so far. These models were historically important because they made context explicit, but they also showed the limitation of compressing long sequences into a moving state.

9.9.2 Attention and Transformers in NLP

Transformers became dominant in NLP because attention allows each token representation to look directly at other tokens that

matter, rather than relying only on a compressed recurrent summary. If a pronoun depends on a noun many words earlier, or a negation changes the sentiment of a later phrase, attention provides a flexible way to connect those pieces.

This is why transformer-based pretraining changed the field so quickly [17, 61]. The important idea is not that attention is magical. It is that contextual representation learning became far more scalable and reusable.

> **Advanced Note.** An earlier chapter already explained attention as a general representation mechanism. In NLP, the central point is task-specific: attention matters because token meaning often depends on distant and selective context, not merely on adjacent words.

9.10 Language Modeling as Sequence Prediction

Definition 9.3 (Language Model)**.** A language model assigns probabilities to sequences of tokens, typically by predicting each next token conditioned on the tokens that came before it.

This chapter uses the classic autoregressive setup because it makes the next-token logic explicit. Masked or denoising objectives are also important in modern NLP, but the next-token factorization is the clearest starting point for the sequence-prediction story here. Once context is represented, generation can be described as repeated prediction under that context.

If a sequence is $(w_1, w_2, \ldots, w_T)$, a standard factorization is

$$P(w_1, w_2, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}).$$

Conceptually, language modeling asks one repeated question: given the text so far, what token should come next?

> **Math Lens.** If the model assigns probabilities $p_t = P(w_t \mid w_1, \ldots, w_{t-1})$ to the observed tokens, then the

sequence probability is $\prod_{t=1}^{T} p_t$. The average negative log-likelihood is

$$\mathrm{NLL} = -\frac{1}{T}\sum_{t=1}^{T} \log p_t,$$

and perplexity is

$$\mathrm{PP} = \exp(\mathrm{NLL}) = \left(\prod_{t=1}^{T} p_t\right)^{-1/T}.$$

So perplexity is the inverse geometric mean probability per token: lower is better because the model assigned higher probability to the observed sequence.

Perplexity comparisons are only meaningful when tokenization and evaluation protocol are held fixed. A model scored on subword units is not directly comparable to a model scored on different tokens unless the setup is normalized carefully.

9.10.1 Worked Example: A Tiny Bigram Model

Suppose a tiny corpus contains the following counts after the token `bank`:

$$\texttt{approved} : 6, \qquad \texttt{loan} : 1, \qquad \texttt{closed} : 3.$$

A simple bigram model estimates

$$P(\texttt{approved} \mid \texttt{bank}) = \frac{6}{10} = 0.6.$$

If the current context ends with `bank`, then `approved` is the most likely next token under this tiny model.

The example is deliberately simple, but it teaches the core logic cleanly. Generation is repeated sequence prediction, not mysterious text magic.

9.10.2 Encoder-Decoder Models as Conditional Sequence Mapping

Not every language task is best understood as open-ended continuation. Translation and summarization are better viewed as conditional sequence mapping: the input sequence is the evidence or source text, and the output sequence is a new sequence whose content must be constrained by that source.

In that setting, the model is not primarily asking, "What token comes next in this text?" It is asking, "Given this source sequence, what target sequence should be produced?" That difference matters because the output is judged against the input's meaning, not only against local continuation plausibility.

Next-token prediction extends text. Conditional sequence mapping transforms one sequence into another.

The architecture can be kept conceptually light: an encoder reads the source, a decoder writes the target, and the source remains available as the main conditioning signal throughout generation. The important modeling decision is not the machinery itself. It is whether the task shape is source-to-target transformation or free continuation.

> **Math Lens.** For a source sequence x and target sequence $y = (y_1, \ldots, y_T)$, the conditional generation objective is
>
> $$P(y \mid x) = \prod_{t=1}^{T} P(y_t \mid y_1, \ldots, y_{t-1}, x).$$
>
> Training therefore rewards target tokens that are correct given both the source and the already-generated prefix. The source is not background context. It is part of the probability model.

Listing 9.1: A minimal encoder-decoder training sketch

```
encode the source sequence x
for each target position t:
    use the source representation and previous target
        tokens
```

```
    predict the next target token y_t
    add loss for the correct y_t
update encoder and decoder parameters
```

Example 9.6 (Tiny Translation Example)**.** Input: `The student missed the lab because of illness.`
Output: `Der Student verpasste das Labor wegen Krankheit.`

The output is not a continuation of the English sentence. It is a constrained transformation into another language. The source sentence provides the evidence that must be preserved, while the target sentence must be fluent in the new language.

Example 9.7 (Tiny Summarization Contrast)**.** Source: `Late work due to illness is possible only after the extension form is filed within forty-eight hours.`
Good summary: `Illness extensions require a form within forty-eight hours.`
Bad summary: `Illness always allows late work.`

The bad summary sounds plausible, but it drops the operational constraint that the source required. This is exactly the kind of failure that sequence transformation can hide if the model is fluent but not faithful.

> **Practical Note.** Choose encoder-decoder structure when the task has a clear source text and a target text with different surface form but preserved content, such as translation, abstractive summarization, or controlled rewriting. Plain continuation is a better fit when the system should extend an already-open prompt, improvise within the prompt's style, or generate text without a fixed source-output boundary.

> **Practical Note.** The evidence question also changes. In continuation, the prompt itself is usually the main context. In sequence transformation, the source text is evidence that constrains the whole output, and any retrieval you add should support that source rather than replace it.

> **Warning.** Encoder-decoder structure does not guarantee faithfulness. If the target is freer than the source permits, the model can compress away conditions, hedge too little, or invent support that was never in the source. High-risk tasks may therefore require extraction, citation, or grounded answer policies even when encoder-decoder modeling fits the task shape well.

9.11 Decoding, Retrieval, and Grounding as Behavior Design

The chapter now shifts from representation and training objectives to system behavior. Once a language model produces next-token probabilities, the system still needs a behavior policy.

9.11.1 Decoding Changes the Behavior

Greedy decoding always picks the highest-probability next token. Sampling introduces randomness. Temperature changes how conservative or diverse the output becomes. Two systems built on the same model can therefore behave very differently depending on decoding choices.

9.11.2 Retrieval and Grounding Change the Claim

Fluent continuation is not enough when correctness depends on trustworthy outside information. In those settings, retrieval and grounding are not optional embellishments. They change what the system is allowed to claim.

This is the deeper reason retrieval-augmented generation matters. Once a language system is required to retrieve evidence before answering, the strongest defensible claim is no longer "the model knows the answer." It is narrower and more technical: given the current document collection, retrieval pipeline, and answer policy, the system can often find the right supporting material and stay faithful to it [43]. That claim is better because it can

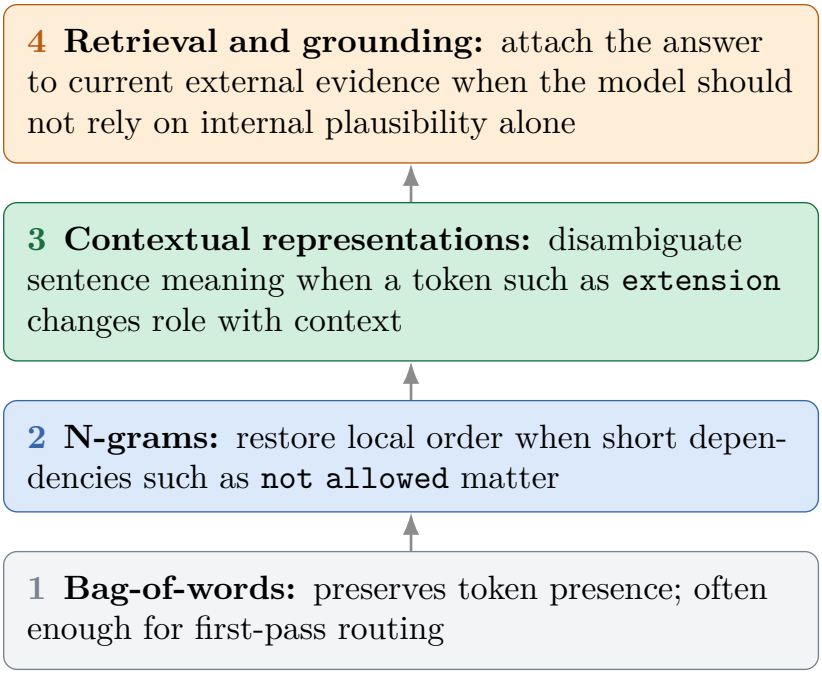


Figure 9.4: The chapter's representation ladder moves from token presence to local order, sentence context, and finally grounded external evidence. The right choice is not always the top rung; it is the smallest rung that preserves the structure the task truly needs.

be audited. It also means many apparent language failures are really retrieval or policy failures earlier in the pipeline.

In NLP, behavior is determined not only by the model, but also by decoding policy and by whether the system is grounded.

Example 9.8 (Grounded Versus Ungrounded Answering)**.** Suppose a student asks a course-support assistant, "I missed Lab 3 because I was sick. Can I submit tonight without penalty?" A pure generator may produce a fluent answer from internal statistical patterns alone, such as "Yes, illness usually allows a late submission." A grounded system instead retrieves the current course late-policy page and conditions the answer on that source. The second system supports a stronger claim because its output can be traced to current evidence rather than to generic linguistic

plausibility.

9.11.3 Retrieval Contract: What Evidence Must Be Present?

In grounded systems, retrieval quality is not only about whether something relevant appears somewhere in the top few results. The stronger question is whether the retrieved evidence is good enough to support the answer policy the system is about to use.

> **Problem-Solving Technique. Retrieval contract.** Before trusting a grounded answer, ask four questions: was the right passage retrieved, was it authoritative, was it current, and was it complete enough to support the answer without hidden conditions or missing exceptions?

This matters because retrieval failure is not only absence. The system can retrieve a plausible-looking but stale page, a related but incomplete rule, or a fragment that omits the condition that changes the whole answer. A top-ranked result may therefore look linguistically relevant while still violating the operational contract of the task.

9.11.4 Retrieval Pipeline Anatomy

A practical retrieval stack usually has five stages:

- **Chunking.** Split documents into passages that can be searched; too coarse hides the right sentence, too fine removes surrounding conditions.
- **Indexing.** Build a lexical or vector index so the system can search quickly.
- **First-pass retrieval.** Pull a broad candidate set using a cheap similarity score.
- **Reranking.** Rescore the candidates with a stronger model or additional features.
- **Grounding and citation.** Answer only from the selected

Contract element	What must be true	Typical failure if it is missing
Relevance	The passage addresses the user's actual policy or factual question	The answer is fluent but about the wrong rule
Authority	The source is one the organization is willing to stand behind	The system cites unofficial or low-trust material
Freshness	The passage is current for the semester, product version, or policy window that matters	The answer is correct for last term and wrong for this one
Completeness	The evidence includes the conditions, exceptions, and escalation path needed for action	The answer omits the one clause that changes what the user should do

Table 9.2: A grounded answer is only as good as the retrieval contract behind it. Hit rate alone does not tell the whole story.

evidence, and expose the source when the user or reviewer needs an audit trail.

> **Practical Note.** If chunking strips away the exception clause, no reranker can recover it later. Retrieval quality starts with how the evidence was segmented.

Once this contract is stated explicitly, retrieval evaluation becomes sharper. A system should not get full credit merely because a vaguely related document appeared somewhere in the ranking. The document must be the right kind of support for the answer the system is going to present.

9.11.5 Retrieval Failure Often Precedes Generation Failure

In grounded language systems, a bad final answer is often a retrieval failure in disguise. The answer may read smoothly while

Pipeline failure	What the answer may look like	What actually failed
No relevant passage retrieved	Fluent but unsupported answer	The generator had no authoritative evidence to ground on
Wrong passage ranked first	Confident answer to the wrong rule	Lexical overlap looked plausible, but the operational intent was missed
Current and stale passages mixed	Contradictory or hedged answer	The evidence set itself was inconsistent
High-risk case not escalated	Polite but unsafe direct answer	The behavior policy failed, not only the language model

Table 9.3: In grounded NLP, many answer failures begin as retrieval or escalation failures before they become generation failures.

being based on the wrong policy passage, an outdated FAQ, or no real support at all. This matters because otherwise teams over-diagnose the generator and under-diagnose the evidence path.

Example 9.9 (Same Generator, Different Evidence)**.** Suppose the same language model is used twice on the question "Can I submit tonight without penalty?" In one run, retrieval returns the current late-policy page and the illness-extension instructions. In another, retrieval returns an outdated FAQ plus a general attendance page. The generator may be identical in both cases, yet the second run is much more likely to produce a polished but operationally wrong answer. That is why grounded systems must be evaluated as pipelines rather than as text generators alone.

9.11.6 Escalation and Abstention Change the Behavior Too

Sometimes the right language-system behavior is not to answer directly at all. A message about visa attendance, harassment, self-harm, disability accommodation, or grade appeal may require specialized context, a stronger audit trail, or institutional authority that the model does not have. In those cases, the right system may classify the request, retrieve supporting information

for the human reviewer, and abstain from final answering.

> **Practical Note.** Generation is only one possible output policy. In support systems, a good abstention or escalation rule can be more valuable than a slightly more fluent answerer.

> **Practical Note.** When evidence is weak, good grounded systems usually tighten behavior rather than loosen it. They answer with explicit support, answer conservatively, or abstain. They should not answer boldly from uncertain retrieval.

> **Failure Box.** Fluency is not proof of truth. A system can produce coherent, confident language while relying on weak context, unsupported continuation patterns, or stale internal knowledge.

9.12 From Language Model to Instruction-Following Model

A language model trained only on next-token prediction learns to continue text in ways that are statistically plausible given its training distribution. That is a useful foundation, but it is not yet a system that follows instructions, answers questions reliably, or behaves consistently across diverse inputs. The gap between a base language model and an instruction-following system requires additional training stages that apply different kinds of behavioral pressure. The previous sections focused on what evidence should enter the system; this section focuses on how the model itself is shaped to respond.

A base language model predicts likely continuations. An instruction-following model responds to requests in a way that is helpful, bounded, and consistent.

This distinction matters because the two systems have different failure modes. A base model produces text that is statistically plausible but may be factually wrong, harmful in some contexts, or unresponsive to the intent behind an input. An instruction-

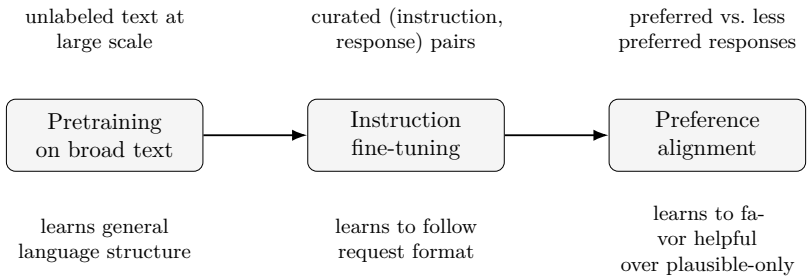


Figure 9.5: Three stages of building an instruction-following language system. Each stage applies a different kind of behavioral pressure. Pretraining gives broad language structure; instruction fine-tuning teaches request-following; preference alignment shapes the quality of responses toward helpful, honest, and bounded behavior.

following model is shaped to interpret and respond to the intent behind a request, but it may still hallucinate, behave inconsistently under paraphrase, or fail when inputs differ structurally from those seen during alignment.

The three stages can be understood using the same vocabulary introduced in the earlier representation-learning chapter: each stage is a different source of training pressure that shapes a different aspect of behavior.

> **Practical Note.** Instruction fine-tuning and preference alignment do not change the fundamental architecture or the next-token prediction mechanism. They change the distribution of text the model is trained to produce. What changes across stages is what kind of continuation is rewarded, not how the model computes it.
>
> **Decision Box.** Use a base language model when the task requires open-ended text continuation, when the full behavioral interface will be designed separately, or when a custom fine-tuning pipeline will be applied. Use an instruction-following model when a behavioral interface that already follows requests in a bounded, consistent way is needed and starting from raw continuation behavior is not practical.

Stage	What the model learns from	What behavior changes
Pretraining	Large collections of unlabeled text	General language structure, vocabulary coverage, and broad world-level patterns
Instruction fine-tuning	Curated (instruction, response) examples	How to interpret and respond to requests in the intended format
Preference alignment	Signals that distinguish preferred from less-preferred responses	How to favor helpful, honest, and bounded behavior over merely plausible continuations

Table 9.4: Each stage applies a different behavioral pressure. The stages are additive: instruction fine-tuning builds on the base model, and preference alignment builds on instruction fine-tuning. Removing a stage does not simply undo the previous one; it leaves the model at an intermediate behavioral state.

9.13 Prompting as a Behavioral Interface

Once a language model can follow instructions, it can be steered at inference time without changing any of its parameters. The mechanism is prompting: structuring the input to the model so that the output is directed toward a desired behavior. This is the inference-time counterpart to the training stages in the previous section.

This is a different kind of adaptation from fine-tuning or transfer learning. Fine-tuning changes model weights using additional training. Prompting leaves the model unchanged. It changes what information the model sees at the moment it generates a response, and therefore what response it produces.

Prompting is a behavioral policy, not a model modification. It shapes output by structuring input, not by adjusting parameters.

Three prompting patterns appear consistently across task settings, but they do not form a single ladder. Zero-shot and few-shot prompting control how many demonstrations are shown. Chain-of-thought prompting controls whether the prompt asks

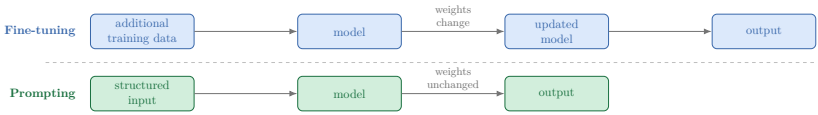


Figure 9.6: Fine-tuning and prompting address behavioral needs through different mechanisms. Fine-tuning changes the model's parameters based on new training data. Prompting leaves the parameters fixed and instead shapes behavior by changing what the model receives as input at inference time.

for intermediate reasoning. A prompt can therefore be zero-shot with chain-of-thought, few-shot without it, or neither.

Example 9.10 (Same Task, Three Prompting Strategies)**.** Suppose the task is to classify a student message into one of three routing categories: *deadline*, *grading*, or *administrative*.

- **Zero-shot**: the input describes the categories and asks for a label.
- **Few-shot**: the input includes two or three labeled examples before the new message.
- **Chain-of-thought variant**: either zero-shot or few-shot, but the input asks the model to first identify the main concern in the message and then select the category.

If the zero-shot result is already reliable under realistic message variation, adding examples or reasoning steps may not improve it. If the model makes consistent errors on edge cases, few-shot examples that cover those cases may help. If the routing logic is genuinely multi-step—for example, distinguishing a visa question from a course question requires identifying a specific entity type first—a chain-of-thought version of either prompt may reduce errors by making the intermediate step explicit. Chain-of-thought is therefore not a third rung above few-shot; it is an extra scaffold that can be paired with different demonstration choices.

> **Problem-Solving Technique. Wrong question → better question.** Wrong: "What is the best prompt?" Better: "What information does the model need at inference time to produce reliable behavior,

Chain-of-thought. Ask the model to produce intermediate reasoning steps before the final answer. This can be added to either zero-shot or few-shot prompts when the task requires multi-step inference or when intermediate steps are part of what needs to be checked.

Few-shot. Include worked input-output examples in the input before the new query. The examples demonstrate the expected format and reasoning pattern.

Zero-shot. Describe the task in natural language and ask for a response. No examples are provided. The model must interpret the task from the description alone.

Figure 9.7: Three common prompting patterns. Zero-shot and few-shot vary the number of demonstrations, while chain-of-thought adds an optional reasoning scaffold that can be paired with either. The simplest structure that produces reliable behavior under realistic input variation is usually preferable.

and what is the simplest prompting structure that provides it under realistic input variation?"

9.13.1 Prompting Interacts with Retrieval and Fine-Tuning

Prompting, retrieval, and fine-tuning address different behavioral problems. A deployed system may combine more than one of them. The decision depends on what kind of change is needed and what evidence supports it.

Decision Box. Start with prompting when the model's existing behavior is already close to the target and the task can be specified clearly at inference time. Add retrieval when correctness must be tied to current, auditable external evidence. Move to instruction fine-tuning when the behavioral format must change substantially and curated examples are available. Add preference alignment only when fine-

Strategy	What it changes	Main advantage	Main risk
Prompting	What information the model sees at inference time	No training needed; fast to adjust and test	Output can be sensitive to phrasing; no parameter control
Retrieval and grounding	What external evidence accompanies the input	Answers can be traced and audited	Retrieval failure propagates to the final answer
Instruction fine-tuning	Model parameters, toward request-following format	Strong behavioral shift; consistent output format	Requires curated examples; costly at scale
Preference alignment	Model parameters, toward preferred over less-preferred responses	Can shape subtle behavioral quality systemically	Requires preference signal; risk of misaligned reward

Table 9.5: Four adaptation strategies address different behavioral problems. The appropriate choice depends on what kind of change is needed, what evidence is available, and what audit trail is required. These strategies can be combined: a retrieval-augmented system may also use prompting to structure how evidence is presented to the model.

grained behavioral quality requires further shaping and preference evidence can be collected and verified reliably.

Failure Box. Hallucination: plausible continuation without factual grounding. A language model trained to predict the next token learns to produce text that is statistically likely given its training distribution. It has no separate mechanism to verify whether its claims match external facts. This means the model can generate fluent, confident text that is factually wrong or entirely unsupported. The failure is not random: the model produces what would be a likely continuation of the input in its training data,

which may coincide with false or fabricated information.

This is why grounding changes the claim being made. A grounded system does not say "the model knows the answer." It says "given this retrieved passage, the answer is." That narrower claim can be audited, corrected, or rejected when the passage is wrong. An ungrounded system cannot offer that audit trail.

Diagnosis. Measure faithfulness to retrieved evidence separately from fluency. A fluent answer that contradicts or ignores the retrieved passage is a hallucination of the synthesis type. A fluent answer generated when no passage was retrieved is a fabrication. Both require different responses: the first requires better answer-composition controls, the second requires stronger grounding or abstention logic.

Failure Pattern. Treating prompting as equivalent to fine-tuning. Prompting shapes behavior by changing what the model sees at inference time. It cannot produce behavior that requires a genuine change in model parameters. A model that consistently produces the wrong format, fails on a structural task, or misses a behavioral constraint under realistic variation is unlikely to be fixed by continued prompt iteration. If prompt variation produces unreliable results across reasonable paraphrases of the same task, the problem may require fine-tuning rather than further prompt design.

9.14 Evaluation Matched to Task

Evaluation in NLP depends sharply on task shape. The evidence must match the behavior the system is supposed to produce. This is where the chapter protects the claim made by the earlier choices about context, grounding, prompting, and escalation.

Task family	Primary evidence	What else to check
Classification	Accuracy, precision, recall, F1	Slice performance, threshold choice, and calibration if decisions matter
Sequence labeling	Token or span F1, exact span match	Boundary errors, rare-label recall, and policy-sensitive slots
Retrieval and ranking	Recall@k, reciprocal rank, nDCG, or hit rate	Source authority, freshness, and coverage of exceptions
Grounded QA	Faithfulness to retrieved text plus retrieval success	Prompt variation, citations, and abstention on weak support
Generation, translation, summarization	Human review and reference metrics where appropriate	Factuality, hallucination rate, domain shift, and safety constraints

Table 9.6: A simple evaluation map helps match task shape to the metric and to the failure slices that matter most.

9.14.1 A Simple Evaluation Map

The point of the map is not that every project needs every row. The point is that the unit of judgment must match the task shape.

9.14.2 Classification and Ranking Metrics

For text classification, standard metrics such as accuracy, precision, recall, and F1 remain appropriate. For retrieval tasks, the key question is whether the system finds the right supporting documents, passages, or ranked results reliably enough to support downstream use.

Example 9.11 (A Tiny Retrieval Check)**.** Suppose a late-submission question has one authoritative syllabus passage, and the system retrieves three passages. If the correct late-policy passage does not appear in the top few retrieved items, then a later generator is already operating on weak evidence no matter how fluent the final answer sounds. For retrieval-heavy systems, one important question is therefore simple: did the right support

appear high enough in the ranking to make a grounded answer possible?

9.14.3 Retrieval and Recommendation Are Top-of-List Problems

Search, document retrieval, FAQ suggestion, and recommendation all live or die by the quality of short ranked lists. Information retrieval is evaluated this way for a reason: users rarely inspect the whole candidate pool, so the top ranks carry most of the practical burden [44].

If there is usually one best supporting document, reciprocal rank or hit rate can be sensible because the first correct result matters most. If several documents could support the answer, precision@k and recall@k become more informative. If relevance is graded rather than binary, such as "excellent answer," "helpful hint," and "barely related passage," then ranking metrics such as normalized discounted cumulative gain (nDCG) are useful because strong early results should count more than weak late ones.

Recommendation systems share the same logic. A system that proposes articles, products, or policy pages is still making a ranking claim: which items deserve scarce user attention near the top of the list? That is why evaluation should report the quality of the exposed list, not only a global score over all candidates.

9.14.4 Perplexity and Likelihood

For language models, one common measure is perplexity:

$$\mathrm{PP} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log P(w_t \mid w_1, \ldots, w_{t-1})\right).$$

Lower perplexity means the model assigns higher probability to the observed text. This can be useful, but it does not directly measure factual accuracy, grounding quality, fairness, or user usefulness.

9.14.5 Generation and Grounding Evidence

Metrics such as BLEU and ROUGE compare generated text to references, but many correct outputs may use different wording. For grounded systems, it is often equally important to ask whether the right supporting material was found and whether the answer stayed faithful to it.

Example 9.12 (Grounded Versus Ungrounded Evaluation)**.** Imagine the question "I missed the lab because of illness. Can I submit tonight?" An ungrounded model answers, "Yes, illness normally means late work is accepted." A grounded system retrieves the current course policy, which says illness extensions require a form submitted within forty-eight hours, and answers, "Only if you submit the illness-extension form within forty-eight hours; the current course policy says late credit is handled through that process." The second system should be judged not only on fluent wording, but on whether it retrieved the correct policy and stayed faithful to it.

> **Evidence Box.** Evidence that an NLP system is genuinely useful should match the task. For classification, that may mean held-out metrics and error slices. For retrieval, it means showing that the right supporting material is found reliably. For generation, it often requires grounded references, factual checks, human review, and tests under prompt variation or domain shift. For routed support systems, it may also require showing that high-risk or institution-specific questions are escalated reliably rather than answered with unjustified confidence.

9.14.6 A Practical Rubric for Grounded Answers

When a document-grounded answer is important enough to review manually, a short rubric is often more informative than one headline score. A reviewer can ask:

- Did the system retrieve the right supporting passage?
- Did the final answer accurately reflect that passage?

- Did it avoid adding unsupported conditions, exceptions, or assurances?
- Did it cite or expose the source clearly enough for audit?
- Did it escalate rather than answer when the evidence or risk profile required it?

This kind of rubric fits the running support-assistant case well. A sentence can be fluent and mostly correct while still failing the review because it cites the wrong rule, omits a necessary condition, or answers a case that should have been deferred.

9.14.7 Prompt Sensitivity and Evaluation Instability

Modern language systems can be sensitive to phrasing, decoding settings, retrieval context, and instruction format. A model that looks strong under one prompt may fail under another. This is why evaluation should include prompt variation and error analysis rather than one benchmark number. Prompting is part of the behavior policy, not only a wrapper around the model [9]. In grounded systems, a practical standard is policy stability: reasonable prompt changes should not swing the system from cautious, cited answers to unsupported direct advice when the underlying evidence is unchanged.

9.14.8 Hallucination Failure Types and Response Design

Students often use the word *hallucination* too broadly. The label becomes more useful when it is split into failure types with different causes and different responses.

This classification is useful because it points to different fixes. Fabrication without evidence is often a grounding or answer-policy failure. Wrong synthesis of correct evidence is more of a reasoning or answer-composition failure. Stale-evidence answers usually indicate retrieval freshness or indexing problems. Overconfident answers under weak support often reflect missing abstention logic or poor uncertainty handling.

Failure type	What went wrong	Better system response
Fabrication without evidence	The answer states a rule or fact that was not supported by retrieved material at all	abstain, retrieve again, or escalate
Wrong synthesis of correct evidence	The right passages were retrieved, but the answer combined them incorrectly or dropped a key condition	cite more explicitly, simplify the answer, or route to review
Stale-evidence answer	The answer is grounded in a source that is outdated for the current policy or context	enforce freshness checks and request current evidence
Overconfident uncertainty failure	The system lacks strong support but answers directly instead of expressing uncertainty or asking for clarification	refuse, ask a follow-up question, or escalate

Table 9.7: Different hallucination failures demand different controls. A single label is too blunt if the goal is to redesign the pipeline.

> **Decision Box.** When a grounded language system fails, do not stop at "the model hallucinated." Ask which failure type occurred and what the system should have done instead: cite, simplify, ask for clarification, abstain, or escalate.

9.15 Bias, Privacy, and Social Meaning Are Part of the Technical Problem

Language technology raises visible social questions because language is tied to identity, culture, and power. A dataset may underrepresent dialects, minority languages, or informal registers. A model may amplify stereotypes, mishandle dialectal text, generate toxic continuations, or expose memorized private text.

These are not separate moral decorations on top of an otherwise technical system. They are representational and evaluation problems. If the model has not learned to treat dialectal or minority forms of language as normal and meaningful, then it has a technical coverage failure with social consequences.

Example 9.13 (A Representation Problem With Social Consequences)**.** If a model is trained mostly on formal standard-language text, it may treat dialectal expressions as rare, low-quality, or anomalous. That is not only a style issue. It is a representational failure that can degrade fairness, usability, and trust.

9.16 Chapter Artifact: An NLP Context-and-Grounding Card

By the end of this chapter, the reader should be able to write down not only which model family they want to use, but what context the task needs and what grounding evidence would justify a strong claim. The framing memo named the decision, the metric worksheet named good behavior, the evaluation plan protected the claim, the baseline sheet justified linearity, the structure diagnosis sheet justified leaving it, the training report justified the learned-model claim, the reuse-and-adaptation plan clarified what transfer could explain, and the vision design card made nuisance variation and deployment realism explicit. This chapter's card adds the language-specific layer: what context must survive, what evidence must be present, when the system should answer versus escalate, and what evaluation would justify that workflow.

> **Problem-Solving Technique. NLP context-and-grounding card.** Before reporting results, write down what context the task needs, whether external grounding is required, how the system will behave at generation time, when it must escalate instead of answering, and what evidence would justify the final claim.

Example 9.14 (Filled NLP Context-and-Grounding Card)**.** For a course-support assistant, a sensible card would specify: task type is *question answering with retrieval and escalation*; output type is grounded generated text or a referral to a human; the unit of evaluation is the final answer plus the retrieved syllabus, policy, or advising passages; bag-of-words is not enough because wording varies and policy references can be indirect, so contextual retrieval is justified; external grounding is required because answers must track current course and university rules; decoding should stay conservative rather than highly sampled; the escalation rule should defer visa, disability-accommodation, harassment, self-harm, and grade-appeal questions to a human workflow; primary evidence should include retrieval success, faithfulness to retrieved text, escalation quality on high-risk messages, and human checks on factual correctness; coverage risks include multilingual student phrasing, distressed messages, and privacy around student records or accommodations.

9.17 Common Mistakes in NLP

Mistake Checklist.

- **Declaring classical methods dead.** Sparse methods and n-grams still matter when they match the task.
- **Treating fluency as truth.** Surface quality is not enough evidence.
- **Ignoring tokenization.** Tokenization changes the effective input space.
- **Using the wrong evaluation for the task.** Perplexity, BLEU, ROUGE, accuracy, retrieval quality, and human preference measure different things.
- **Forgetting that retrieval and generation are different.** Some tasks are about finding trustworthy evidence, not free composition.

- **Conflating a base model with an instruction-following model.** The behavioral gap between them is large; the stages that close it are additional training, not architecture.
- **Using prompt iteration when fine-tuning is needed.** Prompting cannot produce behavior that requires a genuine change in model parameters; repeated prompt redesign on a structurally mismatched task wastes time without diagnosing the real problem.
- **Treating hallucination as a single failure type.** Fabrication, wrong synthesis, stale evidence, and overconfident uncertainty failure require different pipeline responses.

9.18 Looking Ahead

This chapter treated language as context-dependent sequence structure. The next chapter keeps the sequential view but moves to audio and time series, where the central question becomes more explicit still: what information was actually available at the decision moment, and what would count as cheating through time?

Chapter Summary

- NLP begins by deciding what context the task needs, not by choosing an architecture by habit.
- Tokenization, bag-of-words, n-grams, and contextual models preserve different amounts of linguistic structure, so they should be chosen by task need rather than by fashion.
- Task shape matters before representation depth: classification, retrieval, question answering, summarization, and generation produce different outputs and therefore need different evidence.

- Language modeling predicts likely continuations, while instruction following, prompting, retrieval, and fine-tuning each change behavior in different ways.
- Prompting is an inference-time behavioral policy, not a parameter update; retrieval changes the evidence available to the system; fine-tuning changes the model itself.
- Hallucination arises because plausible continuation is not the same as grounded correctness, so grounding, abstention, and escalation are structural design choices rather than afterthoughts.
- Decoding policy and grounding both change system behavior, so fluent output is not enough evidence of trustworthy performance.
- NLP evaluation must match task shape and should often include retrieval quality, factual support, prompt variation, and coverage checks rather than one surface metric alone.
- The main deliverable of the chapter is an NLP context-and-grounding card that makes the required context, evidence, and escalation policy explicit before a system claim is made.

Study Questions

1. Why is a context audit more useful than choosing an NLP architecture by habit?
2. When is bag-of-words a serious baseline rather than a weak placeholder?
3. What do n-grams repair that unigram counts erase?
4. Why does contextual meaning require more than one fixed vector per word type?
5. Why can a fluent answer still be untrustworthy?
6. When should retrieval or external grounding be part of the system design rather than a later add-on?

7. When is reciprocal rank more informative than recall@k for an NLP retrieval task?

8. Why can abstention or escalation be a better language-system behavior than answering directly?

Exercises

1. For a support-email classifier, decide whether bag-of-words, n-grams, or contextual modeling is justified first. Explain what structure the task really needs.

2. Construct two short sentences with identical bag-of-words representations but different meanings. Explain which downstream task would fail under bag-of-words and which might still work.

3. Give one example of an NLP task for which local order is the main missing structure and one for which external grounding is the main missing structure.

4. Explain why a language model can assign high probability to a sentence that is factually false.

5. Design a context audit for a small question-answering system over product manuals, policy documents, or another document collection of your choice.

6. A retrieval system returns ranked passages with relevance labels $[0, 1, 0, 1]$ in the top four positions, and there are three relevant passages in the collection. Compute precision@2, recall@4, and reciprocal rank.

7. For a support assistant, propose two message types that should be answered automatically and two that should be escalated to a human. Justify the boundary technically, not only socially.

8. Give one example in which dialect or register variation creates a technical coverage problem rather than merely a stylistic difference.

9. Use Table 9.8 to sketch a one-page context-and-grounding card for a language task of your choice. End by naming the smallest workflow that could meet the card's requirements. If you are carrying one case through the book, state which parts come over unchanged from the framing memo, evaluation plan, and reliability discipline, and which parts become language-specific here.

Challenge Exercises

1. Show algebraically or with a concrete vector example that two sentences can have identical unigram count vectors and different bigram count vectors.
2. A unigram baseline performs poorly on an order-sensitive classification task, while a bigram logistic-regression model performs very well. Explain why this does not mean deep learning is unnecessary in general.
3. A retrieval-augmented assistant produces better answers than a pure generator. Describe what evidence would support the claim that retrieval, rather than prompt wording alone, caused the gain.
4. A language model achieves low perplexity on a benchmark corpus but frequently invents unsupported details in long-form answers. Explain why these facts can coexist, and describe an evaluation protocol that would surface the problem clearly.

Card field	What must be written clearly
Seven-question focus	Questions 3, 4, 6, and 7: what external evidence is needed, what linguistic structure must survive, what evaluation supports the claim, and how grounded outputs become usable actions
Task type	Classification, sequence labeling, retrieval, question answering, summarization, translation, or open-ended generation
Output type	Label, span, ranked list, or generated text
Unit of evaluation	What object is being judged: document, token, answer, summary, retrieval list, or another output
Representation choice	Whether bag-of-words, n-grams, contextual modeling, or another representation is justified
Grounding requirement	Whether external documents, retrieval, or citations are required for trustworthy behavior
Model behavioral stage	Whether a base language model or an instruction-following model is used, and what alignment stages have been applied
Prompting strategy	Whether zero-shot, few-shot, or chain-of-thought prompting is used, and whether it has been tested under realistic input variation
Adaptation strategy	Whether prompting, retrieval, fine-tuning, or a combination is used, and what evidence justifies that choice
Decoding policy	Greedy decoding, sampling, temperature choice, or another policy if generation is involved
Escalation rule	Which questions must defer to a human, specialist workflow, or safer fallback rather than receive a direct answer
Primary evidence	The main metric or evaluation procedure that should justify the claim
Coverage and risk concerns	Dialect, register, privacy, toxicity, or underrepresentation risks that must be checked

Table 9.8: An NLP context-and-grounding card turns the chapter's ideas into a design protocol before a language-system claim is made.

Chapter 10

Audio, Time, and Prediction-Time Realism

Learning Goals

- Explain why sequential problems should be framed around what information is truly available at the decision moment.
- Use a prediction-time audit to define horizon, allowed context, forbidden future information, and latency constraints.
- Understand waveforms, spectrograms, lag features, trends, seasonality, and regime change as representation choices for sequential data.
- Distinguish speech recognition, keyword spotting, speaker verification, forecasting, and anomaly detection before choosing a model family.
- Evaluate sequential systems with time-faithful splits, realistic baselines, and deployment-aware stress tests.
- Produce a sequential design card before making a serious sequential-model claim.

The language chapter treated meaning as sequence structure shaped by context and grounding. Audio and time-series data

extend that sequential viewpoint into different signal types and operational constraints. Once time enters the problem, examples stop being static records and become evolving processes. That change makes representation, splitting, latency, and leakage part of the modeling claim itself. The chapter therefore moves in a specific order: fix the decision moment and legal context first, separate the task shapes that sequences can support, choose representations and memory mechanisms that respect that boundary, and only then ask what evaluation and deployment evidence would make the claim honest. The chapter culminates in a sequential design card, so the audit stays tied to a reusable design decision rather than remaining a checklist.

> **Problem-Solving Lens.** This chapter asks one practical question: at the moment a sequential system must act, what information is actually available, what past context matters, and what evaluation prevents us from cheating through time? The answer drives representation choice, model family, baselines, and deployment realism.

10.1 Opening Dilemma: Time Turns Examples Into Processes

In a static classification problem, an example is often treated as a fixed object. In a sequential problem, the object unfolds. A waveform is produced over time. Electricity demand evolves hour by hour. A patient-monitoring stream changes as interventions occur. A fraud signal arrives one event at a time. Once time enters the task, the central question becomes immediate:

What did the system know at the decision moment?

That question makes sequential modeling different from ordinary tabular prediction in at least three ways:

- order usually matters,
- available information may be constrained by the prediction moment,

- evaluation can become invalid if future information leaks backward through the split or the feature set.

This is why a strong sequential result is not defined only by model accuracy. It is defined by whether the system used only legitimate information at the right time and still performed well.

Keep two running cases in view throughout this chapter. One is a streaming wake-word detector that must fire within a fraction of a second after hearing a target phrase. The other is a one-hour-ahead electricity-demand forecaster for a campus energy system. One consumes audio and the other consumes numeric measurements, but both are governed by the same practical question: what information was really available at the moment a decision had to be made? Those two cases also give the chapter its shape: audio makes representation and latency vivid, while forecasting makes horizon, baselines, and backtesting vivid.

10.2 Problem-Solving Technique: Prediction-Time Audit

Problem-Solving Technique. Prediction-time audit. Before training a sequential model, write down:

- the decision time,
- the forecast horizon or response delay that matters,
- the past context that is allowed,
- any future information that is forbidden,
- the latency budget,
- the temporal structure the representation must preserve.

This is the chapter's most reusable habit. A forecast, a wake-word detector, a speaker verifier, and an anomaly detector all process sequences, but they do not share the same prediction

moment or the same information constraints. The audit forces those constraints into the open before the model family is chosen.

> **Decision Box.** Sequential claims should always answer three questions early: what is known at decision time, how far into the future the system must reason, and which parts of the past are legitimately available. If those answers are vague, the evaluation can become meaningless even when the model looks strong.

> **Practical Note.** A compact sequential workflow is: state the decision time and horizon, identify the allowed context and forbidden future information, choose a representation that preserves the relevant time structure, match model memory to that context, test a chronological baseline, and only then compare alternatives against the latency budget and drift risks. That is also the chapter order in miniature. The audit and the design card are the same procedure at two different levels of detail.

Example 10.1 (Mini Prediction-Time Audit)**.** For the wake-word detector, the decision time is "right now" on each incoming buffer, the legal context is only the recent audio already heard, future frames are forbidden, and the latency budget is tight enough that the representation must preserve very local temporal structure.

For the one-hour-ahead demand forecaster, the decision time is the top of the hour when the forecast is issued, the legal context includes past load and weather reports that had already been issued by then, realized future demand is forbidden, and the representation must preserve recent trend, daily rhythm, and calendar effects.

That is already enough to rule out several bad experiments before architecture enters the story.

> **Problem-Solving Technique. Tempting mistake → corrected reasoning.** Tempting mistake: "Use every useful-looking field in the dataset and let

Case	Decision time	Allowed context	Forbidden future information	Latency budget
Wake-word detector	every incoming frame or short buffer	recent audio buffer only	any frames that arrive after the current decision moment	very small
One-hour-ahead demand forecast	top of each hour	past load, calendar, and issued weather data up to that hour	realized future demand or revised weather not yet available at issue time	modest

Table 10.1: Audio and forecasting look different on the surface, but the same audit applies: define the decision moment, legal context, forbidden future information, and timing constraint first.

> the model figure it out." Corrected reasoning: "Use only what would legally and physically exist at the decision moment. If a feature, preprocessing step, or split leaks future structure, the experiment is solving an easier problem than deployment."

10.3 Two Domains, One Audit

The chapter is easier to teach if the audio and forecasting examples are kept in the same frame rather than treated as unrelated mini-courses. Table 10.1 shows why.

The prediction-time audit is not a technique specific to audio or time-series forecasting. Any task where an action must be taken at a specific moment, using only the information that legally and physically existed at that moment, is a sequential prediction problem governed by the same audit. The chapter uses audio and

Domain	Decision moment	Allowed context	Forbidden future	Latency pressure
Medical monitoring (ICU early-warning system)	each vital-sign reading	all prior readings for that patient	any readings that occur after the current sample	tight
Fraud detection (online transaction scoring)	transaction time	prior transactions for that account up to the current event	any future transac-tions	very tight
Predictive maintenance (sensor anomaly detection)	each sensor cycle	recent sensor history up to the current cycle	future sensor values not yet recorded	modest
Recommendation (next-item prediction)	page load	user's prior interaction history up to that moment	any inter-actions that occur after page load	tight

Table 10.2: The prediction-time audit applies across sequential domains far beyond audio and forecasting. The questions are the same; only the answers change.

demand forecasting as its running cases because they contrast well in latency pressure and signal type, but the discipline applies equally to the domains shown in Table 10.2. This shared audit is what lets the chapter move from two domains into one modeling discipline rather than splitting into unrelated application stories.

This comparison is the real center of the chapter. The wake-word system fails if it silently uses future audio. The forecaster fails if it uses tomorrow's realized demand or a revised weather report that would not have existed when the forecast was issued. In both cases the main danger is the same: the experiment starts solving an easier problem than the real deployment.

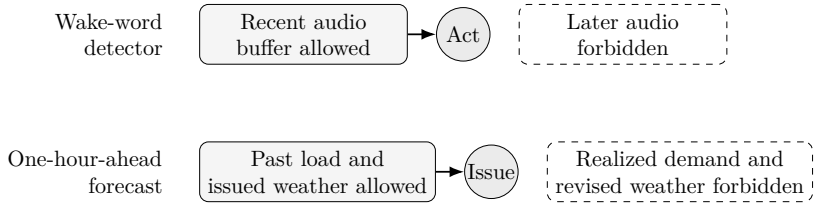


Figure 10.1: The same prediction-time boundary governs both running cases. The legal context sits to the left of the decision moment, while information created after that moment belongs to an easier problem than the one the deployed system must actually solve.

10.4 Task Shape Before Model Family

Before talking about sequence architectures, it helps to ask what kind of output the task really needs. A transcript, a trigger, an identity judgment, a forecast, and an anomaly alarm do not ask the same question of time.

10.4.1 Speech Recognition

Speech recognition maps audio to text. This is not just classification. The output length may differ from the input length, alignments are uncertain, and acoustic evidence must be combined with linguistic structure.

10.4.2 Keyword Spotting and Audio Classification

Keyword spotting asks whether a short target command such as `stop` or `wake` appears in an audio clip. More general audio classification may predict labels such as traffic, applause, machinery, or rain. These tasks often depend strongly on robustness to noise, microphone changes, and speaker variation.

10.4.3 Speaker Verification

Speaker verification asks whether the current voice matches a claimed identity or an enrolled reference voice, rather than what is being said. This is useful because it shows that the same signal can support different tasks with different invariances. A

speech-recognition system should usually ignore speaker identity. A speaker-verification system must preserve it.

10.4.4 Forecasting and Anomaly Detection

Forecasting predicts future values from information available up to the present. Anomaly or event detection searches for rare failures, abrupt changes, or unusual states in a process. These are not the same task. Forecasting is about expected continuation; anomaly detection is about deviation from expected continuation.

Example 10.2 (Why Task Shape Comes First)**.** An audio stream from a smart home could support at least three different tasks. It could classify the room scene, detect a wake word, or verify that the current speaker is an authorized user. The same raw signal appears in all three cases, but the relevant invariances, latency demands, and evaluation standards are different.

Once the task shape is fixed, the next question is representational: what view of the sequence preserves the structure that task needs?

10.5 Audio as a Representation Problem

An audio waveform is a sequence of amplitudes sampled over time. That representation is faithful to the signal, but not always to the task. Many audio tasks depend less on raw amplitude and more on how frequency content changes over time.

10.5.1 Waveforms and Frequencies

Pure tones are well described by frequency. Speech and music are not pure tones, but they still contain informative mixtures of frequencies that evolve over time. This is why audio is often represented not only in the time domain but also in a time-frequency domain.

Definition 10.1 (Spectrogram)**.** A spectrogram is a time-frequency representation showing how much energy a signal con-

tains at different frequencies over successive time windows.

One way to build a spectrogram is to take short overlapping windows of the waveform and compute a frequency decomposition within each window. The result is a matrix whose horizontal axis is time, whose vertical axis is frequency, and whose values indicate energy.

Depending on the construction, those values may be magnitude, power, or log-power rather than raw energy.

> **Prerequisite Bridge.** You do not need full Fourier-analysis machinery to use the idea of a spectrogram. The practical picture is enough: take a short chunk of sound, ask which frequencies are strong, slide forward a little, and repeat. The result shows how frequency content changes over time.

10.5.2 Worked Example: A Tone Change Over Time

Imagine a two-second clip. In the first second it contains a low tone at 4 Hz, and in the second second it contains a higher tone at 10 Hz. If we summarize the whole clip at once, both frequencies appear somewhere, but the timing is lost.

Now suppose we compute short-window frequency summaries. Early windows show strong energy near 4 Hz. Later windows show strong energy near 10 Hz. The representation now answers the useful question: not just which frequencies exist, but *when* they are active.

A spectrogram is not the sound itself; it is a task-shaped lens on the sound.

> **Math Lens.** If the analysis window has duration T, then time localization improves as T gets smaller, while frequency resolution improves as T gets larger. In the simplest intuition, short windows catch brief events but smear stable frequencies; long windows separate frequencies more cleanly but blur when they

occurred. So window choice is really a task choice: onset detection prefers timing, while tone or harmonic analysis prefers frequency precision.

Example 10.3 (Window Choice Depends on the Decision)**.** For the running wake-word case, short windows may preserve the onset of consonants and brief timing cues that matter for fast detection, while longer windows may smear those cues even if they give cleaner frequency estimates. The right representation is therefore tied not only to the signal, but to the timing of the action the system must take.

10.5.3 Time-Frequency Tradeoffs

Short windows give better time localization but blur frequency precision. Longer windows improve frequency resolution but blur timing. This tradeoff matters because different tasks value timing and frequency detail differently. A transient command and a slowly changing tone are not equally well served by the same window.

Advanced Note. The short-time Fourier transform can be written schematically as

$$S(t, f) = \left| \sum_n x[n]\, w[n-t]\, e^{-i2\pi f n} \right|^2 ,$$

where $x[n]$ is the waveform, w is a local window, and f is a frequency or frequency-bin variable. The key idea is not the formula itself, but the fact that local frequency analysis depends on both a signal and a windowing choice.

Math Lens. If a window has length L and the step size is $s < L$, then adjacent windows overlap by $L - s$ samples. For example,

$$W_t = (y_t, \ldots, y_{t+L-1}),$$
$$W_{t+s} = (y_{t+s}, \ldots, y_{t+s+L-1}),$$

so the overlap fraction is $(L - s)/L$. If a random split places one overlapping window in train and the next

in test, the test example already contains much of the same raw signal. That is why overlapping windows from one continuous segment should usually stay in the same temporal block or group.

10.5.4 Helpful but Incomplete

Spectrograms are attractive because they let audio benefit from architectural ideas developed for images. But they also hide structure. Phase information is often compressed or ignored. Windowing can smear sharp events. Reverberation and background noise can distort the display. As elsewhere in the book, the representation is never neutral. The next section carries the same lesson into forecasting and monitoring, where lags, gaps, and horizons play the role that windows and frequencies played in audio.

10.6 Time Series as Evolving Process

Audio is one kind of sequence. Forecasting and monitoring create another. Here the goal is often to predict future values, detect events, or classify temporal patterns from evolving measurements. The signal is different, but the teaching move is the same: decide what temporal structure must survive before deciding how to model it.

10.6.1 Trend, Seasonality, Lagged Dependence, and Regime Change

Several temporal structures appear repeatedly:

- **trend**: long-term upward or downward movement,
- **seasonality**: recurring cycles such as daily, weekly, or yearly patterns,
- **lagged dependence**: present values that depend on recent past values,
- **regime change**: a shift in how the process behaves.

These are not only descriptive words. They affect feature design, baseline choice, and split logic. A forecasting model that ignores seasonality may look unstable. A model that cannot survive regime change may look strong in the lab and fail in deployment. Forecasting texts treat trend, seasonality, horizon, and structural change as core parts of the problem definition rather than as afterthoughts [34].

Definition 10.2 (Forecasting)**.** Forecasting is the task of predicting future values of a sequence from information available up to the present time.

10.6.2 Worked Example: Lag Features by Hand

Suppose we want to predict electricity demand one hour ahead. A simple forecaster might use the most recent three observations:

$$x_t = \begin{bmatrix} y_{t-2} \\ y_{t-1} \\ y_t \end{bmatrix}.$$

Let

$$y_{t-2} = 98, \qquad y_{t-1} = 101, \qquad y_t = 103.$$

Suppose a linear forecasting model is

$$\hat{y}_{t+1} = 0.2y_{t-2} + 0.3y_{t-1} + 0.5y_t.$$

Then

$$\hat{y}_{t+1} = 0.2(98) + 0.3(101) + 0.5(103) = 19.6 + 30.3 + 51.5 = 101.4.$$

The example is simple on purpose. It shows that sequential prediction begins by deciding what past context matters and how it should enter the prediction. This is the forecasting counterpart to window choice in audio: both are ways of deciding what local past is allowed to matter. More advanced models learn richer context automatically, but this conditioning logic remains central.

10.6.3 Irregular Sampling and Missingness as Signal

Many real sequential problems are not sampled on a neat fixed grid. Clinical measurements arrive at uneven times. User events occur in bursts and then disappear for hours. Sensors drop readings. Support messages appear only when a student decides to write. In such settings, the timestamp pattern is part of the representation, not just metadata around the representation.

This matters because missingness is often informative. A missing sensor value may indicate device failure. A long gap since the last event may signal inactivity, recovery, or disengagement. A support system that has not received follow-up messages for several days may be seeing a different situation from one that receives repeated messages within an hour. Treating every gap as a nuisance to be smoothed away can remove exactly the structure the model needs.

> **Decision Box.** Before filling missing values or resampling to a regular grid, ask whether the absence itself carries signal. If the answer may be yes, preserve time gaps, missingness flags, or event counts explicitly instead of pretending the process was evenly observed all along.

A practical workflow is to separate three questions:

- is the process truly continuous but only observed intermittently?
- is the missing value a measurement failure?
- is the missing value itself a meaningful event, such as “no message arrived” or “no transaction occurred”?

Different answers lead to different designs. Sometimes resampling to hourly or daily bins is fine. Sometimes the right move is to keep a feature for time since last observation. Sometimes the right representation is event based rather than grid based. The core modeling habit is to stop treating timestamps and gaps as bookkeeping and start treating them as part of the signal.

10.6.4 Horizon Is Part of the Task Definition

Predicting one step ahead is not the same as predicting a day, a week, or a month ahead. The relevant context, the uncertainty, and the operational meaning all change with the horizon. Once the horizon is fixed, the next question is what kind of memory the model needs in order to carry the right past forward.

Horizon is part of the task definition, not a detail added after modeling.

Example 10.4 (One-Hour-Ahead and Day-Ahead Are Different Problems)**.** For the campus energy system, a one-hour-ahead forecast can rely heavily on the most recent load values and near-term conditions. A day-ahead forecast may depend more on calendar effects, planned building usage, and weather forecasts issued much earlier. Calling both tasks "forecasting" is true but incomplete. They are different operational decisions with different legal context and different uncertainty.

10.7 Sequence Models as Memory Choices

The purpose of a sequential model is not merely to read one item after another. It is to preserve the forms of memory that matter for the task. Architectures enter only after the audit, task shape, representation choice, and horizon are already on the page.

10.7.1 Hidden Markov Models and Structured State

Earlier sequence models such as hidden Markov models treated the system as evolving through hidden states that generate observations. These models remain conceptually useful because they force us to think about latent state, transitions, and uncertainty explicitly [51].

10.7.2 Recurrent Models and LSTMs

Recurrent neural networks process sequences step by step, updating a hidden state that summarizes past context. LSTMs and

related gated models were influential because they made it easier to retain useful information across longer spans than simple recurrent networks could manage reliably.

10.7.3 Transformers and Flexible Context Access

Transformers matter in sequential work for the same broad reason they matter in language: they allow more flexible access to distant context than a purely recurrent hidden state. When enough data and compute are available, that flexibility can be valuable for long-range dependencies [61].

Example 10.5 (A Toy Memory Contrast)**.** Consider a toy task: output `1` at the current time step only if the tone heard *eight* steps earlier matches the tone heard now; otherwise output `0`. A short-window model that only sees the last three steps cannot solve this task for the right reason because the decisive information lies outside its memory. A recurrent model may solve it if its hidden state preserves the earlier tone reliably. A transformer-style model can compare the current step directly with the earlier relevant step more explicitly. The important lesson is not that one architecture is always better. It is that the task is really a question about memory access.

> **Math Lens.** Suppose a predictor only retains the last m observations, so its state at time t is a function of $(y_{t-m+1}, \ldots, y_t)$. If the correct output at time $t+1$ depends on y_{t-h} with $h > m$, then two sequences that match on the last m steps but differ at y_{t-h} produce the same state but may require different outputs. No model with only m-step memory can solve that task perfectly in all cases. This is the formal reason horizon and memory length have to be matched to the task.

Example 10.6 (Which Past Matters?)**.** For keyword spotting, the last second of audio may matter most. For electricity demand, the same hour on previous days may matter strongly. For clinical monitoring, sudden short-term changes may matter more than long-run averages. Different tasks reward different notions of

memory.

Sequence architectures differ mainly in how they represent and access memory.

10.8 Evaluation That Respects Time

This is the chapter's emotional center. Time is one of the easiest ways to fool yourself experimentally. The earlier sections chose representations and memory stories; this section decides whether those choices are being tested honestly.

10.8.1 Chronological Splits

Forecasting and many monitoring tasks require training on earlier periods and testing on later ones. Random splits can create unrealistic settings in which information from future regimes effectively leaks into the training distribution [53].

Definition 10.3 (Chronological Split)**.** A chronological split separates training and testing by time order so that evaluation uses only future observations relative to the training period.

10.8.2 Why Random Splits Can Mislead

Suppose a time series is converted into overlapping windows and then randomly split. Very similar local histories may appear in both train and test sets. The test set becomes easier than real deployment because the model sees nearly the same recent patterns during training. If the process later changes regime, the random split may hide the real difficulty of future prediction.

10.8.3 A Temporal Leakage Taxonomy

Students often learn the warning "do not use the future" and still miss how many different ways the future can sneak in. A more useful habit is to name the leakage pattern explicitly, because different patterns require different fixes.

Common temporal leakage patterns include:

- **future-feature leakage**: a predictor uses information created after the decision moment, such as tomorrow's realized demand or a laboratory value measured after triage;
- **window-overlap leakage**: train and test windows share nearly the same recent history, making the test set much easier than the true future problem;
- **preprocessing leakage**: normalization, imputation, seasonal adjustment, or feature construction is fit using future periods and then applied backward into training;
- **label-definition leakage**: the target itself is defined using future information in a way that quietly contaminates features or split logic;
- **entity or source carryover**: the split mixes recordings, devices, speakers, patients, or sensors across time so heavily that the model is tested on near-duplicates of what it already saw.

These are not all the same mistake. Future-feature leakage breaks the prediction-time boundary directly. Window-overlap leakage creates an unrealistically easy interpolation problem. Preprocessing leakage often looks harmless because it happens before the model is trained, yet it can still let future structure shape the representation. Label-definition leakage is especially dangerous because it can make the whole experiment answer a different question from the deployment task.

> **Decision Box.** When you suspect leakage, do not stop at "the split seems wrong." Ask which leakage family is present, what information crossed the boundary, and whether the fix belongs in the split, the feature pipeline, the label definition, or the grouping logic.

10.8.4 Available Information at Prediction Time

Chronology alone is not enough. The feature set itself must be realistic. A feature derived from tomorrow's average demand is invalid if the goal is to predict tomorrow before it happens. A

Feature example	Legal?	Why
Past load at issue time	Yes	It existed when the forecast was issued and can be used without looking ahead
Issued weather forecast report	Yes	The report was already published at decision time
Realized future demand	No	It is the target the model is supposed to predict, not an input available beforehand
Revised weather bulletin posted later	No	The later revision would not have been available when the forecast was made
Recent audio buffer for wake-word detection	Yes	It is part of the current signal at the decision moment
Audio frames that arrive after the trigger point	No	They cross the prediction-time boundary

Table 10.3: A legal-vs-illegal feature table makes the prediction-time boundary concrete across audio and forecasting.

clinical laboratory value timestamped after triage is illegal for a triage-time prediction. In audio, future frames may be acceptable for offline transcription but forbidden for a low-latency streaming detector.

> **Failure Box.** Sequential leakage does not always look like copying answers. It often appears as future-adjacent windows, mixed regimes, illegal timestamps, or features that would not exist at the real decision point. The result can still look impressive while meaning very little.

Example 10.7 (Real-World Callout: Leakage Hides in Structure)**.** Roberts et al. argue that cross-validation for temporally, spatially, or otherwise structured data must respect the real dependency structure, because ordinary random splits can turn a hard generalization problem into an easier interpolation problem [53]. Bernett et al. make a closely related point in biological

Evaluation design	Mean absolute error
Random split of overlapping windows from the same semester	2.1
Chronological split with the next month held out	5.8
Chronological split with a later semester held out	7.0

Table 10.4: A sequential model can look much better under a random split than under the deployment-faithful chronological split that actually matters.

machine learning: leakage often enters through preprocessing, grouping mistakes, or shared entities that quietly connect train and test data even when nobody intended to cheat [5]. The practical lesson for sequential modeling is blunt: if the split ignores how examples are connected through time, source, or measurement process, the reported result may answer the wrong question.

10.8.5 Worked Example: A Split That Solves the Wrong Problem

Suppose the campus demand forecaster is trained on overlapping windows and evaluated in two ways:

The random split looks best because nearby windows share almost the same recent history and regime. The model is effectively tested on patterns that are only slightly shifted versions of what it already saw during training. The chronological split is harder because it asks the real question: can the system predict later demand under changed conditions without borrowing information from the future?

The same logic appears in audio. A wake-word detector evaluated with the same speakers, rooms, and devices mixed across train and test may look strong for the wrong reason. If deployment requires new speakers or new microphones, the honest split should separate those conditions too. The central lesson is not "always use one specific split." It is "use the split that matches the real prediction story."

Evidence Box. Evidence for a sequential model should answer at least these questions:

- Was the split chronological or otherwise faithful to deployment?
- Were all features available at the decision moment?
- Does the model beat a realistic naive baseline?
- What happens under noise, latency limits, or regime shift?

10.8.6 Backtesting Regimes and Horizon-Specific Claims

Chronological evaluation still leaves an important choice open: how should the model be rolled forward through time during evaluation? That choice is the backtesting regime, and it should mirror how the system will actually be maintained. This is where evaluation becomes an operational simulation rather than a single held-out score.

In practice, model selection often uses rolling or walk-forward backtesting rather than a single chronological cut, especially when the deployed system will be retrained repeatedly.

Three common regimes appear often:

- **single holdout**: train on an early block and test on one later block; useful for a one-time launch check but weak if performance varies over time;
- **expanding-window backtest**: train on an initial period, test on the next block, then expand the training history and repeat; useful when the real system will keep accumulating data;
- **sliding-window backtest**: always train on the most recent fixed window and test on the next block; useful when old data becomes stale and retraining emphasizes recent behavior.

None of these is universally best. The right regime depends on

the deployment cadence. If the real forecasting service retrains every week on the latest semester of data, then a one-shot historical split is not the most faithful evaluation. If the real system is updated rarely, an expanding-window simulation may overstate adaptation. Backtesting is therefore part of the operational claim, not just part of the statistical protocol.

Horizon belongs in the same sentence. A one-hour-ahead model and a day-ahead model may need different baselines, different context windows, and different retraining schedules even when they use the same raw signal. "This model works for forecasting" is too broad a claim. A scientifically honest sentence is closer to: "under weekly expanding-window retraining, the model improves one-hour-ahead demand forecasts, but the gain shrinks sharply at the 24-hour horizon." That is a usable operational claim.

> **Decision Box.** Choose the backtesting regime by asking how the deployed system will actually be refreshed. Then state the horizon in the claim itself, because a win at one horizon does not automatically transfer to another.

10.9 Naive Baselines as Honesty Checks

Sequential tasks often have deceptively strong simple baselines. In forecasting, predicting that the next value equals the current value can be hard to beat on stable series. In seasonal data, repeating the value from the same time yesterday or last week may be stronger still. The companion forecasting demo makes this visible: once the split is honest, a naive rule can recover much of the apparent gain claimed by a more complex model.

In sequential tasks, the first baseline is often not weak at all. It is a realism check.

A complicated sequential model that cannot beat a naive baseline under an honest split is not a near-miss. It is a warning sign that the representation, split logic, or claim may be wrong.

10.10 Deployment Realism: Noise, Latency, and Drift

A sequential system is not useful just because it is accurate offline. It must also act in time, survive noise, and remain relevant as the process changes. These pressures are not cleanup tasks after modeling. They decide whether the same prediction-time assumptions still hold once the system leaves the notebook.

10.10.1 Noise and Channel Variation

Audio systems must cope with microphone differences, compression artifacts, room acoustics, overlap from other speakers, and background noise. Sensor systems must cope with missing readings, calibration changes, and hardware variation. Robustness is often the real difficulty.

10.10.2 Latency

Some systems can use the whole sequence before deciding. Others must act online. A wake-word detector cannot wait until the conversation ends. A fraud detector cannot wait until tomorrow. Latency is therefore part of the modeling problem, not just a later optimization concern.

Example 10.8 (Offline Accuracy Can Hide an Illegal Delay)**.** Suppose a wake-word system achieves excellent accuracy by using one full second of future audio after the target phrase ends. That may be acceptable for offline transcription, but it is not acceptable for a device that must wake immediately while the user is still speaking. The model may be statistically impressive and operationally invalid at the same time.

10.10.3 Concept Drift

Processes change. Speakers change microphones. Consumption patterns shift across seasons. Traffic changes after roadwork. Financial behavior changes across market regimes. A model that was sound yesterday may become stale even if its training procedure was technically correct at the time.

For the running demand-forecast case, drift may come from new building schedules, extreme weather, or policy changes that alter campus energy usage. For the wake-word case, drift may come from microphone replacements, firmware updates, background-noise shifts, or changing speaking habits. Sequential systems therefore need not only a one-time evaluation, but a plan for noticing when the old prediction-time assumptions stop matching the new world.

> **Advanced Note.** Many sequential deployments are partly monitoring problems. The system must detect drift, report it, and possibly trigger retraining or fallback behavior over time. This connects naturally to the later chapters on experiments, reliability, and end-to-end systems.

10.11 Chapter Artifact: A Sequential Design Card

By the end of this chapter, the reader should be able to write down not only which sequence model they want to use, but what was truly available at decision time and what evaluation would make the claim honest. Earlier artifacts in the book named the decision, the metric, the evaluation plan, the baseline, the structure diagnosis, the training claim, the reuse strategy, the vision deployment constraints, and the language grounding contract. This chapter's card adds the sequential layer: decision time, legal context, backtesting regime, latency, and drift.

> **Problem-Solving Technique. Sequential design card.** Before reporting results, write down the decision time, horizon, allowed context, forbidden future information, timestamp and missingness handling, baseline family, split and backtesting logic, latency budget, operating mode, and likely drift risks.

Example 10.9 (Filled Sequential Design Card)**.** For a wake-word detector, the card might read as follows: decision time is every incoming audio frame; horizon is immediate detection

within a fraction of a second; allowed context is the recent audio buffer only; forbidden future information is any audio that arrives after the decision moment; timestamp and missingness handling should treat dropped or clipped frames as explicit quality signals rather than pretending the stream was complete; task type is keyword spotting; key invariances include speaker identity and mild background noise but not arbitrary microphone failure; the baseline family may include a simple spectral classifier over short windows; split and backtesting logic should separate speakers, devices, and recording sessions, then replay held-out sessions in streaming order so the evaluation matches the deployed trigger setting; latency budget is very small because delayed wake-word detection is unusable; operating mode is streaming rather than offline full-sequence scoring; major risks are room acoustics, new microphones, household noise, and gradual drift in speaking style.

Example 10.10 (Filled Forecast Design Card)**.** For the one-hour-ahead campus demand forecaster, the card might read as follows: decision time is the top of each hour; horizon is one hour ahead; allowed context is past load, calendar, and weather data that were already issued by that hour; forbidden future information is realized future demand or revised weather posted later; timestamp and missingness handling should preserve gaps, outages, and imputed values explicitly; task type is forecasting; key invariances include scale shifts and seasonal repetition but not arbitrary future regime changes; the baseline family should include persistence and seasonal-repeat checks; split and backtesting logic should use chronological holdout plus expanding-window or sliding-window backtests; latency budget is modest because the forecast can be issued on a schedule; operating mode is offline batch scoring with periodic refresh; major risks are holidays, extreme weather, building schedule changes, and policy shifts.

10.12 Common Mistakes in Sequential Modeling

The mistakes in this chapter are surface forms of one deeper error:

pretending time does not change what counts as a valid claim.

Mistake Checklist.

- **Randomly splitting a time series.** Random windows often make the problem look easier than real deployment.
- **Ignoring the forecast horizon.** One-step and multi-step forecasting are different tasks.
- **Using features from the future.** Even subtle timestamp mistakes can invalidate an experiment.
- **Confusing representation with reality.** A spectrogram or lag vector is a view, not the whole process.
- **Forgetting deployment constraints.** Accuracy offline does not help if the model is too slow, too noise-sensitive, or too brittle under drift.

10.13 Looking Ahead

This chapter treated sequences as processes, so evaluation had to respect prediction time. The next chapter generalizes that discipline into experiment claims themselves: once the protocol stops cheating through time, we still have to ask what any score actually justifies and what rival explanation survives.

Chapter Summary

- Sequential modeling begins by asking what was known at the decision moment rather than by choosing an architecture by habit.
- A prediction-time audit clarifies horizon, allowed context, forbidden future information, latency, and temporal structure before modeling begins.
- Spectrograms are useful because they expose time-frequency

structure, but they remain task-shaped lenses rather than neutral reality.

- Forecasting depends on lagged context, trend, seasonality, horizon, and regime change.
- Sequence model families differ mainly in how they represent and access memory.
- Chronological evaluation and realistic feature timing are essential for honest sequential experiments.
- Naive baselines, noise robustness, latency constraints, and drift handling are realism checks, not optional extras.
- The prediction-time audit is not specific to audio or forecasting: any sequential decision system must define what information existed at the decision moment, making this discipline applicable across medical monitoring, fraud detection, maintenance, recommendation, and many other domains.

Study Questions

1. Why should a sequential model be judged partly by what information was available at prediction time?
2. Why is a spectrogram best understood as a task-shaped lens rather than as the sound itself?
3. Why is forecast horizon part of the task definition?
4. How do sequence architectures differ as memory choices rather than as brand names?
5. Why can random splits be especially misleading for time-series tasks?
6. Why are naive baselines unusually important in sequential problems?
7. Why can an offline sequential result be statistically strong but operationally invalid?

Exercises

1. Write a prediction-time audit for a wake-word detector and for a day-ahead electricity forecast. What differs?
2. Explain in words why a short analysis window gives different information from a long analysis window in a spectrogram.
3. Construct a tiny forecasting example with three lag features and compute one prediction by hand.
4. Give one example of a task where speaker identity should be preserved and one where it should be ignored.
5. Describe a realistic form of leakage that could occur in a hospital time-series prediction system.
6. Explain why a model with strong clean-audio accuracy might still fail under realistic deployment noise.
7. For a one-hour-ahead demand forecast, write one illegal future feature and one legal feature that might look superficially similar. Explain the difference in timing.
8. Use Table 10.5 to sketch a one-page sequential design card for a streaming or forecasting task of your choice. If you are carrying one case through the book, state what changed in the earlier framing and evaluation artifacts once decision time, legal context, and latency became central.

Challenge Exercises

1. A forecasting model performs much better under a random split than under a chronological split. Give three technically plausible explanations, and state what additional experiment would test each one.
2. Explain carefully why the same audio representation might be useful for keyword spotting but insufficient for speaker verification, or vice versa.
3. A lab reports excellent anomaly-detection performance on a rare-event dataset. Describe an evaluation protocol that

would convince you the result is real rather than an artifact of leakage, class imbalance, or unstable labeling.

4. Design a sequential design card for a streaming medical alert system and explain which fields are hardest to specify honestly.

Card field	What must be written clearly
Seven-question focus	Questions 1, 3, 4, 6, and 7: what decision or action moment matters, what information is really available then, what sequential representation is legal, what evaluation is honest through time, and how latency shapes action
Decision time	The moment the system must act or emit a prediction
Horizon	How far into the future the prediction or detection target lies
Allowed context window	What past observations, frames, or tokens are legitimately available
Forbidden future information	What timestamps, summaries, or future windows must never enter the model
Timestamp and missingness handling	Whether the stream is regular or irregular, and whether gaps or missing values carry signal that must be represented explicitly
Task type	Recognition, keyword spotting, speaker verification, forecasting, anomaly detection, or another sequence task
Key invariances	What noise, channel variation, or temporal variation should be ignored
Baseline family	Persistence, seasonal repeat, simple lag model, or another realism-check baseline
Split and backtesting logic	Chronological holdout, grouped-by-source, expanding-window, sliding-window, streaming, or another deployment-faithful evaluation design
Latency budget	How quickly the system must respond
Operating mode	Whether the system is offline with full-sequence access or streaming with only partial context
Noise and drift risks	What channel variation or process change is likely in deployment

Table 10.5: A sequential design card turns prediction-time realism into a concrete protocol before a sequential-model claim is made.

Part IV

Evidence, Reliability, and Systems

Chapter 11

Experiments, Error Analysis, and Scientific Claims

Learning Goals

- Explain why an experimental result is only meaningful relative to the claim it is supposed to support.
- Use a claim audit to distinguish the strongest justified conclusion from stronger conclusions that remain unsupported.
- Understand reproducibility, baselines, repeated runs, ablations, and slice diagnosis as controls on alternative explanations.
- Use cross-validation when data are limited and selection must stay nested.
- Compute and interpret simple summary statistics across repeated runs.
- Bound the scope of an empirical claim rather than overstating it.
- Produce an experiment claim sheet before reporting a serious

System	Held-out accuracy	Gain
Lexical retrieval + template answer	0.842	–
Contextual retrieval + grounded generator	0.846	+0.004

Table 11.1: A tiny improvement on a grounded course-support assistant is not yet a scientific claim. Its meaning depends on protocol, variability, slices, and controls.

result.

The previous chapter made experimental discipline especially visible by showing how easy it is to cheat through time. This chapter generalizes that lesson. Across all AI domains, a score matters only if the protocol behind it is trustworthy and the scientific claim attached to it is disciplined. The chapter therefore moves in a specific order: name the claim, protect the premises, test whether the apparent gain is stable, ask what caused it, inspect where it fails, and only then decide what sentence is actually honest. The chapter culminates in an experiment claim sheet, so the early claim audit should be read as the first draft of that final artifact.

> **Problem-Solving Lens.** This chapter asks one central question: what exactly does this experiment justify, and what alternative explanation still survives? The chapter treats empirical work as scientific judgment rather than scoreboard reading.

11.1 Opening Dilemma: The Score Improved, So What?

Suppose an experiment table reports the following:

At first glance, the new system looks better. But many different interpretations are still possible:

- the baseline may have been too weak for a grounded support task,

- the gain may vanish across different seeds, prompts, or retrieval tie breaks,
- the overall score may hide failure on high-risk slices such as visa or accommodation questions,
- the generator may improve fluent phrasing while harming faithfulness to retrieved policy text,
- the protocol may have mixed document versions, reused the test set, or allowed hidden leakage from policy updates.

This is why a result table should never be read as self-explanatory. The chapter's job is to train a stronger reflex:

a number is not evidence until we know what claim it supports and what rival explanation still survives.

> **Problem-Solving Technique. Mini claim sheet.** Before discussing why a score changed, write one line for the claim, one for the baseline, one for the primary metric, one guardrail metric, and one slice where failure would matter most. The full experiment claim sheet at the end of the chapter is an expanded version of this early discipline.

Throughout this chapter, keep one experimental story in view: the grounded course-support assistant introduced in the language chapter. It receives student questions, routes them, retrieves syllabus or policy passages, sometimes escalates them to humans, and sometimes generates a grounded answer. This case is useful because a tiny numerical gain can still hide weak controls, retrieval errors, unsafe slices, or overconfident generation. It also gives the chapter its spine: the same small gain is revisited under stronger protocol control, repeated runs, ablations, slice analysis, and finally a bounded written claim.

When this opening section says *accuracy*, read it narrowly: the fraction of held-out questions judged correct under a fixed document snapshot and grading rubric. Later sections will ask whether that narrow metric hides weaker faithfulness, weaker slice performance, or unstable wins across repeated runs.

To keep the chapter from reading as if these habits belong only to educational NLP, occasionally cross-check the same questions against the earlier dashboard-camera pedestrian detector. A small gain on familiar daytime campus footage may still disappear under night rain, unseen cameras, or tighter latency constraints. The scientific discipline is the same even though the modality is different.

11.2 Problem-Solving Technique: Claim Audit

Problem-Solving Technique. Claim audit. Before reading the result table, write the intended claim in one sentence. Then ask:

- What weaker claim is definitely supported?
- What stronger claim is still not supported?
- What alternative explanation would survive even if the score improved?

This is the chapter's master habit. Once students adopt it, experiments stop looking like a stream of numbers and start looking like arguments with premises, controls, and scope limits. The seven-step protocol below is simply the chapter-wide expansion of that one habit.

11.2.1 A Seven-Step Protocol for Scientific Claims

The rest of the chapter can be read as one protocol:

1. state the claim,
2. protect the protocol,
3. choose the control,
4. quantify uncertainty,
5. isolate the mechanism,
6. inspect the failures,

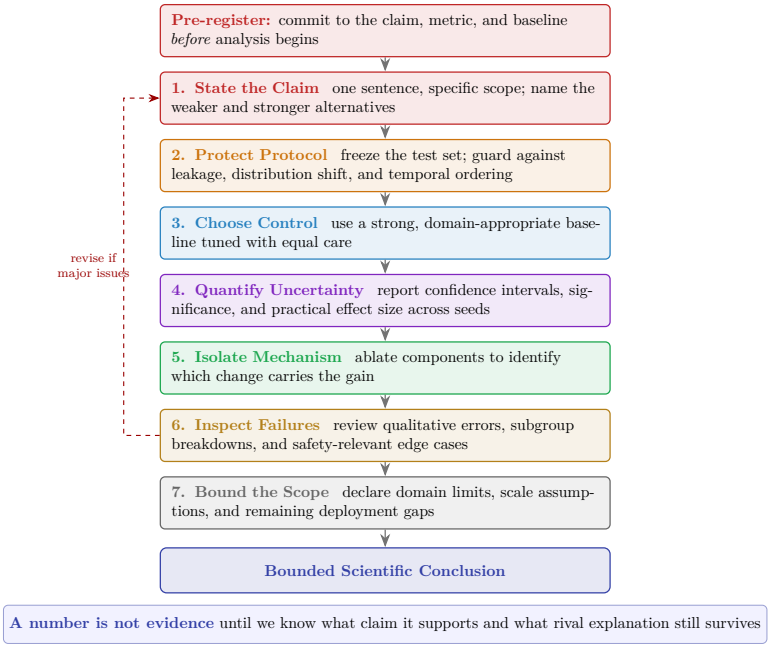


Figure 11.1: The seven-step scientific experiment workflow transforms scores into bounded claims by systematically blocking rival explanations. Each checkpoint prevents common failure modes, while decision diamonds and feedback loops ensure experimental rigor before deployment.

7. bound the scope.

Each step exists because a different rival explanation can otherwise survive. A good experiment does not remove every uncertainty, but it should make the remaining uncertainties visible rather than hiding them. Later sections follow this order closely, so the reader can treat the rest of the chapter as one pass through the same argument rather than as a list of disconnected evaluation tricks.

> **Practical Note.** A compact claim-audit workflow is: write the claim in one sentence, freeze the protocol and strongest baseline, measure run-to-run variation, isolate the mechanism with ablations, inspect the worst slices, and then write the strongest honest

Field to write before running	Why it matters
Claim in one sentence	prevents the result table from silently expanding into a stronger claim later
Primary metric and any guardrail metric	stops the team from highlighting whichever metric looks best after the fact
Strongest meaningful baseline	forces the comparison to be informative rather than convenient
Critical slices	commits the experiment to inspect where failure would matter most in deployment
Randomness plan	states in advance whether multiple seeds, repeated runs, or paired comparisons are required
What result would change your mind	turns the experiment into a test rather than a search for confirmation

Table 11.2: A rough pre-run claim sheet is a lightweight way to block hindsight narratives before the strongest-looking result has already been selected.

sentence the evidence supports. The workflow figure and the claim sheet are two views of the same discipline.

11.2.2 Write the Claim Sheet Before the Result Exists

One way to prevent post hoc storytelling is to write down the experimental claim structure before the final result is known. This does not need to be a formal research pre-registration in the strongest institutional sense. For this book, it is enough to adopt the habit of committing early to the main claim, baseline, metric, slices, and uncertainty plan before tuning has already suggested the winning story.

This habit is especially valuable when the expected gain is small. Tiny improvements are exactly where prompt search, hyperparameter luck, weak baselines, or selective reporting can create persuasive but fragile stories. Writing the rough claim sheet early does not remove every bias, but it makes later scope drift easier

to detect.

> **Practical Note.** If the claimed win is only a few tenths of a point, write down before the run which metric counts, which slices matter, how many seeds will be used, and what baseline must be beaten. Otherwise the comparison is too easy to rewrite after the numbers arrive.

11.2.3 Worked Claim Audit for the 0.004 Gain

Return to Table 11.1, where the new support assistant appears to beat the baseline by 0.004 accuracy points. The seven-step protocol turns that table into a disciplined judgment:

1. **State the claim.** A narrow starting claim might be: under the current question set, document snapshot, and evaluation rubric, the contextual retrieval plus grounded-generation system outperformed the lexical retrieval plus template baseline.

2. **Protect the protocol.** We must confirm that both systems used the same question split, the same policy-document snapshot, the same grading rubric, and the same test discipline, and that the test set was not reused during development.

3. **Choose the control.** We must ask whether the lexical-template system is actually a serious grounded baseline or merely an easy system to beat.

4. **Quantify uncertainty.** A gain of 0.004 may disappear entirely across seeds, so repeated runs are essential.

5. **Isolate the mechanism.** If retrieval, generation, prompt wording, and postprocessing all changed at once, we still do not know which change produced the gain.

6. **Inspect the failures.** The overall score may hide a collapse on multilingual, distressed, or high-risk advising slices that matter in deployment.

7. **Bound the scope.** Even if every earlier step looks clean, the supported claim is still local unless broader external evidence

is added.

So the raw table does *not* yet justify "the new support assistant is better." At best, it motivates the next experiments needed to decide whether that sentence can be defended.

> **Problem-Solving Technique. Tempting mistake → corrected reasoning.** Tempting mistake: "The score improved, so the method works." Corrected reasoning: "A score change is only one premise. The claim still depends on protocol control, baseline strength, uncertainty, mechanism isolation, failure analysis, and bounded scope."
>
> **Problem-Solving Technique. Signal to trust → signal to doubt.** Trust: the gain survives a strong baseline, repeated runs, mechanism isolation, and failure inspection, and the final claim stays tightly bounded to the evidence. Doubt: the win is tiny, only one run is shown, several components changed at once, and the paper or report jumps from score movement to a broad conclusion.
>
> **Decision Box.** Before running an experiment, decide what result would genuinely support the claim and what alternative explanation would remain possible even if the metric improved.

11.3 Experimental Protocol as Control of Premises

Definition 11.1 (Reproducibility)**.** Reproducibility is the ability for another person, or the same person later, to repeat an experiment with the documented setup and obtain comparable results.

Reproducibility is not paperwork. It is control over the premises of the argument. If the dataset version is unclear, the preprocessing is undocumented, the split procedure is vague, or the seeds are missing, then the scientific premise of the result is unstable.

Field-scale reproducibility efforts have made this expectation explicit by asking authors to document data, code, training details, and evaluation choices clearly [50].

11.3.1 What Must Be Recorded

A serious experiment record should include at least:

- the dataset source and version,
- preprocessing and feature-construction steps,
- train, validation, and test split logic,
- model family and important hyperparameters,
- random seeds or other sources of stochasticity,
- evaluation metrics and how they were computed,
- any postprocessing, thresholding, or filtering steps.

In small projects this may be a careful notebook and a script. In larger systems it may involve experiment tracking tools, model registries, and configuration files. The scale changes, but the principle does not: another practitioner should be able to understand how the result came to exist.

> **Practical Note.** If a result can be reproduced only by the original author from memory, it is not a robust experimental result. It is a fragile anecdote.

Example 11.1 (Real-World Callout: Reproducibility Changed the Standard)**.** Henderson et al. showed that deep reinforcement learning results could shift substantially under different seeds, code details, and evaluation choices, making apparently clear model comparisons much less stable than many papers suggested [29]. Pineau et al. then described the NeurIPS reproducibility program as a field-level response: stronger documentation of data, code, hyperparameters, and experimental procedure became part of what counts as serious empirical work [50]. The point is not limited to reinforcement learning. Once a field sees

how easily results can move under weak controls, better experimental reporting stops looking optional.

11.3.2 Split Logic Is Part of the Premise

Chapter 3 introduced protected evaluation, and Chapter 9 showed how split logic can break sequential claims entirely. The same principle applies broadly. If the validation set guides model development, then it is part of development. If the test set influences design choices, it is no longer a clean test. Leakage does not need to be dramatic to be destructive. Protocol control is what makes the later questions about baselines, uncertainty, and mechanism worth asking at all.

11.4 Baselines and Controls: Better Than What?

No model can be interpreted in isolation. A result only becomes informative once it survives a control that makes the claim hard to win.

A result is not informative until it has survived a control that makes the claim hard to win.

11.4.1 Task-Dependent Strong Baselines

The right baseline depends on the task:

- for classification, majority-class predictors, linear models, or prior task-standard methods,
- for forecasting, persistence or seasonal-repeat rules,
- for retrieval, lexical baselines,
- for deep systems, smaller or simpler models with comparable data access.

This is why the book has repeatedly returned to linear models, classical methods, and naive forecasting rules. They are not museum pieces. They are controls.

Example 11.2 (A Strong Baseline for the Running Support Case)**.** For the course-support assistant, a meaningful baseline is not an ungrounded generator that answers from internal statistics alone. A much stronger control is a lexical retriever over current syllabus and policy documents plus a conservative template or extractive answerer. That baseline already respects the chapter's grounding requirement. Beating it means more than beating a fluent but ungrounded chatbot.

11.4.2 Weak Controls Produce Weak Claims

Beating a deliberately weak baseline may still be useful as a sanity check, but it does not justify a strong scientific story. A baseline should make unearned claims harder, not easier.

> **Failure Box.** Reporting only the best new model, with no meaningful control and no explanation of what simpler alternatives can or cannot do, is not showing discipline. It is hiding the context required for interpretation.

Once protocol and controls are credible, the next question is whether the apparent gain is stable or merely fortunate.

11.5 Repeated Runs and Uncertainty: Stable or Lucky?

Many machine-learning systems are stochastic. Initialization, data order, subsampling, augmentation randomness, and hardware nondeterminism can all change the final result. This means a single run can be a poor basis for a scientific claim, a point made especially visible by reproducibility analyses in deep reinforcement learning [29]. This section is the chapter's stability check: after asking "better than what?" it asks "better often enough to believe?"

11.5.1 Why One Number Is Not Enough

Suppose model A achieves accuracy 0.846 on one run and model B achieves accuracy 0.842. It is tempting to announce that A is better. But if each model varies by several points across seeds, the difference may be too small to justify that conclusion.

> **Prerequisite Bridge.** If repeated runs produce metrics $m_1, m_2, \ldots, m_S$, the sample mean is
>
> $$\bar{m} = \frac{1}{S}\sum_{s=1}^{S} m_s.$$
>
> The sample standard deviation is
>
> $$\sigma = \sqrt{\frac{1}{S-1}\sum_{s=1}^{S}(m_s - \bar{m})^2}.$$
>
> The mean summarizes typical performance; the standard deviation summarizes instability across runs.

11.5.2 Worked Example: Clean Versus Shifted Digits Across Seeds

Consider repeated results on a digit-classification task under two slices: clean images and shifted images.

Without augmentation, the clean accuracies across five seeds are:

$$0.947,\ 0.956,\ 0.944,\ 0.956,\ 0.958,$$

with mean about 0.952.

The shifted accuracies for the same model are:

$$0.153,\ 0.124,\ 0.142,\ 0.107,\ 0.098,$$

with mean about 0.125.

With shift augmentation, the clean accuracies become:

$$0.871,\ 0.882,\ 0.833,\ 0.882,\ 0.878,$$

with mean about 0.869, while the shifted accuracies become:

$$0.844,\ 0.858,\ 0.856,\ 0.873,\ 0.844,$$

with mean about 0.855.

This is a much better scientific result than one headline number. It shows:

- the baseline is excellent on clean data,
- the same baseline collapses on a realistic robustness slice,
- augmentation changes the tradeoff rather than uniformly improving everything,
- the tradeoff is stable enough across seeds to support a real claim.

The honest conclusion is not "augmentation is better." It is narrower: in this setup, augmentation greatly improves shifted-data robustness at a measurable clean-data cost. The companion demo is useful here because it makes the same point computationally: repeated runs are valuable partly because they reveal tradeoffs, not only because they stabilize one favorite metric.

> **Evidence Box.** When a result changes the tradeoff rather than improving every metric, report the tradeoff explicitly. That is stronger science than hiding the uncomfortable metric.

11.5.3 Uncertainty Is Part of the Finding

If a method varies widely across seeds, that instability is itself part of the result. A model that sometimes wins and sometimes collapses may be less useful than a slightly weaker but more stable alternative.

> **Advanced Note.** More advanced work may report confidence intervals, bootstrap uncertainty, or paired tests. The basic principle is unchanged: quantify uncertainty instead of pretending that one run is the whole story.

11.5.4 Cross-Validation for Limited Data

When data are scarce, a single train/validation/test split can make the claim depend too much on which examples happened to land in each partition. Cross-validation reduces that sensitivity by rotating which examples play the held-out role. It is not a shortcut around experimental discipline. It is a different evaluation design for limited data.

> **Practical Note.** A compact cross-validation workflow is:
>
> 1. fix the outer folds before looking at results,
> 2. tune only on the training side of each fold,
> 3. if search is needed, nest an inner validation loop inside the outer fold,
> 4. score each outer held-out fold once,
> 5. summarize the fold scores with mean and spread,
> 6. explain any fold that behaves differently from the rest.

This matters because the outer fold is part of the claim, while the inner fold is part of selection. If the held-out fold influences prompt choice, hyperparameter choice, or model choice, then the fold is no longer a clean test. The claim supported by cross-validation should therefore name the search protocol as well as the average score: under this specified nested procedure, the method appears to generalize about this well on new data from the same source.

Example 11.3 (Cautionary Example)**.** A small medical-support dataset has only 120 labeled cases. A team tries three prompt variants and four retrieval depths, then keeps the combination with the best average across all folds and reports that number as the final result. The score may be real, but the selection protocol has leaked into the evaluation protocol. The honest claim is weaker unless the winner was chosen inside the training side of each fold and evaluated on untouched outer folds.

Problem-Solving Technique. Three uncertainty sources. Do not lump every kind of uncertainty together:

- **Seed-to-seed variability** changes because initialization, sampling, or optimization randomness changes.
- **Finite test-set uncertainty** changes because the held-out set is only a sample of the deployment world.
- **Human-rater uncertainty** changes because reviewers, rubrics, and borderline cases can disagree.

The right remedy depends on which source dominates: more runs help with seeds, more held-out data helps with test noise, and better rubric design helps with rater disagreement.

Math Lens. If repeated runs produce scores $x_1, \ldots, x_n$, the sample standard deviation is

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}, \qquad \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

The standard error of the mean is then $s/\sqrt{n}$, which shrinks as the number of runs grows. A rough undergraduate confidence interval for the mean is

$$\bar{x} \pm 2\frac{s}{\sqrt{n}},$$

which is only an approximation, but it gives the right intuition: a result is less convincing when the run-to-run spread is large compared with the reported gain.

11.5.5 Worked Example: Mean Improvement Is Not the Whole Uncertainty Story

Suppose the support-assistant team runs the same comparison over six seeds and records answer accuracy:

$$\text{baseline}: 0.842,\ 0.839,\ 0.847,\ 0.841,\ 0.844,\ 0.838$$

$$\text{new system}: 0.846,\ 0.845,\ 0.850,\ 0.844,\ 0.848,\ 0.843$$

The new system still looks better on average, but now the evidence is more honest. The baseline mean is about 0.842, the new-system mean is about 0.846, and the average improvement is again roughly 0.004. That result is more credible than the single-run table because it survives repeated stochastic variation. But the scientific judgment is still incomplete until we ask how variable the run-to-run gap itself is.

If one or two seeds erase the gain entirely, the correct interpretation changes. The practical question is not only whether the new mean is larger. It is whether the improvement is stable enough that another practitioner running the documented pipeline should expect a similar outcome rather than an arbitrary reversal.

> **Practical Note.** When the claimed improvement is small, the standard of evidence should become stricter rather than looser. Tiny gains are exactly where seed luck, evaluation noise, or minor implementation asymmetries can create the strongest false stories.

11.5.6 Paired Comparison Usually Matters More Than Separate Means

Many model comparisons are naturally paired because both systems are evaluated on the same examples. This is important. If the new support assistant and the baseline answer the same n student questions, then each question contributes a paired difference rather than two unrelated observations.

> **Math Lens.** Suppose six shared questions give per-example differences
>
> $$d = [0, +1, -1, 0, 0, +1],$$
>
> where $+1$ means the new system is correct and the baseline is wrong, -1 means the reverse, and 0 means a tie. Then
>
> $$\bar{d} = \frac{1}{6}.$$

> The mean gain is real, but the negative entry still matters if it occurs on a high-risk question. Paired evaluation keeps that structure visible.

Suppose that across 10 held-out questions, the new system improves on four, ties on four, and becomes worse on two. The average gain may still be positive, but the example-level pattern reveals more:

- Are the wins concentrated on routine low-stakes questions while the losses occur on high-risk cases?
- Are the wins only stylistic while the losses are factual or policy-related?
- Are the same difficult items unstable across seeds?

Paired reasoning is often scientifically stronger because it asks where one system changed the actual decision outcome rather than merely comparing two aggregate means as if the evaluations were independent. In language and retrieval work, paired designs are especially useful because the same examples often vary greatly in difficulty.

> **Math Lens.** If the baseline and new system are both scored on the same examples, define the per-example difference $d_i = a_i - b_i$. The comparison then reduces to one average difference,
>
> $$\bar{d} = \frac{1}{n}\sum_{i=1}^{n} d_i,$$
>
> with uncertainty governed by the spread of the differences rather than by two separate totals. That is stronger because shared example difficulty often cancels out in d_i, so the noise term is smaller than in an unpaired comparison. In short: paired evaluation asks, example by example, what actually changed.

Example 11.4 (Paired Wins Are Not All Equal)**.** Imagine that a new grounded generator answers five extra routine deadline questions correctly but also gives two unsupported answers on

visa-status questions that the baseline had escalated safely. A raw net gain of three examples does not settle the story. The paired reading is better because it reveals that the gain came from low-risk convenience while the losses hit high-risk cases. This is precisely the kind of result that should weaken a deployment claim even if the mean score improves.

11.5.7 Confidence Intervals and Practical Importance

Confidence intervals, bootstrap summaries, or paired significance tests can help quantify uncertainty more formally. The book does not need every mathematical detail, but the interpretation habit matters. A narrow interval around a tiny improvement can still describe a result that is scientifically real but operationally trivial. Conversely, a practically important difference may remain uncertain if the study is too small or too noisy.

> **Decision Box.** Ask two separate questions. First, is the result stable enough that the claimed direction of improvement is believable? Second, even if it is believable, is the gain large enough on the right metrics and slices to matter?

This distinction is especially important in applied AI. A statistically stable gain of 0.003 in generic answer quality may matter far less than a stable reduction in unsafe direct answers on escalation-required cases. Good experimental work therefore combines uncertainty summaries with task-specific consequence analysis rather than treating "significant" as a synonym for "important."

11.5.8 Hyperparameter Budget and Selection Protocol

One of the easiest ways to produce a misleading result is to tune the new system heavily while giving the baseline only a casual configuration. The final table may still look clean, but the comparison is no longer fair. What won was not only the method. It was also the unequal search effort behind the method.

This issue appears in small student projects and in serious industrial experiments alike. Online experimentation teams learned long ago that apparently clear wins can dissolve once logging bugs, metric definitions, or implementation asymmetries are checked carefully [37, 38]. The same lesson applies to offline machine-learning comparisons. If one model receives broader threshold tuning, more prompt variants, more retrieval settings, or more careful early stopping than another, then part of the reported gain may simply reflect evaluation effort.

Modern language-model comparisons make this risk even sharper. A new method may appear to win because it received more prompt search, more retrieval-query variants, or more aggressive filtering of failed generations than the baseline. Once search effort becomes asymmetric, the table is no longer only comparing methods. It is comparing methods plus unequal experimentation budgets.

Practical Note. A fair search protocol is:

1. declare the search space and budget before seeing the outcome,
2. give each candidate the same number of trials, seeds, or folds,
3. use validation data for selection and keep the final test untouched,
4. record which choices were fixed in advance and which were tuned,
5. report the full search path, not only the winning setting.

In claim terms, the search protocol is part of the premise. If method A got more tuning opportunities than method B, then the result supports a claim about method A under a larger search budget, not a clean claim that the method itself is better. Equal tuning budget is therefore a fairness condition, not a bookkeeping detail.

Failure Box. "New method beats baseline" is a

weak sentence if the baseline received less tuning, fewer seeds, weaker prompts, or inferior access to the same information. Fair comparison requires roughly comparable optimization effort, not only identical test data.

Problem-Solving Technique. Fair comparison checklist. Before trusting a comparison, verify:

- same split,
- same document or data snapshot,
- same tuning budget,
- same prompt budget,
- same retrieval budget,
- same stopping and model-selection rule.

If one side got more search effort, say so explicitly; otherwise the table is comparing method plus extra optimization effort, not method alone.

11.5.9 Multiple Comparisons and Researcher Degrees of Freedom

Unequal budget is not the only way experiments become misleading. Even when two methods receive similar tuning effort, the final reported result may still be inflated if the team tries many prompts, several retrieval settings, multiple seeds, different stopping rules, a few model sizes, and then reports only the most flattering comparison. This is a version of the same scientific problem that appears in other fields under names such as multiple comparisons or researcher degrees of freedom. The more chances we give ourselves to find a lucky win, the easier it becomes to mistake search noise for method evidence.

Math Lens. If one comparison has a false-alarm rate α, then m independent attempts have probability

$$1 - (1 - \alpha)^m$$

of producing at least one apparently significant win by luck alone. For small α, this is approximately $m\alpha$. So a 5% false-alarm rate becomes much less reassuring once the team tries many prompts, thresholds, or seeds and only reports the best one.

Current AI practice creates this risk constantly. A system may be tested with several prompt phrasings, retrieval depths, reranking choices, decoding temperatures, and context-window policies before one configuration is declared the result. That exploration is often necessary. The problem is not exploration itself. The problem begins when the search process is hidden, when only the best outcome survives into the table, or when one method quietly receives more opportunities to look good than another.

Decision Box. Before trusting an improvement, ask: how many comparison opportunities existed? If the team tried many prompts, thresholds, seeds, or model variants, the write-up should say so and should explain which decisions were fixed in advance, which were tuned on validation data, and which were only inspected after the result appeared.

A practical student protocol is simple:

- record the full comparison space, not only the winning setting;
- separate validation-time choices from test-time reporting;
- avoid switching baselines after seeing which one loses most cleanly;
- report enough of the search path that a reader can tell whether the gain survived reasonable alternatives.

This does not require turning every student project into a clinical trial. It means making the scientific opportunity structure visible. When a gain appears only after many attempts, the honest claim is weaker than when the same gain appears under a narrow, pre-committed comparison plan.

11.5.10 Human Evaluation Can Also Be an Experimental Protocol

For generated answers, summaries, or grounded responses, some important qualities are not well captured by one automatic metric. Faithfulness, clarity, escalation appropriateness, and policy compliance may require human judgment. But human evaluation is not an escape from rigor. It is another protocol that needs controls. Once a result looks real under the right protocol, the next chapter question is no longer only whether the gain exists, but what mechanism actually produced it.

A serious human evaluation design usually specifies:

- what exact question the reviewer is answering,
- whether judgments are blind to system identity,
- whether reviewers see the retrieved evidence,
- how disagreements are resolved,
- whether the rubric separates factual support from stylistic preference.

This matters because otherwise human review can collapse into impressionistic scoring. A fluent answer may be preferred even when it is less faithful to the retrieved policy. A shorter answer may look safer simply because reviewers do not see the omitted requirement. The same scientific discipline from earlier sections applies here: define the claim, make the rubric legible, and inspect disagreement rather than hiding it.

11.6 From Improvement to Mechanism: Ablation

Definition 11.2 (Ablation Study)**.** An ablation study isolates the contribution of one component or design choice by removing or changing it while holding the rest of the system as fixed as possible.

Benchmarking tells us that a bundle improved. Ablation begins to tell us why. This is the transition from evidence of improvement to evidence about mechanism.

11.6.1 What Good Ablation Looks Like

A useful ablation is:

- targeted at a specific claim,
- comparable to the full system in all other important respects,
- evaluated with the same metrics and splits,
- interpreted carefully when components interact.

For example, if a system uses pretraining, augmentation, regularization, and a new architecture block, an honest ablation plan tests each element separately and then in combination. This still may not resolve every interaction, but it is far stronger than presenting only the best final bundle.

11.6.2 Mechanism Evidence Is Stronger Than Bundle Evidence

If five changes are introduced at once, the engineering result may still be useful, but the scientific explanation remains weak. Ablation cannot guarantee full explanation, yet it is one of the best tools for reducing unexplained bundle effects.

11.6.3 Worked Example: What Actually Caused the Gain?

Suppose the course-support team later runs a fuller ablation on a larger follow-up benchmark using the same task definition as the earlier pilot result:

This is a much stronger scientific story than a single final comparison. The table suggests that contextual retrieval accounts for most of the improvement, while generation adds some additional answer quality once the evidence is strong enough. It also shows a tradeoff: the more flexible generator is not as faithful as

System	Answer accuracy	Faithfulness pass rate	What the result suggests
Lexical retrieval + template answer	0.842	0.989	strong grounded baseline, limited flexibility
Contextual retrieval + template answer	0.861	0.990	retrieval quality alone explains much of the gain
Lexical retrieval + grounded generator	0.848	0.944	generation helps little when retrieval evidence is weak
Contextual retrieval + grounded generator	0.872	0.962	best overall performance, but generation introduces some faithfulness cost

Table 11.3: Ablation is stronger when it isolates whether the gain came from better retrieval, better answer generation, or the combination. This table is a follow-up benchmark, not the original +0.004 pilot comparison.

the template answerer, even when the retriever is improved. The right conclusion is therefore narrower than "generation is better." A stronger conclusion is: in this setup, better retrieval explains most of the gain, and grounded generation adds some value while also increasing unsupported wording risk.

11.7 Error Analysis and Slice Diagnosis: Where Does It Fail?

Aggregate metrics hide structure. This is one of the most important lessons in the whole book. Ablation asks why the system improved; error analysis asks where that story breaks.

> **Problem-Solving Technique. Slice diagnosis.** Do not trust one aggregate score until you inspect where performance fails, what kinds of mistakes dom-

inate those slices, and whether those failures matter for deployment.

Definition 11.3 (Error Analysis)**.** Error analysis is the structured inspection of model failures to understand where, how, and why predictions go wrong.

11.7.1 Worked Example: Overall Accuracy Hides a Failure

Suppose the course-support assistant is evaluated on two slices:

- Slice A: routine deadline and grading questions, with 95 correct out of 100 examples,
- Slice B: high-risk visa and accommodation questions, with 15 correct out of 30 examples.

The overall accuracy is

$$\frac{95+15}{100+30} = \frac{110}{130} \approx 0.846.$$

An overall score of 84.6% may sound respectable. But Slice B accuracy is

$$\frac{15}{30} = 0.5.$$

If Slice B contains questions that should have been escalated or handled with much stronger caution, then the overall score is concealing a deployment-critical failure. A system that looks respectable in aggregate may still be unsafe exactly where the institutional stakes are highest.

Example 11.5 (Contrast Case: A Vision Gain That Fails on the Slice That Matters)**.** Suppose a new pedestrian detector raises daytime campus-camera average precision from 0.91 to 0.93, but on the night-rain slice from the earlier vision chapter it drops from 0.74 to 0.58. The honest conclusion is not that the detector improved. It is that the new detector improved one familiar slice while becoming worse on a deployment-critical slice. That is exactly the kind of result a claim audit is supposed to surface before launch language appears.

11.7.2 Useful Slices

Useful slices usually come from one of four sources:

- deployment conditions such as device type, environment, or latency,
- known heterogeneity such as subgroup, dialect, sensor, or source domain,
- known difficulty factors such as blur, noise, rare classes, or sequence length,
- patterns discovered through manual error inspection.

11.7.3 Mistake Reading Is Part of Scientific Interpretation

Error analysis is not just subgroup tabulation. It also means reading false positives, false negatives, calibration failures, confusing examples, and cases where the model is confidently wrong.

> **Practical Note.** If you know only how many mistakes occurred, but not what kinds of mistakes they were, you still do not understand the model well enough to make a strong claim.

11.7.4 From Error Clusters to the Next Experiment

Error analysis becomes much more valuable when it leads to the next controlled experiment rather than a vague impression that "the model struggles on hard cases." The purpose of reading failures is to propose a sharper rival explanation and then test it.

This table embodies a habit worth reusing across the whole book. A failure pattern should usually trigger one of three responses:

- a cleaner evaluation slice,
- a sharper ablation,
- a redesign of the pipeline or decision policy.

Observed failure pattern	Likely explanation to test	Next experiment or redesign step
Wrong policy passage retrieved for paraphrased student questions	retrieval is too lexical or query rewriting is weak	compare lexical retrieval, contextual retrieval, and query expansion on a paraphrase-heavy slice
Fluent answers add unsupported conditions	generation policy is too free or prompt instructions are underspecified	compare extractive, templated, and grounded generative answerers under the same retrieval evidence
Aggregate score is stable but multilingual slice is weak	coverage or representation gap in data and retrieval index	build slice-specific evaluation and test targeted data expansion or retrieval normalization
Night-rain detector failures despite strong daytime metrics	shortcut learning to familiar illumination or camera conditions	add corruption or domain-shift evaluation and compare augmentation or fallback policies

Table 11.4: Error analysis is strongest when it generates the next falsifiable experiment rather than only a narrative about what looked difficult.

What it should *not* trigger is immediate storytelling about the model's "understanding" without a follow-up test.

11.7.5 When the Right Conclusion Is to Change the Claim

Sometimes error analysis does not suggest a fix strong enough to preserve the original claim. That is still scientifically valuable. If a support assistant remains brittle on institution-specific advising cases, the right move may be to narrow the claim from "automated answering" to "retrieval support plus escalation." If a detector fails under the visibility conditions that matter most, the right conclusion may be to redesign the operating envelope rather than to celebrate average performance.

> **Evidence Box.** Strong experimental work does not insist that every result end with a larger claim. Some-

> times the highest-quality outcome of a chapter's protocol is a smaller, better-defended claim or a decision not to deploy yet.

11.8 Claim Scope: Local, External, and Negative Results

After the protocol, controls, uncertainty, mechanism evidence, and failure analysis are in place, one final question remains:

What is the largest honest sentence you are allowed to say?

11.8.1 Local Claims Versus Broad Claims

If an experiment compares two methods on one dataset under one evaluation protocol, the strongest supported claim is usually local:

> Under this held-out course-support dataset, document snapshot, grading rubric, and implementation, contextual retrieval plus grounded generation outperformed the lexical-template baseline.

That is already useful. What it does *not* automatically support is a broad claim such as:

> Retrieval-augmented generation is generally better for student support, policy answering, or language understanding in the wild.

11.8.2 External Validity

A result may be internally valid and still have weak external validity. It may hold on one benchmark but transfer poorly to new domains, institutions, devices, or future data. Claims about robustness, fairness, or general usefulness usually require broader evidence than one benchmark score.

11.8.3 Negative and Partial Results

A result showing failure under noise, subgroup shift, or strict chronological evaluation is still a result. Negative findings often clarify the real limits of an approach and may be scientifically more valuable than another narrow leaderboard gain.

11.9 Chapter Artifact: An Experiment Claim Sheet

By the end of this chapter, the reader should be able to write down not only what result they obtained, but what scientific sentence it actually supports. Earlier artifacts in the book named the decision, the metric, the evaluation boundary, the baseline family, the structure diagnosis, the training claim, the reuse plan, the modality-specific deployment constraints, and the sequential prediction-time contract. This chapter's sheet adds the scientific-argument layer: which premise is fixed, which rival explanation is blocked, and what scope remains.

> **Problem-Solving Technique. Experiment claim sheet.** Before reporting a result, write the claim, the control, the protocol premises, the uncertainty summary, the key ablations, the critical slices, and the strongest supported conclusion.

Example 11.6 (Filled Experiment Claim Sheet)**.** For the 0.004 gain in Table 11.1, a disciplined sheet might read as follows: claim in one sentence is "the contextual-retrieval support assistant improves held-out answer accuracy over the lexical-template baseline on this question set"; strongest baseline is the grounded lexical-template system rather than an ungrounded chatbot; protocol assumptions are identical question splits, frozen policy-document snapshot, fixed grading rubric, and no test-set reuse; sources of randomness include initialization, retrieval tie breaking, and generation stochasticity; repeated-run summary now shows a modest mean gain across six seeds, so the pilot no longer rests on one lucky run; essential ablations are still missing for that pilot comparison, so the mechanism behind the gain remains un-

Sheet field	What must be written clearly
Seven-question focus	Questions 5 and 6 most directly: which baseline or control matters, and what evidence the experiment truly justifies
Claim in one sentence	The exact scientific or engineering claim the experiment is meant to support
Strongest meaningful baseline	The control the method must beat for the claim to be informative
Protocol assumptions	Dataset version, split logic, preprocessing, and other premises of the experiment
Sources of randomness	Seeds, stochastic training choices, and anything else that can move the result
Repeated-run summary	Mean, variability, and any important tradeoff across runs
Essential ablations	Which components must be removed or changed to test the mechanism claim
Critical slices	Which deployment-relevant slices must be inspected beyond the aggregate score
Remaining alternative explanations	What rival explanation still survives after the current experiment
Strongest supported claim	The largest honest sentence the evidence supports now
Stronger claim still unsupported	What the result does *not* justify yet

Table 11.5: An experiment claim sheet turns a result table into an explicit scientific argument with scope and limits.

resolved; critical slices include multilingual questions, distressed messages, and high-risk advising requests where aggregate accuracy may hide failure; remaining alternative explanations include weak controls, better retrieval rather than better generation, and unsafe slice tradeoffs; strongest supported claim right now is that under this protocol the contextual-retrieval system achieved a modest accuracy gain on this benchmark; stronger unsupported claim is that the new system is generally better for course support or safer for deployment.

11.10 Common Mistakes in Experimental Work

These mistakes are surface forms of the chapter's main enemy:

confusing improved scores with justified explanation.

Mistake Checklist.

- **Changing too many things at once.** This may improve the final system, but it weakens the explanation.
- **Choosing weak baselines.** A poor control can make a weak gain look impressive.
- **Reporting only the best run.** The best run is often the least informative summary when the method is stochastic.
- **Using only aggregate metrics.** Aggregate numbers can hide subgroup failures and deployment tradeoffs.
- **Making claims larger than the evidence.** A strong-sounding conclusion can still be scientifically weak if the experiment is narrow.
- **Confusing benchmarking with explanation.** A benchmark tells us which number is larger, not why.

11.11 Looking Ahead

This chapter treated experiments as scientific arguments. The next chapter extends the same discipline to bounded failure: once a claim is experimentally honest, we still have to ask for whom it fails, under what shifts, with what privacy exposure, and with what safety margin.

Chapter Summary

- A score improvement is not self-explanatory; its meaning depends on the claim, protocol, controls, and remaining rival explanations.
- Claim audit is the master habit of scientific judgment: state the claim, test what weaker claim is supported, and bound what stronger claim remains unsupported.
- Reproducibility protects the premises of the empirical argument.
- Strong baselines and controls are necessary because no model is meaningful in isolation.
- Repeated runs reveal whether a result is stable or merely fortunate.
- Ablation begins to explain why a system improved rather than only showing that the bundle improved.
- Error analysis and slice diagnosis reveal failures hidden by aggregate metrics.
- Scientific claims should be bounded by the actual evidence rather than inflated beyond it.

Study Questions

1. Why is it useful to think of an experiment as an argument rather than as a scoreboard entry?
2. What information is necessary to protect the premises of an experiment?
3. Why can a single run be a poor basis for a scientific claim?
4. What does ablation begin to tell us that plain benchmarking cannot?
5. Why can overall accuracy hide a deployment-critical failure?

6. What is the difference between a local supported claim and a broader unsupported claim?
7. Why is a grounded baseline stronger than an ungrounded baseline for a retrieval-based support assistant?

Exercises

1. Write a one-sentence claim for a model comparison on one dataset, then rewrite it into a weaker but clearly justified version.
2. Suppose a model obtains validation accuracies 0.81, 0.84, 0.79, and 0.82 across four seeds. Compute the sample mean.
3. Give one example of a strong baseline for a classification task and one for a forecasting task. Explain why each is meaningful.
4. Construct a simple two-slice evaluation example where the overall metric looks strong but one slice is clearly unacceptable.
5. Describe an ablation plan for a system with pretraining, augmentation, and a postprocessing rule.
6. State one broader claim that would remain unsupported even if a method beat a baseline on one benchmark.
7. For a retrieval-augmented support assistant, name one slice where generation quality matters most and one slice where escalation quality matters most. Explain why the evidence should differ.
8. A pedestrian detector improves on familiar daytime footage but degrades on night-rain or unseen-camera slices. State the strongest honest claim and one deployment recommendation that follows.
9. Use Table 11.5 to sketch an experiment claim sheet for a system of your choice. If you are carrying one case through the book, state which earlier artifact supplies the baseline, the

critical slices, and the strongest remaining deployment limitation.

Challenge Exercises

1. A research team reports that a new model is better because its best run beats the baseline by 0.4 points. Give three reasons why this claim may still be weak, and state what additional evidence would strengthen it.
2. A model improves overall performance but worsens a critical subgroup. Explain how you would report this result scientifically and what deployment recommendation might follow.
3. Take the experiment claim sheet you wrote in the standard exercises and identify the field that was hardest to fill honestly. Explain what additional evidence would be needed to defend it.
4. A result survives strong baselines and repeated runs but has no ablation and no slice analysis. Explain exactly what claim remains possible and what claim is still too strong.

Chapter 12

Reliability: Bias, Robustness, Privacy, and Safety

Learning Goals

- Understand reliability as a property of the full pipeline rather than only the trained model.
- Explain how subgroup evaluation, robustness checks, privacy protections, and safety constraints all operationalize reliability.
- Compute and interpret basic subgroup metrics such as accuracy, true positive rate, and false positive rate.
- Distinguish distribution shift, spurious correlation, privacy leakage, and safety failure as different reliability problems.
- Learn when a system should be redesigned, constrained, or rejected rather than simply tuned harder.

The previous chapter argued that experimental results only matter when claims match evidence. Reliability extends that discipline into higher-stakes questions and turns it into a deployment

constraint. It asks not only whether a model works on average, but for whom, under what shifts, with what privacy costs, and with what safety margins when failure matters. The chapter therefore moves in a specific order: identify who is unevenly served, test how those failures change under shift, trace what information might leak, bound the failures that are unacceptable, and then decide what monitoring and fallback make deployment defensible. The chapter culminates in a reliability review card, so the early slice dashboards and harm models should be read as preparation for one launch decision rather than as a detached ethics checklist.

> **Problem-Solving Lens.** This chapter widens the seven-question framework from average performance to bounded failure. The question is no longer only "does it work?" but "for whom, under what changes, and with what unacceptable risks?"

12.1 Reliability Is a Pipeline Property

Every AI system simplifies reality. The question is not whether simplification occurs, but how its consequences are distributed and whether the resulting failures are acceptable. Reliability is therefore not a property that can be read off one model checkpoint. It emerges from data collection, labeling, measurement, representation, objective choice, thresholding, interface design, monitoring, and deployment context. This full-pipeline framing matches work that locates harm across the machine-learning life cycle rather than only in the final trained model [59].

This chapter groups together bias, robustness, privacy, and safety because they are often separated artificially in teaching. In practice they interact. A subgroup failure may be caused by data imbalance, but it may only become visible under a domain shift. A privacy mechanism may reduce memorization risk but also alter utility. A safety guardrail may depend on uncertainty estimation and fallback design. The unifying idea is that reliability means the system continues to behave within acceptable bounds under the conditions that matter.

Decision Box. Do not ask only "how accurate is the model?" Ask "for whom does it work, under what changes does it fail, what information might it expose, and what happens when it is wrong?" Reliability begins when those questions become part of the core technical design.

Practical Note. A compact reliability workflow is: name the harm model, identify operational slices, measure subgroup metrics, stress the system under realistic shifts, decide what fallback or no-launch condition is required, and then record that decision on the review card. That is also the chapter order in miniature. The rest of the chapter fills in the evidence each step needs.

Problem-Solving Technique. Wrong question → better question. Wrong: "Is the model fair, robust, private, and safe?" Better: "For whom can it fail, under what shifts, what information might leak, and which failures are unacceptable enough to block deployment?"

Problem-Solving Technique. Common failure mode → recovery move. Failure mode: computing one fairness or robustness number, seeing that it looks acceptable, and stopping the review there. Recovery move: specify the harm model, evaluate operational slices separately, inspect severe failures directly, and tighten the fallback or redesign the pipeline when any subgroup or shift still faces unacceptable risk.

Problem-Solving Technique. Signal to trust → signal to doubt. Trust: the harm model is explicit, operational slices are reported separately, severe failures are inspected directly, and fallback or escalation paths are tightened where risk remains. Doubt: the system looks acceptable only in aggregate, subgroup or shift behavior is unreported, and privacy or safety is mentioned as a principle rather than reviewed as

an operational failure channel.

Keep one running case in view throughout this chapter: the grounded course-support assistant from the earlier language and experiments chapters. It routes student questions, retrieves policy passages, sometimes generates grounded answers, and sometimes escalates to humans. This case is useful because it looks ordinary enough to tempt careless deployment, yet it concentrates all four reliability themes in one pipeline: subgroup gaps in language coverage, robustness failures when wording or policy context shifts, privacy exposure through sensitive student records, and safety failures when the system answers questions that should have been escalated. It also gives the chapter its spine: slice gaps first, then shifts, then exposure, then bounded-failure design.

To keep the reliability story visibly cross-domain, this chapter also returns to the dashboard-camera pedestrian detector from Chapter 8. That contrast case moves the same ideas into sensor shift, bystander privacy, and bounded physical risk. The point is not that the domains are identical. It is that the same reliability questions survive domain change.

12.2 Bias as a Full-Pipeline Failure

Bias enters through many stages:

- **data collection**: some groups or conditions are underrepresented
- **labeling**: annotation quality differs across subpopulations
- **measurement**: sensors or interfaces capture some groups less accurately
- **representation**: useful distinctions for one group are blurred for another
- **objective design**: optimizing one metric may hide unequal error patterns
- **deployment**: a model is used in a setting unlike the one it was built for

Reliability Evaluation Matrix: Full-Pipeline Assessment

	Data & Labels	Model & Training	Inference & Decision	Deployment & Monitor
BIAS	**Representation** Coverage gaps Label quality Annotation bias	**Learning** Objective imbalance Feature selection Group interactions	**Allocation** Unequal outcomes Service quality gaps Threshold effects	**Operational** Feedback loops Drift monitoring Harm detection
ROBUST.	**Distribution** Domain coverage Shortcut risks Spurious patterns	**Generalisation** Regularisation Augmentation policy Hold-out domains	**Shift Handling** New conditions Edge cases Confidence scores	**Adaptation** Performance drift Graceful degradation Human handoff
PRIVACY	**Collection** Sensitive attributes Consent boundaries Retention policies	**Training** Memorisation risk Inference attacks Differential privacy	**Output** Information leakage Re-identification risk Output sanitisation	**Lifecycle** Log management Access controls Deletion schedules
SAFETY	**Assumptions** Task boundaries World-model limits Scope validation	**Constraints** Bounded outputs Adversarial robustness Red teaming	**Failure Modes** Escalation triggers Abstention policies Worst-case analysis	**Operations** Incident response Human oversight Kill switches

Rows name reliability dimensions; columns name pipeline stages.
Each cell lists checks to perform, not a deployment-status label.

Each cell requires specific checks — no single metric captures full reliability

Figure 12.1: The reliability evaluation matrix shows that each reliability dimension (bias, robustness, privacy, safety) requires specific checks across every pipeline stage. No single metric captures full reliability; systematic assessment across all cells prevents deployment of systems with hidden failure modes.

This is why bias should not be treated as a moral adjective attached only to the model. It is a structured failure mode of the whole system.

12.2.1 Harm Taxonomy Before Metrics

Students often encounter reliability as a list of metrics and then try to choose one as if the metric itself decided the ethical and engineering question. That is backwards. The first step is to name the kind of harm that matters, because different harms require different evidence, different mitigations, and sometimes different system designs altogether.

A useful working taxonomy is:

- **allocation harm**: the system grants or withholds opportunities unevenly, such as who receives a scholarship flag, a manual review, or a recommended intervention;
- **quality-of-service harm**: the system serves some users less accurately, less clearly, or less reliably than others, even if the final binary decision looks similar;
- **representation harm**: the system erases, stereotypes, or misdescribes people or groups in the language, labels, or categories it uses;
- **privacy harm**: sensitive information is exposed, inferable, retained too long, or reused outside the justified purpose;
- **safety or operational harm**: the system creates failures whose consequences are materially serious, such as bad medical triage, unsafe control behavior, or unbounded advice in a high-risk support context.

These categories overlap, but they are still useful because they keep the reliability review from collapsing into one vague fairness score. A course-support assistant might show quality-of-service harm for multilingual students, privacy harm if sensitive records leak into prompts or logs, and safety harm if high-risk messages are answered directly instead of escalated. One dashboard number cannot summarize all three responsibly.

> **Practical Note.** Ask "what kind of harm are we trying to rule out?" before asking "which metric should we compute?" Reliability metrics become more meaningful once the harm model is explicit.

12.2.2 Overall Performance Can Hide Unequal Harm

A system can achieve strong average metrics while failing badly for a subgroup. If the subgroup is small relative to the full dataset, the overall score may barely move even when the subgroup problem is severe. This is not a side issue. In many de-

ployments, the subgroup may correspond to the most vulnerable, least represented, or most important users. Cases such as Gender Shades made this risk visible by showing large intersectional error gaps hidden inside commercial systems that might otherwise have looked acceptable in aggregate [10].

Definition 12.1 (Subgroup Evaluation)**.** Subgroup evaluation measures model performance separately for distinct groups, conditions, or slices rather than only in aggregate.

12.2.3 A Subgroup Reliability Dashboard

One fairness metric in isolation is rarely enough. A subgroup can look acceptable on accuracy while still receiving overconfident automation, too many unsafe direct answers, or too little human review. A better habit is to maintain a small subgroup dashboard that keeps several reliability questions visible at once.

For each operationally relevant subgroup, record at least:

- sample size, so small groups are not overinterpreted or ignored;
- task metric, such as accuracy, recall, or error rate for the decision that matters;
- calibration or confidence quality, if confidence affects escalation or abstention;
- abstention or escalation rate, if the system is allowed to defer;
- severe-failure count, if some mistakes matter much more than others.

> **Practical Note.** Small slices are noisy. If a subgroup has only a few dozen examples, a one- or two-case change can move the metric materially. Treat large gaps as real signals, but treat small gaps on tiny slices as hypotheses until they survive more data or repeated evaluation.

This format matters because subgroup disparities do not always appear in the same column. One group may have similar accuracy but much worse calibration. Another may receive far more

abstentions because the system is less certain, which may be a safe design choice or a service-quality problem depending on the application. Yet another may look fine on average while concentrating the worst errors in exactly the cases that should have triggered escalation.

> **Decision Box.** Do not ask only whether subgroup metrics are equal. Ask whether each subgroup receives an acceptable combination of accuracy, confidence quality, escalation behavior, and severe-failure exposure.

For the running support-assistant case, the dashboard might track dominant-register versus multilingual messages, routine policy questions versus high-risk escalation cases, and short messages versus long narrative messages. For the pedestrian detector, the analogous dashboard might track day versus night, dry versus wet conditions, and child versus adult pedestrian slices. The principle is the same: reliability review should compare operational slices with enough structure to reveal different failure channels, not just one averaged score per group. The next section keeps the same slice discipline but asks what happens once the environment shifts.

12.2.4 Worked Example: Accuracy Is Not the Whole Story

Suppose the course-support assistant is evaluated on two groups of messages. Group A contains questions written in the dominant formal register well represented in training data. Group B contains questions written in a less represented informal or multilingual register.

For Group A:

$$\mathrm{TP} = 45, \quad \mathrm{FN} = 5, \quad \mathrm{FP} = 10, \quad \mathrm{TN} = 40.$$

For Group B:

$$\mathrm{TP} = 8, \quad \mathrm{FN} = 12, \quad \mathrm{FP} = 2, \quad \mathrm{TN} = 18.$$

Group A accuracy is

$$\frac{45+40}{45+5+10+40} = \frac{85}{100} = 0.85.$$

Group B accuracy is

$$\frac{8+18}{8+12+2+18} = \frac{26}{40} = 0.65.$$

Now consider the true positive rate:

$$\text{TPR} = \frac{\text{TP}}{\text{TP}+\text{FN}}.$$

Group A has

$$\text{TPR}_A = \frac{45}{45+5} = 0.90,$$

while Group B has

$$\text{TPR}_B = \frac{8}{8+12} = 0.40.$$

This is a much sharper picture than overall accuracy alone. If the task is routing students to the right policy or escalation path, the gap in true positive rate may matter more than the average score. A system that misses the right route for underrepresented language varieties is not merely slightly worse. It is less reliable for a specific slice of real users.

> **Prerequisite Bridge.** Accuracy counts all correct predictions equally. True positive rate isolates how often truly positive cases are detected. False positive rate isolates how often negative cases are incorrectly flagged. Different applications care about these errors differently.

12.2.5 Fairness Metrics Are Tools, Not Magic

Fairness metrics such as subgroup accuracy, true positive rate, false positive rate, demographic parity, or equalized odds are useful because they force modelers to examine uneven behavior explicitly [26]. But no single metric captures all relevant harms.

Different applications justify different tradeoffs, and some fairness objectives conflict with one another when base rates differ.

> **Advanced Note.** Equalized odds asks whether groups have similar true positive and false positive rates. Demographic parity asks whether groups receive positive predictions at similar rates. These may conflict, which is one reason reliability work requires judgment rather than metric worship.
>
> One compact way to see the tension is to write, for group $A = a$ with base rate $\pi_a = P(Y = 1 \mid A = a)$,
>
> $$P(\hat{Y} = 1 \mid A = a) = \mathrm{TPR}_a\, \pi_a + \mathrm{FPR}_a\, (1 - \pi_a).$$
>
> If equalized odds holds across groups, then TPR_a and FPR_a are the same for each group, so different base rates π_a still produce different positive-prediction rates. In that case, demographic parity usually cannot hold at the same time unless the groups have equal base rates or the classifier is trivial.

Example 12.1 (Real-World Callout: Aggregate Performance Hid Unequal Failure)**.** Buolamwini and Gebru showed that commercial gender-classification systems could look broadly capable while producing substantially higher error rates for darker-skinned women than for lighter-skinned men [10]. That result is exactly why subgroup evaluation matters: an aggregate success story can coexist with a severe failure for a specific population. Fairness metrics such as equal opportunity or equalized odds do not remove the need for judgment, but they make this kind of uneven behavior visible enough to challenge a misleading average-case narrative [26].

12.2.6 Calibration Gaps Can Also Be Reliability Gaps

Reliability is not only about whether predictions are correct. It is also about whether confidence is interpretable. If a system produces confidence scores that are systematically too high for

one subgroup, then the system can look equally accurate in aggregate while still being more dangerous for that subgroup in practice. Modern neural systems are often poorly calibrated by default, which means their stated confidence can be higher than their true correctness rate [25].

This matters immediately in the running support-assistant case. Suppose the model attaches a confidence score to whether a message can be answered automatically or should be escalated. If messages written in a dominant formal register receive well-calibrated confidence but multilingual or distressed messages receive overconfident scores, then the same threshold can produce uneven harm even before the final answer is generated.

Example 12.2 (Same Threshold, Different Reliability)**.** Imagine two groups of support messages. In Group A, messages scored at confidence 0.9 are actually correct about 90% of the time. In Group B, messages scored at confidence 0.9 are correct only about 65% of the time. A single confidence threshold may then look principled while silently exposing Group B users to much riskier automation. The engineering lesson is simple: reliability review should inspect calibration and threshold behavior by slice, not only global score distributions.

> **Practical Note.** If a model's confidence is used to decide escalation, triage, or human review, calibration becomes part of the system contract rather than a cosmetic diagnostic.

12.3 Robustness Under Change

Definition 12.2 (Robustness)**.** Robustness is the ability of a model or system to continue behaving reasonably when inputs, environments, or usage conditions differ from the development setting.

Robustness is not the same as average test accuracy. A model may perform well on one benchmark distribution and fail under small, predictable changes. This can happen because the model relied on shortcuts, because the representation is brittle,

or because the deployment domain has shifted in ways the model never learned to handle. Benchmarks based on common corruptions made this problem concrete by evaluating the same classifier under systematic input degradation rather than only on clean test images [30]. After asking "for whom is the system uneven?" the reliability review now asks "what happens when the world changes in expected ways?"

12.3.1 Common Forms of Shift

Several reliability-relevant shifts appear repeatedly:

- benign covariate shift such as lighting, microphone, or interface changes
- subgroup-specific measurement degradation
- domain shift across institutions, devices, or demographics
- wording or policy shift after a new semester, new interface, or updated documentation
- adversarial perturbation intended to trigger failure
- concept drift, where the task itself changes over time

12.3.2 Spurious Correlations

Many robustness failures are caused by spurious correlations. A model learns a pattern that predicts well in training but is not the intended signal. Snow in wolf photos, scanner artifacts in medical data, room acoustics in keyword detection, or formatting quirks in text can all become accidental shortcuts. In a support assistant, a model may over-rely on course codes, platform names, or the phrasing habits of one institution rather than on the underlying request. The system may appear successful until the spurious cue disappears.

Figure 12.2 gives a concrete hospital-shift example from chest-radiograph modeling. In the original study, MSH refers to Mount Sinai Hospital and NIH refers to the NIH ChestXray dataset. As the relative pneumonia prevalence changes across those sources,

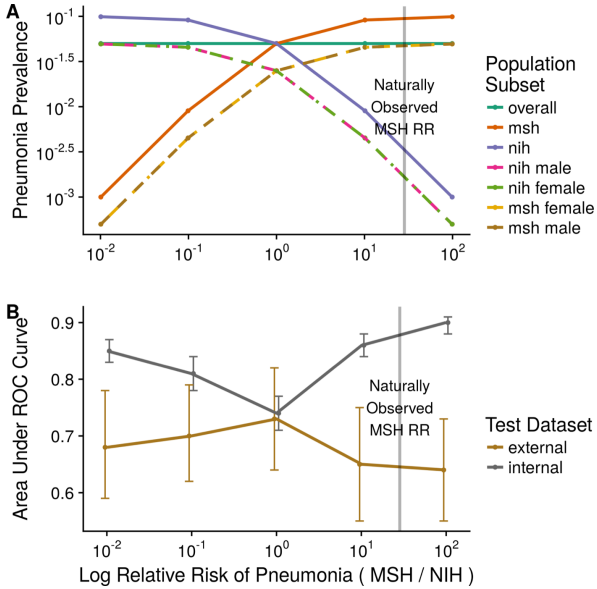


Figure 12.2: Institutional shift can reverse the story told by internal validation. In this open-access figure from Zech et al. [63], internal AUC and external AUC diverge as the relative pneumonia prevalence between hospital sources changes. Reproduced from a PLOS Medicine article distributed under CC BY 4.0.

internal and external performance move in different directions. That is exactly what shortcut learning under domain shift looks like: the model appears to improve under one validation story while becoming less reliable under the deployment-relevant one.

> **Failure Box.** Reliability failures often look like competence until the environment changes. This is why a clean benchmark can be misleading: it may reward shortcut use instead of robust understanding of the intended task.

12.3.3 Worked Example: A Reliability Tradeoff

Suppose a course-support assistant is evaluated under two conditions:

- familiar wording from the current semester's policies and stu-

dent messages

- shifted wording from a new semester, with revised policy language and more varied student phrasing

Across repeated runs, a baseline support model achieves mean familiar-wording accuracy about 0.914 but mean shifted-wording accuracy only about 0.627. After adding paraphrase augmentation and more diverse retrieval-side query rewriting during training, the model's mean familiar-wording accuracy drops to about 0.892 but shifted-wording accuracy rises to about 0.812.

This is a reliability result, not merely an accuracy result. The right claim is not that augmentation is universally better. It is that the new training procedure improves robustness to the specific wording shift of interest while trading away some performance on the familiar slice. Reliability often lives in these tradeoffs. The companion demo is useful for exactly this reason: mitigation may reduce one subgroup weakness or one shift failure without making the whole problem disappear.

12.3.4 Stress Tests Should Be Designed, Not Improvised

Robustness evaluation becomes much stronger once teams name the changes they actually expect in deployment and test them deliberately. For image models, this may mean blur, noise, corruption, compression artifacts, weather, or camera shift. For language systems, it may mean paraphrase, spelling variation, multilingual phrasing, outdated policy pages, or retrieval-index changes. Benchmarks based on common corruptions became influential precisely because they turned vague robustness talk into a repeatable stress protocol [30].

The chapter's broader lesson is that robustness tests should be justified by a deployment story. A benchmark corruption that never appears in practice may still be pedagogically useful, but the strongest reliability evidence comes from changes that the system is actually likely to face.

System type	Stress scenario	What the test is trying to reveal
Course-support assistant	paraphrased or multilingual student phrasing	whether routing and retrieval depend too heavily on familiar lexical forms
Course-support assistant	updated policy page structure or terminology	whether retrieval and answer generation stay grounded after document drift
Pedestrian detector	night rain, glare, or new camera placement	whether visual competence relies on familiar conditions rather than robust cues
Forecasting system	delayed sensor updates or seasonal regime change	whether the model has learned stable signal or only local historical regularities

Table 12.1: Stress tests are more informative when they are named in advance and tied to realistic deployment changes rather than chosen after failure is observed.

12.4 Privacy as a Modeling Constraint

Privacy is not merely a legal layer added after technical work. It shapes what data may be collected, stored, linked, and learned from in the first place. After asking how the system fails for different users and under different conditions, the chapter now asks what the system might expose along the way.

In the running support-assistant case, this point becomes immediate. Student messages may mention illness, disability accommodations, visa status, financial hardship, grade disputes, or identifying numbers. A team that casually logs and reuses such messages for model improvement may create privacy risk long before deployment decisions become visible.

The vision contrast case makes the same point differently. Dashboard-camera footage can capture faces, license plates, home addresses, and repeated movement patterns of bystanders who never consented to becoming training data. A perception

task that seems less personal than text can still create serious privacy exposure if collection, retention, or reuse rules are weak.

12.4.1 Privacy Threat Models Before Privacy Tools

Privacy review improves once it stops asking only "which protection should we use?" and starts asking "who could learn what, through which access path, and with what consequence?" That is a threat-model question. Without it, privacy discussion can become a vague checklist of fashionable defenses rather than an engineering analysis of actual exposure.

A compact privacy threat model names:

- **the sensitive asset**: what information must be protected, such as health disclosures, identifiers, location traces, or accommodation status;
- **the attacker or observer**: who might gain access, such as internal reviewers, external users, data brokers, compromised services, or other institutions;
- **the access path**: logs, prompts, retained training data, retrieval indexes, debugging tools, model outputs, or linkage across datasets;
- **the consequence**: embarrassment, discrimination, financial loss, legal exposure, or physical risk.

For the support-assistant case, one threat model might say: sensitive student records and messages are the asset; internal debugging access and long-lived prompt logs are the path; unauthorized staff or compromised tooling are the observer; and the consequence is exposure of disability, immigration, or health information. For the pedestrian-detector case, the asset may be identifiable faces or movement patterns; the path may be retained video and annotation tooling; and the consequence may be long-term surveillance or unconsented secondary use.

> **Decision Box.** Before choosing a privacy method, write one sentence of the form: "We are trying to stop *this observer* from learning *this sensitive fact* through

> *this data path.*" If that sentence is vague, the privacy design is probably vague too.

12.4.2 Why Models Can Leak Information

Training data may contain names, messages, health records, locations, or other sensitive content. A model can leak such information by memorizing rare examples, exposing hidden attributes through inference, or enabling linkage across datasets. Even if the model is not explicitly designed to reveal private information, its outputs or internals may still create privacy risk.

12.4.3 Common Privacy Responses

Several technical responses are common:

- **data minimization**: collect and retain only what is necessary
- **access control and secure storage**: reduce unnecessary exposure
- **differential privacy**: constrain what the model can reveal about any individual example
- **federated or distributed learning**: keep raw data decentralized in some workflows
- **redaction and filtering**: remove especially sensitive fields before modeling

12.4.4 Privacy Has Tradeoffs

These techniques are not free. Stronger privacy constraints can reduce utility, increase computation, or complicate debugging. But that tradeoff is part of the real engineering problem. A system that performs well only by exposing people to unacceptable privacy risk is not a reliable system in any serious sense.

> **Advanced Note.** Differential privacy gives a formal guarantee that the output distribution of an algorithm changes only slightly when one training ex-

ample is added or removed [18]. The precise definition is more advanced than this chapter requires, but the intuition is important: privacy can be formalized as a bound on how much an individual's data can influence the released result.

Formally, a randomized mechanism M is ϵ-DP if for all neighboring datasets D, D' and all events S,

$$P(M(D) \in S) \leq e^{\epsilon} P(M(D') \in S).$$

A useful undergraduate intuition is sensitivity: if a query q changes by at most Δq when one record changes, then adding noise on the order of $\Delta q/\epsilon$ makes any single person's contribution hard to detect while preserving the coarse signal.

12.4.5 Privacy Risk Lives in the Whole Data Path

Privacy review should follow the data path end to end:

- what raw fields are collected,
- what fields are joined or linked later,
- what gets logged during inference,
- who can inspect those logs,
- what examples are retained for future training,
- what generated outputs might reveal or reconstruct.

This framing helps students avoid a common mistake. They imagine privacy as a property of the final model weights alone. In real systems, the privacy risk may be dominated by preprocessing dumps, support logs, annotation tools, cached prompts, or debugging traces rather than by the final deployed checkpoint.

Example 12.3 (A Privacy Failure Without an Exotic Attack)**.** A support assistant may never explicitly emit a student's name in the final answer yet still create serious privacy exposure if raw messages containing health details and accommodation requests are retained in accessible logs for prompt debugging. That is a

systems failure, not only a modeling failure. The safest model architecture in the world will not repair careless retention and access rules.

12.5 Safety and Bounded Failure

Safety becomes especially important when AI systems influence high-stakes decisions, interact with the physical world, or generate open-ended outputs that can trigger harmful actions. This is the chapter's most explicit bounded-failure step: which failures are unacceptable even if the system often looks competent?

Definition 12.3 (Safety)**.** Safety concerns whether a system's failures are anticipated, bounded, monitored, and handled in ways that keep harms within acceptable limits.

This is broader than model accuracy. A medical triage assistant, an autonomous-driving perception module, and a general-purpose language assistant each face different safety problems. But they share a systems lesson: safety depends on fallback behavior, monitoring, escalation paths, human review, and clear operational limits.

The course-support assistant is a useful reminder that safety is not limited to robots or hospitals. A confident answer about visa attendance, disability accommodation, self-harm disclosures, or disciplinary policy can still create severe harm if the system answers when it should have escalated.

Example 12.4 (Contrast Case: Safety in a Perception System)**.** Suppose a shuttle's pedestrian detector looks strong on familiar daylight footage but misses close-range pedestrians in headlight glare or night rain. The safety problem is not captured by average detection quality alone. It also depends on whether the vehicle slows down, hands control back to a human, or triggers a conservative fallback whenever visibility or model confidence becomes poor.

12.5.1 Hazards, Not Just Errors

An error becomes a safety issue when the consequences matter. Misclassifying one image in a photo app is not the same as missing a critical pathology finding. Safety analysis therefore begins by identifying hazards:

- what types of error are possible?
- which of them are severe?
- how likely are they?
- what mitigations reduce their impact?

12.5.2 Safety Cases Make Bounded Failure Concrete

Once hazards are named, the next step is to write a small safety case. The point is not to imitate a certification bureaucracy. It is to force the team to connect hazard, trigger, mitigation, fallback, and monitoring in one coherent argument.

A lightweight safety case can be written with five fields:

- **hazard**: what harmful failure must be bounded;
- **trigger condition**: under what inputs, environments, or uncertainty patterns the hazard is likely to appear;
- **mitigation**: what model-side or pipeline-side design reduces the chance of that failure;
- **fallback**: what the system does instead when the hazard cannot be ruled out confidently;
- **monitoring signal**: what evidence after launch would indicate that the safety case is weakening.

For the course-support assistant, one safety case might center on high-risk student messages being answered directly when they should be escalated. The trigger condition could be a message that mixes routine academic language with distress, immigration,

disability, or disciplinary cues. The mitigation could include high-risk intent detection, retrieval grounding, and conservative escalation thresholds. The fallback is immediate routing to trained human support. The monitoring signal is any rise in escalation misses or unsafe direct answers on audited high-risk slices.

For the pedestrian detector, the same pattern appears in another domain: the hazard is a missed close-range pedestrian; the trigger condition is glare, rain, or unusual night geometry; the mitigation is stress-tested sensing and conservative policy thresholds; the fallback is slowdown or human takeover; the monitoring signal is rising miss rate under those visibility conditions. The lesson is the same in both domains. Safety is strongest when failure handling is specified before deployment rather than improvised after the first incident.

> **Evidence Box.** "The model is accurate" is not a safety case. A safety case must say what the dangerous failure is, when it is likely, what reduces it, how the system falls back, and what signal would warn the team that the protection is no longer holding.

12.5.3 Human Oversight and Fallbacks

Some systems should assist rather than automate. A model may provide ranked options, highlight uncertainty, or request human confirmation when confidence is low or context is unusual. In other settings, rule-based guardrails or conservative thresholds may be more important than squeezing out another percentage point of average performance.

Example 12.5 (A Safety Failure in an Ordinary Support System)**.** Suppose a student writes a message that mixes deadline language with a severe personal-risk signal or an immigration-status concern. A superficially competent assistant might answer only the visible deadline question because that is what the lexical content most strongly resembles. A safer system would route the message to a human support path, attach the relevant policy information, and decline to answer beyond its authority.

12.5.4 Selective Automation Requires Usable Uncertainty

Reliability often improves not by forcing the model to answer everything, but by giving it permission to abstain on the cases it does not understand well enough. Selective classification makes this logic explicit: the system chooses when to predict and when to defer, trading coverage against risk [22]. Closely related ideas such as conformal prediction aim to provide uncertainty sets or calibrated coverage guarantees rather than one forced point prediction [58].

If a selector is $c(x) \in \{0, 1\}$, where $c(x) = 1$ means "answer," then coverage is

$$\phi = \mathbb{E}[c(X)] \approx \frac{1}{n} \sum_{i=1}^{n} c_i,$$

and selective risk is

$$R_{\text{sel}} = \frac{\sum_{i=1}^{n} c_i \, \ell_i}{\sum_{i=1}^{n} c_i},$$

the average loss only on answered cases, provided at least one case is answered so that $\sum_i c_i > 0$. A simple threshold rule makes the tradeoff concrete: answer only when $\max_k p(y = k \mid x) \geq \tau$. Higher τ usually lowers coverage and can lower selective risk when the confidence score orders easier cases above harder ones and is reasonably calibrated.

This does not mean every application needs advanced uncertainty machinery. The operational lesson is more basic. If the system will be trusted differently at confidence 0.55 and 0.95, then confidence quality matters. Calibration, abstention rules, and human review thresholds become central parts of the reliability design rather than optional research extras.

> **Decision Box.** When the cost of a confident mistake is high, do not ask only how to improve the prediction. Ask how to make low-confidence or unfamiliar cases fall out of automation gracefully.

Example 12.6 (Coverage Versus Safety)**.** Suppose the support assistant can answer 82% of incoming questions automatically

with very low factual error when it abstains on low-confidence or high-risk cases, but reaches 96% automation only by answering many uncertain messages directly. The second system may look more efficient, yet the first may be far more reliable because its failures are bounded by design. The same principle appears in perception systems that slow down, request takeover, or switch to a conservative fallback when visibility is poor.

> **Evidence Box.** Evidence for safety should include more than nominal test performance. It should include severe-failure analysis, operating thresholds, fallback behavior, monitoring plans, and documentation of what the system is *not* intended to do.

12.5.5 Reliability Is Sometimes a Reason to Decline Deployment

This is an important professional lesson. Some systems should not be deployed yet. If subgroup failures are severe, privacy risks are unacceptable, or fallback behavior is missing in a high-stakes setting, the correct action may be to redesign the system or refuse deployment. Reliability is not only about mitigation. It is also about knowing when the evidence is not good enough.

12.6 Monitoring Reliability After Launch

Reliability does not end when the model leaves the notebook. Post-deployment monitoring is where many apparently careful systems begin to fail. User populations change, document collections drift, sensors degrade, labels arrive late, and human reviewers work under different load than anticipated. A system that was reliable at launch can become unreliable quietly unless the team watches the right signals. Monitoring is where the earlier harm model, shift scenarios, privacy paths, and safety cases become operational rather than hypothetical.

For the running support-assistant case, useful monitoring signals might include subgroup routing accuracy, retrieval success on current policy passages, escalation miss rate on high-risk messages,

citation coverage for grounded answers, and sudden changes in the distribution of incoming questions. For the pedestrian detector, useful signals might include weather-conditioned miss rates, takeover frequency, latency spikes, or changes in camera installation geometry.

> **Problem-Solving Technique. Monitoring question.** For every major reliability risk named before launch, specify one observable post-launch signal that would warn the team early if that risk were getting worse.

Monitoring also connects directly to rollback and narrowing decisions. If a support assistant begins failing after a policy-site redesign, the right response may be to disable direct answers temporarily and revert to retrieval-plus-escalation. If a perception module shows degraded performance under a new camera firmware version, the right response may be to reduce automation authority until the shift is understood. A reliable system is therefore not just one that works initially. It is one whose failure can be noticed and bounded before harm compounds.

12.7 Why Ethics and Engineering Cannot Be Separated

One common mistake in AI education is separating ethics from engineering so completely that students discuss harms in one room and technical design in another. That structure quietly teaches the wrong lesson. It suggests that fairness, privacy, and safety are optional reflections added after the real work.

A better educational pattern is to show that:

- subgroup metrics are technical measurements,
- robustness tests are experimental design,
- privacy protection constrains data and training,
- safety analysis shapes thresholds and system architecture,

- documentation and human oversight are engineering choices.

Ethical reflection remains broader than these tools, but reliability work is one place where ethical obligations become operational technical practice.

12.8 Chapter Artifact: A Reliability Review Card

By the end of this chapter, the reader should be able to produce a one-page reliability review card before saying a system is ready for serious use. Earlier artifacts in the book named the decision, the metric, the evaluation boundary, the baseline, the structure diagnosis, the experiment claim, and the deployment-facing risks of specific modalities. This review card adds the bounded-failure layer: who is at risk, what shift matters, what may leak, what hazard is unacceptable, and what condition blocks launch.

> **Problem-Solving Technique. Reliability review card.** Before calling a system reliable, write down the critical slices, shift scenarios, privacy exposures, severe hazards, fallback path, monitoring triggers, and the condition under which deployment should be delayed or refused.

Example 12.7 (Filled Reliability Review Card)**.** For the course-support assistant, a disciplined review card would name critical slices such as multilingual phrasing, underrepresented language registers, visa-related questions, disability-accommodation requests, and distressed student messages; shift scenarios such as a new semester, revised syllabus wording, new learning-platform terminology, or policy-page reorganization; spurious cues such as course codes or standard-form wording that may not survive outside the training setting; privacy exposures such as student identifiers, health disclosures, and accommodation records; severe hazards such as answering high-risk questions directly when escalation is required; fallback path as human advising or specialist support teams; monitoring triggers as subgroup degradation, retrieval failures on policy pages, or rising escalation misses; and

a no-deploy condition such as unacceptable failure on high-risk slices even when overall metrics remain strong.

12.9 Common Mistakes in Reliability Work

These mistakes are surface forms of the chapter's main enemy: confusing improved scores with justified explanation. Read them as one reliability checklist rather than five separate mini-topics.

Mistake Checklist.

- **Reporting only overall metrics.** Overall numbers can hide subgroup failures and robustness weaknesses.
- **Treating bias as only a dataset issue.** Bias can enter through labels, measurements, thresholds, deployment conditions, and user interaction.
- **Confusing robustness with clean accuracy.** A model may be accurate on a benchmark and brittle under predictable shift.
- **Treating privacy as an afterthought.** Late privacy fixes are often expensive or insufficient.
- **Thinking safety is just the model's job.** Fallback logic, escalation, monitoring, and interface design often matter more than one extra point of accuracy.

12.10 Looking Ahead

This chapter treated reliability as an engineering discipline across the full pipeline. The final chapter asks the launch question that follows from that discipline: once contracts, monitoring, fallback behavior, queue limits, and ownership are included, is the whole system actually ready to operate as a real service?

Chapter Summary

- Reliability is a property of the full pipeline, not just the trained model.
- Subgroup evaluation is necessary because overall metrics can conceal unequal failures.
- Fairness metrics such as accuracy, true positive rate, and false positive rate provide useful but incomplete views of reliability.
- Robustness concerns how performance changes under realistic shift, noise, shortcut failure, or concept drift.
- Privacy constrains what data can be used and what the model is allowed to reveal.
- Safety depends on bounded failure, fallback behavior, monitoring, and human oversight.
- Reliability is where ethical obligations and technical design become inseparable in practice.
- The main deliverable of the chapter is a reliability review card that makes bounded-failure evidence explicit before deployment claims are made.

Study Questions

1. Why should bias be treated as a property of the full pipeline rather than only the model?
2. Why can subgroup true positive rate matter more than overall accuracy in some applications?
3. How is robustness different from average test performance?
4. Why does privacy count as a modeling constraint rather than only a policy issue?
5. What makes an error a safety issue rather than only a performance issue?

6. Why can refusing deployment be the correct reliability decision?
7. Why might a support assistant with good average performance still be unreliable on the questions that matter most?

Exercises

1. Compute the true positive rate and false positive rate for a small confusion matrix of your own design.
2. Give one example of a subgroup failure that could be hidden by a strong overall metric.
3. Describe one realistic spurious correlation for a vision, language, or audio task.
4. Explain one case in which a privacy-preserving design choice might reduce raw utility but still improve the overall system.
5. Give one example of a model that should assist a human rather than automate the decision completely.
6. Describe a deployment situation in which robustness to distribution shift matters more than clean benchmark accuracy.
7. For a support assistant or a dashboard-camera pedestrian detector, list one subgroup slice, one shift scenario, one privacy exposure, and one severe hazard that should appear on the reliability review card.
8. Use Table 12.2 to sketch a one-page reliability review card for a system of your choice. If you are carrying one case through the book, write it for the same system as your experiment claim sheet and state which empirical claim survives this reliability review unchanged.

Challenge Exercises

1. A classifier has similar overall accuracy across two groups but very different false positive rates. Explain why this can still

be a serious reliability concern, and give a domain in which it would matter.

2. A team proposes deploying a model with excellent benchmark results but no subgroup analysis, no privacy review, and no fallback behavior. Write the strongest technical objection you can in one paragraph.

3. Compare two mitigation strategies for a minority-group performance gap: collecting better data versus threshold adjustment. Explain when each might help, what it cannot fix, and what evidence would be needed to judge it.

Review field	What must be written clearly
Seven-question focus	Questions 3, 6, and 7 most directly: what data and deployment conditions create risk, what evidence bounds failure, and what review or fallback keeps the system acceptable
Critical users or slices	Which groups, contexts, or operating conditions must be checked separately
Shift scenarios	Which realistic changes in device, environment, source, or user population could break the system
Spurious-cue or measurement risks	What accidental shortcuts or measurement failures may be carrying the score
Privacy threat model	What sensitive fact is being protected, from whom, through which data path
Severe hazards	Which failures are unacceptable even if overall metrics stay strong
Safety case	For each severe hazard: trigger condition, mitigation, fallback, and monitoring signal
Fallback and review path	What abstention, escalation, or human-review route exists when the system is uncertain or high risk
Monitoring triggers	What post-deployment signals would indicate reliability degradation
No-deploy condition	What finding would be strong enough to delay, narrow, or reject deployment

Table 12.2: A reliability review card turns fairness, robustness, privacy, and safety into one bounded-failure protocol rather than four disconnected checklists.

Chapter 13

From Models to Systems

Learning Goals

- Understand why a deployed AI system must be designed as a pipeline with requirements, interfaces, monitoring, and fallback behavior rather than as a model checkpoint alone.
- Distinguish offline model quality from system quality, including latency, capacity, coverage, calibration, and review workflow constraints.
- Explain training-serving skew and why data contracts and feature consistency matter as much as model architecture.
- Analyze selective prediction, human review, and threshold choice as system design decisions rather than only model-evaluation details.
- Learn when an AI system should be simplified, constrained, or rejected because the full operational setting does not support reliable deployment.

The previous chapter argued that reliability lives in the full pipeline, not only in the model. This final chapter turns that reliability review into a deployment decision. A deployed AI system is a combination of data collection, preprocessing, model inference, thresholds, interfaces, logging, monitoring, fallback be-

havior, human review, and organizational ownership. A model can be mathematically elegant and still fail as a system because one of those surrounding pieces is missing or poorly designed. This systems view matches long-running warnings about hidden technical debt and the software-engineering realities of ML production [2, 57]. The chapter route is deliberate: define the service objective, state the operational requirements, lock the input contract, turn scores into actions, specify monitoring and staged release, and then decide whether launch is justified. The chapter culminates in a system readiness brief, so the service-design sections should be read as steps toward that launch decision.

This chapter therefore changes the unit of analysis. Earlier chapters often asked whether a model learned a useful mapping from inputs to outputs. Here the question becomes broader: does the entire decision pipeline improve the real task under the actual constraints of deployment? That is the question students must learn to ask if they want to build trustworthy systems rather than impressive demos.

> **Problem-Solving Lens.** This chapter completes the book's seven-question framework by asking how prediction becomes action in a real organization. The unit of success is no longer the model checkpoint, but the full system that surrounds it.
>
> **Problem-Solving Technique. Pipeline realism.** Ask what would break when a notebook result becomes a service: contracts, preprocessing, thresholds, review queues, monitoring, rollback, ownership, and organizational support.
>
> **Problem-Solving Technique. Common failure mode → recovery move.** Failure mode: trying to launch a strong offline model before the system has an input contract, escalation path, monitoring triggers, and a named owner. Recovery move: write the service blueprint first, including validation, thresholds, fallback behavior, review-capacity limits, alert conditions, and rollback authority, then judge the model

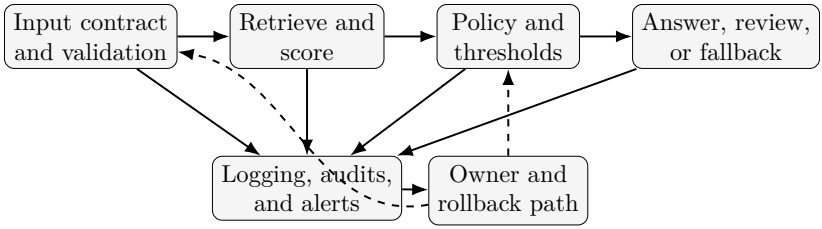


Figure 13.1: A deployment system is not only a forward path from validated inputs to decisions. Monitoring must feed a named owner who can disable, revert, or tighten the operating policy when alerts or audits show the service drifting out of bounds.

only inside that operational design.

Keep the same running case in view from the earlier language, experiments, and reliability chapters: a grounded course-support assistant that routes student messages, retrieves institutional policy, drafts bounded answers for routine cases, and escalates uncertain or high-risk cases to humans. This chapter asks whether that system is ready to launch as an actual service rather than as an impressive demo.

That language workflow remains the primary capstone because it makes contracts, retrieval, escalation, and review paths easy to see. It also gives the chapter its spine: service objective first, then requirements and contracts, then action policy, then monitoring and launch readiness. But the same systems questions should also survive outside one domain, so this chapter will repeatedly contrast it with the dashboard-camera pedestrian detector from Chapter 8. The aim is to make the systems lesson clearly broader than educational NLP.

Example 13.1 (A First End-to-End Service Sketch)**.** For the course-support assistant, the system story can already be written in one pass. Validate the incoming message and context. Retrieve only approved current policy sources. Score whether the case is routine enough to answer automatically. If retrieval is weak, the topic is high risk, or the automation-safety score is too low, escalate to a human instead of answering. Log the decision, the evidence used, and the outcome. Monitor overrides, backlog,

and citation failures, and give one named owner the authority to tighten the policy or roll back the service.

13.1 Systems Thinking Changes the Question

When students first learn machine learning, it is natural to think of the model as the product. In practice, the model is only one component inside a larger service. The service has users, deadlines, costs, review paths, logging requirements, failure cases, and operating assumptions. A classifier with strong test accuracy might still be unusable if it is too slow, if it produces too many low-confidence cases for human review capacity, or if it depends on features that are not available at prediction time.

13.1.1 From Model Objective to Service Objective

A model objective is usually narrow: minimize a training loss, maximize validation accuracy, reduce error on a benchmark. A service objective is broader: improve a workflow while respecting operational limits. The two are related, but they are not identical.

Consider the course-support assistant. The institution does not really want confidence scores or generated answers for their own sake. It wants a process that gives students fast, reliable help on routine questions while preserving human advising capacity for ambiguous, sensitive, or high-risk cases. If the assistant escalates too many messages, advisors cannot keep up. If it escalates too few, the system may answer questions that require human judgment or authority. If its behavior shifts every time a policy page is updated, staff will stop trusting it. Good system design begins when the modeling problem is tied back to that service objective.

13.1.2 Service-Level Objectives Make the Objective Operational

One useful way to make the service objective concrete is to write service-level objectives rather than stopping at general hopes

such as "be accurate" or "be helpful." A service-level objective states what the workflow must achieve under real operating conditions.

- maximum response latency for routine questions,
- minimum escalation recall on high-risk messages,
- maximum average human-review backlog,
- minimum grounded-citation coverage on automated answers,
- maximum severe-error rate before rollback is required.

These are not replacements for model metrics. They are the bridge between model behavior and operational value. A model can raise offline answer accuracy slightly while violating the system's response-time or review-capacity objectives. In that case the model improved, but the service did not.

> **Practical Note.** If the team cannot name the service-level objectives, it is still too early to say whether a threshold, model family, or deployment plan is good. The model objective becomes meaningful only when it is tied to a workflow target the organization actually cares about.
>
> **Decision Box.** Before asking "which model performs best?", ask "what decision or workflow is being improved, what capacity exists to act on the output, and what kinds of mistakes are acceptable?" The model can only be judged inside that broader operational context.
>
> **Practical Note.** A compact deployment workflow is: state the service objective, write the input contract, choose the operating policy and fallback path, test latency and capacity limits, define monitoring and rollback triggers, and only then decide whether launch is justified. That sequence is also the chapter order in miniature. The readiness brief at the end is the written form of that workflow.

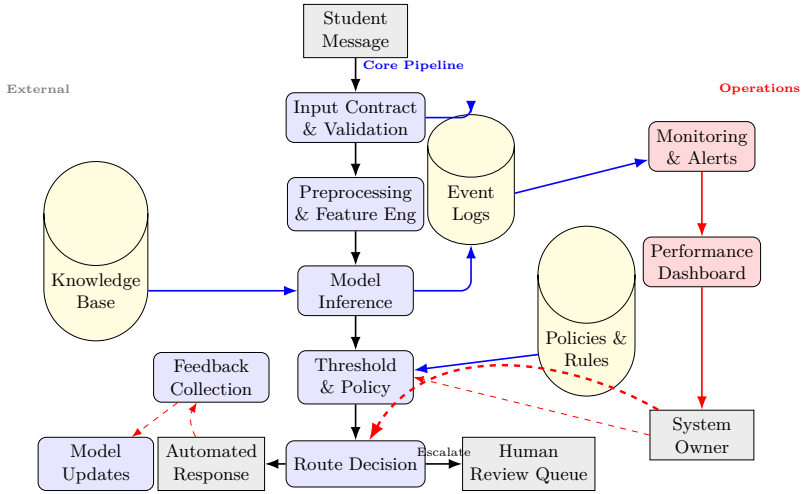


Figure 13.2: Complete system architecture for production AI deployment. The system includes not only the core ML pipeline but also operational components for monitoring, alerting, feedback collection, and rollback capability. A named system owner can tighten thresholds, update validation rules, and disable the service when alerts indicate that it is drifting out of bounds.

13.1.3 Success Includes More Than Accuracy

Several non-accuracy constraints routinely matter:

- **latency**: how quickly must a prediction be returned?
- **throughput**: how many predictions must the system handle?
- **review capacity**: how many uncertain cases can humans inspect?
- **cost**: what computation, storage, or annotation effort is affordable?
- **stability**: do outputs fluctuate so much that users lose trust?
- **auditability**: can later investigators explain what happened?

These constraints are not secondary implementation details. They change which model class, threshold, interface, or deployment pattern is appropriate. The next section starts with the

first deployment question in that workflow: what must the service achieve before any model can be called good enough?

13.2 Requirements Before Training

Well-designed systems begin with requirements. A requirement is a statement about what the system must accomplish or avoid under real operating conditions. Some requirements are quantitative, such as a maximum response time or minimum recall on a high-risk class. Others are structural, such as requiring human confirmation for severe decisions.

Definition 13.1 (Operational Requirement)**.** An operational requirement is a concrete condition that the deployed system must satisfy, such as a latency bound, a review-capacity limit, a minimum recall target, or a mandatory fallback path.

13.2.1 Worked Example: Queue Capacity Changes the Threshold

Suppose the advising team behind the course-support assistant can manually inspect at most 30 escalated messages per day. The system assigns each incoming message an *escalation score*, where larger values mean stronger reason to defer to a human. On a 200-message day, a lower escalation threshold sends 78 messages to humans. A stricter escalation threshold sends 26.

If the team deploys the more aggressive policy because it appears safer in isolation, the queue may overload the staff and slow all responses, including the truly urgent ones. The seemingly more cautious threshold can therefore produce the weaker service. The stricter threshold may let more medium-risk cases stay automated, but it also produces a queue the organization can actually process.

> **Math Lens.** Let q_t be the review backlog after day t, let r be daily human capacity, and let $a(\tau)$ be the number of cases escalated by threshold τ. Then
>
> $$q_{t+1} = \max\{0,\ q_t + a(\tau) - r\}.$$

> A lower escalation threshold can destabilize the queue if it makes $a(\tau) > r$, because the backlog then grows instead of clearing.

This is not a trick example. Many deployments face exactly this issue. The useful prediction is not the one that looks best in isolation. It is the one that leads to the best action process under real constraints.

Example 13.2 (Contrast Case: Capacity Looks Different in Vision)**.** For a pedestrian-alert system, the same requirement may appear as braking latency or the maximum number of false alerts per hour before drivers stop trusting the system. The resource is not an advising queue, but the systems lesson is identical: operating policy must fit the real capacity of the downstream process.

> **Prerequisite Bridge.** Precision and recall describe statistical behavior. Capacity tells us whether the downstream organization can act on the predictions. A system designer must connect the two. A threshold that increases recall may still be unacceptable if it creates more positive predictions than the workflow can absorb.

13.2.2 Requirements Should Be Written Down Early

Teams often delay requirement writing because it feels less exciting than training models. That is backwards. If severe errors, response-time limits, or review paths are not defined early, then later evaluation becomes ambiguous. A model cannot be "good enough" until the system has stated what "enough" means. Once those requirements are explicit, the next systems question is whether the served inputs actually satisfy the assumptions under which the model was evaluated.

13.3 Data Contracts and Training-Serving Consistency

Many system failures occur even when the model itself is reasonable. The problem is that the data seen at serving time does not match what the training pipeline assumed. Features may be computed differently, missing values may be encoded differently, text may be tokenized inconsistently, or a timestamp field may be shifted by one unit. These failures are often called *training-serving skew* [57].

Definition 13.2 (Training-Serving Skew)**.** Training-serving skew is a mismatch between the data representation or feature-processing pipeline used during model development and the one used during deployment.

13.3.1 Why Small Pipeline Mismatches Matter

Modern models are sensitive to representation. A linear classifier expects features on the scale it was trained on. A tree expects the same fields and semantics it saw during fitting. A neural model expects the same tokenization, image normalization, or waveform preprocessing. If the serving pipeline silently changes one of those steps, the deployed system is effectively running a different model from the one that was evaluated offline.

> **Math Lens.** If training standardizes a feature as $z = (x - \mu_{\text{tr}})/\sigma_{\text{tr}}$ but serving uses $\tilde{z} = (x - \mu_{\text{sv}})/\sigma_{\text{sv}}$, then
> $$\tilde{z} = \frac{\sigma_{\text{tr}}}{\sigma_{\text{sv}}} z + \frac{\mu_{\text{tr}} - \mu_{\text{sv}}}{\sigma_{\text{sv}}}.$$
> For a linear score $s = wz + b$, the served score becomes $\tilde{s} = w\tilde{z} + b$, so a changed mean or scale shifts the effective decision boundary even though the weights are unchanged.

13.3.2 Worked Example: A Retrieval Contract Bug

Suppose the course-support assistant was evaluated with a retrieval pipeline that indexed the current semester's official policy

pages, chunked them by section, and attached course and term metadata to each chunk. At serving time, a new ingestion job accidentally mixes last semester's pages with the current ones and drops part of the course metadata. The same student question about an extension now retrieves an outdated late-policy passage because the serving index no longer matches the evaluated one.

The consequences can be serious. A message that should have triggered a current-semester escalation path may now receive a confident but stale answer. Nothing is wrong with the language model weights. The system failed because the evidence contract was broken.

> **Failure Box.** Training-serving skew is dangerous because it often leaves the model file unchanged. Teams may say "we deployed the validated model" even though the effective input representation at serving time is different from the one that was validated.

Example 13.3 (Real-World Callout: Hidden Technical Debt)**.** Sculley et al. argue that many machine-learning failures come not from the model alone, but from undeclared data dependencies, brittle feature pipelines, feedback loops, and glue code that quietly changes the effective system [57]. Amershi et al. describe similar production realities in large software organizations, where data verification, configuration management, and monitoring often dominate the real engineering burden [2]. The practical lesson is that a clean notebook result can still hide a brittle service.

13.3.3 Data Contracts Reduce Ambiguity

A practical response is to define data contracts. These specify what features exist, how they are computed, what units they use, which values are allowed to be missing, how categorical values are encoded, and what validation checks must pass before inference occurs. In mature systems, model deployment is blocked when these contracts fail. Requirements state what the service must achieve; contracts state what must be true before the model is allowed to act.

> **Practical Note.** Many deployment bugs are ordinary software bugs: wrong field order, missing normalization, stale joins, silently changed units, or inconsistent tokenization. Systems engineering matters because these bugs can dominate model quality in production.

13.4 Prediction Is Not the Decision

One of the most important system lessons is that a model output is not the final decision. Most models return a score, probability, rank, or generated candidate. The deployed system must still decide what action to take with that output. After the service objective, requirements, and input contract are fixed, this is the next design boundary: how does a score become an action in the real workflow?

13.4.1 Thresholds Are Policies

Choosing a threshold is not a purely mathematical cleanup step after training. It is a policy that trades off false positives, false negatives, workload, and risk. In a low-stakes recommendation setting, one threshold may be acceptable. In a high-stakes screening setting, a different threshold or a mandatory human review path may be required.

13.4.2 Queueing, Review Capacity, and Human Bottlenecks

Once abstention or escalation is available, the threshold also becomes a queueing decision. Suppose the course-support assistant handles about 120 messages on a busy day and the advising team can review at most 20 escalated cases before the backlog starts harming response time. A lower threshold might escalate 36 messages, while a stricter threshold escalates 18.

If the lower threshold improves severe-error control only slightly but creates a daily backlog of 16 unresolved messages, then after five similar days the system accumulates roughly 80 delayed

cases. At that point the system may be statistically cautious and operationally failing. Students should recognize this as a systems error, not merely as a staffing footnote.

> **Math Lens.** The same queue can be written as $q_{t+1} = \max\{0,\ q_t + a_t - r\}$, where a_t is the day-t deferral count. Stability requires average arrivals below capacity, $\mathbb{E}[a_t] < r$. If a lower escalation threshold raises deferred cases from 18 to 36 while $r = 20$, then the expected drift becomes positive and the backlog grows.

The right threshold is therefore constrained by both mistake costs and queue capacity. A policy that looks safer on isolated examples may produce a weaker service once review load, delay, and reviewer fatigue are counted as part of the outcome.

> **Decision Box.** When human review is part of the system, do not choose the threshold from precision and recall alone. Estimate how many cases the policy will defer per hour or per day, compare that with real review capacity, and ask what backlog the organization can tolerate before service quality degrades.

13.4.3 Ranking Allocates Exposure

In search and recommendation systems, the deployed action is often not "approve or reject" but "which items will users actually see, and in what order?" A ranked list allocates attention. Items shown higher receive more clicks, views, purchases, or follow-up actions partly because they were exposed earlier.

Even the running support-assistant case contains a smaller version of this issue: ranking retrieved passages changes which evidence the generator or the human reviewer sees first, so exposure logic is not only a recommender-system problem.

This makes ranking systems different from ordinary static prediction tasks. Joachims et al. show that clickthrough data is informative but biased by position and presentation [35]. Chapter 1 made the same point from the data side for recommendation

logs: observed interactions are filtered by earlier exposure decisions rather than sampled neutrally from all possible user-item pairs [15]. A deployed ranking therefore changes the future data that the next model version will see.

The practical consequence is that teams should not monitor only click-through or engagement rate. They should also monitor exposure concentration, subgroup or catalog coverage, repeated overexposure of already popular items, and whether the ranking is narrowing what users get to see. In systems language, a recommender or retrieval ranker is not merely predicting preference. It is shaping the information environment.

13.4.4 Selective Prediction and Abstention

Sometimes the right system does not force the model to answer every case. Instead, it allows abstention: confident cases are automated, while ambiguous cases are deferred to humans or alternative procedures [22]. In the running support-assistant case, the real decision is often among answer, refuse, and escalate rather than simply positive versus negative.

The word *confidence* needs discipline here. It should not mean a vague feeling that the model "sounds certain." It should mean an estimate tied to the action policy. For the support assistant, a useful score would be something like the estimated probability that an automatic answer will stay in scope, remain grounded in approved evidence, and avoid an escalation-worthy mistake.

This score direction is deliberately different from the earlier queue examples. There the score measured escalation pressure, so a lower threshold produced more human review. Here the score measures automation safety, so a lower threshold accepts more cases for automatic answering.

> **Math Lens.** Let $a(x) \in \{0, 1\}$ indicate whether the system answers on input x, and let $\ell(f(x), y)$ be the loss on answered cases. Then coverage is $\phi = \mathbb{E}[a(x)]$ and selective risk is
>
> $$R_\phi(f) = \frac{\mathbb{E}[a(x)\,\ell(f(x), y)]}{\mathbb{E}[a(x)]}.$$

> On this automation-safety score, lowering the threshold usually raises coverage, but it can also raise selective risk if the newly accepted cases are too uncertain. This is why thresholding works best when confidence is reasonably calibrated.

Definition 13.3 (Selective Prediction)**.** Selective prediction is a system design in which the model predicts only on cases where it is sufficiently confident and abstains or defers on the rest.

13.4.5 Worked Example: Confidence, Coverage, and Review

Suppose the course-support assistant receives 100 messages:

- 60 are routine questions with strong retrieval agreement and an estimated automation-safety score above 0.90
- 25 have moderate scores between 0.60 and 0.90 because the wording is unusual or the retrieved evidence is incomplete
- 15 contain conflicting signals, policy ambiguity, or high-risk language and are escalated regardless of score

If the system answers only the most confident 60 cases automatically and defers the remaining 40, then automation coverage is 0.60. If the automated subset is highly accurate and well grounded, and the human review process handles the deferred cases well, this design may outperform full automation on severe-error control even though fewer cases are answered automatically.

Notice what the score means in this example. A low automation-safety score does not mean "negative class" in the ordinary classification sense. It means "not safe enough to answer automatically under the current policy."

The right question is therefore not "can the model answer every message?" but "which messages should the system answer on its own, and which should trigger review or fallback?" That is a systems question. Once that action policy is written, the next question is how the team will notice when the live system drifts away from the assumptions that made the policy look acceptable.

> **Advanced Note.** Selective prediction works best when confidence scores are reasonably calibrated. If predicted probabilities do not reflect real uncertainty well, then thresholding by confidence may not separate easy and hard cases as cleanly as expected.

13.5 Monitoring After Deployment

Deployment is not the end of evaluation. Once a system is live, the environment changes. User behavior changes, interfaces change, sensors degrade, workload grows, and downstream policies change. Monitoring is the mechanism by which the system notices that reality has drifted away from the assumptions made during development. It is what turns pre-deployment assumptions into live checks rather than launch-time promises.

13.5.1 What Should Be Monitored

Useful monitoring often includes:

- **input quality**: missing fields, schema violations, extreme values
- **distribution change**: shifts in feature statistics or class balance
- **prediction behavior**: score distributions, abstention rates, subgroup gaps
- **exposure dynamics**: position bias, concentration of traffic, subgroup or catalog coverage in ranked systems
- **operational metrics**: latency, throughput, system errors, review backlog
- **outcome metrics**: post-deployment accuracy or task success when labels eventually arrive

For the course-support assistant, useful monitors would include policy-retrieval hit rate, citation coverage in answered messages, escalation rate by subgroup, advisor override rate, backlog length, and complaint or re-contact rate after automated replies.

13.5.2 Delayed Labels Make Monitoring Harder

Many real systems do not receive immediate ground truth. A support assistant may learn that an answer was inadequate only when an advisor corrects it later, when a student asks again, when a complaint arrives, or when a periodic audit catches a policy mistake. This delay means teams must monitor proxy signals as well as eventual outcomes. A silent increase in retrieval misses, escalation backlog, or answer overrides can be an early warning even before formal labels arrive.

> **Evidence Box.** Evidence for deployment should include a monitoring plan, alert thresholds, and a rollback path. A system without post-deployment visibility is not really under control, even if its offline evaluation was careful. Production rubrics such as the ML test score make monitoring, rollback, and ownership first-class deployment requirements rather than cleanup tasks [8].

13.5.3 Rollback and Recovery

Good systems assume that some deployments will go wrong. They therefore include ways to disable the model, revert to an earlier version, or fall back to a safer baseline. In some settings the fallback may be human review. In others it may be a conservative rule-based heuristic. Recovery planning is part of reliability, not a sign that the model is weak. The next section asks how to generate that post-launch evidence carefully instead of exposing the full user population all at once.

13.6 Post-Deployment Experimentation

Launching a new model to everyone at once creates two avoidable problems. First, it maximizes the blast radius if the new system behaves badly. Second, it makes it harder to learn whether the change actually improved the workflow under real usage. A safer pattern is staged release: observe the system without acting, expose a small slice of traffic first, and expand only when

the evidence remains favorable.

13.6.1 Shadow Mode and Canary Release

One common first step is *shadow mode*. The new model runs on live inputs, but its outputs are logged rather than used to drive the user-facing decision. This allows the team to compare the new system with the current one under realistic traffic, latency, and data-quality conditions before users are exposed to the behavior. Shadow mode is especially useful for catching missing fields, stale joins, calibration drift, or review-load surprises that were not visible offline.

If shadow results are acceptable, the next step is often a *canary release*: route a small fraction of real traffic to the new system while monitoring severe-error indicators, latency, override rate, and rollback triggers closely. The point is not to celebrate partial launch. The point is to limit damage while testing whether the deployment assumptions survive contact with production.

13.6.2 Online Experiments and Guardrails

For product changes where randomized exposure is acceptable, online controlled experiments provide the strongest evidence about whether a deployment helps real users. Kohavi et al. argue that trustworthy online experiments are valuable precisely because intuition and offline metrics can be misleading once a change reaches live traffic [37, 38]. In this chapter's language, an A/B test is not only a product-growth tool. It is a way to test whether a systems change improves the real workflow without silently degrading something else.

That requires guardrails. A system should not be judged only on one headline metric such as click-through rate, average response time, or overall resolution rate. A rollout scorecard should also track severe failures, subgroup disparities, complaint rate, human-review backlog, safety-rule violations, and any metric that would justify immediate rollback. A small gain on a surface metric does not justify launch if a guardrail metric gets worse.

Decision Box. Do not ship a new model to everyone simply because offline validation improved. Ask what staged release plan will test the change safely: shadow evaluation, canary traffic, randomized comparison, guardrail thresholds, and rollback conditions.

13.6.3 Not Every System Should Be Randomized

Some systems are too high-stakes, too asymmetric in harm, or too constrained by policy to support ordinary online experiments. In those settings the right pattern may be shadow evaluation, restricted rollout to low-risk cases, or delayed deployment until stronger evidence is available. The systems lesson is the same either way: deployment should generate evidence deliberately, not by exposing everyone to a poorly understood change.

13.7 Human-in-the-Loop Design

Human-in-the-loop design is often misunderstood. It does not simply mean adding a human somewhere in the pipeline. The question is whether the model and the human are arranged so that the combined system is better than either one alone. In this chapter's workflow, human review is not an afterthought added to rescue a weak model. It is one of the main ways a system turns uncertain predictions into bounded actions.

13.7.1 Humans Need the Right Interface

If the system presents confusing scores, hides uncertainty, or overloads reviewers with too many cases, then human oversight may be ineffective. A reviewer needs the right information at the right time: the model score, the reason for referral, the relevant features or examples, and the allowed actions. In the support-assistant case, that means the student's message, the retrieved policy passages, the draft answer if one exists, and the rule or uncertainty trigger that caused escalation.

13.7.2 Humans Also Introduce Failure Modes

People can become over-reliant on model outputs, ignore contradictory evidence, or become inconsistent under time pressure. This is sometimes called automation bias. A badly designed interface can therefore make the combined system worse than either the human or the model on its own.

> **Decision Box.** Adding a human reviewer does not automatically make a system safe. The review step must have clear triggers, enough time, adequate information, and authority to override the model when necessary.

13.8 Documentation, Ownership, and Iteration

A system that nobody owns will decay. Inputs change, staff turnover occurs, and the assumptions that were obvious during development are forgotten. Documentation helps preserve institutional memory, but documentation alone is not enough. Teams also need clear ownership of retraining, alert handling, audit requests, and rollback decisions. This is how a launch plan becomes a maintained service rather than a one-time demo.

13.8.1 What Documentation Should Capture

Useful documentation records:

- the task definition and intended use
- the training data sources and known limitations
- the feature pipeline and preprocessing assumptions
- the evaluation protocol and operating threshold
- the staged release plan, guardrail metrics, and rollback criteria
- subgroup, robustness, privacy, and safety results
- the monitoring plan and fallback behavior

This is not bureaucracy for its own sake. It is part of making the system understandable, maintainable, and auditable.

13.8.2 Iteration Should Be Scientific

Teams often respond to a production issue by making a fast change and hoping for improvement. A better pattern is scientific iteration: state the failure, hypothesize the cause, change one part of the system deliberately, reevaluate, and monitor whether the change improved the right metric without creating a worse problem elsewhere.

13.9 Chapter Artifact: A System Readiness Brief

By the end of this chapter, the reader should be able to produce a one-page system readiness brief before claiming that a model is deployable. The brief is the deployment counterpart to the earlier framing memo, evaluation plan, claim sheet, and reliability review card: it asks whether those earlier decisions still hold once the model becomes a service. It also condenses the same production questions emphasized by deployment checklists such as the ML test score [8].

> **Problem-Solving Technique. System readiness brief.** Before deployment, write down the decision owner, the operational requirement, the input contract, the operating policy, the release plan, the human review path, the monitoring and rollback plan, and the exact finding that would block launch.

Example 13.4 (Filled System Readiness Brief)**.** For the course-support assistant, a strong readiness brief would identify the primary user as the student-support organization rather than the model team; the operational requirement as bounded response time plus zero tolerance for unauthorized answers on high-risk categories; the input contract as current message text, course or program context when available, approved policy sources, and validated document freshness; the model plus fallback as

Brief field	What must be written clearly
Seven-question focus	All seven questions are reconciled operationally here, especially the decision being supported, the evidence required, and how prediction becomes action
Decision and primary user	Who acts on the output and what workflow is being improved
Operational requirement	Latency, capacity, severe-error bound, or other non-negotiable service constraint
Input contract	What fields, units, preprocessing rules, and validation checks must hold at serving time
Model plus fallback	What model is being used and what simpler fallback exists if conditions are not met
Operating policy	Threshold, ranking, abstention, or routing rule that turns scores into actions
Release plan	Whether launch begins in shadow mode, canary traffic, or randomized comparison, and what guardrails block expansion
Human review path	When a human is required, what they see, and what authority they have
Monitoring and rollback	What alerts, drift checks, and disable-or-revert path exist after launch
Owner and no-launch trigger	Who is responsible for the system and what finding would block deployment

Table 13.1: A system readiness brief forces the final chapter to answer the full deployment question, not merely to restate model quality.

a retrieval-backed assistant with a rule-based refusal or routing fallback when evidence is weak; the operating policy as "answer only when retrieval, confidence, and risk checks all pass"; the release plan as shadow mode first, then a limited canary on routine questions only, with rollback on citation-quality or override failures; the human review path as escalation to advisors or specialist staff with the retrieved evidence attached; the monitoring plan as drift checks on message wording, citation coverage, override rate, subgroup gaps, and backlog; and the no-launch trigger as any unresolved failure on severe slices such as visa, accommodation, or crisis-related messages.

Example 13.5 (Real-World Callout: Readiness Is a Launch Gate)**.** Breck et al. propose the ML test score as a production rubric precisely because offline accuracy is not enough to justify launch [8]. Their checklist asks whether the system has testable data assumptions, serving consistency, monitoring, reproducibility, and operational ownership before users are exposed to it. In this chapter's language, the readiness brief is not classroom paperwork. It is a compact no-launch gate for catching missing rollback paths, broken contracts, or absent owners before deployment turns those omissions into user-facing failures.

13.10 When Not to Build an AI System

The final systems lesson is restraint. Not every decision problem should become a machine-learning deployment. A strong readiness process must allow "do not launch" as a technical conclusion rather than treating deployment as the default reward for finishing a model.

13.10.1 Simple Rules Sometimes Win

If a stable rule solves the task well, then adding ML may only increase complexity, opacity, and maintenance burden. This is especially true when the input space is narrow and the desired behavior is already well specified.

13.10.2 The Organization May Lack the Needed Support

Even a promising model should not be deployed if the surrounding organization lacks feature quality checks, review capacity, monitoring, or ownership. In that case the limiting factor is not algorithmic performance. It is system readiness.

13.10.3 High Stakes Raise the Evidence Bar

Some settings demand stronger evidence, stronger fallback behavior, or stronger privacy protection than a team can currently

provide. In those settings, the correct decision may be to delay deployment, narrow the use case, or reject the system entirely.

13.11 Worked System Walkthrough: Course Support

To make the full chapter concrete, consider the grounded course-support pipeline developed across the earlier language, experiments, and reliability chapters. The walkthrough follows the same order as the readiness brief so that the chapter ends by assembling one coherent launch argument rather than introducing a separate example.

13.11.1 Step 1: Define the Service Objective

The institution's goal is not simply to maximize answer accuracy. It wants to help students get fast, reliable help on routine questions while reserving human attention for ambiguous, sensitive, or high-risk cases. The service succeeds only if response time improves, advisor workload stays manageable, and severe escalation misses are kept acceptably low.

13.11.2 Step 2: Define the Input Contract

The system receives the student's message text, course or program context when available, the current term, and a validated set of approved policy documents. Each field has a missing-value rule, format requirement, and freshness check. If the current-semester policy pages are stale, a course identifier is absent, or a document source fails validation, the system should not quietly answer anyway.

13.11.3 Step 3: Train and Evaluate the Components

The team evaluates routing, retrieval, and answer quality chronologically rather than randomly, because the real task predicts future student traffic from past semesters. It reports subgroup metrics, calibration or confidence quality, citation quality, and

robustness under wording shift, document reorganization, and missing-context scenarios.

13.11.4 Step 4: Choose the Action Policy

The system does not answer every message. It answers only when the request is in scope, the retrieved evidence comes from approved current sources, the confidence is high enough, and no escalation rule fires. Messages with policy ambiguity, weak evidence, unusual phrasing, or severe-risk content are routed to human staff. This policy is chosen because advising capacity is limited and because ambiguity is exactly where human authority matters.

13.11.5 Step 5: Design the Review Interface

When the system escalates a case, the advisor should not receive only a bare risk score. The review interface should show the student's message, the retrieved policy passages, the draft answer if one was prepared, the uncertainty or rule trigger, and the permitted actions. Otherwise the review step becomes a bottleneck or a rubber stamp rather than a real safeguard.

13.11.6 Step 6: Monitor and Iterate

After deployment, the team monitors document freshness, retrieval hit rate, subgroup escalation rates, advisor override frequency, backlog growth, and eventual quality audits. If the percentage of escalated cases rises sharply, that may indicate drift, a source-ingestion bug, or a confidence-calibration failure. The team can then investigate before performance collapses.

This walkthrough shows the chapter's central point: the useful system is not just the language model or retriever. It is the model plus contracts, routing policy, review interface, monitoring, and organizational ownership.

13.11.7 Contrast Case: A Pedestrian-Alert Service

The same readiness questions survive in the dashboard-camera pedestrian detector from Chapter 8. The service objective is no longer "answer routine questions quickly." It is "surface likely pedestrian hazards early enough to support a safe braking or alerting decision without flooding the driver or controller with false alarms." The input contract includes camera calibration, frame rate, lens geometry, synchronization, and health checks for glare or dropout. The operating policy might require agreement between detection and short-term tracking before a high-confidence alert is issued, with conservative slowdown behavior when the visual evidence degrades.

Monitoring also looks different without changing the systems logic. Useful signals include night-rain miss rate, alert rate by lighting condition, camera-health failures, latency spikes, and disagreement between the detector and downstream tracker. A no-launch trigger might be unacceptable failure on close-range pedestrians under dusk, glare, or unseen-camera slices even if average daytime metrics remain strong. The details differ from the support assistant, but the readiness brief still asks the same questions about requirements, contracts, fallback behavior, monitoring, and ownership.

13.12 Common Mistakes in Systems Work

Mistake Checklist.

- **Treating deployment as a final packaging step.** Deployment is part of the design problem from the beginning.
- **Optimizing the model while ignoring the workflow.** A slightly better benchmark score may be irrelevant if the system cannot be run or reviewed safely.
- **Ignoring training-serving skew.** A mismatch between training and production preprocessing

can break an otherwise good model.

- **Equating confidence with correctness.** Confidence only helps when scores are calibrated and fallback exists.
- **Deploying without monitoring and ownership.** A system without alerts, rollback, and owners is not robust enough for long-term use.
- **Shipping to everyone at once.** Expansion should earn its way through shadow evaluation or canary traffic.

13.13 Closing Perspective

This book began with problem framing and ends with system framing. That is a deliberate arc. Learning algorithms matter, but they are not the whole discipline. A mature understanding of AI connects representation, optimization, evaluation, reliability, and deployment into one coherent practice. The framing memo from Chapter 1, the evaluation plan, the claim sheet, the reliability review card, and the system readiness brief are not separate bureaucratic forms. They are the same discipline viewed at progressively more realistic levels, from defining the task to deciding whether the full service should exist at all. The final professional standard is not "can I train a model?" It is "can I build a system that improves the real task under the real constraints without hiding its limitations?"

Chapter Summary

- A deployed AI system includes data pipelines, feature contracts, thresholds, interfaces, monitoring, fallback behavior, and ownership in addition to the model itself.
- Operational requirements such as latency, review capacity, and severe-error control should be defined before training begins.

- Training-serving skew occurs when the deployed input pipeline differs from the evaluated one and can break a system even if the model file is unchanged.
- Thresholds and selective prediction are policy choices that trade off automation, workload, and risk.
- Ranking and recommendation systems shape future data because what they expose affects later clicks, interactions, and measured feedback.
- Monitoring, alerting, and rollback are part of the technical design of a reliable system.
- Staged release strategies such as shadow mode, canary rollout, and online experiments help teams learn safely before full launch.
- Human-in-the-loop design succeeds only when the review step has clear triggers, adequate information, and real authority.
- Some tasks should be simplified, delayed, or rejected because the full system evidence is not strong enough.
- The main deliverable of the chapter is a system readiness brief that forces the deployment claim to be written before launch.

Study Questions

1. Why can a high-performing offline model still produce a poor deployed system?
2. What makes an operational requirement different from an ordinary evaluation metric?
3. Why is training-serving skew a systems problem rather than only a modeling problem?
4. What is the purpose of selective prediction or abstention?
5. Why is click-through rate not the same thing as unbiased user preference in a ranked system?

6. Why is monitoring necessary even after careful pre-deployment evaluation?
7. Why might a team choose shadow mode or a canary release before a full rollout?
8. In what situations is refusing deployment the correct systems decision?
9. Why can a grounded support assistant with good answer quality still be unfit to launch as a real service?

Exercises

1. Give one example of a system requirement that cannot be captured by accuracy alone.
2. Explain how a review-capacity limit could change the best operating threshold for a classifier.
3. Describe one realistic source of training-serving skew for a text, vision, or tabular pipeline.
4. Give one example in which a human-in-the-loop system could still fail because of poor interface design.
5. Describe a monitoring signal that could warn of trouble before true labels are available.
6. For a recommendation feed or document-ranking system, name one guardrail metric you would monitor besides click-through rate and explain why.
7. Sketch one rollout plan that uses shadow mode, canary traffic, or both for a model update of your choice.
8. Give one case in which a rule-based system would be preferable to a learned model.
9. For a grounded support assistant or a dashboard-camera pedestrian detector, write one launch rule and one no-launch trigger.

10. Use Table 13.1 to sketch a one-page system readiness brief for a system of your choice. If you are carrying one case through the book, write it for the same system as your reliability review card and name one sentence from Chapter 1's framing memo that is now confirmed, narrowed, or rejected.

Challenge Exercises

1. A team proposes deploying a highly accurate model, but they have no clear feature contract and no rollback plan. Write the strongest technical objection you can in one paragraph.
2. Suppose a model is well calibrated and a review team can inspect only 20% of cases. Explain how you would design a selective-prediction policy and what evidence you would require before deployment.
3. Compare two failure responses to post-deployment drift: retraining the model immediately versus disabling the model and falling back to a simpler policy. Explain when each is justified and what risks each creates.
4. A team wants to replace its current model with a new one that improves offline quality but has never seen live traffic. Design the strongest staged-release plan you can, including one guardrail metric that would block expansion.
5. A university wants to launch a course-support assistant before the start of term, but the latest policy pages are still changing daily. Explain the strongest systems argument for delaying launch or narrowing the use case.

Appendix A

Mathematical Foundations for the Main Text

This appendix is not meant to replace a full course in calculus, linear algebra, probability, or statistics. Its purpose is narrower and more practical. The main chapters of this book are written at a later-bachelor level, but mathematically ambitious readers and self-learners may still want a compact bridge into the notation and habits of university-level machine learning. This appendix provides that bridge.

The right attitude toward this material is not to memorize every formula in isolation. The goal is to become comfortable reading notation inside an argument. If a chapter uses a dot product, a conditional probability, a loss function, a gradient, or a confidence interval, the reader should be able to answer three questions:

- what object is being described,
- what role the object plays in the model or evaluation,
- what the expression means in plain language.

Prerequisite Bridge. The appendix is designed

for legibility, not for maximum formal completeness. If you can read the symbols in context and connect them to the modeling story, then it is doing its job.

A.1 Scalars, Vectors, and Matrices

Machine learning is mostly about structured collections of numbers. Three objects appear constantly:

- a **scalar** is one number, such as a probability p, a loss value $\mathcal{L}$, or a coefficient β,
- a **vector** is an ordered list of numbers, such as a feature vector x or a parameter vector w,
- a **matrix** is a rectangular grid of numbers, such as a dataset X with one row per example and one column per feature.

If an example has d features, we may write it as

$$x = [x_1, x_2, \ldots, x_d]^\top.$$

This notation simply means that the example has d coordinates or feature values. The superscript $\top$ indicates transpose and is often used to write the vector as a column rather than a row.

If another vector of the same length is

$$w = [w_1, w_2, \ldots, w_d]^\top,$$

then their dot product is

$$w^\top x = \sum_{j=1}^{d} w_j x_j.$$

This is one of the most important operations in the book. A dot product is a weighted sum. In linear models, the vector w contains coefficients and the vector x contains features. The dot product therefore combines many signals into a single score.

Prerequisite Bridge. If the notation $w^\top x$ feels abstract, read it as "multiply each feature by its weight, then add the results." That informal sentence is often all you need in early chapters.

A.1.1 Matrices as Datasets

If a dataset has n examples and d features, a common representation is

$$X \in \mathbb{R}^{n \times d}.$$

This means that X is a matrix with n rows and d columns. The entry in row i and column j is often written as x_{ij}.

It is useful to keep the two index roles separate:

- i usually identifies the example,
- j usually identifies the feature.

So a formula such as

$$\hat{y}_i = \sum_{j=1}^{d} w_j x_{ij}$$

should be read in two layers. The subscript i identifies the example, and the subscript j moves across the features of that example. This is just a more explicit way of writing a dot product.

A.1.2 Matrix Multiplication as Many Dot Products at Once

If we multiply the dataset matrix X by the parameter vector w, we get

$$Xw = \begin{bmatrix} x_1^\top w \\ x_2^\top w \\ \vdots \\ x_n^\top w \end{bmatrix}.$$

This means that matrix multiplication is often just a compact way to compute one linear score for each example in the dataset. In practice, a great deal of machine-learning notation is simply compressed bookkeeping for many repeated dot products or weighted sums.

A.2 Summation, Means, and Indexing

Machine-learning formulas often use summation notation because datasets contain many examples and each example contains many features. For instance,

$$\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

means: for each example indexed by i from 1 to n, compute the squared difference between the true target y_i and the prediction $\hat{y}_i$, then add all those values together.

Several recurring symbols are worth reading fluently:

- $\sum$ means add many terms,
- $\prod$ means multiply many terms,
- $\frac{1}{n}\sum_{i=1}^{n} a_i$ means the average of the values $a_1, \ldots, a_n$.

The sample mean of numbers $a_1, \ldots, a_n$ is

$$\bar{a} = \frac{1}{n}\sum_{i=1}^{n} a_i.$$

This appears everywhere: average loss, average accuracy over runs, average error on a slice, or average confidence of a prediction rule.

A.2.1 Why Means Matter

Many quantities in machine learning are empirical averages. For example:

- training loss is usually the average loss across training examples,
- validation accuracy is the average correctness across validation examples,
- repeated-run summaries often report the average metric across seeds.

This is conceptually helpful. A machine-learning objective often looks complicated, but many of them are simply averages of per-example penalties.

A.3 Functions That Appear Everywhere

A.3.1 Logarithms and Exponentials

Logarithms and exponentials appear throughout machine learning because they are tightly connected to probabilities, growth, and optimization.

The natural exponential function is written as e^z, and its inverse is the natural logarithm $\log z$. Two properties matter constantly:

$$\log(ab) = \log a + \log b, \qquad \log\left(\frac{a}{b}\right) = \log a - \log b.$$

These properties make logarithms useful for turning products of probabilities into sums, which are easier to analyze and optimize.

A.3.2 The Sigmoid Function

The sigmoid function is

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

It turns any real number into a value between 0 and 1. Large positive values of z map to probabilities near 1, large negative values map to probabilities near 0, and $z = 0$ maps to 0.5. This is why logistic regression uses the sigmoid: it converts a linear score into a probability-like output.

A.3.3 The Softmax Function

For multiclass prediction, a model often needs a probability distribution over several classes rather than one binary probability. If a model produces class scores $z_1, \ldots, z_K$, the softmax is

$$\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{\ell=1}^{K} e^{z_\ell}}.$$

This transforms arbitrary real-valued scores into nonnegative values that sum to 1. Conceptually, the softmax says: larger scores receive larger probabilities, but all class probabilities must fit into one normalized distribution.

> **Prerequisite Bridge.** You do not need to memorize the softmax formula immediately. It is enough at first to know that it turns class scores into a probability distribution over classes.

A.4 Probability Language

If a model predicts a probability, the reader should know what that means. A probability such as $P(y = 1 \mid x)$ is read as "the probability that $y = 1$ given x." The vertical bar means "given" or "conditioned on."

Conditional probability matters because machine learning almost always predicts targets *given* inputs. A spam classifier does not estimate the overall fraction of spam in the universe. It estimates the probability that a message is spam *given the features of that message.*

Several ideas return frequently:

- a probability lies between 0 and 1,
- conditional probabilities depend on what information is available,
- a model can assign probabilities even when it is uncertain, and that uncertainty can itself be useful.

A.4.1 Random Variables and Distributions

A **random variable** is a quantity whose value is uncertain. In the book, Y often denotes the target variable and X the input. A distribution describes how likely different values are. For a binary target, we might write

$$P(Y = 1) = p, \qquad P(Y = 0) = 1 - p.$$

For a multiclass target with classes $1, \ldots, K$, a model might output probabilities

$$P(Y = 1 \mid x),\ P(Y = 2 \mid x),\ \ldots,\ P(Y = K \mid x),$$

which together sum to 1.

A.4.2 Joint, Marginal, and Conditional Probability

Three related probability ideas appear often:

- a **joint probability** such as $P(X, Y)$ describes two variables together,
- a **marginal probability** such as $P(Y)$ ignores other variables,
- a **conditional probability** such as $P(Y \mid X)$ asks about one variable given another.

The relation

$$P(X, Y) = P(Y \mid X)P(X)$$

is one of the basic identities that underlies probabilistic modeling. It simply says that the probability of both events occurring can be decomposed into first seeing X and then seeing Y given X.

A.4.3 Bayes' Rule

Bayes' rule is

$$P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)}.$$

This matters because it connects a posterior probability, which is often what we want, to a likelihood and a prior. In practical machine learning, Bayes' rule is most important conceptually. It reminds us that predictions can change when either the evidence or the prior assumptions change.

A.5 Expectation, Variance, and Covariance

A.5.1 Expectation

The expectation of a random variable is its long-run average or average value under the distribution. For a discrete variable Z taking values z,

$$\mathbb{E}[Z] = \sum_z z\, P(Z = z).$$

In machine learning, expectation appears whenever we speak about expected loss, expected error, or average performance under a data-generating process. Empirical averages over a dataset are often used as estimates of these theoretical expectations.

A.5.2 Variance

Variance measures spread:

$$\mathrm{Var}(Z) = \mathbb{E}\left[(Z - \mathbb{E}[Z])^2\right].$$

If the variance is small, values tend to stay near the mean. If it is large, values vary more widely.

This matters for repeated runs, prediction uncertainty, and confidence intervals. A model with strong mean performance but high variability may be less dependable than a slightly weaker but more stable model.

A.5.3 Covariance and Correlation

Covariance measures whether two quantities move together:

$$\mathrm{Cov}(X, Y) = \mathbb{E}\left[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\right].$$

If the covariance is positive, larger values of one variable tend to occur with larger values of the other. Correlation rescales covariance into a unitless measure between -1 and 1.

This matters because machine-learning features are rarely independent in practice. Correlated features can make coefficient interpretation harder, can change optimization behavior, and can

explain why a model seems to discover signal that is really a proxy for something else.

A.6 Loss Functions and Objectives

Learning usually means choosing parameters that make a loss function small. A **loss** assigns a penalty to one prediction on one example. An **objective** is often an average of those losses across the dataset.

A.6.1 Squared Error

For regression, one common loss is squared error:

$$\ell(y, \hat{y}) = (y - \hat{y})^2.$$

The average squared error across the dataset is

$$\mathcal{L}(w) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2.$$

Large errors are penalized more heavily than small ones because of the squaring.

A.6.2 Log Loss and Cross-Entropy

For binary classification, if a model outputs predicted probability $\hat{p}_i$ for the positive class, the log loss on example i is

$$\ell_i = -\left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right].$$

This loss becomes very large when the model assigns low probability to the true label. That is one reason log loss is useful: it punishes confident mistakes strongly.

For multiclass classification, the same idea generalizes to cross-entropy. Conceptually, cross-entropy rewards assigning high probability to the true class and penalizes spreading too much probability mass onto wrong classes.

Math Lens. Loss functions are not just optimization devices. They encode what kinds of mistakes the training process will care about. That is why Chapter 2 treats metric choice and Chapter 11 treats bounded-failure choice as part of the real modeling problem.

A.6.3 Regularized Objectives

Often we do not minimize only the data-fit term. We add a regularization penalty:

$$\mathcal{L}_{\text{reg}}(w) = \frac{1}{n}\sum_{i=1}^{n} \ell_i + \lambda\Omega(w),$$

where $\Omega(w)$ measures the size or complexity of the parameter vector and λ controls how strongly we penalize it.

This is a clean mathematical way to express a modeling preference: fit the data, but do not do so with unnecessarily extreme or unstable parameter values.

A.7 Derivatives, Gradients, and the Chain Rule

Modern machine learning relies heavily on optimization, and optimization relies heavily on derivatives. A derivative measures how a quantity changes when one variable changes slightly. If a function $f(x)$ has derivative $f'(x)$, then $f'(x)$ tells us the local slope of the function at x.

A.7.1 Partial Derivatives and Gradients

In many dimensions, the analogue of the derivative is the gradient. If a function depends on many parameters, the gradient collects the partial derivatives with respect to each parameter:

$$\nabla_\theta \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2}, \dots\right]^\top.$$

Why is this useful? Because the gradient points in the direction of increasing loss. If we want to reduce the loss, we usually move in the opposite direction.

A.7.2 Gradient Descent

The basic gradient-descent update is

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L},$$

where η is the learning rate. The symbol $\leftarrow$ means "update the old value to the new value."

This rule says: compute how the loss changes with respect to each parameter, then move the parameter vector a little in the direction that reduces the loss. Nearly all modern neural-network training is a scaled-up variant of this idea.

A.7.3 Why the Learning Rate Matters

If the learning rate is too small, optimization moves slowly. If it is too large, updates may overshoot and become unstable. This is one reason Chapter 5 emphasizes training curves and optimization diagnosis. The math is simple; the behavior can still be tricky.

A.7.4 The Chain Rule

The chain rule explains how derivatives pass through compositions of functions. If

$$z = g(x) \qquad \text{and} \qquad y = f(z),$$

then

$$\frac{dy}{dx} = \frac{dy}{dz}\frac{dz}{dx}.$$

This matters because neural networks are layered compositions of simpler transformations. Backpropagation is largely an organized application of the chain rule to many nested functions.

Prerequisite Bridge. You do not need to derive backpropagation from scratch to read the main chapters. At first, it is enough to know that gradients tell the model how to adjust parameters so the loss goes down, and that the chain rule is what lets that signal move backward through many layers.

A.8 Norms, Distance, Similarity, and Regularization

When the book discusses regularization, representation size, or nearest neighbors, it often refers to norms and distances.

A.8.1 Norms

Two especially common norms are:

$$\|w\|_1 = \sum_{j=1}^{d} |w_j|, \qquad \|w\|_2^2 = \sum_{j=1}^{d} w_j^2.$$

The first adds absolute values. The second adds squared values. These are ways of measuring the size of the coefficient vector. In regularized models, we penalize large norms because we want to discourage overly extreme solutions unless the data truly supports them.

A.8.2 Distance

The Euclidean distance between two vectors x and x' is

$$\|x - x'\|_2 = \sqrt{\sum_{j=1}^{d} (x_j - x'_j)^2}.$$

This matters in nearest-neighbor methods, clustering, and geometric intuition about representation spaces.

A.8.3 Cosine Similarity

Cosine similarity between vectors x and x' is

$$\cos(\theta) = \frac{x^\top x'}{\|x\|_2 \|x'\|_2}.$$

It measures directional similarity rather than absolute magnitude similarity. This is useful in text retrieval and embedding-based similarity, where the angle between vectors can matter more than their lengths.

A.9 Statistics for Evaluation

The book repeatedly argues that model claims depend on disciplined evidence. That requires a little statistical language.

A.9.1 Sample Mean and Sample Standard Deviation

If repeated runs produce metrics $m_1, \ldots, m_S$, the sample mean is

$$\bar{m} = \frac{1}{S} \sum_{s=1}^{S} m_s.$$

The sample standard deviation is

$$\sigma = \sqrt{\frac{1}{S-1} \sum_{s=1}^{S} (m_s - \bar{m})^2}.$$

The mean summarizes typical performance. The standard deviation summarizes instability across runs.

A.9.2 Standard Error and Confidence Intuition

The **standard error of the mean** is roughly

$$\mathrm{SE} = \frac{\sigma}{\sqrt{S}}.$$

This quantity gets smaller as we average more repeated runs, all else equal. A confidence interval uses the standard error to express uncertainty about the mean. The exact formulas depend

on assumptions, but the core interpretation is simple: the estimate is not exact, and uncertainty should be reported rather than ignored.

> **Math Lens.** The practical lesson is more important than the formula. If one method beats another by a tiny amount and the repeated-run uncertainty is similar in size, then the scientific conclusion should stay cautious.

A.9.3 Confusion Matrix Language

For binary classification, four counts recur:

- true positives (TP),
- false positives (FP),
- true negatives (TN),
- false negatives (FN).

From these counts, several metrics are built:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}},$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

These formulas matter because they reveal which kinds of error the metric is sensitive to. Accuracy can look strong even when recall on a critical minority class is poor. That is why later chapters insist on slice analysis and metric choice tied to the application.

A.9.4 Thresholds

Many probabilistic classifiers produce a score or probability and then turn it into a discrete decision using a threshold. For example, a system may predict class 1 when

$$P(Y = 1 \mid x) > 0.5.$$

But 0.5 is not sacred. If false negatives are more costly than false positives, a lower threshold may be justified. If automation should be conservative, a higher threshold may be justified. Threshold choice is therefore part of the decision policy, not just a final formatting step.

A.9.5 Calibration

Students should also distinguish probability from frequency in one sample. If a model assigns probability 0.8 to many similar cases, we hope that about 80% of them truly belong to the positive class. This is the intuition behind calibration.

Calibration matters especially when scores drive downstream action. If a support assistant uses confidence to decide whether to answer directly or escalate to a human, then badly calibrated confidence can become a safety problem, not just a cosmetic modeling problem.

A.10 Linear Algebra for Representation Intuition

Some of the book's deeper modeling chapters use representation language that becomes easier if a few linear-algebra ideas feel natural.

A.10.1 Linear Maps

A matrix A can be read as a linear transformation. When we multiply a vector x by A, we get a new vector Ax. In machine learning, this often means one representation is being transformed into another:

- raw features become hidden features,
- embeddings are projected into a new space,
- a classifier turns a representation into class scores.

A.10.2 Projection Intuition

If a vector is projected onto a direction, we are asking how much of the vector lies along that direction. This is one reason dot products are so central: they measure alignment. In a rough geometric sense, linear models ask how strongly an example aligns with a learned direction in feature space.

A.10.3 Eigenvectors and Principal Components

You do not need a full course in spectral methods to read this book, but one intuition is useful. Principal component analysis searches for directions in the data that capture high variance. Those directions are related to eigenvectors of a covariance matrix. The important modeling lesson is not the algebraic machinery itself. It is that high-dimensional data often contains lower-dimensional structure, and linear algebra provides tools for describing it.

A.11 A Tiny Bridge to Optimization Practice

The theory above becomes more useful when tied to the practical questions that appear in the main chapters:

- if training loss falls but validation loss rises, think about overfitting rather than only optimization success,
- if gradients vanish or explode, think about scale, architecture, and training stability,
- if repeated runs vary widely, think about variance and uncertainty rather than only mean performance,
- if a score threshold changes the operational behavior, think about decision policy rather than only model output.

This is why the book does not teach mathematics as a separate ceremonial layer. The point of the symbols is to sharpen judgment about models, evidence, and deployment claims.

A.12 How to Use This Appendix

If a chapter feels mathematically dense, do not stop at the first unfamiliar symbol. Instead, ask three questions:

1. What object is this notation describing: an example, a feature, a weight, a prediction, a loss, a probability, or an update?
2. Is the formula adding things, comparing things, transforming one quantity into another, or summarizing uncertainty?
3. What does the formula mean in plain language?

That habit matters more than speed. A mathematically ambitious reader or later-bachelor reader does not need the book to be simpler. They need the book to make each technical step legible. This appendix exists to support exactly that.

Conclusion

This book has argued for a particular way of teaching the machine-learning core of modern AI. Start from problems, not fashions. Treat evaluation as a first-class idea. Connect classical methods with deep learning through the common language of representation and generalization. Teach modalities as variations on shared modeling questions rather than as isolated silos. End not with a model in isolation, but with a system whose data contracts, thresholds, review paths, monitoring, and limits are part of the technical design.

If the book has succeeded, a reader should now be able to take a vague AI idea and turn it into a bounded machine-learning proposal. That means naming the decision being supported, choosing a defensible proxy task, inspecting what the dataset actually represents, selecting a serious baseline, deciding what evidence would justify a claim, and tracing how a prediction becomes an action inside a real workflow. Those are not final-specialization skills. They are the habits that make later specialization trustworthy.

That is why the book has spent so much time on framing, baselines, leakage, calibration, slices, claim scope, reliability review, and deployment readiness. These topics are sometimes treated as secondary to the "real" content of machine learning. They are not secondary. They are the discipline that keeps modeling from collapsing into score chasing or software theater.

The scope has also been deliberate. This is not a full survey of artificial intelligence. It does not try to cover search, planning,

robotics, reinforcement learning, causal inference, or advanced probabilistic modeling at reference-book depth. Its job has been narrower and more practical: to give students a coherent first serious path through machine learning, evaluation, reliability, and systems thinking. A reader who finishes this book should be better prepared to study deeper mathematics, stronger deep-learning texts, specialized modality courses, or production ML systems because the conceptual spine is already in place.

The next step depends on the reader's goal. A mathematically ambitious reader can continue into probability, optimization, and advanced learning theory. A systems-oriented reader can go deeper into data engineering, experimentation, deployment, monitoring, and governance. A modality-focused reader can push further into vision, language, speech, or sequential decision-making. But whichever path comes next, the standard should remain the same: make the problem explicit, make the claim precise, and make the evidence do the work.

If students finish this book with one enduring habit, it should be this: whenever they encounter an AI system, they should ask what was learned, from which data, under which assumptions, for which decision, through which pipeline, and with what evidence. Those questions do not merely criticize AI. They make real understanding possible, and they are what allow a model builder to become a reliable practitioner.

Bibliography

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

[2] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*, 2019.

[3] Apache Software Foundation. Apache spamassassin. `https://spamassassin.apache.org/`, 2026. Accessed March 6, 2026.

[4] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013.

[5] Johannes Bernett, David B. Blumenthal, Dennis G. Grimm, et al. Guiding questions to avoid data leakage in biological machine learning applications. *Nature Methods*, 21:1444–1453, 2024. doi: 10.1038/s41592-024-02362-y.

[6] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[7] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al.

On the opportunities and risks of foundation models. Technical report, Center for Research on Foundation Models, Stanford University, 2021.

[8] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. What's your ml test score? a rubric for ml production systems. In *Reliable Machine Learning in the Wild Workshop at NeurIPS*, 2016.

[9] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

[10] Joy Buolamwini and Timnit Gebru. Gender shades: Intersectional accuracy disparities in commercial gender classification. In *Proceedings of Machine Learning Research*, volume 81, pages 1–15, 2018.

[11] Mathilde Caron, Hugo Touvron, Ishan Misra, Herve Jegou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9650–9660, 2021.

[12] Centers for Disease Control and Prevention. Conducting trend analyses of yrbs data. `https://www.cdc.gov/yrbs/media/pdf/2023/2023_yrbs_conducting_trend_analyses.pdf`, 2023. Accessed March 6, 2026.

[13] Centers for Disease Control and Prevention. Marginal effects: Quantifying the effect of changes in risk factors in logistic regression models. `https://stacks.cdc.gov/view/cdc/222733`, 2026. CDC Stacks, accessed March 6, 2026.

[14] Centers for Disease Control and Prevention. Rapid influenza diagnostic tests. `https://www.cdc.gov/flu/hcp/testing-methods/clinician_guidance_ridt.html`, 2026. Accessed March 6, 2026.

[15] Jiawei Chen, Hande Dong, Yang Qiu, Xiangnan He, Xin Xin, Liang Chen, Guli Lin, and Keping Yang. Autodebias: Learn-

ing to debias for recommendation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 21–30, 2021. doi: 10.1145/3404835.3462919.

[16] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, pages 1597–1607, 2020.

[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[18] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography*, volume 3876 of *Lecture Notes in Computer Science*, pages 265–284. Springer, 2006. doi: 10.1007/11681878_14.

[19] Manuela Ekowo and Iris Palmer. The promise and peril of predictive analytics in higher education: A landscape analysis. `https://www.newamerica.org/education-policy/policy-papers/promise-and-peril-predictive-analytics-higher-education/`, 2016. New America policy paper.

[20] Montgomery L. Flora, Corey K. Potvin, Patrick S. Skinner, Shawn Handler, and Amy McGovern. Using machine learning to generate storm-scale probabilistic guidance of severe weather hazards in the warn-on-forecast system. *Monthly Weather Review*, 149(5):1535–1557, 2021. doi: 10.1175/MWR-D-20-0194.1.

[21] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. Datasheets for datasets. *arXiv preprint arXiv:1803.09010*, 2018.

[22] Yonatan Geifman and Ran El-Yaniv. Selective classification

for deep neural networks. *arXiv preprint arXiv:1705.08500*, 2017.

[23] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[24] Jean-Bastien Grill, Florian Strub, Florent Altche, et al. Bootstrap your own latent: A new approach to self-supervised learning. *Advances in Neural Information Processing Systems*, 33:21271–21284, 2020.

[25] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330, 2017.

[26] Moritz Hardt, Eric Price, and Nati Srebro. Equality of opportunity in supervised learning. In *Advances in Neural Information Processing Systems*, volume 29, pages 3315–3323, 2016.

[27] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2 edition, 2009.

[28] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9729–9738, 2020.

[29] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[30] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. *arXiv preprint arXiv:1903.12261*, 2019.

[31] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, et al. Parameter-efficient transfer learning for NLP. *Proceedings*

of the 36th International Conference on Machine Learning, pages 2790–2799, 2019.

[32] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, pages 328–339, 2018.

[33] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[34] Rob J. Hyndman and George Athanasopoulos. Forecasting: Principles and practice. `https://otexts.com/fpp3/`, 2021. 3rd edition, OTexts, Melbourne, Australia; accessed March 12, 2026.

[35] Thorsten Joachims, Laura Granka, Bing Pan, Helene Hembrooke, and Geri Gay. Accurately interpreting clickthrough data as implicit feedback. In *Proceedings of the 28th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 154–161, 2005. doi: 10.1145/1076034.1076063.

[36] Bryan Klimt and Yiming Yang. The enron corpus: A new dataset for email classification research. In *Machine Learning: ECML 2004*, pages 217–226, 2004. doi: 10.1007/978-3-540-30115-8_22.

[37] Ron Kohavi, Alex Deng, Brian Frasca, Roger Longbotham, Toby Walker, and Ya Xu. Trustworthy online controlled experiments: Five puzzling outcomes explained. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 786–794, 2012.

[38] Ron Kohavi, Alex Deng, Brian Frasca, Toby Walker, Ya Xu, and Nils Pohlmann. Online controlled experiments at large scale. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1168–1176, 2013.

[39] Simon Kornblith, Jonathon Shlens, and Quoc V. Le. Do better ImageNet models transfer better? *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2661–2671, 2019.

[40] Neil Kumaran. New gmail protections for a safer, less spammy inbox. `https://blog.google/products-and-platforms/products/gmail/gmail-security-authentication-spam-protection/`, 2023. Google blog, accessed March 6, 2026.

[41] Jakub Kuzilek, Martin Hlosta, and Zdenek Zdrahal. Open university learning analytics dataset. *Scientific Data*, 4: 170171, 2017. doi: 10.1038/sdata.2017.171.

[42] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[43] Patrick Lewis, Ethan Perez, Aleksandara Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[44] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schutze. *Introduction to Information Retrieval.* Cambridge University Press, 2008.

[45] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[46] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction.* MIT Press, 2022.

[47] NOAA National Severe Storms Laboratory. Probability forecasting. `https://www.nssl.noaa.gov/~brooks/prob/Probability.html`, 2026. Accessed March 6, 2026.

[48] Judea Pearl. *Causality: Models, Reasoning, and Inference.* Cambridge University Press, 2 edition, 2009.

[49] Matthew E. Peters, Mark Neumann, Mohit Iyyer, et al.

Deep contextualized word representations. In *Proceedings of NAACL-HLT*, pages 2227–2237, 2018.

[50] Joelle Pineau, Philippe Vincent, Shagun Gugger, David Azria, Victor C. Romero, and Yoshua Bengio. Improving reproducibility in machine learning research: A report from the neurips 2019 reproducibility program. *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[51] Lawrence R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.

[52] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763, 2021.

[53] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, Jose J. Lahoz-Monfort, Boris Schroder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017. doi: 10.1111/ecog.02881.

[54] Aaron Rotenberg. Syntax tree.svg. `https://commons.wikimedia.org/wiki/File:Syntax_tree.svg`, 2008. Wikimedia Commons, public domain, accessed March 6, 2026.

[55] Nithya Sambasivan, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora M. Aroyo. "everyone wants to do the model work, not the data work": Data cascades in high-stakes ai. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–15, 2021.

[56] Maarten Sap, Swabha Swayamdipta, Laura Vianna, Xuhui Zhou, Yejin Choi, and Noah A. Smith. Annotators with attitudes: How annotator beliefs and identities bias toxic language detection. In *Proceedings of the 2022 Conference of*

the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 5884–5906, 2022. doi: 10.18653/v1/2022.naacl-main.431.

[57] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[58] Glenn Shafer and Vladimir Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9:371–421, 2008.

[59] Harini Suresh and John V. Guttag. A framework for understanding sources of harm throughout the machine learning life cycle. In *Proceedings of the 2021 ACM Conference on Equity and Access in Algorithms, Mechanisms, and Optimization*, 2021. doi: 10.1145/3465416.3483305.

[60] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[62] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, pages 3320–3328, 2014.

[63] John R. Zech, Marcus A. Badgeley, Manway Liu, Anthony B. Costa, Joseph J. Titano, and Eric Karl Oermann. Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study. *PLOS Medicine*, 15(11):e1002683, 2018. doi: 10.1371/journal.pmed.1002683.